

无线电干扰源的快速自动定位与清除

——基于集员估计的交会定位、稳健选址与方向完备在线清除

摘 要

附录中“同一地点重复检测不会改变检测误差”一句决定了全文的误差范式：测向误差是与地点绑定的有界确定性质，而非可由重复测量平均掉的随机噪声，靠 $1/\sqrt{N}$ 降噪的路径被堵死，以分布假设为前提的最小二乘、极大似然点估计失去依据。本文因而全程采用**集员估计**：每次测向给出干扰源必然落入的角域，角域之交即位置的保证集，全部结论都是可严格证明的最坏情形结论。四问由同一范式贯穿：前两问给出可证明的几何结论，后两问把这些结论装配成在线控制系统，并把“可证明的保证”与“演练标定的工程参数”严格分开。

针对**第一问**，先把 $\pm 1^\circ$ 示向度误差化为顶点在 S_i 、张角 2° 的角域，再用二维叉积把角域等价改写为两个线性半平面，一举绕开斜率发散与 $0^\circ/360^\circ$ 跨界两类退化。定位区域 Ω 是 $2n$ 个闭半平面之交，必为含真实源的闭凸集；由回收锥证得有界的充要条件是 n 段示向弧之交为空。算法用 Sutherland-Hodgman 裁剪求有序顶点，凭“凸集直径必在极点对取得”把连续优化降为有限枚举，再以旋转卡壳在 $O(m)$ 时间扫描对踵点，总复杂度 $O(n^2)$ ；两检测点情形另给出闭式解，与裁剪结果的相对偏差不超过 10^{-13} 。对“以直径为直径的圆能否覆盖定位区域”，本文给出**完全判定**：覆盖圆圆心只能是最远点对中点，充要条件为“存在顶点对使其余各顶点对之张角均不小于 90° ”，并导出一眼可用的推论——只要有三个顶点内角小于 90° ，覆盖必然失败。两检测点情形的充要条件为交会角 $\gamma \notin (90^\circ, 92^\circ)$ ，算例 $\gamma = 91^\circ$ 中有顶点超出直径圆 0.307 m（相对 1.76%），构成确凿反例；三点绕源 120° 布设时 Ω 退化为近似等边三角形，最小覆盖圆半径达 $D/2$ 的 1.133 倍，几乎取到 Jung 界 $2/\sqrt{3} \approx 1.1547$ 。故工程上必须以**最小覆盖圆**而非直径圆作为不确定性包络。

针对**第二问**，困难在于目标函数依赖未知量：单次测向只给方向不给距离，而 D 同时依赖 γ, r_1, r_2 。本文把源位置不确定性写成先验可行集 $\mathcal{U}$ （长 1480 m、末端半宽仅 26 m 的极细扇形），以最坏源位置下的定位直径 J_{rob} 为目标、以“ $\mathcal{U}$ 中每一点都被保证接收”为硬约束，建立可靠性约束下的稳健优化模型。三重几何约束交出候选区域：保证接收透镜、领结形交会禁区、稳健直径的近优水平集。求解揭示出一对清晰的对峙：**约束由近端源决定，目标由远端源决定**。最优点落在近端接收圆的边界弧上，距 S_1 约 1016 m、与示向方向成 34.7° ， $J_{\text{rob}} = 119.4$ m；候选区域为关于示向线对称的两支窄带，区内最坏定位直径不超过最优值的 1.05 倍。权衡分析表明，若允许 1 个百分点的保证接收比例损失，稳健直径可降至 99.4 m（改善 16.8%），且该让步带确定性回退分支——收不到信号恰好证明源距 S_1 不足 151 m，沿示向线前进即可完成定位。

针对**第三问**，难点从“已知观测求最优几何”变为“未知源数下的在线决策”：源数 $N \in [10, 16]$ 、频道、位置与有效接收半径全部未知，每一次信息获取都要付出时间代价。本文把完备性与效率分离处理：完备性由可证明的几何与逻辑保证，效率由在线启发式逼近。取原点与半径 1010 m 外环上七个等角点构成八点搜索骨架，证明其最坏覆盖距离为 992.057 m，严格小于全向源最小有效接收半径 1000 m，故任何频道若在八点全部无信号即可判定为空；由可行性判别式还可证明该族的最小骨架为七点。沿用集员范式维护每个频道的位置后验，圆盘一律用外接正多边形离散，使 $\mathcal{F}_{\text{true}} \subseteq \mathcal{F}_{\text{code}}$ 恒成立，从而“最小包围圆半径 ≤ 20 m 则在圆心清除必成功”成为严格结论。注意到清除成本高度不对称（成功 5 s、失败仅 3 s），本文导出试探清除的有利条件 $p > 3/(\Delta T + 3)$ ，仅需约 6% 的成功率即划算，据此设置三类试探清除。在线调度把所有动作折算为“距离减等效奖励”统一排序，再对当前强制任务集合用 bitmask 动态规划精确求开放最短 Hamilton 路径，仅当路径 regret 不小于 250 m 时才覆盖原决策。三次正式测试中清除数与认证空频道数之和均为 20，被清除干扰源个数的比例均为 100%，平均定位清除时间分别为 228.15、236.49、238.82 s。

针对**第四问**，定向源的引入使第三问的空频道判据失效：某频道在全部检测点无信号，既可能无源，也可能所有检测点都落在其辐射半平面之外。本文由此把完备性要求从“覆盖位置”加强为“覆盖位置与方向的乘积空间”，并证明一条关键等价：一组检测点能对位于 G 的任意朝向定向源保证可见，当且仅当 G 落在这组点中距 G 不超过 $R_{\min}$ 的那些点的凸包内。据此构造**中心 1 点 + 990 m 内环 7 点 + 外接正十四边形外环 14 点**的 22 点方向完备骨架，并以 2 m 网格离线证书证明：目标圆内任意位置的 997 m 邻域搜索点集，其凸包以不小于 8.6016 m 的余量包含该格点

(距离余量 1.586 m、凸包余量 7.187 m)，从而恢复严格判空。同时证明仅由目标区域内部点构成的骨架不可能方向完备，外环是必需的。针对定向源失联，给出“前向对 + 侧向对”四点恢复扇及其可见性证明；针对方向后验难以收缩到 20 m 的情形，给出步长 27.72 m 的确定性网格覆盖清除，给出定位阶段的有限步终止上界。调度上采用固定最短搜索环、插入机会成本判据与搜索后开放 2-opt，最后叠加只吃**严格局部支配**收益的 SAFE 层：仅当替换点仍严格保证清除、且两动作总移动严格更短时才挪动清除点，并强制锚定原调度的下一动作，从而保证“同一前状态 $\rightarrow$ 更短或相等路径 $\rightarrow$ 同一后状态”，绝不以长尾退化换平均收益。42 轮演练共清除 535 个源，加权平均 489.89 s/源；三次正式测试清除干扰源个数分别为 16、11、12，清除比例均为 100%，平均定位清除时间分别为 354.23、572.39、497.01 s。

本文特色：范式选择由题目原话推出，结论皆可证明；覆盖性给出充要判定并指出 Jung 界几乎可取到；第二问揭示的“近端定约束、远端定精度”分离结构把高维搜索降为低维问题；第四问把方向完备性化归为凸包包含，给出可机器校验的离线证书，并严格区分“可证明的保证”与“演练标定的工程参数”。

关键词：集员估计；交会定位区域；半平面交与旋转卡壳；直径圆覆盖判定；最小覆盖圆；稳健选址；覆盖搜索完备性；方向完备与凸包判据；在线调度；局部支配优化

1 问题重述与问题分析

1.1 问题重述

干扰源分布区域为半径 $R_T = 1800$ m 的圆域 $\mathcal{D}$ ，取圆心为原点、正东为 x 轴正向、正北为 y 轴正向。测向机在检测点给出干扰源的示向度，读数与真实方位角之差落在 $[-1^\circ, 1^\circ]$ 内；有效接收半径在 1000 至 1500 m 之间；机器狗以 5 m/s 直线行进，单次检测耗时 5 s、切换频道 1 s，光学精确定位需在 20 m 以内（定位 3 s、清除 2 s，清除成功计 5 s、失败计 3 s）。

- **问题一** 已知若干检测点坐标及同一干扰源关于这些点的示向度，给出用交会定位法计算多边形定位区域直径的算法，并回答以该直径为直径的圆能否覆盖此定位区域。
- **问题二** 已知某全向干扰源在一个检测点处测得的示向度，给出第二个检测点的选择策略与候选区域。
- **问题三** 目标区域内均为全向干扰源，总数在 10 至 16 之间但具体个数未知。机器狗自原点出发、初始频道为 1，需在确保所有干扰源被清除的前提下尽量缩短总时间。要求给出自动搜索、定位与清除的策略及算法，通过模拟器演练测试并给出清除比例与平均定位清除时间，最后报告三次正式测试的结果。
- **问题四** 目标区域中既有全向干扰源，又有定向干扰源，总数仍为 10-16 个之间，但具体个数未知，定向干扰源的个数及定向方向也未知，其他条件与问题三相同。定向干扰源的有效覆盖角度范围为定向方向两侧各 90° （含），即一个以源为顶点的闭半平面。请在此条件下给出检测与清除的策略及算法，并通过模拟器进行演练测试，验证其有效性，并报告三次正式测试的结果。

1.2 误差范式的选择：为什么必须用集员估计

附录 2(1) 中“同一地点重复检测不会改变检测误差”这句话信息量很大：误差与地点绑定，是有界确定性误差，重复测量不产生新增信息，靠 $1/\sqrt{N}$ 平均降噪的路径被堵死，以随机分布为前提的最小二乘、极大似然点估计失去依据。全文因而采用**集员估计**——每次测向给出干扰源必然落入的确定性集合，全部集合之交为位置的保证集，其直径即最坏情形下定位不确定性的上界。与随机范式相比，输出由“点估计加置信椭圆”换成“凸多边形加最坏界”，结论由概率成立变为确定成立，可给严格证明。这一选择贯穿四问：第一问的 Ω 是保证集，第二问的目标函数 J_{rob} 与约束 p_{safe} 也都取最坏情形而非期望；第三、四问的空频道判据与清除判据同样是最坏情形下成立的严格命题，而非置信度陈述。

1.3 全文的建模思路与技术路线

第一问分四步：把 $\pm 1^\circ$ 误差化为角域并用叉积改写为线性半平面（绕开斜率发散与象限分段）；证明非空性、凸性与有界性充要条件（这是直径存在的前提，也是仅用外框截断的朴素做法必须补全的一环）；半平面交裁剪求顶点、旋转卡壳求直径；最后由圆心唯一性与张角判据给出覆盖

性的充要条件。覆盖性不能只给结论：断言“能覆盖”则等边三角形即为反例，断言“不能覆盖”则 $\gamma = 75^\circ$ 的算例又确实可覆盖，唯一正确的处理是给出充要条件并在具体构形下化为可检验的形式。

第二问的困难在于目标函数依赖未知量：单次测向只给方向不给距离，而 D 同时依赖 γ, r_1, r_2 ，三者都随源的真实位置变化。先假定距离再优化，所得点对其他距离可能很差；只追求 $\gamma \approx 90^\circ$ ，又可能把机器狗送到收不到信号的地方。因此把源的位置不确定性显式写成先验可行集，对候选点同时考察“到了那里能不能测到”与“测到了精度够不够”，采用可靠性约束下最小化最坏直径，而非把两者简单加权相加。前两问的技术路线见图 1。

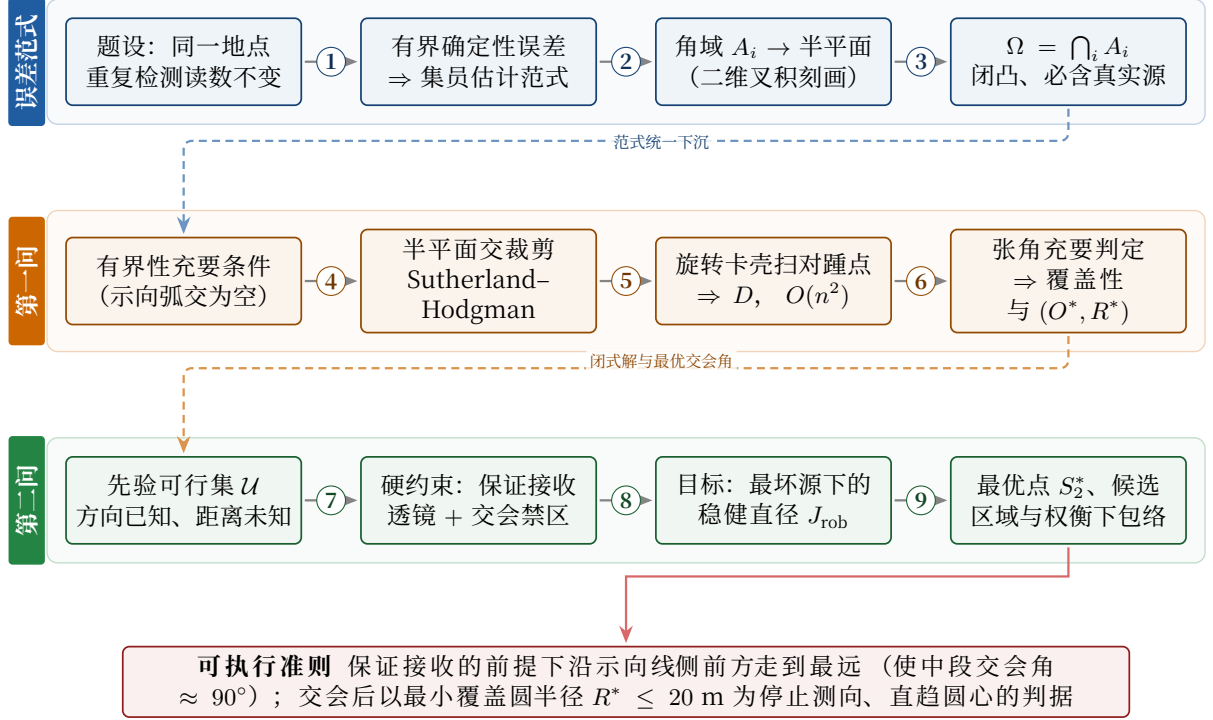


图 1 全文技术路线：由题设“重复检测读数不变”推出有界确定性误差与集员估计范式，再由同一范式统一导出两问的模型、算法与最终可执行准则

第三、四问不再是离线几何优化，而是信息不完全的在线决策问题。二者共用同一分层架构：可证明的完备搜索 $\rightarrow$ 保守后验刻画 $\rightarrow$ 保证清除与试探清除 $\rightarrow$ 在线优先级调度 $\rightarrow$ 路径保护。二者唯一的本质差别在第一层：全向源只需**距离完备**（存在检测点使 $\|E - G\| \leq R_{\min}$ ），定向源则需**方向完备**（对源的任意朝向都存在可见的检测点），后者由第 6 节给出的凸包充要条件刻画。第三问自身新增完备搜索骨架、清除的风险-收益判据与任务顺序的路径保护；第四问在此基础上替换两个模块——搜索完备性判据与失联恢复机制——并新增只吃确定性收益的 SAFE 局部支配层，其余框架整体复用。

2 模型假设与符号说明

- (A1) 检测点坐标精确，其误差相对测向误差可忽略；机器狗按指令位置精确到位。
- (A2) 测向误差为有界确定性误差， $|\theta_i - \theta_i^*| \leq \delta = 1^\circ$ ，不设任何分布；同一地点重复测量读数相同。
- (A3) 参与同一次交会的读数对应同一干扰源（由附录 1(1) 的频道唯一性保证）；角域边界上的点视为满足约束。
- (A4) 测向机与干扰源共面（附录 2(7)）；机器狗沿直线匀速移动，移动时不检测；干扰源静止不动。
- (A5) 全向源的有效接收半径 R 未知但 $R \in [R_{\min}, R_{\max}] = [1000, 1500]$ m，且在一次测试中不随时间变化。
- (A6) 第二问中源位于 $\mathcal{D}$ 内，且在第一条角域中按面积均匀分布，即无其他信息时不偏向任何距离。
- (A7) （第四问）定向源以未知单位向量 u 为指向，检测点 E 能接收到它当且仅当 $\|E - G\| \leq R$ 且

$(E - G) \cdot u \geq 0$; 其示向度误差界同 (A2), 有效接收半径范围同 (A5); 全向源可形式地视为对一切 u 都可见。

(A8) 每个源的频道互不相同且不随时间变化, 第三、四问中源总数 $N \in [10, 16]$ 。

假设 (A6) 的均匀先验只用于把“保证接收比例”这类面积指标定义清楚, 主决策采用最坏情形准则, 对先验形式并不敏感, 详见第 7 节。主要符号见表 1。

表 1 全文主要符号及其含义 (长度单位均为 m, 角度单位均为度)

符号	含义	符号	含义
S_i, θ_i	第 i 个检测点及其示向度	$\mathcal{U}$	源位置的先验可行集
θ_i^*	源相对 S_i 的真实方位角	$r_0, R_{\max}$	先验距离下界 20 与上界 1500
δ	测向误差上界, $\delta = 1^\circ$	p_{safe}	保证接收比例 (安全覆盖率)
α_i, β_i	$\theta_i - \delta$ 与 $\theta_i + \delta$	q_{fail}	必然失效比例
$d(\gamma)$	单位向量 $(\cos \gamma, \sin \gamma)$	J_{rob}	最坏情形稳健定位直径
A_i, Ω	角域与定位区域 $\bigcap_i A_i$	φ_k	Ω 在顶点 P_k 处的内角
$P_1, \dots, P_m$	Ω 的逆时针有序顶点	ϵ, ϵ^*	可靠性损失及其自动选定值
D	定位区域直径	η, ρ_0	直径近优容差与边际率阈值
$(P_a, P_b), O_D$	最远点对及其中心点	L_ϵ, C^*	接收透镜与候选区域
(O^*, R^*)	最小覆盖圆圆心与半径	ℓ, ψ	S_2 相对 S_1 的距离与方位偏角
γ, r_i	交会角与源到 S_i 的距离	(x_2, h)	S_2 沿/垂直示向线的分量
$B_L, \mathcal{D}$	外框 $[-L, L]^2$ 与目标圆域	ε	数值容差 (以米为量纲)
$\mathcal{E}, E_k$	搜索骨架及其第 k 点	r_{cov}	骨架的最坏覆盖距离
$\mathcal{F}_{\text{true}}, \mathcal{F}_{\text{code}}$	真实可行域与程序后验外包	$r_{\text{MEC}}, C_{\text{MEC}}$	后验最小包围圆半径与圆心
ρ_{work}	后验直径的一半 (工作半径)	$\beta(a)$	动作 a 的等效距离奖励
$\mathcal{M}$	当前强制任务集合	$L^*, \mathcal{R}$	开放最短路长与路径 regret
found	已发现的有源频道数	$\bar{T}$	平均定位清除时间
u	定向源的指向单位向量	$N_R(G)$	G 的 R 邻域内的搜索点集
E^{22}	第四问 22 点方向完备骨架	$\text{conv}(\cdot)$	凸包

3 第一问：定位区域直径的计算模型与覆盖性完全判定

3.1 角域约束与半平面等价刻画

若测向无误差, 干扰源位于射线 $S_i + td(\theta_i)$, $t \geq 0$ 上。由 (A2), 记 $\alpha_i = \theta_i - \delta$ 、 $\beta_i = \theta_i + \delta$, 单次测量把源限制在以 S_i 为顶点、张角 2δ 的平面角域内:

$$A_i = \{S_i + rd(\phi) : r \geq 0, \alpha_i \leq \phi \leq \beta_i\}, \quad \Omega = \bigcap_{i=1}^n A_i. \quad (1)$$

用斜率描述边界会遇到 $\theta_i \rightarrow 90^\circ, 270^\circ$ 时斜率发散、以及 $0^\circ/360^\circ$ 跨界需分段两类退化, 故引入二维叉积 $a \times b = a_x b_y - a_y b_x$, 其正号表示 b 在 a 的逆时针侧。

命题 3.1 (角域的半平面等价刻画). 设 $0 < 2\delta < 180^\circ$, 则 $P \in A_i$ 当且仅当 $d(\alpha_i) \times (P - S_i) \geq 0$ 且 $d(\beta_i) \times (P - S_i) \leq 0$ 。

证明. 设 $P - S_i = rd(\phi)$ 。由 $d(\gamma) \times d(\phi) = \sin(\phi - \gamma)$, 两式等价于 $r \sin(\phi - \alpha_i) \geq 0$ 与 $r \sin(\phi - \beta_i) \leq 0$ 。 $r = 0$ 时恒成立; $r > 0$ 时二者分别等价于 $\phi - \alpha_i \in [0, 180^\circ]$ 与 $\phi - \beta_i \in [-180^\circ, 0]$ (模 360°) , 其交恰为 $\phi \in [\alpha_i, \beta_i]$, 此处用到 $\beta_i - \alpha_i = 2\delta < 180^\circ$ 。论证只依赖角度之差, 与象限无关, 且因周期性自动处理 $0^\circ/360^\circ$ 跨界。□

把第二式取负, 可统一写成“保留有向直线左侧”的形式:

$$g_{i,1}(P) = d(\alpha_i) \times (P - S_i) \geq 0, \quad g_{i,2}(P) = -d(\beta_i) \times (P - S_i) \geq 0. \quad (2)$$

于是 Ω 是 $2n$ 个闭半平面之交, 为凸多面集; 又因 d 是单位向量, $g(P)$ 恰为点到边界直线的有符号距离, 量纲为米, 数值容差可直接按米设定。

3.2 定位区域的基本性质：非空、凸与有界

定理 3.2 (非空性与凸性). 在 (A2)(A3) 下 Ω 为闭凸集, 且真实源 $G \in \Omega$, 故 $\Omega \neq \emptyset$.

证明. 每个 A_i 是两闭半平面之交, 闭凸集的任意交仍为闭凸集。由 (A2) 有 $\theta_i^* \in [\alpha_i, \beta_i]$, 故对一切 i 有 $G \in A_i$. $\square$

定理 3.2 有两点用处: 算法若求得空集, 必是输入不一致 (混用了不同源的读数, 或误差超出 1°), 可直接作为数据自检准则; Ω 是保证集, 取其中任意点作估计, 定位误差必不超过 D , 这正是“定位区域直径”的物理意义。但非空不蕴含有界: $n = 1$ 时 $\Omega = A_1$ 沿测向方向无限延伸。若仅用外框截断, 会把外框角点误当作顶点, 输出虚假的有限直径。

定理 3.3 (有界性的充要条件). 记弧 $I_i = [\theta_i - \delta, \theta_i + \delta] \subset S^1$, 则 Ω 有界当且仅当 $\bigcap_{i=1}^n I_i = \emptyset$ 。特别地 $n = 2$ 时有界当且仅当两示向度的圆周角差 $\Delta\theta > 2\delta = 2^\circ$ 。

证明. 非空闭凸集有界当且仅当回收锥 $0^+\Omega = \{0\}$ 。 A_i 的回收锥为方向角落在 I_i 内的射线全体 $K(I_i)$, 而 $0^+(\bigcap_i A_i) = \bigcap_i 0^+A_i = K(\bigcap_i I_i)$, 故 $0^+\Omega = \{0\}$ 当且仅当 $\bigcap_i I_i = \emptyset$ 。 $n = 2$ 时两条长 2δ 的弧不相交当且仅当圆心角差大于 2δ 。 $\square$

注 3.4. 其工程含义是: 两次测向方向若几乎相同 (角差不超过 2°), 交会定位在原理上失效。算法仍以 B_L 起裁, 但必须做外框接触检测: 若有顶点落在 ∂B_L 上则返回 **unbounded**。

3.3 半平面交裁剪与直径计算

采用 Sutherland–Hodgman 型凸多边形裁剪, 依次用 (2) 的 $2n$ 个半平面裁剪当前多边形。设有向边为 UV 、约束为 $g(P) \geq 0$, 由 g 的仿射性 $g(U + t(V - U)) = (1 - t)g(U) + tg(V)$, 令其为零得

$$t^* = \frac{g(U)}{g(U) - g(V)}, \quad I = U + t^*(V - U). \quad (3)$$

规则为: $g(U) \geq 0$ 保留 U ; $g(U), g(V)$ 异号则插入 I 。由于 2δ 只有 2° , Ω 极细长、长宽比常超过 50, 实现中取相对容差 $\varepsilon = 10^{-10} \max(L, \max_i \|S_i\|)$ 放松内侧判据, 删去间距小于 ε 的相邻顶点, 并对每个输出顶点回代全部 $2n$ 个约束做后验校验——两条边界直线的交点未必满足其余约束, 也可能落在边界射线的反向延长线上, 裁剪法天然只产生有效交点, 这正是它优于“枚举全部直线交点再取凸包”的根本原因。其几何见图 2。

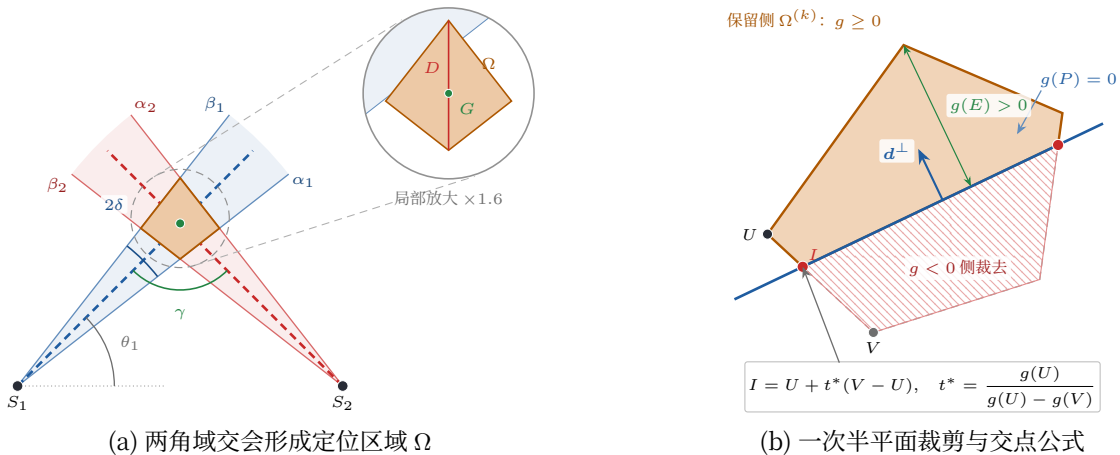


图 2 示向度误差所张角域的交会 (左; 半张角按 7° 夸张绘制, 实际仅 1° , 右上角为 Ω 的局部放大) 与单次半平面裁剪的几何 (右)

引理 3.5 (直径在顶点对取得). 设 Ω 为有界闭凸多边形、顶点集 $V = \{P_1, \dots, P_m\}$, 则 $D = \max_{P, Q \in \Omega} \|P - Q\| = \max_{i < j} \|P_i - P_j\|$ 。

证明. 固定 Q , $P \mapsto \|P - Q\|$ 为凸函数, 凸函数在紧凸集上的最大值必在极点处取得, 而有界凸多边形的极点集恰为 V ; 再对 Q 重复同一论证。 $\square$

由 Ω 至多由 $2n$ 条直线加外框四条边界定, 有 $m \leq 2n + 4$ 。本文采用旋转卡壳: 凸多边形的直径必由一对对踵点实现, 对踵点对共 $O(m)$ 个, 可在 $O(m)$ 时间内遍历完毕; 又因裁剪输出的顶点已逆时针有序, 无需再做凸包排序, 直径计算总代价降为 $O(m)$ 。

3.4 两检测点交会的闭式解

两检测点是本题最常用的构形, 也是第二问的全部内容。下面给出 Ω 的顶点与直径的闭式解, 它既是算法的独立校核基准, 又是覆盖性判定的解析基础。

命题 3.6 (两检测点情形的顶点与对角线闭式). 设 $n = 2$, 取 $G \in \Omega$, 记 $r_i = \|G - S_i\|$, 交会角 $\gamma = \angle S_1 G S_2 \in (2\delta, 180^\circ - 2\delta)$ 。不妨设 S_2 位于示向线 $S_1 G$ 的逆时针侧。令

$$A = r_1 \sin(\gamma - \delta) + r_2 \sin \delta, \quad B = r_1 \sin(\gamma + \delta) - r_2 \sin \delta, \quad (4)$$

并设 $A, B > 0$ (本题构形恒满足)。则 Ω 的四个顶点为 $V_{st} = S_1 + t_{st}d(\theta_1 + s\delta)$, $s, t \in \{+, -\}$, 其中

$$t_{+-} = \frac{A}{\sin(\gamma - 2\delta)}, \quad t_{-+} = \frac{B}{\sin(\gamma + 2\delta)}, \quad t_{--} = \frac{A}{\sin \gamma}, \quad t_{++} = \frac{B}{\sin \gamma}, \quad (5)$$

两条对角线的长度为

$$|d_1|^2 = (t_{+-} - t_{-+})^2 + 4t_{+-}t_{-+} \sin^2 \delta, \quad |d_2|^2 = (t_{++} - t_{--})^2 + 4t_{++}t_{--} \sin^2 \delta, \quad (6)$$

且 $D = \max(|d_1|, |d_2|)$ 。

证明. 取 G 为原点、 $\theta_1 = 0$, 则 $S_1 = -r_1 d(0)$ 、 $S_2 = -r_2 d(\gamma)$ 。顶点 V_{st} 是过 S_1 方向 $s\delta$ 的直线与过 S_2 方向 $\gamma + t\delta$ 的直线之交。设 $V_{st} = S_1 + t_{st}d(s\delta) = S_2 + v d(\gamma + t\delta)$, 两端与 $d(\gamma + t\delta)$ 作叉积消去 v , 并用 $d(\mu) \times d(\nu) = \sin(\nu - \mu)$, 得 $t_{st} \sin[\gamma + (t - s)\delta] = r_1 \sin(\gamma + t\delta) - r_2 \sin(t\delta)$, 代入 s, t 的四种取值即得诸式。 V_{+-}, V_{++} 落在方向 $+\delta$ 的射线上, V_{-+}, V_{--} 落在方向 $-\delta$ 的射线上, 两射线夹角为 2δ , 由余弦定理可得对角线公式; 写成 $(t - t')^2 + 4tt' \sin^2 \delta$ 而非 $t^2 + t'^2 - 2tt' \cos 2\delta$ 是为了避免 t, t' 数百米量级而 D 只有数十米时的灾难性抵消 (3000 组随机构形对照裁剪法, 相对偏差不超过 9.3×10^{-14})。最后由推论 3.11, Ω 的直径必由两条对角线之一实现。□

3.5 直径圆覆盖性的完全判定

定理 3.7 (覆盖圆圆心的唯一性). 设 Ω 的直径为 D 、 (P_a, P_b) 为任一最远点对。若存在半径 $D/2$ 的圆盘 $B(O, D/2) \supseteq \Omega$, 则必有 $O = O_D := \frac{1}{2}(P_a + P_b)$ 。若存在两组中点不同的最远点对, 则不存在以 D 为直径的覆盖圆。

证明. 由三角不等式 $D = \|P_a - P_b\| \leq \|P_a - O\| + \|O - P_b\| \leq D/2 + D/2 = D$, 两处不等号须同时取等, 故 O 为 $P_a P_b$ 中点。若两组最远点对中点不同, O 需同时等于两个不同的点, 矛盾。□

引理 3.8 (凸多边形的张角上界). 设 Ω 为凸多边形, X 为其顶点、内角为 φ_X , 则对 Ω 的任意两个其他顶点 P, Q 有 $\angle P X Q \leq \varphi_X$; 当 P, Q 恰为 X 的两个相邻顶点时取等。

证明. 由凸性, Ω 的全部顶点落在 X 处由两条相邻边张成的闭凸锥内, 该锥的张角即 φ_X ; 锥内任意两射线的夹角不超过锥的张角。□

定理 3.9 (直径圆覆盖的充要条件). 以 D 为直径的圆覆盖凸多边形 Ω , 当且仅当存在顶点对 (P_a, P_b) 使其余每个顶点 X 都满足 $\angle P_a X P_b \geq 90^\circ$ 。此时必有 $\|P_a - P_b\| = D$, 且该圆即以 $P_a P_b$ 为直径的圆。

证明. 充分性: 设这样的 (P_a, P_b) 存在, 记 B 为以 $P_a P_b$ 为直径的闭圆盘。由 Thales 定理, $\angle P_a X P_b \geq 90^\circ$ 当且仅当 $X \in B$, 故全部顶点落入 B ; B 为凸集, 因此 $\Omega = \text{conv}\{P_k\} \subseteq B$, 于是 $D = \text{diam } \Omega \leq \text{diam } B = \|P_a - P_b\| \leq D$, 两端相等, 故 $\|P_a - P_b\| = D$, B 正是直径圆。

必要性: 设直径圆 $B(O, D/2) \supseteq \Omega$ 。由引理 3.5 取最远点对 (P_a, P_b) , 由定理 3.7 有 $O = O_D$ 。任一顶点 $X \in \Omega \subseteq B(O_D, D/2)$, 由 Thales 定理得 $\angle P_a X P_b \geq 90^\circ$ 。□

推论 3.10 (锐角计数判据). 若 Ω 有三个或三个以上顶点的内角小于 90° , 则不存在以 D 为直径的覆盖圆。

证明. 顶点对 (P_a, P_b) 至多含两个顶点, 故必有某个内角小于 90° 的顶点 $X \notin \{P_a, P_b\}$ 。由引理 3.8, $\angle P_a X P_b \leq \varphi_X < 90^\circ$, 与定理 3.9 矛盾。□

推论 3.11 (对角线与三角形情形). 凸四边形的直径必由某条对角线实现; 三角形的直径圆能覆盖该三角形当且仅当它不是锐角三角形。

两检测点情形的完全刻画。由命题 3.6, Ω 的四条边界直线两两夹角只取 $\gamma, \gamma \pm 2\delta$ 三个值, 四个内角依次为 $\varphi_1 = \gamma - 2\delta, \varphi_2 = 180^\circ - \gamma, \varphi_3 = \gamma + 2\delta, \varphi_4 = 180^\circ - \gamma$ 。

定理 3.12 (两检测点情形的完全刻画). 设 $2\delta < \gamma < 180^\circ - 2\delta$, 则以 D 为直径的圆能覆盖 Ω 当且仅当 $\gamma \notin (90^\circ, 90^\circ + 2\delta)$, 本题即 $\gamma \notin (90^\circ, 92^\circ)$ 。并且 $\gamma \leq 90^\circ$ 时 $D = |d_1|$, $\gamma \geq 90^\circ + 2\delta$ 时 $D = |d_2|$ 。

证明. 必要性: 设 $\gamma \in (90^\circ, 90^\circ + 2\delta)$, 则 $\varphi_1 = \gamma - 2\delta < 90^\circ, \varphi_2 = \varphi_4 = 180^\circ - \gamma < 90^\circ$, 有三个内角小于 90° , 由推论 3.10 不可覆盖。

充分性: 若 $\gamma \leq 90^\circ$, 取 $(P_a, P_b) = (P_1, P_3)$, 由引理 3.8 取等情形, $\angle P_1 P_2 P_3 = \varphi_2 \geq 90^\circ, \angle P_1 P_4 P_3 = \varphi_4 \geq 90^\circ$, 由定理 3.9 可覆盖且 $D = |P_1 P_3| = |d_1|$ 。若 $\gamma \geq 90^\circ + 2\delta$ 同理可覆盖且 $D = |d_2|$ 。□

注 3.13. 失效窗口宽度恰为 2δ , 且与 r_1, r_2 完全无关: 以 0.02° 步长、 r_2/r_1 从 1:0.001 取到 1:7 的全参数扫描, 失效区间稳定为 $[90.02^\circ, 91.98^\circ]$ 。窗口左端点 $\gamma = 90^\circ$ 恰是使定位区域最小的最优交会角, 也就是说, 在工程上最希望采用的构形附近, 判据恰处于临界失效状态, 绝不能依赖直径圆作为不确定性包络。

三检测点: Jung 界几乎可以取到。 取三个检测点绕真实源成 120° 均匀布设、距离均为 $r = 600$ m, 当三次测向误差取到边界值时, Ω 退化为三角形, 内角为 $58^\circ, 60^\circ, 62^\circ$ ——一个几乎等边的锐角三角形。此时 $D = 24.692$ m 而最小覆盖圆半径 $R^* = 13.983$ m, $R^*/(D/2) = 1.1326$, 而 Jung 界为 $2/\sqrt{3} \approx 1.1547$, 二者只差 1.9%: Jung 不等式的极值构形 (等边三角形) 在本题中是物理上完全自然的布点方案, 而非人为构造的病态例子。由平面 Jung 定理, $D/2 \leq R^* \leq D/\sqrt{3}$, 右端在等边三角形处取等, 故以 $2D/\sqrt{3} \approx 1.155D$ 为直径的圆必能覆盖, 且该系数不可改进。

定理 3.14 (最优交会角). 在细长条带近似 $D \approx \frac{2 \tan \delta}{\sin \gamma} \sqrt{r_1^2 + r_2^2 + 2r_1 r_2 |\cos \gamma|}$ 意义下, 对任意固定 $r_1, r_2 > 0$, $D(\gamma)$ 在 $\gamma = 90^\circ$ 取唯一最小值 $D_{\min} = 2 \tan \delta \sqrt{r_1^2 + r_2^2}$; $r_1 = r_2 = r$ 时 $D_{\min} = 2\sqrt{2} r \tan \delta \approx 0.04938 r$ 。

证明. 令 $u = \cos \gamma$ 、 $A_0 = r_1^2 + r_2^2$, 则 $D^2 = 4 \tan^2 \delta \cdot f(u)$, $f(u) = \frac{A_0 + 2r_1 r_2 |u|}{1 - u^2}$ 。在 $u \in (0, 1)$ 上 $f'(u) > 0$; 在 $u \in (-1, 0)$ 上 $f'(u) < 0$ 。故 f 在 $u = 0$ 即 $\gamma = 90^\circ$ 处取唯一极小。□

3.6 算法总流程

程序最终直接用 Welzl 随机增量法计算最小覆盖圆, 期望复杂度 $O(m)$: 它恒存在且唯一, 半径最小给出最紧包络, 圆心 $O^* = \arg \min_P \max_{Q \in \Omega} \|P - Q\|$ 又是极小极大意义下的最优位置估计, 正是机器狗应前往的目标点。复杂度: 弧交判定 $O(n \log n)$; 每次裁剪至多使边数加一, 故 $m \leq 2n + 4$, $2n$ 次裁剪与后验校验合计 $O(nm) = O(n^2)$; 直径、覆盖性判定与最小覆盖圆各 $O(m)$ 。故总时间复杂度 $O(n^2)$ 、空间 $O(n)$ 。本题 n 通常为 2 至 5, 单次求解在微秒量级, 可嵌入在线决策循环。

3.7 数值实验与定量规律

取真实源 $G = (300, 400)$, 四个检测点的读数由真值叠加 $[-1^\circ, 1^\circ]$ 误差生成, 逐次增加检测点, 定位区域面积由 506.8 降至 353.9 再降至 139.6 m^2 , 直径由 35.075 降至 33.514 再降至 18.975 m, 三种情形下 Ω 均含 G , 验证了定理 3.2。四点交会时最小覆盖圆 $O^* = (297.942, 403.606)$ 、 $R^* = 9.487$ m; 由 $R^* < 20$ m, 机器狗前往 O^* 必定落入光学定位范围。

算法 1: 交会定位区域的直径计算与直径圆覆盖性判定

输入: 检测点 $S_1, \dots, S_n$; 示向度 $\theta_1, \dots, \theta_n$; 误差界 $\delta = 1^\circ$; 外框半边长 $L = 1800$
输出: 状态 st ; 顶点 $\{P_k\}$; 直径 D ; 最远点对 (P_a, P_b) ; 覆盖标志 Cov ; 最小覆盖圆 (O^*, R^*)

- 1 若 $\bigcap_i [\theta_i - \delta, \theta_i + \delta] \neq \emptyset$ (S^1 上弧交, $O(n \log n)$) 则
- 2 **return** unbounded; // 定理 3.3
- 3 $\Omega \leftarrow$ 外框 $[-L, L]^2$ 的四顶点 (逆时针); $\varepsilon \leftarrow 10^{-10} \max(L, \max_i \|S_i\|)$;
- 4 **对** $i \leftarrow 1$ **to** n **执行**
- 5 $\Omega \leftarrow \text{ClipLeft}(\Omega, S_i, d(\theta_i - \delta)); \Omega \leftarrow \text{ClipLeft}(\Omega, S_i, -d(\theta_i + \delta));$ // 式 (2)
- 6 **若** $\Omega = \emptyset$ **则**
- 7 **return** empty; // 数据不一致, 触发自检
- 8 删除相邻重复顶点; 每个顶点回代全部 $2n$ 个约束做后验校验;
- 9 **若** 存在顶点落在 ∂B_L 上 **则**
- 10 **return** unbounded
- 11 旋转卡壳遍历全部对踵点对, 得 $D \leftarrow \max \|P_i - P_j\|$ 及 (P_a, P_b) ; // 引理 3.5, $O(m)$
- 12 计算各顶点内角 φ_k ; **若** $\#\{k : \varphi_k < 90^\circ\} \geq 3$ **则**
- 13 $Cov \leftarrow \text{false}$; // 推论 3.10, 快速否决
- 14 **否则**
- 15 $O_D \leftarrow (P_a + P_b)/2$; $Cov \leftarrow (\max_k \|P_k - O_D\| \leq D/2 + \varepsilon)$; // 定理 3.9
- 16 $(O^*, R^*) \leftarrow \text{Welzl}(\{P_1, \dots, P_m\})$; **return** bounded, $\{P_k\}, D, (P_a, P_b), Cov, (O^*, R^*)$;

表 2 印证定理 3.12: $\gamma = 91^\circ$ 时四个内角为 $89^\circ, 89^\circ, 93^\circ, 89^\circ$, 其中三个小于 90° , 由推论 3.10 即可无需计算直接否决; 实测顶点到 O_D 的最大距离 17.760 m 超出半径 17.452 m 达 0.307 m (相对超出 1.76%), 直径圆无法覆盖。而 $\gamma = 90^\circ$ 时四顶点恰好共圆, 处于临界可覆盖状态——这解释了失效窗口为何从 90° 开始。

表 2 覆盖性对照算例 ($S_1 = (0, 0)$, $\theta_1 = 45^\circ$, $S_2 = (1000, 0)$)

$\gamma/^\circ$	内角/ $^\circ$	D/m	$\max_k \ P_k - O_D\ $	覆盖
75	73, 105, 77, 105	46.862	23.431	是
90	88, 90, 92, 90	34.921	17.460	临界
91	89, 89, 93, 89	34.905	17.760	否
93	91, 87, 95, 87	34.947	17.474	是

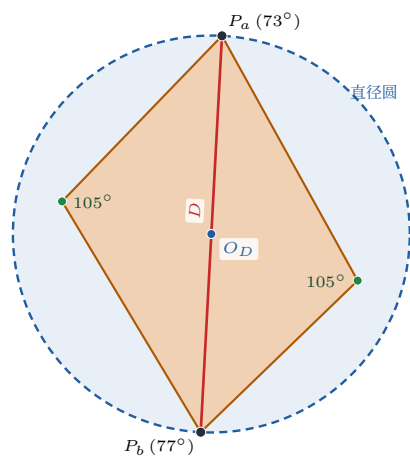
在 $[-1800, 1800]^2$ 内随机取检测点、 $[-1200, 1200]^2$ 内随机取源, 各约 2×10^4 组有界算例的直径圆失效率为: $n = 2$ 时 1.14%, $n = 3$ 时 14.5%, $n = 4$ 时 23.3%, $n = 5$ 时 29.4%; 失效时顶点超出量的中位数达半径的 9%, 最大达 72%, 绝非数值噪声。同一批算例中 $R^*/(D/2)$ 的最大值: $n = 2$ 时仅 1.0005, 而 $n \geq 3$ 时可达 1.147, 已逼近 Jung 界。

另一条规律是边际收益递减: 把 n 个检测点均布环绕在距目标 $r = 600$ m 处, D 依次为 29.631, 24.189, 22.673, 22.025, 21.685, 21.485, 21.357 m ($n = 2, \dots, 8$), 单调趋近硬下界 $D \geq 2r \tan \delta = 20.951$ m, 两次良好交会就已达到与理论极限仅差 41% 的精度, 继续增加测向次数收益急剧递减——这正是第二问只讨论“第二个”检测点的依据。

3.8 第一问小结

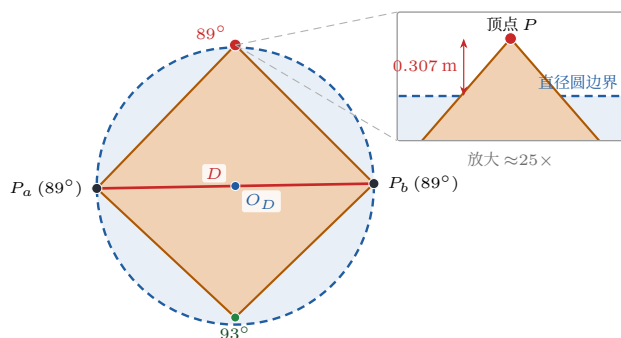
直径算法: 角域经叉积化为线性半平面 (命题 3.1); 先由定理 3.3 判有界性, 再用半平面交裁剪求有序顶点, 最后由引理 3.5 与旋转卡壳在 $O(m)$ 时间内得 D , 总复杂度 $O(n^2)$; 两检测点时另有闭式解可直接求值并作校核。覆盖性答案是不一定: 直径圆覆盖 Ω 当且仅当存在顶点对使其余顶点对之张角均不小于 90° (定理 3.9), 一条一眼可用的必要条件是内角小于 90° 的顶点不超过两个 (推论 3.10)。两检测点时充要条件为 $\gamma \notin (90^\circ, 92^\circ)$, $\gamma = 91^\circ$ 的算例中有顶点超出直径圆 0.307 m, 构成确凿反例; 三点 120° 布设时 $R^*/(D/2) = 1.133$ 已逼近 Jung 界 1.1547, 说明直径圆的不足在本题中不是边缘现象。工程上应以最小覆盖圆作为不确定性包络, 其圆心即极小极大最优位置估计。定理 3.14 把选点从定性经验上升为显式优化问题

$$\min_{S_2} D(\gamma(S_2), r_1, r_2(S_2)) \quad \text{s.t.} \quad \gamma > 2\delta, r_2 \leq R, S_2 \in \mathcal{D},$$



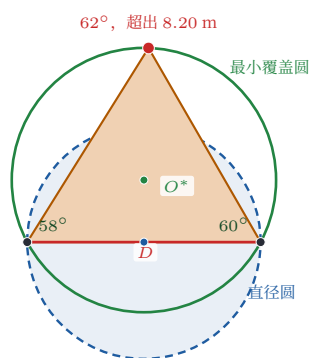
$D = 46.862 \text{ m}$, $R^* = D/2$: 直径圆即最小覆盖圆

(a) $\gamma = 75^\circ$: 可覆盖



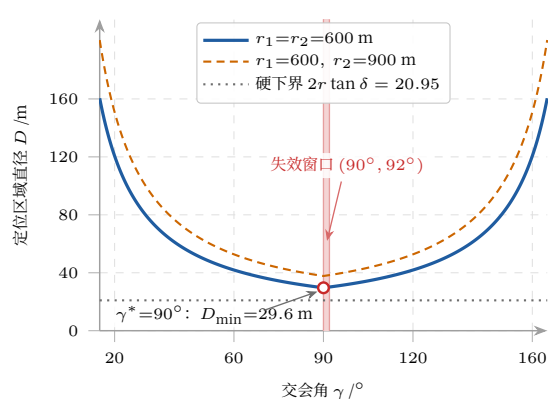
四个内角中有三个 $< 90^\circ$: 由推论 3.10 直接否决

(b) $\gamma = 91^\circ$: 顶点超出 0.307 m, 不可覆盖



$R^*/(D/2) = 1.133$, Jung 界为 1.1547

(c) 三点 120° 布设: 几乎取到 Jung 界



(d) D 随交会角的变化与失效窗口

图 3 直径圆覆盖性的完全判定: (a)(b) 按真实比例绘制, 蓝色虚线圆为直径圆; (b) 中三个内角小于 90° , 顶点超出直径圆 0.307 m (见局部放大); (c) 三点 120° 布设时 Ω 近似等边三角形, 绿色实线圆为最小覆盖圆; (d) 失效窗口 $(90^\circ, 92^\circ)$ 恰位于最优交会角右侧

第二问在此骨架上补齐题目要求必须处理的两件事： r_1 未知， R 也只知落在 $[1000, 1500]$ 内。

4 第二问：第二个检测点的选择策略与候选区域

4.1 先验可行集与决策变量

在 S_1 处能读到示向度，说明信号强度高于灵敏度，故 $r_1 \leq R \leq R_{\max} = 1500$ m；又由附录 2(9)， $r_1 \leq 5$ m 时读数失效，而 $r_1 \leq 20$ m 时本就直接转入光学定位，故取 $r_0 = 20$ m 为下限。结合角域与目标区域约束，得源位置的先验可行集

$$\mathcal{U} = \{S_1 + rd(\phi) : r_0 \leq r \leq R_{\max}, |\phi - \theta_1| \leq \delta\} \cap \mathcal{D}, \quad (7)$$

它是长 1480 m、末端半宽仅 $R_{\max} \tan \delta = 26.2$ m 的极细扇形——长宽比超过 28 : 1。这一悬殊比例是后文一切化简的出发点：在涉及数百米尺度的接收与几何约束时，可用中轴线段 $\Sigma = [r_0, R_{\max}] \times \{0\}$ 代替 $\mathcal{U}$ ，引入的误差不超过 26 m，远小于决策量的分辨率；而在计算定位直径时则必须保留 δ ，因为 D 本身正比于 $\tan \delta$ 。

模型对刚体运动等变，故不妨取 $S_1 = (0, 0)$ 、 $\theta_1 = 0^\circ$ ，一般情形只需把结果整体旋转 θ_1 并平移至真实 S_1 。记决策变量 $S_2 = (x_2, h)$ ， h 为到示向线的有符号垂距，并记 $\ell = \|S_2 - S_1\|$ 、 $\psi = \angle(S_2 - S_1, d(\theta_1))$ 为机器狗的移动距离与转向角，移动时间为 $\ell/5$ s。

4.2 硬约束：保证接收透镜与交会禁区

(1) **保证接收透镜**。 R 未知但 $R \geq R_{\min} = 1000$ m，故若 $r_2(u) = \|u - S_2\| \leq R_{\min}$ ，则无论 R 取何值都一定收得到，称 u 被保证接收。主模型要求 $\mathcal{U}$ 中每一个可能位置都被保证接收。由于对一条线段而言，以其上各点为心的同半径圆盘之交等于两端点圆盘之交，得

$$L_0 = B((r_0, 0), R_{\min}) \cap B((R_{\max}, 0), R_{\min}), \quad (8)$$

其角点为 $\left(\frac{r_0 + R_{\max}}{2}, \pm \sqrt{R_{\min}^2 - \left(\frac{R_{\max} - r_0}{2}\right)^2}\right) = (760, \pm 672.6)$ m，非空条件 $R_{\max} - r_0 \leq 2R_{\min}$ 即 $1480 < 2000$ 满足。同时定义必然失效比例 $q_{\text{fail}}(S_2) = |\{u \in \mathcal{U} : \|u - S_2\| > R_{\max}\}|/|\mathcal{U}|$ ，它刻画“无论 R 取何值都收不到”的硬失败； $S_2 \in L_0$ 时显然 $q_{\text{fail}} = 0$ 。

(2) **交会禁区**。由定理 3.3，若对某个可能的源位置交会角退化，则定位区域无界，交会在原理上失效。对 $u = (r, 0) \in \Sigma$ ，可得 $\gamma(r) \leq 2\delta$ 当且仅当 $r \geq x_2 + |h|/\tan 2\delta$ ， $180^\circ - \gamma(r) \leq 2\delta$ 当且仅当 $r \leq x_2 - |h|/\tan 2\delta$ 。要求二者对全部 $r \in [r_0, R_{\max}]$ 都不发生，得禁区之补的精确解析条件

$$|h| > \tan(2\delta) \cdot \max(R_{\max} - x_2, x_2 - r_0), \quad \tan 2^\circ = 0.034921. \quad (9)$$

禁区 $\mathcal{N}$ 因而是以示向线为轴、在 $x_2 = 760$ m 处最窄（半宽 25.8 m）、向两端线性张开的领结形细带，在 $x_2 \in [-1800, 2500]$ 上半宽始终不超过 120 m。可见禁区在几何上几乎不构成限制——只要不把第二检测点放在示向线的延长线上，交会就不会退化；真正决定候选区域形状的是接收约束与精度目标。

4.3 稳健目标与“近端定约束、远端定精度”的分离结构

对候选点 S_2 与任一可能源位置 $u \in \mathcal{U}$ ，记 $r_1(u) = \|u - S_1\|$ 、 $r_2(u) = \|u - S_2\|$ 、 $\gamma(u) = \angle S_1 u S_2$ ，由命题 3.6 可精确求得该情形下的定位直径 $D(u; S_1, S_2)$ 。定义最坏情形稳健直径

$$J_{\text{rob}}(S_2) = \max_{u \in \mathcal{U}} D(u; S_1, S_2), \quad (10)$$

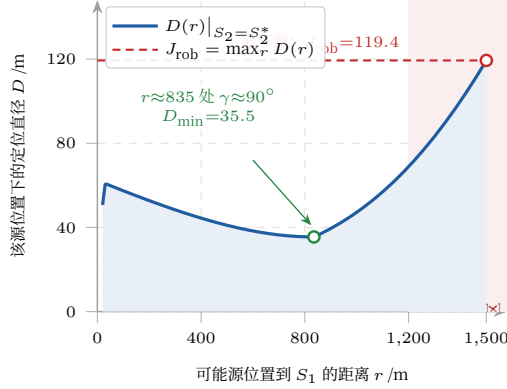
它是“无论源实际落在何处，两次交会后定位不确定性都不会超过”的确定性上界。于是主模型为

$$\min_{S_2} J_{\text{rob}}(S_2) \quad \text{s.t.} \quad S_2 \in L_0 \cap \mathcal{D} \setminus \mathcal{N}. \quad (11)$$

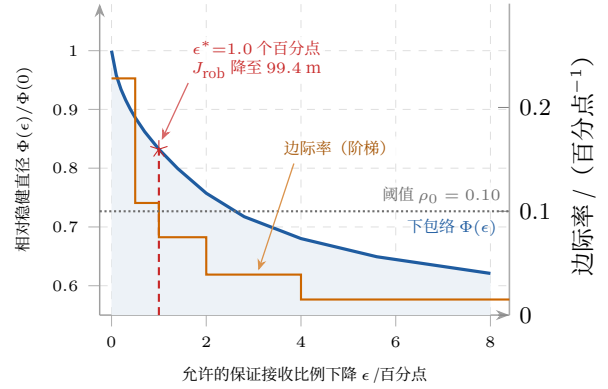
命题 4.1 (最坏源位于远端). 在 $L_0 \setminus \mathcal{N}$ 上（且 $|h| \geq 100$ m）， $r \mapsto D((r, 0); S_1, S_2)$ 的最大值在 $r = R_{\max}$ 处取得，即 $J_{\text{rob}}(S_2) = D((R_{\max}, 0); S_1, S_2)$ 。

理由与验证：由细长条带近似， D 随 r_1 单调增。而在 L_0 内 S_2 的横坐标 $x_2 \leq R_{\max}$ ，故当 r 由 r_0 增至 $R_{\max}$ 时，源先向 S_2 靠近（ D 下降），越过 x_2 后又远离（ D 加速上升），两段的净效应由远端主导。以 10 m 网格遍历 $L_0 \setminus \mathcal{N}$ 的全部 2243 个可行点、每点对 r 取 201 个样本验证：除 $|h| \leq 40$ m 的 48 个贴近禁区边缘的劣点外，最大值均在 $r = R_{\max}$ 处取得。算法本身仍按 (10) 对整段取最大值，故最终结论不依赖本命题，它只用于解释结构并把搜索降维。

与此相对，约束却由近端决定： L_0 的两个圆盘中，以 $(r_0, 0)$ 为心的那个才是真正卡住 S_2 的——机器狗要往侧前方走远以改善远端交会角，而走得越远，离 S_1 附近的可能源位置就越远，接收保证越吃紧。于是有近端源决定“能走多远”（约束），远端源决定“精度多好”（目标）。这条对峙直接给出了最优解的位置：最优点必落在近端接收圆的边界弧 $\partial B((r_0, 0), R_{\min})$ 上，因为只要约束不紧，沿着增大 $|h|$ 与 x_2 的方向移动总能继续改善远端的交会角。



(a) 最优点 S_2^* 处的定位直径剖面



(b) 可靠性与精度的权衡下包络

图4 第二问的两条核心曲线：(a) 定位直径在中段最小、在远端最大，故最坏源必位于 $r = R_{\max}$ ；(b) 下包络单调下降且边际收益递减，橙色阶梯为相邻层级的边际率，其首次跌破阈值 $\rho_0 = 0.10$ 处即自动选定的 $\epsilon^* = 1$ 个百分点

4.4 最优解、候选区域与算法

以 2 m 网格在 $L_0 \setminus \mathcal{N}$ 上枚举 S_2 ，对每个候选点沿 Σ 取 301 个源样本、用命题 3.6 的闭式精确求 D 并取最大值，得

$$S_2^* \approx (836, 578) \text{ m}, \quad \ell^* = 1016 \text{ m}, \quad \psi^* = 34.7^\circ, \quad J_{\text{rob}}^* = 119.4 \text{ m}. \quad (12)$$

该点到 $(r_0, 0)$ 的距离恰为 1000.0 m，近端接收约束严格取等，印证了上节的分离结构；对最坏源（ $r = 1500$ m）有 $r_2 = 880.3$ m、 $\gamma = 41.0^\circ$ ，对中段源（ $r \approx 835$ m）则 $\gamma \approx 90^\circ$ ，恰为最优交会角。机器狗从 S_1 走到 S_2^* 需 $1016/5 \approx 203$ s，加上一次检测 5 s，总计约 3.5 min。

目标函数在最优点附近极为平坦：以 S_2^* 为中心 15 m 邻域内 J_{rob} 的变化不足 0.2 m，说明数值结果稳健，也直接说明题目要求“候选区域”而非单个坐标是合理的。取近优容差 $\eta = 5\%$ ，定义

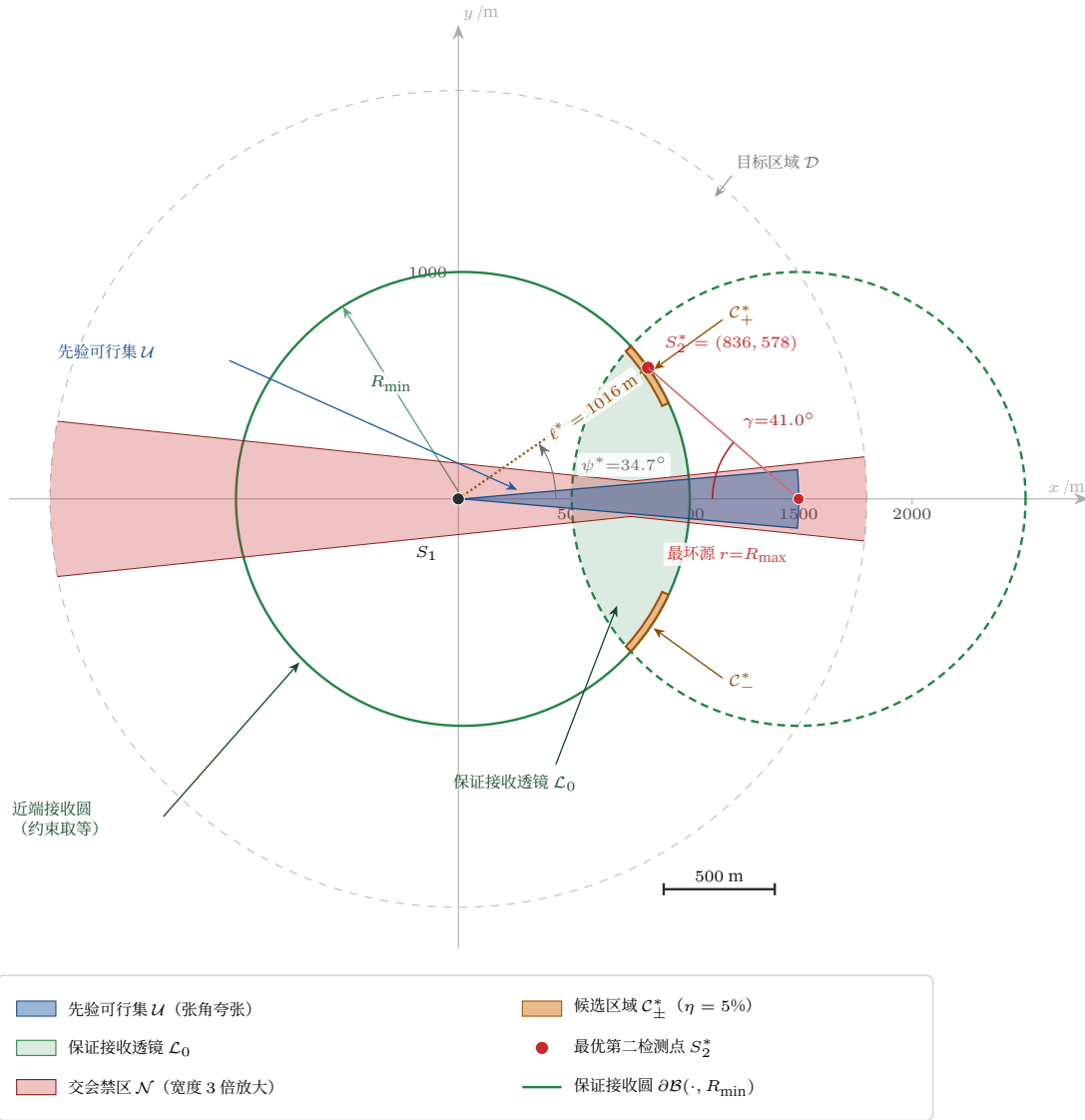
$$C^* = \left\{ S_2 \in L_0 \cap \mathcal{D} : |h| > \tan(2\delta) \max(R_{\max} - x_2, x_2 - r_0), J_{\text{rob}}(S_2) \leq (1 + \eta) J_{\text{rob}}^* \right\}. \quad (13)$$

数值求解表明 C^* 由关于示向线对称的两支构成，每一支都是贴着近端接收圆边界弧的窄带，用以 $(r_0, 0)$ 为极点的极坐标描述最为简洁：

$$C_{\pm}^* : \|S_2 - (r_0, 0)\| \in [969, 1000] \text{ m}, \quad \arg(S_2 - (r_0, 0)) \in \pm[25.0^\circ, 42.2^\circ]; \quad (14)$$

换算到以 S_1 为原点即 $\ell \in [985, 1018] \text{ m}$ 、 $\psi \in \pm[24.5^\circ, 41.4^\circ]$ 。两支在数学上完全等价，执行时按移动距离与地形可通行性任选其一；若 S_1 靠近目标区域 $\mathcal{D}$ 的边界使某一支被裁掉，另一支恰好构成天然备份。

为什么是 34.7° 而不是 90° ？直觉上应让交会角等于 90° ，但对哪个源等于 90° 并无先验答案：若源距 d 已知，使 $\gamma = 90^\circ$ 的 S_2 构成过该源且垂直于示向线的整条直线； d 未知时这族平行线扫过示向线一侧的全部半平面，单凭交会角无法定出任何一点。真正把候选区域钉死在这条窄带上的是两件事：其一，最坏源在远端（命题 4.1），故应优先改善远端的交会角，这要求 S_2 尽量向东而非



算法 2: 第二检测点的稳健选址与候选区域生成

单纯向北；其二，近端接收圆封住了向外扩张的余地。二者的切点即 (12)。一句话概括策略：在保证任何可能位置的源都收得到信号的前提下，尽量沿示向方向的侧前方走远。

复杂度：设网格点数 N_g 、源采样数 N_u 。可行性判别每点 $O(1)$ ； J_{rob} 每点 $O(N_u)$ ，闭式求值无迭代；故主流程总复杂度 $O(N_g N_u)$ ，空间 $O(N_g)$ 。本文取 $\Delta = 2 \text{ m}$ 、 $N_u = 301$ ，在标准笔记本上数十秒内完成，且因闭式解的存在而完全不需要在内层反复调用算法 1。

4.5 可靠性与精度的权衡：一个带确定性回退分支的决策

主模型要求 $\mathcal{U}$ 中每一点都被保证接收，这是最保守的选择。若允许放弃一小段近端——即接受“万源就在 S_1 近旁 $r < r_\epsilon$ 处，在 S_2 可能收不到”——则 L_0 的近端圆心东移， S_2 可以走得更远，精度随之改善。由 (A6) 的面积均匀先验，扇形中距离小于 r 的部分占面积比为 $(r^2 - r_0^2)/(R_{\text{max}}^2 - r_0^2)$ ，是 r 的二次函数，这意味着放弃近端一小段几乎不花费可靠性：

$$r_\epsilon = \sqrt{r_0^2 + \epsilon(R_{\text{max}}^2 - r_0^2)}, \quad L_\epsilon = B((r_\epsilon, 0), R_{\text{min}}) \cap B((R_{\text{max}}, 0), R_{\text{min}}). \quad (15)$$

记权衡下包络 $\Phi(\epsilon) = \min\{J_{\text{rob}}(S_2) : S_2 \in L_\epsilon \setminus \mathcal{N}\}$ ，它关于 ϵ 单调不增。 Φ 没有平台，而是持续下降但边际收益迅速递减，因此阈值不应靠“找拐点是否水平”来定，而应由边际率准则给出：取阈值 $\rho_0 = 0.10$ （每让出 1 个百分点的保证接收比例，至少换回 10% 的相对精度改善），则

$$\epsilon^* = \max\left\{\epsilon_j : -\frac{\Phi(\epsilon_j) - \Phi(\epsilon_{j-1})}{\Phi(0)(\epsilon_j - \epsilon_{j-1})} \geq \rho_0\right\} = 1.0 \text{ 个百分点}, \quad (16)$$

对应 $r_{\epsilon^*} = 151 \text{ m}$ ， $S_2 \approx (1012, 509) \text{ m}$ ($\ell = 1133 \text{ m}$ ， $\psi = 26.7^\circ$)， $J_{\text{rob}} = 99.4 \text{ m}$ ，比零损失方案改善 16.8%。这正是确定性范式相较概率范式的优势：“失败”本身也是一条可用的负信息，而不是一次无法收拾的意外——若在 S_2 收不到信号，恰恰确定性地证明源距 S_1 不超过 151 m，机器狗沿示向线前进即可完成定位，故策略是带确定性回退分支的决策树。

4.6 第二问小结

(1) **选择策略**。以“ $\mathcal{U}$ 中每一点都被保证接收”和“避开交会禁区 (9)”为硬约束、以最坏源位置下的定位直径 J_{rob} 为目标求解 (11)。策略可概括为：在保证任何可能位置的源都收得到信号的前提下，沿示向方向的侧前方走到接收约束允许的最远处；其几何实质是“近端源定约束、远端源定精度”，最优点必落在近端接收圆的边界弧上。

(2) **候选区域**。由 (13) 给出，解析形式为 (14)：以 $(r_0, 0)$ 为极点，半径 969 至 1000 m、幅角 $\pm 25.0^\circ$ 至 $\pm 42.2^\circ$ 的两段对称环带；以 S_1 为原点即距离 985 至 1018 m、与示向方向成 24.5° 至 41.4° 。区内任一点的最坏定位直径不超过最优值 119.4 m 的 1.05 倍， $q_{\text{fail}} \equiv 0$ 。两支等价，可按地形任选，互为备份。

(3) **精度水平与后续接口**。任何两点交会都不可能突破 $2r \tan \delta$ 的硬下界，故若要把不确定性压到 20 m 光学定位阈值以内，不能靠继续挑选第二检测点，而应先沿示向方向前进以缩短 r_1 。由 $\tan \delta \sqrt{r_1^2 + r_2^2} \leq 20 \text{ m}$ 及 $R^* \approx D/2$ ，充分条件为

$$\sqrt{r_1^2 + r_2^2} \leq 1145.8 \text{ m} \quad (r_1 = r_2 \text{ 时即 } r \leq 810.2 \text{ m}). \quad (17)$$

5 第三问：全向源的搜索-定位-清除一体化在线策略

5.1 问题分析与总体框架

第三问与前两问的性质截然不同：源的个数 $N \in [10, 16]$ 未知、所占频道未知、位置未知、有效接收半径未知，机器狗只能通过“移动-切频-检测”逐步获取信息，且每一次信息获取都要付出时间代价。目标函数

$$\bar{T} = \frac{T_{\text{total}}}{N_{\text{cleared}}} = \frac{T_{\text{move}} + T_{\text{switch}} + T_{\text{measure}} + T_{\text{clear}}}{N_{\text{cleared}}} \quad (18)$$

同时受两类地位并不对等的约束支配：**完备性是硬约束**，必须由可证明的几何与逻辑保证；**时间效率**则只能由在线启发式逼近，因为最优解需要预知全部源的位置，属于事后信息。本文因此严格区分“可证明的保证”与“演练标定的工程参数”，并明确声明：本文并不声称求得全局最优策略——全局最优要求对未知源分布求解随机动态规划，其状态空间随频道数与后验多边形呈指数增长，在程序运行限制下不可行。

5.2 完备搜索骨架与严格覆盖定理

只要选定一组检测点 $\mathcal{E} = \{E_0, \dots, E_{m-1}\}$ 使 $\mathcal{D} \subseteq \bigcup_k B(E_k, R_{\min})$, 就可断言目标区域内任何内向源至少在一个 E_k 处可被接收到。本文取原点与半径 $r = 1010$ m 圆周上等角分布的 7 个点, 共 $m = 8$ 个搜索点 (下称 EW 点) :

$$E_0 = (0, 0), \quad E_k = \left(r \cos \frac{2\pi(k-1)}{7}, r \sin \frac{2\pi(k-1)}{7} \right), \quad k = 1, \dots, 7. \quad (19)$$

定理 5.1 (八点严格覆盖). 记 $\alpha = \pi/7$, 则

$$r_{\text{cov}} := \max_{p \in \mathcal{D}} \min_k \|p - E_k\| = \sqrt{R_T^2 + r^2 - 2R_T r \cos \alpha} = 992.057 \text{ m} < R_{\min} = 1000 \text{ m}, \quad (20)$$

因此 $\mathcal{D} \subseteq \bigcup_k B(E_k, R_{\min})$ 。

证明. 若 $\|p\| \leq R_{\min}$, 则 $\min_k \|p - E_k\| \leq \|p\| \leq 1000$ 。若 $R_{\min} < \|p\| \leq R_T$, 记 p 与最近外环点的圆心角为 $\theta \leq \alpha$; 由余弦定理 $\|p - E_k\|^2 = \rho^2 + r^2 - 2\rho r \cos \theta$ 关于 $\theta \in [0, \alpha]$ 单调增, 故最坏取 $\theta = \alpha$; 此时 $g(\rho) = \rho^2 + r^2 - 2\rho r \cos \alpha$ 的对称轴 $\rho = r \cos \alpha = 910.0 < R_{\min}$, 故 g 在 $(R_{\min}, R_T]$ 上单调增, 最坏取 $\rho = R_T$, 代入 $R_T = 1800, r = 1010, \cos(\pi/7) = 0.900969$ 得 $r_{\text{cov}} = 992.057 < 1000$ 。□

骨架点数的最小性. 在 “一个中心点 + n 个等角外环点 (半径 r) ” 这一族构形内, 覆盖条件等价于

$$r^2 - 2R_T r \cos \frac{\pi}{n} + (R_T^2 - R_{\min}^2) \leq 0, \quad (21)$$

判别式 $\Delta_n = 4R_T^2 \cos^2(\pi/n) - 4(R_T^2 - R_{\min}^2)$ 。计算得 $\Delta_5 = -4.78 \times 10^5 < 0$, 故 $n = 5$ (共 6 点) 无论 r 取何值都不可能覆盖; 而 $\Delta_6 = 7.60 \times 10^5 > 0$, 可行半径区间为 $r \in [1122.96, 1994.74]$ m。因此该族中的最小骨架为 $1 + 6 = 7$ 个点。本文最终程序仍采用 $1 + 7 = 8$ 点, 理由有二: 其一, 外环七点相邻夹角 51.4° , 为同一源提供的交会基线在角度上更密, 配合后文的顺路共享测量可显著提高早期交会角质量, 而六点方案的基线夹角为 60° 、外环半径被迫增大到 1130 m 以上; 其二, $n = 7, r = 1010$ m 的覆盖余量为 8.0 m, 且 r 的可行区间为 $[997.20, 2246.29]$ m, 对实现中的数值取整更稳健。把外环减到六点是本策略明确的后续压缩方向。

推论 5.2 (空频道证明). 若某频道 c 在全部八个 EW 点的检测结果均为 no_signal, 则目标区域内该频道不存在干扰源。

证明. 反证。设源 $G \in \mathcal{D}$ 位于频道 c 。由定理 5.1, 存在 k 使 $\|G - E_k\| \leq 992.057 < R_{\min} \leq R$, 即 G 在 E_k 处位于有效接收范围内, 检测结果必为 direction 或 near, 与假设矛盾。□

程序据此把该频道标记为 CERTIFIED_EMPTY。这是 “不漏源” 的逻辑闭环: 清除计数与空频道证明之和达到 20 时, 二十个频道全部有定论。

found = 16 的提前终止. 由附录 1(1), 各源频道互异; 由题设 $N \leq 16$ 。记 found 为已实际收到过 direction 或 near 的互异频道数, 则

$$\text{found} = 16 \wedge N \leq 16 \implies N = 16 \implies \text{全部仍为 UNKNOWN 的频道皆空}. \quad (22)$$

此时程序立即把全部未知频道置为 CERTIFIED_EMPTY 并清空剩余 EW 队列, 只保留已发现源的定位与清除任务。这是严格逻辑推理而非概率剪枝, 不损害完备性, 却可在高源数案例中省去数站 EW 访问。

5.3 频道状态机与保守位置后验

每个频道 $c \in \{1, \dots, 20\}$ 维护一个四状态记录: UNKNOWN (尚未在任何检测点收到该频道信号, 且尚未完成八点扫描)、LOCALIZING (已收到至少一次 direction/near, 正在收缩后验)、CLEARED (已成功清除)、CERTIFIED_EMPTY (由推论 5.2 或式 (22) 判定为空)。只有后两者是终止状态, 且分别由推论 5.2 与稍后的定理 5.4 保证正确性。LOCALIZING 状态另设一个 lost 标志: 若在某点对已处于 LOCALIZING 的频道检测得 no_signal, 说明该点已跑出该源的有效接收

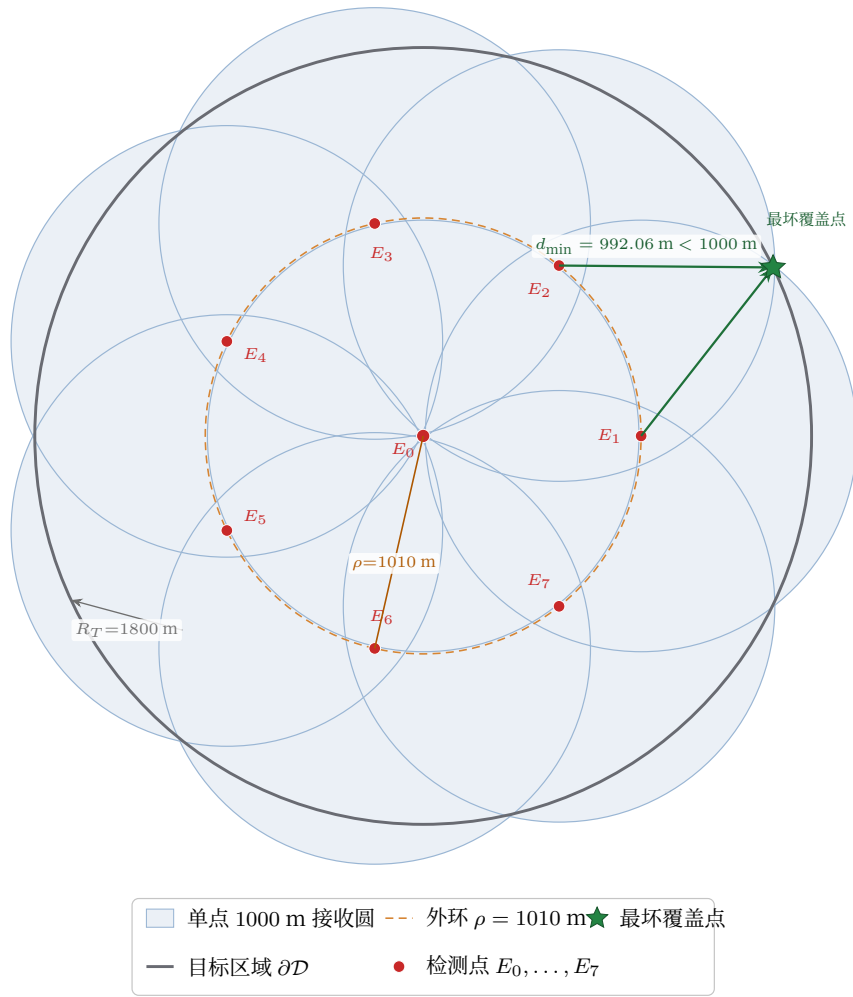


图6 八点完备搜索骨架：原点 E_0 与半径 $\rho = 1010\text{ m}$ 外环上的七个等角点 $E_1, \dots, E_7$ ，其 1000 m 接收圆（浅蓝）的并集严格包含目标区域。最坏覆盖点位于目标圆边界、相邻两外环点的角平分方向上，到最近检测点的距离为 $992.057\text{ m} < 1000\text{ m}$ ，故任一干扰源必被至少一个检测点接收

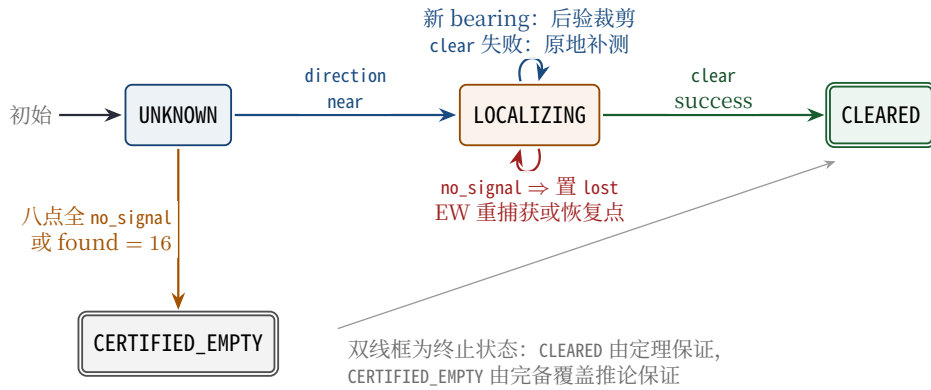


图7 单频道状态机。只有两条出边指向终止状态（双线框），且两者分别由推论 5.2 与定理 5.4 保证正确性

半径，此时不修改后验（no_signal 只给出 $\|G - p\| > R \geq R_{\min}$ 这一非凸约束，本文出于保守性不将其并入凸后验），而是置 lost，并优先等待后续 EW 点重新捕获，状态转移见图 7。

设频道 c 已在检测点 $S_1, \dots, S_q$ 获得示向度 $\theta_1, \dots, \theta_q$ ，沿用集员范式，真实源必落在

$$\mathcal{F}_{\text{true}} = \mathcal{D} \cap \bigcap_{j=1}^q [A_j \cap B(S_j, R_{\max})], \quad A_j = \{S_j + \rho d(\phi) : \rho \geq 0, |\phi - \theta_j| \leq \delta\}, \quad (23)$$

其中 A_j 由命题 3.1 化为两个线性半平面；两处圆盘约束则用外接正多边形离散（目标圆 72 边形、接收圆 32 边形）。

命题 5.3 (后验的保守性). 记程序维护的多边形为 $\mathcal{F}_{\text{code}}$ ，则 $\mathcal{F}_{\text{true}} \subseteq \mathcal{F}_{\text{code}}$ 。

证明. 外接正多边形包含被它外接的圆盘：边数为 s 、外接于半径 R 的圆的正多边形，其内切圆半径恰为 R ，故该多边形 $\supseteq B(\cdot, R)$ 。半平面约束是精确的。凸集之交对包含关系单调，故逐条裁剪得到的 $\mathcal{F}_{\text{code}}$ 包含由精确约束得到的 $\mathcal{F}_{\text{true}}$ 。□

命题 5.3 是全部“保证”类结论的基石：一切离散化误差都被推向保守一侧，程序绝不会因为把圆盘换成多边形而误认为定位精度已经达标。对多边形 $\mathcal{F}_{\text{code}}$ 记直径端点中点 C 与工作半径 $\rho_{\text{work}} = D(\mathcal{F}_{\text{code}})/2$ 、最小包围圆 $(C_{\text{MEC}}, r_{\text{MEC}})$ 。由第一问的 Jung 界讨论， $\rho_{\text{work}} \leq r_{\text{MEC}} \leq \frac{2}{\sqrt{3}}\rho_{\text{work}}$ ，两者在细长后验下几乎相等，而在接近等边的后验下差异可达 15.5%——这正是第一问“必须用最小覆盖圆而非直径圆作为不确定性包络”这一结论在第三问中的直接兑现：若此处误用 $\rho_{\text{work}} \leq 20$ 作为清除判据，将产生虚假确定性。

5.4 清除判据：严格保证清除与试探清除

定理 5.4 (MEC 清除判据). 若 $r_{\text{MEC}} \leq 20$ m，则在 C_{MEC} 处执行光学定位与清除必然成功。

证明. 由最小包围圆定义 $\mathcal{F}_{\text{code}} \subseteq B(C_{\text{MEC}}, r_{\text{MEC}}) \subseteq B(C_{\text{MEC}}, 20)$ 。由命题 5.3, $G \in \mathcal{F}_{\text{true}} \subseteq \mathcal{F}_{\text{code}}$ ，故 $\|G - C_{\text{MEC}}\| \leq 20$ 。由附录 2(8)，光学精确定位与清除的成败只取决于清除点与源的距离， ≤ 20 m 即成功。□

同理，若某次检测返回 near，由附录 2(9) 知 $\|G - p\| \leq 5$ m，在原地清除亦必然成功。这两类动作构成保证清除集合 $\mathcal{M}_{\text{clear}}$ 。

由附录 2(8)，一次清除尝试的时间代价是非对称的： $T_{\text{clear}}^{\text{succ}} = 3 + 2 = 5$ s, $T_{\text{clear}}^{\text{fail}} = 3$ s。设放弃试探而走完严格流程的追加成本为 ΔT ，试探成功概率为 p ，则试探清除有利当且仅当

$$p \Delta T > (1 - p) \cdot 3 \iff p > \frac{3}{\Delta T + 3}. \quad (24)$$

即使只考虑一次额外测向与 100 m 往返 ($\Delta T \approx 45$ s)，阈值也仅 $p > 6.3\%$ 。这一悬殊的不对称性说明：在后验已经较小但尚未严格达标时，敢清在期望意义上几乎总是划算的。据此设置三类试探清除（均不承担完备性证明）：

类型	触发条件	清除点
Strong Probe	$q \geq 2$ 且 $\rho_{\text{work}} \leq 80$ m	C
Weak Probe	$q \geq 2$ 且 $\rho_{\text{work}} \leq 180$ m	C
Geometric Probe	$ \sin \phi \geq 0.65$ 且 $E \leq 24$ m 且交点 $\in \mathcal{F}_{\text{code}}$	两中心线交点 $\hat{G}$

其中 Geometric Probe 使用两条示向中心线的交点，设两线交会角为 ϕ 、交点到两检测点的距离为 r_1, r_2 ，定义误差代理

$$E = \tan \delta \cdot \frac{r_1 + r_2}{|\sin \phi|}. \quad (25)$$

必须强调， E 不是定位误差的严格上界（严格上界由 r_{MEC} 给出），它只用于从几何上筛选低风险的试探点；因此即使 $E \leq 24$ m，该动作仍按 Probe Clear 处理：成功则频道置 CLEARED，结

束；失败则损失 3 s、就地补一次 measure 以收缩后验。“失败即就地测量”把一次失败转化为一次信息获取：机器狗已经在该点，补测只需 5 s 而无移动成本，且新 bearing 的顶点恰在旧后验中心附近，裁剪效率很高。程序另记录每个频道的失败清除点，后续候选点若距任一失败点不足 8 m 则被剔除，避免在同一位置重复试探。

三次正式测试共尝试清除 60 次，其中成功 43 次、失败 17 次，失败合计仅耗 51 s，占三局总虚拟时间 10087.3 s 的 0.51%——与式 (24) 的判断一致。

5.5 在线优先级调度

每个决策时刻，程序把所有可执行动作放入同一候选池，包括：剩余 EW 搜索点；各 LOCALIZING 频道的保证清除点、三类试探清除点、第二检测点 (Q2-fast)、后验中心逼近点；以及 lost 频道的恢复点。对候选动作 a ，设其目标点为 $p(a)$ ，评分取

$$\text{score}(a) = \|X - p(a)\| - \beta(a), \quad a^{\text{greedy}} = \arg \min_a \text{score}(a), \quad (26)$$

其中 X 为当前位置， $\beta(a) \geq 0$ 是以米为单位的等效距离奖励。把优先级折算成距离而非时间，是因为 $T_{\text{move}} = \|\cdot\|/v$ 与距离成正比，量纲统一且便于标定。各项取值见表 3，全部由演练测试标定，不是理论最优值。奖励的相对大小体现了三条设计意图：保证清除的 320 m 远高于其余项，意味着只要存在可严格清除的目标，机器狗宁可绕行数百米也先去把它清掉——因为清除是唯一能减少任务总量的动作；试探清除按“几何质量”排序 (Geometric > Strong > Weak)；EW 只给很小的推进奖励，因为它是必须完成的任务，其顺序另由下节的保护器统一管理。

第一次获得 bearing 后，理论上应按第二问的稳健选址模型求第二检测点，但那是离线的单源模型，若对每个源都走 1016 m，移动成本将完全失控。因此程序采用轻量化近似：取当前后验中心 C 与第一条示向线的单位法向 n ，令 $S_2^\pm = C \pm 80n$ ，并以 $|\sin \phi|$ 作为信息增益，按 $\beta = 45|\sin \phi|$ 参与统一竞争。这一选择保留了第二问的核心结论——侧向偏移以改善交会角——但把偏移量压缩到 80 m 量级。

机器狗到达某个 EW 点后，除扫描仍为 UNKNOWN 的频道外，还会顺路对若干正在 LOCALIZING 的频道补测一次：此时移动成本为零，一次补测只需 5 s 检测加至多 1 s 切频，而 EW 点位于外环、与多数后验中心的基线很长，恰是式 (25) 中 $|\sin \phi|$ 较优的位置。候选按基线长度 (≥ 220 m)、交会角 ($|\sin \phi| \geq 0.20$) 与后验中心距离 (≤ 1650 m) 筛选并排序，每站最多 4 个；当 found ≥ 13 时放宽到 5 个。

表 3 第三问的等效距离奖励与演练标定参数 (均为工程取值，非理论最优)

动作/参数	性质	触发条件	β 或取值
保证清除 (near 原地 / $r_{\text{MEC}} \leq 20$)	严格保证	定理 5.4	320 m
Geometric Probe Clear	几何依据的启发式	$ \sin \phi \geq 0.65, E \leq 24$ m	170 m
Strong Probe Clear	启发式	$q \geq 2, \rho_{\text{work}} \leq 80$ m	150 m
Weak Probe Clear	启发式	$q \geq 2, \rho_{\text{work}} \leq 180$ m	60 m
Q2-fast 第二检测点	信息奖励	$q = 1$, 侧移 80 m	$45 \sin \phi $
EW 搜索点	完备性任务	尚未访问	15 m
后验中心 refine / lost 恢复	兜底	恒可用 / EW 队列已空	0
外环半径 / 点数	覆盖余量 8.0 m	定理 5.1	1010 m / 7
顺路复测门槛	剔除低信息量补测	基线 ≥ 220 m, $ \sin \phi \geq 0.20$	每站 ≤ 4 (5)
失败清除点回避半径	防止同点重复试探	—	8 m
路径 regret 阈值 R_0	保证保护器低介入	式 (29)	250 m (= 50 s)

5.6 强制任务最短路保护器

演练与正式测试均表明，移动时间占总虚拟时间的比例接近 80% (三次正式测试为 78.1%、80.7%、78.5%)。因此继续增加测量或精调阈值的边际收益有限，主要瓶颈在于任务访问顺序。式 (26) 只比较“当前一步”的代价，对若干无论如何都必须完成的任务无法保证整体顺序合理。本文因此在不改动式 (26) 的前提下，外加一层保守的顺序保护。

定义当前强制任务集合 $\mathcal{M} = \mathcal{M}_{\text{EW}} \cup \mathcal{M}_{\text{clear}}$ ，其中 $\mathcal{M}_{\text{EW}}$ 为尚未访问的 EW 点， $\mathcal{M}_{\text{clear}}$ 为已满足定理 5.4 的保证清除点。二者的共同特征是：在未触发式 (22) 的前提下，它们最终都必须被执

行。而 Q2-fast、中心 refine、三类 Probe Clear 与 lost 恢复不属于强制任务——它们具有信息获取或试探性质，是否执行本身就是决策变量，不应为了缩短路径而被强行重排。

设当前位置为 X ，强制任务点为 $P_1, \dots, P_N$ 。定义访问顺序 π 的开放路径长度

$$L(\pi) = d(X, P_{\pi_1}) + \sum_{k=1}^{N-1} d(P_{\pi_k}, P_{\pi_{k+1}}), \quad L^* = \min_{\pi} L(\pi). \quad (27)$$

程序仅在 $2 \leq N \leq 12$ 时启动该优化，故可用 bitmask 动态规划精确求解而无需最近邻等近似。设 $f(i, S)$ 为“当前位于任务 i 、尚需访问集合 S ”的最短剩余路程：

$$f(i, S) = \min_{j \in S} [d(P_i, P_j) + f(j, S \setminus \{j\})], \quad f(i, \emptyset) = 0, \quad (28)$$

于是以任务 i 为第一站的最优总路径为 $L_i = d(X, P_i) + f(i, \mathcal{M} \setminus \{i\})$ ，且 $L^* = \min_i L_i$ 。状态数 $N \cdot 2^N$ 、转移 $O(N)$ ， $N \leq 12$ 时不超过 5.9×10^5 次基本运算。

程序首先完全按式 (26) 得到 a^{greedy} ；仅当 $a^{\text{greedy}} \in \mathcal{M}$ 时，计算其路径 regret $\mathcal{R} = L_{a^{\text{greedy}}} - L^* \geq 0$ 。只有 $\mathcal{R} \geq R_0 = 250 \text{ m}$ 时，才把本次动作替换为开放最短路的第一站；否则完全保持原选择。完整触发条件为（四者需同时满足）：

$$(i) a^{\text{greedy}} \in \mathcal{M}; \quad (ii) 2 \leq N \leq 12; \quad (iii) \text{found} < 15; \quad (iv) \mathcal{R} \geq 250 \text{ m}. \quad (29)$$

条件 (iii) 在 $\text{found} \geq 15$ 时主动关闭保护器：此时已非常接近式 (22) 的严格提前终止条件，若仍按静态强制任务集合重排搜索顺序，可能推迟第 16 个源的发现。三次正式测试中 `route_guard_overrides` 均为 0，说明在这三个案例上式 (26) 的贪心顺序未出现 250 m 以上的确定性绕路，保护器保持“低介入”；其价值在于对少数不利案例提供上界保护，代价仅为每次决策不超过 10^{-3} s 的额外计算。

注 5.5 (不能声称全局最优). 式 (28) 的求解对当前时刻的强制任务集合是精确的，但执行下一个动作后，可能出现新的 bearing、新的保证清除点、新的活跃源、 $\text{found} = 16$ 或 EW 被批量取消，任务集合本身会动态变化。因此正确的表述是：本文用 bitmask DP 精确求解当前强制任务集合的开放最短访问路径，并将其作为在线贪心调度的保护层，而非用 TSP 取代调度器，更不是求出了全局最优路线。

5.7 完备性、正确性与终止性

定理 5.6 (策略的完备性与正确性). 在附录 1-2 的模型假设下，若机器狗执行完全部八个 EW 点并对每个进入 LOCALIZING 的频道执行到严格清除判据成立，则：(G1) 不漏源：任何位于 $\mathcal{D}$ 内的全向源所在频道都不会被误判为空（定理 5.1 与推论 5.2）；(G2) 清除不误判：每次由 $r_{\text{MEC}} \leq 20 \text{ m}$ 或 `near` 触发的清除必然成功（命题 5.3 与定理 5.4）；(G3) 提前终止不损完备性： $\text{found} = 16$ 时取消剩余 EW 不会遗漏任何源（式 (22)）。因此终止时有 $N_{\text{cleared}} + N_{\text{empty}} = 20$ ，且 $N_{\text{cleared}} = N$ ，即被清除干扰源个数的比例为 100%。

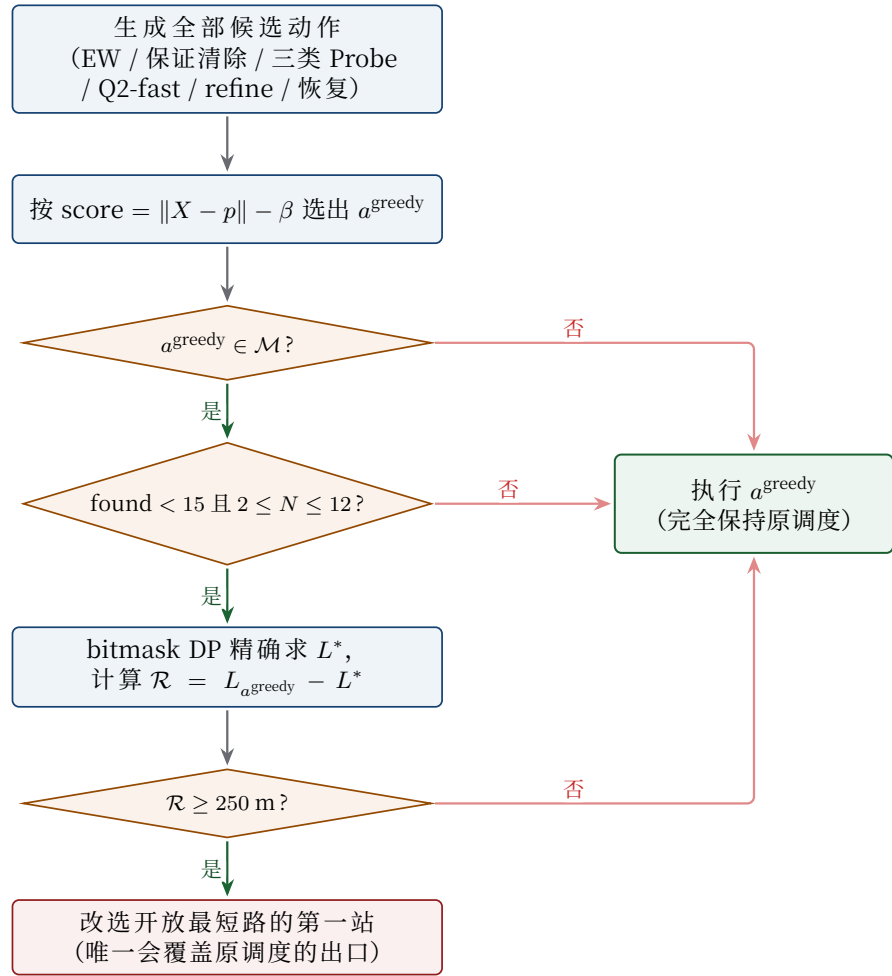
关于终止性，需要区分两个层次。(a) 搜索阶段严格终止：每访问一个 EW 点， $|\mathcal{M}_{\text{EW}}|$ 严格减一，至多八步耗尽。(b) 定位阶段的单调性只是弱单调：每次新的 bearing 使 $\mathcal{F}_{\text{code}}$ 单调收缩， r_{MEC} 单调不增，故 $r_{\text{MEC}} \leq 20$ 迟早成立；但试探清除失败后的重测、lost 恢复等分支无法给出显式步数上界。为此程序设置逻辑动作数硬上限 2500 与官方 25 分钟窗口作为兜底，并在每次 clear 失败后强制就地补测以确保后验实际收缩。三次正式测试的实际动作数远低于该上限，退出原因均为主动调用 `/exit`。

5.8 算法总流程

5.9 正式测试结果

三次正式测试的完整统计见表 4（格式与题目表 1 一致）。三次测试中 $N_{\text{cleared}} + N_{\text{empty}}$ 均等于频道总数 20，由定理 5.6 得被清除干扰源个数的比例

$$\frac{N_{\text{cleared}}}{N} = 100\% \quad (\text{三次均如此}), \quad (30)$$



绿色路径=四个条件全部满足（唯一覆盖分支）；红色路径=任一条件不满足，立即回落到默认调度

图 8 强制任务最短路保护器的决策流程：默认路径是右侧“完全保持原调度”，只有四个条件同时满足时才覆盖

算法 3: 第三问：搜索-定位-清除一体化在线策略

输入: $R_T = 1800$, $R_{\min} = 1000$, $R_{\max} = 1500$, $\delta = 1^\circ$, $v = 5$ m/s; 奖励表 3; $R_0 = 250$ m
输出: 全部频道的终态、清除数 N_{cleared} 、总虚拟时间 T_{total}

```

1 按式 (19) 建立 EW 队列; 全部频道置 UNKNOWN; 调用 /enter;
2 当 存在非终态频道 且 动作数 < 2500 执行
3   若 found = 16 则
4     全部 UNKNOWN  $\rightarrow$  CERTIFIED_EMPTY; 清空 EW 队列; // 式 (22)
5   对八点均 no_signal 的频道执行空频道认证; // 推论 5.2
6   生成候选池: EW、保证清除、Geometric/Strong/Weak Probe、Q2-fast、refine、恢复;
7    $a \leftarrow \arg \min_a (\|X - p(a)\| - \beta(a))$ ; // 式 (26)
8   若  $a \in \mathcal{M}$  且 found < 15 且  $2 \leq N \leq 12$  则
9     由式 (28) 精确求  $L^*$ ;  $\mathcal{R} \leftarrow L_a - L^*$ ;
10    若  $\mathcal{R} \geq R_0$  则
11       $a \leftarrow$  开放最短路的第一站; // 保护器覆盖
12  分支  $a$  的类型 执行
13    情形 EW 执行 移动至该点; 扫描全部 UNKNOWN 频道; 顺路补测至多 4 (5) 个
        LOCALIZING 频道;
14    情形 MEASURE 执行 移动并检测; direction  $\Rightarrow$  按式 (23) 裁剪后验; near  $\Rightarrow$  原地保证清
        除; no_signal  $\Rightarrow$  置 lost;
15    情形 CLEAR 执行 移动并清除; 失败则记录失败点并就地补测一次; // 式 (24) 后处理
16    ;
17 调用 /exit, 输出统计量;
```

平均定位清除时间分别为 228.15、236.49、238.82 s, 三次均值 234.49 s。移动时间占虚拟总时间 78.1% ~ 80.7% (三次合计 79.09%), 移动距离合计 39891.6 m, 切频与清除失败等固定开销合计不足总时间的 3%, 保护器覆盖次数三次均为 0, 说明贪心调度已足够接近强制任务的最短访问顺序。

表 4 问题 3 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
测试 1 CKY9-797J-MU37-S6FW	14	228.15 s	1.24 s
测试 2 MWYP-QZBV-656B-ZQMG	14	236.49 s	1.22 s
测试 3 JUV3-CTUY-Q87V-9C5Y	15	238.82 s	1.31 s

三次结果的横向比较给出两点可解释的规律。其一, 平均定位清除时间与源数正相关而非反相关 (14 $\rightarrow$ 228.2 s、15 $\rightarrow$ 238.8 s), 这与“固定成本被更多源摊薄”的朴素直觉相反: 八点骨架的搜索成本是固定的, 但源越多, 需要交会定位的频道越多、顺路共享的机会被每站上限约束, 额外的专程测量与清除往返就越多, 后者的增量超过了固定成本的摊薄效应。其二, 测试 3 的清除失败次数 (7 次) 高于另两次, 但其额外代价仅 6 s, 不足总时间的 0.2%, 再次印证式 (24)。

5.10 第三问小结

以“原点 + 半径 1010 m 外环七点”的八点骨架保证搜索完备性 (定理 5.1); 以集员范式维护每个频道的保守外包后验 (命题 5.3); 以最小包围圆半径 ≤ 20 m 作为严格清除判据 (定理 5.4), 并在成本高度不对称的前提下使用三类试探清除加速; 用等效距离奖励式 (26) 统一在线调度; 最后用强制任务集合上的开放最短路修正大尺度路径冗余。三次正式测试的被清除干扰源个数比例均为 100% ($N_{\text{cleared}} + N_{\text{empty}} = 20$), 平均定位清除时间分别为 228.15、236.49、238.82 s, 均值 234.49 s; 移动时间占比 79.1%, 说明策略的时间成本已由信息获取转移到不可压缩的空间遍历, 进一步改进应来自骨架点数 (1 + 6 亦可严格覆盖) 与任务顺序的全局优化。

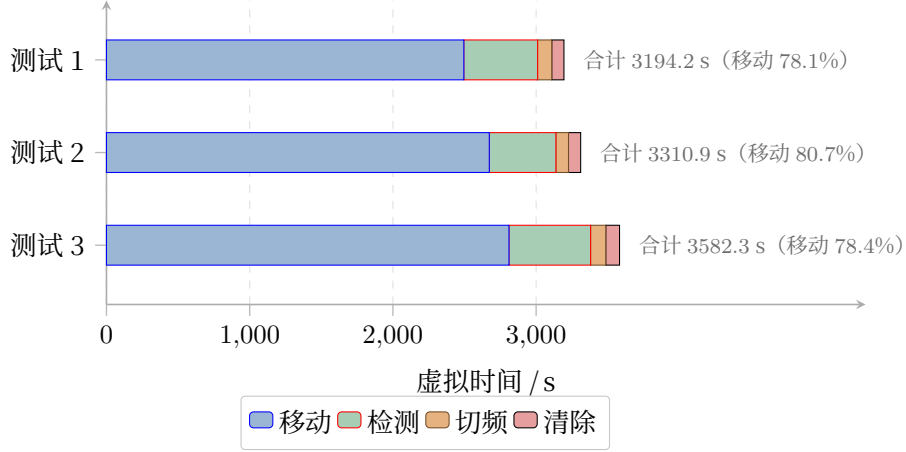


图 9 三次正式测试的时间构成。移动时间稳定占 78% ~ 81%，清除仅占约 2.6%，是 § 5.6 把优化重点放在路径顺序、§ 5.4 容忍试探清除失败的直接依据

6 第四问：定向源的方向完备搜索与确定性清除

6.1 定向源破坏了哪些推理链

由 (A7)，检测点 E 能接收到位于 G 、指向 u 的定向源当且仅当

$$\text{vis}(E; G, u) \iff \underbrace{\|E - G\| \leq R}_{\text{距离}} \wedge \underbrace{(E - G) \cdot u \geq 0}_{\text{半平面}}. \quad (31)$$

把第三问的三条推理链逐条对照 (31)，可以精确地看清哪些必须重做、哪些可以原样继承。

(1) 搜索完备性：整体失效，必须重做。 第三问的八点骨架只保证距离覆盖 $\mathcal{D} \subseteq \bigcup_k B(E_k, R_{\min})$ 。对定向源，距离覆盖不再蕴含可接收：取 $G = (1800, 0)$ 、 $u = (1, 0)$ （指向区域外），则 $(E - G) \cdot u \geq 0$ 要求 $E_x \geq 1800$ ，而八点骨架的全部点都满足 $\|E_k\| \leq 1010$ ，故该源在任何一个 EW 点都收不到。于是推论 5.2 的空频道证明彻底失效——“八点全无信号”既可能是无源，也可能是所有检测点都落在辐射扇区之外。

(2) no_signal 的语义：由单一约束退化为析取。 第三问中 no_signal 至少意味着 $\|G - p\| > R$ ；第四问中它只意味着“ $\|G - p\| > R$ 或 $(p - G) \cdot u < 0$ ”。这是一条析取约束，既非凸也不能分支穷举（ u 是连续未知量），因此本文完全不用 no_signal 裁剪后验，只用它触发失联标志。随之而来的新问题是：第三问“跑出接收半径则向源靠近即可重新接收”的直觉不再成立，必须给出对任意指向都有效的恢复构造。

(3) 位置后验与清除判据：原样继承。 一次 direction 读数的含义没有变化——收到信号本身已蕴含检测点位于辐射半平面内，读数仍满足 $|\theta - \theta^*| \leq \delta$ 。因此角域约束 (23)、保守外包命题 5.3、MEC 清除定理 5.4 与 near 原地清除全部逐字成立，无需任何修改。清除本身也只取决于距离，与指向无关。

综上，第四问 = 用“方向完备”替换“距离完备”，用“恢复扇”替换“单点重捕获”，其余框架整体复用。

6.2 方向完备性的充要条件

定义 6.1 (方向完备). 称点集 $\mathcal{E}$ 在 $G \in \mathcal{D}$ 处方向完备，若对任意单位向量 u 都存在 $E_k \in \mathcal{E}$ 使 $\text{vis}(E_k; G, u)$ 成立；称 $\mathcal{E}$ 方向完备，若它在每个 $G \in \mathcal{D}$ 处方向完备。

记 $N_R(G) = \{E_k \in \mathcal{E} : \|E_k - G\| \leq R\}$ 。

定理 6.2 (方向完备的凸包充要条件). $\mathcal{E}$ 在 G 处对有效接收半径 R 方向完备，当且仅当

$$G \in \text{conv } N_R(G). \quad (32)$$

特别地，由于 $R \geq R_{\min}$ 蕴含 $N_{R_{\min}}(G) \subseteq N_R(G)$ ，条件 $G \in \text{conv } N_{R_{\min}}(G)$ 对一切可能的 R 一致充分；而当 $R = R_{\min}$ 时它同时必要。

证明. 充分性: 设 $G = \sum_k \lambda_k E_k$, $\lambda_k \geq 0$, $\sum \lambda_k = 1$, 求和只取 $N_R(G)$ 中的点。对任意单位向量 u , $\sum_k \lambda_k (E_k - G) \cdot u = (\sum_k \lambda_k E_k - G) \cdot u = 0$, 故不可能所有项都严格为负, 即存在 k 使 $(E_k - G) \cdot u \geq 0$; 又该 $E_k \in N_R(G)$ 满足距离条件, 故 $\text{vis}(E_k; G, u)$ 成立。

必要性 (逆否): 设 $G \notin \text{conv } N_R(G)$ 。 $\text{conv } N_R(G)$ 是有限点集的凸包, 为紧凸集 (可能为空), 由严格分离定理存在单位向量 u 与 $c \in \mathbb{R}$ 使 $x \cdot u < c < G \cdot u$ 对一切 $x \in \text{conv } N_R(G)$ 成立, 即 $(E_k - G) \cdot u < 0$ 对一切 $E_k \in N_R(G)$ 成立。于是指向 u 的源在距离 R 内的所有检测点都处于背面, 在距离 R 外的点又收不到, $\mathcal{E}$ 在 G 处不方向完备。 $\square$

定理 6.2 把一个“对连续的指向变量取全称量词”的条件化成了一个纯粹的、可用有限点集检验的凸包包含关系, 这是第四问全部严格结论的来源。

推论 6.3 (必须在目标圆外布点)。任何方向完备骨架都必须包含满足 $\|E\| > R_T$ 的检测点。

证明. 取 $G = (R_T, 0)$ 、 $u = (1, 0)$ 。可接收要求 $(E - G) \cdot u = E_x - R_T \geq 0$, 即 $E_x \geq R_T$, 从而 $\|E\| \geq R_T$, 等号仅当 $E = G$ 。若全部检测点都满足 $\|E\| \leq R_T$ 且不与 G 重合, 则该源不可被接收; 由于 G 可取遍整条边界, 检测点不可能逐一与之重合, 故必须存在目标圆外的点。 $\square$

这条推论解释了为什么第三问的骨架无论如何加密都救不回来: 它的全部点都在半径 1010 m 的圆内。

命题 6.4 (目标圆外点数的下界)。任何方向完备骨架至少含 7 个满足 $\|E\| \geq R_T$ 的检测点。

证明. 对边界方向 φ 取 $G(\varphi) = R_T d(\varphi)$ 、 $u = d(\varphi)$ 。设检测点 $E = \rho d(\varphi + \Delta)$ 能服务该对, 则须同时满足 $\rho \cos \Delta \geq R_T$, $\rho^2 + R_T^2 - 2R_T \rho \cos \Delta \leq R_{\min}^2$, 即 $\cos \Delta \geq \max\{R_T/\rho, (\rho^2 + R_T^2 - R_{\min}^2)/(2R_T \rho)\} =: c(\rho)$ 。两式在 $\rho = \rho^* = \sqrt{R_T^2 + R_{\min}^2} = 2059.13$ m 处相等, 且 $c(\rho)$ 在 ρ^* 处取最小值 $R_T/\rho^* = 0.87416$, 故 $|\Delta| \leq \Delta_{\max} = \arccos(R_T/\sqrt{R_T^2 + R_{\min}^2}) = 29.055^\circ$ 。即每个检测点最多服务长度 $2\Delta_{\max} = 58.11^\circ$ 的边界方向窗口, 且必满足 $\|E\| \geq R_T$ 。覆盖 $[0^\circ, 360^\circ)$ 需要至少 $\lceil 360/58.11 \rceil = 7$ 个这样的点。 $\square$

6.3 22 点方向完备骨架及其网格证书

构造. 取中心一点、半径 990 m 的内环七点、以及内切圆半径为 $R_T + 10 = 1810$ m 的正十四边形外环十四点 (外接半径 $\rho_{\text{out}} = 1810/\cos(\pi/14) = 1856.548$ m, 整体旋转相位 0.7° 以避免与内环共线), 共

$$\mathcal{E}_{22}: 1 + 7 + 14 = 22 \text{ 点.} \quad (33)$$

三项参数各自对应一条设计约束:

(i) 外环为什么在圆外、为什么取十四点。由推论 6.3 必须出圆; 取正十四边形的内切圆半径 $1810 > R_T$ 保证整个目标圆落在外环凸包内部并留 10 m 余量。由命题 6.4 的计算, $\rho_{\text{out}} = 1856.548$ m 的单点边界服务半窗口为 $\arccos(R_T/\rho_{\text{out}}) = 14.178^\circ$, 而十四点的相邻半间距为 $180^\circ/14 = 12.857^\circ < 14.178^\circ$, 恰好覆盖全部边界外向方向并留 1.32° 余量; 若减到十二点则半间距 $15^\circ > 14.178^\circ$, 边界外向方向出现漏洞。

(ii) 内环为什么取七点、半径 990 m。中心点与外环之间存在半径 $1000 \sim 1800$ m 的环带, 其中的点若只有中心与个别外环点在 1000 m 内, 局部凸包会退化成线段而不含 G 。内环相邻点间距 $2 \cdot 990 \sin(\pi/7) = 859.1$ m、内环到中心 990 m、内环到最近外环点约 866 m, 三者均小于 1000 m, 使得相邻搜索点两两互为“1000 m 邻居”, 局部凸包不会退化。

(iii) 阶段划分。实际执行时先访问中心与内环 (阶段 A, 共八点), 再开放外环十四点 (阶段 B)。阶段 A 的距离覆盖半径为 $\max\{990/(2 \cos(\pi/7)), \sqrt{R_T^2 + 990^2 - 2R_T \cdot 990 \cos(\pi/7)}\} = 1004.513$ m > 1000 m, 它连全向源的距离完备都不满足, 更不方向完备, 因此阶段 A 只是快速筛查而不承担任何证明。之所以仍然保留它, 是因为 42 轮演练中前八个检测点已经发现了 $480/535 = 89.72\%$ 的源: 早发现意味着后续的外环访问可以顺路完成大量交会测量。完备性由 22 点整体承担。

定理 6.5 (22 点方向完备证书). 设 Λ 为步长 $a = 2$ m 的方格网, $\Lambda \cap \mathcal{D}$ 中每个格点 p 都满足

$$\text{dist}(p, \partial \text{conv } N_{997}(p)) \geq m^* = 8.6016 \text{ m}, \quad (34)$$

则 $\mathcal{E}_{22}$ 方向完备, 即对任意 $G \in \mathcal{D}$ 与任意指向 u , 22 点中至少一点可接收。

证明. 任取 $G \in \mathcal{D}$, 取与之最近的格点 p , 则 $\|G - p\| \leq a/\sqrt{2} = 1.4142$ m。

第一步 (距离): 对任意 $E \in N_{997}(p)$ 有 $\|E - G\| \leq \|E - p\| + \|p - G\| \leq 997 + 1.4142 = 998.414 < 1000 \leq R$, 故 $N_{997}(p) \subseteq N_R(G)$, 从而 $\text{conv } N_{997}(p) \subseteq \text{conv } N_R(G)$ 。

第二步 (凸包): 由 (34), 闭球 $B(p, 1.4142) \subseteq \text{conv } N_{997}(p)$, 而 $G \in B(p, 1.4142)$, 故 $G \in \text{conv } N_{997}(p)$ 。

两步合并得 $G \in \text{conv } N_R(G)$, 由定理 6.2 即证。□

证书的两个余量都被程序在每局开始时打印并校验:

$$\underbrace{1000 - 997 - \frac{a}{\sqrt{2}}}_{\text{距离余量}} = 1.5858 \text{ m}, \quad \underbrace{m^* - \frac{a}{\sqrt{2}}}_{\text{凸包余量}} = 7.1874 \text{ m}. \quad (35)$$

二者均为正是定理 6.5 成立的充分条件; 程序在启动时重新核对搜索参数元组与证书版本号, 一旦点数、半径或相位被改动即拒绝运行, 避免沿用失效的证书。

独立复核与最坏点. 用另一套网格锚定独立重算 (32): 对每个格点计算 997 m 邻域内各搜索点相对该点的方位角, 判据 $G \in \text{conv } N$ 等价于排序后的最大角间隙不超过 180° 。全网格的最大角间隙为 $177.69^\circ < 180^\circ$, 最小局部凸包余量 8.31 m。最坏点位于 (772, 1628) 附近, 即目标圆边界上相邻两外环点的角平分方向, 该处 997 m 内只有 3 个搜索点, G 恰在这个细长三角形的内部。

最小性检验. 对当前 22 点骨架的全部 $\binom{22}{21} = 22$ 个 21 点子集重新执行同一 2 m / 997 m 证书, 22 种删除方式全部失败。因此不存在“直接删掉当前某一个点且仍沿用严格证书”的安全捷径; 这不等于 22 是全局最小方向完备点数, 但说明当前构造在这一族内是局部不可约的。

推论 6.6 (定向情形下空频道的严格判定). 若某频道在全部 22 个骨架点的检测结果均为 no_signal, 则目标区域内该频道不存在干扰源 (无论它是全向源还是任意指向的定向源)。

证明. 反证。若存在源 (G, u) , 由定理 6.5 存在 E_k 使 $\text{vis}(E_k; G, u)$ 成立, 该点的检测结果必为 direction 或 near, 矛盾。□

推论 6.6 把第三问中被破坏的“不漏源”逻辑闭环完整地恢复了: 清除计数与认证空频道数之和达到 20 时, 二十个频道全部有定论。式 (22) 的 found = 16 提前终止只用到频道唯一性与 $N \leq 16$, 与源的指向无关, 故在第四问中原样有效。

6.4 失联与四点恢复扇

设频道最近一次成功测向发生在 S 、示向度为 θ , 记 $u = d(\theta)$ 、 $n = d(\theta + 90^\circ)$ 。此后若在其他位置得到 no_signal, 程序置 lost。第三问的做法 (向源靠近以重新进入接收半径) 在这里可能完全无效: 若从背面靠近, 走到源脚下也收不到。

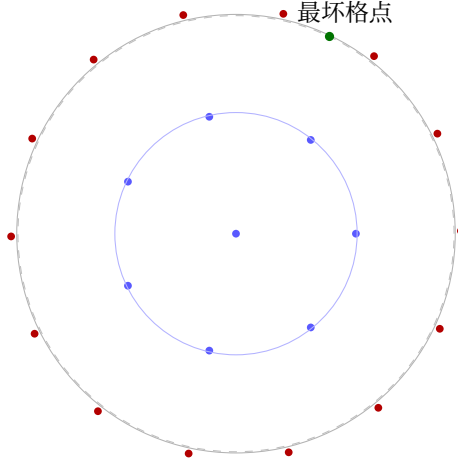
命题 6.7 (四点恢复扇). 在示向读数精确的理想情形下, 四个候选点

$$A_{\pm} = S + 200u \pm 350n, \quad B_{\pm} = S \pm 350n \quad (36)$$

中至少有一个可以接收到该源。

证明. 以 S 为原点、 (u, n) 为正交基。源位于 $G = ru$, $r \in (0, 1500]$, 且由“ S 处曾成功接收”知 S 在辐射半平面内, 即 $e \cdot (S - G) \geq 0$, 其中 $e = \cos \beta u + \sin \beta n$ 为指向。代入 $S - G = -ru$ 得 $-r \cos \beta \geq 0 \iff \cos \beta \leq 0$ 。候选点 $P = S + au + bn$ 满足半平面条件当且仅当 $e \cdot (P - G) = (a - r) \cos \beta + b \sin \beta \geq 0$ 。

情形 1: $r \geq 200$ 。取 $a = 200$, 则 $a - r \leq 0$, 故 $(a - r) \cos \beta \geq 0$; 再取 $b = 350 \text{sgn}(\sin \beta)$, 则 $b \sin \beta \geq 0$, 两项之和非负, 故 A_+ 或 A_- 之一在辐射半平面内。距离方面 $\|P - G\|^2 = (r - 200)^2 +$



22 点方向完备骨架：中心 + 内环 7（蓝）+ 外环 14（红），外环内切圆 $1810 > 1800$ m

图 10 22 点方向完备骨架的构造。判据不是“ G 离某点足够近”，而是“ G 被 1000 m 内的搜索点包住”——这正是定理 6.2 的凸包条件

350^2 ：若 $r \leq 1000$ 则 $\|P - G\| \leq \sqrt{800^2 + 350^2} = 873.2 < 1000 \leq R$ ；若 $r > 1000$ ，由 $R \geq r$ 只需 $(r - 200)^2 + 122500 \leq r^2$ ，即 $r \geq 406.25$ ，成立。

情形 2： $r < 200$ 。取 $a = 0$ ，则 $-r \cos \beta \geq 0$ ，同样取 $b = \pm 350$ 使 $b \sin \beta \geq 0$ ，故 B_+ 或 B_- 之一在半平面内；距离 $\|P - G\| = \sqrt{r^2 + 350^2} < \sqrt{200^2 + 350^2} = 403.1 < 1000 \leq R$ 。□

两对候选的分工是清晰的：前向对 $A_{\pm}$ 处理“源在测量轴上的投影不小于 200 m”的远源，原站侧向对 $B_{\pm}$ 处理近源。 $\pm 1^\circ$ 示向误差下命题 6.7 仍然成立：程序对 r 以 3 m 步长取遍 $[6, 1500]$ 、示向误差以 0.01° 步长取遍 $[-1^\circ, 1^\circ]$ 、指向 β 以 3° 步长取遍全圆（只枚举使 S 可见的 β ），逐例验证四点中至少一点满足 (31)，全部通过，作为运行前的几何自检。执行时四点按“先前向对、后侧向对，对内走近者优先”的固定顺序逐一尝试；若四点全部失败，则与 180° 覆盖及 $R \geq R_{\min}$ 的题设矛盾，程序抛错而非静默继续。

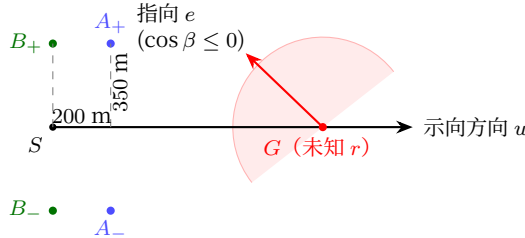


图 11 四点恢复扇：由“ S 曾成功接收”推出 $\cos \beta \leq 0$ ，前向平移 $A_{\pm}$ 处理远源（ $r \geq 200$ m），侧向平移 $B_{\pm}$ 处理近源（ $r < 200$ m），红色扇形为源的辐射半平面

6.5 后验、清除判据与有限步终止

位置后验与清除判据整体继承：后验按 (23) 维护，命题 5.3 与定理 5.4 逐字成立，near 后原地清除必成功。试探清除的成本判据 (24) 也不依赖指向，但第四问把几何 Probe 的门槛从第三问的 $(|\sin \phi| \geq 0.65, E \leq 24 \text{ m})$ 放宽到 $(|\sin \phi| \geq 0.45, E \leq 50 \text{ m})$ 并把奖励从 170 提到 260 m。理由是第四问的移动成本显著更高（22 点骨架约 1.4×10^4 m）， ΔT 随之增大，由 (24) 有利阈值 $3/(\Delta T + 3)$ 进一步下降；实测中几何 Probe 也确实是最主要的清除方式（三次正式测试的 40 次清除尝试中有 27 次是几何 Probe）。

定向情形下真正新增的风险是定位阶段可能不收敛：由于不能使用 no_signal 裁剪，若源恰好位于若干条示向线交会质量都不佳的位置，后验可能长期停在几十米量级而 r_{MEC} 始终大于 20 m。为此引入后验网格确定性清除。

定理 6.8 (后验网格清除的有限步保证). 设当前后验 $\mathcal{F}_{\text{code}}$ 的轴对齐包围盒为 $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ ，以步长 $h = 20\sqrt{2} \times 0.98 = 27.7186$ m 在其上布置方格网并逐点执行清除，则至多 $(\lceil \frac{x_{\max} - x_{\min}}{h} \rceil + 1)(\lceil \frac{y_{\max} - y_{\min}}{h} \rceil + 1)$ 次清除必有一次成功。

证明. 包围盒内任一点到最近网格点的距离不超过半个格对角线 $h/\sqrt{2} = 19.6 < 20$ m。由命题 5.3, 真实源 $G \in \mathcal{F}_{\text{true}} \subseteq \mathcal{F}_{\text{code}}$, 故 G 落在包围盒内, 于是存在网格点 P 使 $\|P - G\| \leq 19.6 \leq 20$ m, 在 P 处清除必成功。□

程序在“方向观测数 ≥ 6 且 $\rho_{\text{work}} \leq 120$ m”时才启用该网格（奖励 210 m），此时包围盒边长约 240 m，网格点不超过 10×10 ，最坏代价约 100×3 s 加上若干十米级移动，远低于反复远程往返测量的代价；而由定理 6.8，它给出了定位阶段的**确定性终止上界**。

6.6 在线调度：固定搜索环、机会成本与搜索后 2-opt

第四问的搜索骨架从 8 点增到 22 点，移动量约为第三问的 1.8 倍，因此调度层的目标非常明确：在不破坏完备性的前提下，把定位与清除动作尽可能“顺路”地嵌进固定的搜索路线。调度分三层。

(1) 固定最短搜索环。 对当前阶段（A 或 B）的待访问点，枚举全部旋转与两个方向共 $2m$ 条闭环顺序，取从当前位置出发的开放路线最短者；若最短长度并列，再以“顺路可形成大交会角的频道尽量提前”为严格次级排序准则。固定环一旦生成即被缓存，避免每次定位动作后重新贪心造成折返。

(2) 搜索期插入判据：现在做还是以后顺路做。 第三问用固定米数门槛判断是否插入非搜索动作，第四问改为机会成本比较。设动作目标点为 q ，当前位置为 X 、下一搜索点为 W ，定义当前插入绕路与未来最优插入绕路

$$\Delta_{\text{now}}(q) = \|X - q\| + \|q - W\| - \|X - W\|, \quad (37)$$

$$\Delta_{\text{fut}}(q) = \min \left\{ \min_{(W_i, W_{i+1})} [\|W_i - q\| + \|q - W_{i+1}\| - \|W_i - W_{i+1}\|], \|W_{\text{end}} - q\| \right\}, \quad (38)$$

其中 (W_i, W_{i+1}) 遍历剩余搜索路线的各条边，最后一项对应“搜索结束后专程前往”。记该动作的评分为 $s = \Delta_{\text{now}}(q) - \beta(a)$ （奖励见表 5），则执行判据为

$$s \leq \kappa(a) \cdot \Delta_{\text{fut}}(q), \quad (39)$$

即“现在的净代价不高于以后最优插入代价”； $\kappa(a) \in (0, 1]$ 是该类动作的经验成功率（表 5），用于折算“以后再做”的期望价值。若存在零移动成本的未来 EW 复测机会，则对 MEASURE 与 RECOVERY 类动作取 $\Delta_{\text{fut}} = 0$ ，即一律推迟到那次零成本复测；清除动作不能被一次复测等价替代，故仍按真实插入量比较。

必须诚实标注： κ 来自明文轨迹的观测成功率，它只影响调度顺序，不参与存在性、后验或清除正确性的任何证明；把它设为 1 只会让策略更保守。

(3) 搜索结束后的开放 2-opt。 搜索队列清空后，对每个活动频道取当前可执行的最佳节点，用“最近邻初解 + 开放路线 2-opt”求访问顺序并执行其第一站。为防止为了全局路线而牺牲眼前高价值任务产生长尾，设置首步 regret 保护：若 2-opt 首站的评分或距离比纯贪心动作差超过 100 等效米，则回退到贪心动作。

6.7 SAFE 局部支配层：只吃确定性收益的路径优化

42 轮诊断显示，即便有上述调度，移动仍占总时间的 75% ~ 84%。进一步压缩有两条路：一是改变调度顺序（例如调节可靠性、清除时机、重规划门槛），二是在不改变任何决策的前提下缩短移动。前者能用样本统计估计平均收益，却会改变观测顺序与未来状态，无法对每一个实例证明不退化；本文最终全部放弃第一条路，只保留第二条，并把它形式化为“局部支配”。42 轮最终诊断数据显示，V5 的主要时间仍由严格搜索与移动构成：此前尝试过的会改变调度顺序的优化（如调节可靠性常数、清除时机、重新规划门槛）虽有统计依据，但无法对每个实例证明不会退化，因此最终版不修改 22 点方向完备搜索骨架、阶段 A/B 结构、UNKNOWN 判空条件、opportunistic measure 规则、Q2-fast 候选、四点恢复扇、Geometric/Strong/Weak Probe 门槛、所有 bonus、所有旧可靠性常数、搜索期插入判据与搜索后 2-opt 策略——最终策略是“V5 原策略 + 只在能形成严格局部支配时缩短移动”，而不是重新训练或重新调度。

定义 6.9 (严格保证清除区域). 设频道后验多边形为 $\mathcal{P}$ （顶点 $V_1, \dots, V_s$ ），最小包围圆为 $B(c, r_{\text{MEC}})$ ，取数值余量 $\varepsilon = 0.05$ m，定义

$$K = \left\{ P : \|P - V_i\| \leq 20 - \varepsilon, i = 1, \dots, s \right\} \cup \left\{ P : \|P - c\| \leq 20 - r_{\text{MEC}} - \varepsilon \right\}. \quad (40)$$

表 5 第四问的等效距离奖励、触发门槛与调度可靠性（均为演练标定的工程取值）

动作类型	触发条件	β / m	κ
保证清除 (near 或 $r_{\text{MEC}} \leq 20 \text{ m}$)	定理 5.4	320	1.00
Geometric Probe Clear	$ \sin \phi \geq 0.45$, $E \leq 50 \text{ m}$, 交点 $\in \mathcal{F}_{\text{code}}$	260	0.88
后验网格清除 Sweep	方向数 ≥ 6 且 $\rho_{\text{work}} \leq 120 \text{ m}$	210	1.00
Strong Probe Clear	$q \geq 2$ 且 $\rho_{\text{work}} \leq 80 \text{ m}$	150	0.65
Weak Probe Clear	$q \geq 2$ 且 $\rho_{\text{work}} \leq 180 \text{ m}$	60	0.50
Q2-fast 第二检测点	$q = 1$, 后验中心 $\pm 80 \text{ m}$ 法向	$45 \sin \phi $	0.45
定向源四点恢复扇	lost 标志	0	0.70
内环 EW / 外环 EW	尚未访问 (阶段 A / 阶段 B)	500 / 300	1.00
后验中心 refine	恒可用 (兜底)	0	1.00
顺路复测门槛	基线 $\geq 220 \text{ m}$, $ \sin \phi \geq 0.15$, 中心距 $\leq 1200 \text{ m}$	—	
失败清除点回避半径	防止同点重复试探	8 m	
搜索后 2-opt 首步 regret 上限	防止长尾退化	100 m	
SAFE 清除余量 ε / 最小采纳增益	数值安全	0.05 m / 0.01 m	

命题 6.10 (K 中任一点清除必成功). 若 $r_{\text{MEC}} \leq 20$ 且 $P \in K$, 则在 P 处清除必然成功。

证明. 第一个集合: $G \mapsto \|P - G\|$ 是凸函数, 在凸多边形 $\mathcal{P}$ 上的最大值必在顶点取得, 故 $\max_{G \in \mathcal{P}} \|P - G\| = \max_i \|P - V_i\| \leq 20 - \varepsilon < 20$; 由命题 5.3, 真实源 $G \in \mathcal{P}$ 。第二个集合: 由 $\|c - G\| \leq r_{\text{MEC}}$ 与三角不等式 $\|P - G\| \leq \|P - c\| + \|c - G\| \leq 20 - \varepsilon < 20$ 。□

注意 K 的第一个集合是 s 个等半径圆盘之交, 为凸集且通常严格大于以 MEC 中心为心的安全内圆——后验越细长, 可自由移动的范围越大, 这正是收益的来源。

定理 6.11 (两动作局部支配). 设原调度器在状态 σ 下选定动作 $A = \text{“在 } C \text{ 处清除频道 } c\text{”}$ 且该动作严格保证成功, 并设其后继动作为 B (绝对目标点 Q)。若把 A 的执行点改为 $P \in K$ 且强制下一动作仍为 B , 则

$$\|X - P\| + \|P - Q\| \leq \|X - C\| + \|C - Q\|. \quad (41)$$

则两动作结束后的系统状态与原策略完全相同, 而总移动不增。

证明. 由命题 6.10, 在 P 处清除的结果与在 C 处一样是确定的 CLEARED, 故频道状态相同; 机器狗随后前往同一点 Q 执行同一类型、同一频道的动作 B 。若 B 是测量, 由 (A4) 源静止、由 (A2) 同一地点重复检测读数不变, 故在相同的 (频道, Q) 上得到相同读数, 后验裁剪结果相同; 若 B 是清除, 其成败同样只取决于 $\|Q - G\|$, 与路径无关。位置、当前测量频道、后验、搜索队列与候选缓存因而逐项相同, 即到达 Q 后两条轨迹重新汇合; 而移动量由假设不增。□

定理 6.11 给出的是逐实例的支配关系“同一前状态 $\rightarrow$ 更短或相等路径 $\rightarrow$ 同一后状态”, 而不是“平均意义上可能更好”。为使“同一后状态”成立, 实现上用一个影子调度器在 C 处虚拟执行清除并调用原动作选择函数得到 B 与全部策略缓存 (planned_ew、恢复队列、网格队列), 再把这些缓存与 B 一起锚定到下一轮强制执行。

零绕路清除. 最有价值的特例是 K 与线段 $X \rightarrow Q$ 相交: 此时取交点 P 有 $\|X - P\| + \|P - Q\| = \|X - Q\|$, 即保证清除被“免费”嵌进原本就要走的路线。程序对每个后验顶点建立半径 $20 - \varepsilon$ 的圆盘约束, 求这些圆盘与线段的参数区间之交, 非空即得到 P ; 否则退化为在 MEC 安全内圆内搜索更优点, 并以原 MEC 中心为兜底。采纳阈值取 $L_{\text{new}} < L_{\text{old}} - 0.01 \text{ m}$, 故浮点或采样误差最多导致“不采用优化”, 绝不会导致采用更长路径。若清除已是整局最后一个动作 (无后继锚点 Q), 则只要求 $\|X - P\| < \|X - C\|$ 。

防御性回退. 理论上 SAFE 清除必成功, 但程序仍设防: 若接口意外返回失败, 立即取消锚定的下一动作、放弃影子状态, 下一轮重新进入普通调度。因此即使模拟器实现与题面严格模型存在未预期差异, 也不会沿错误的影子状态继续运行。

交付前的一致性检查. 为保证 SAFE 层确实没有改变原调度, 提交前自动比较了原程序 choose_action 与最终程序 _choose_action_v5 的抽象语法树, 除函数名外完全一致; 所有可解析的策略常数逐一比对一致, 唯一改变的原有常量是版本字符串, 新增常量只控制安全层。

6.8 算法总流程

算法 4: 第四问：定向源的搜索-定位-清除一体化在线策略 (V5 SAFE FINAL)

输入: $R_T = 1800$, $R_{\min} = 1000$, $R_{\max} = 1500$, $\delta = 1^\circ$, $v = 5 \text{ m/s}$; 奖励与门槛表 5
输出: 全部频道终态、清除数 N_{cleared} 、总虚拟时间 T_{total}

- 1 运行几何自检：校证书参数元组、22 点方向完备余量（式 (34)）与四点恢复扇枚举；不通过则拒绝运行；
- 2 建立 22 点骨架，阶段 A（中心+内环）优先；全部频道置 UNKNOWN；调用 /enter；
- 3 **当** 存在非终态频道 **执行**
- 4 **若** found = 16 **则**
- 5 全部 UNKNOWN $\rightarrow$ CERTIFIED_EMPTY，清空搜索队列； // 式 (22)
- 6 **若** 搜索队列非空且当前阶段无固定环 **则**
- 7 枚举旋转与方向，生成最短开放路线
- 8 生成候选池：EW、保证清除、几何/强/弱 Probe、网格清除、Q2-fast、refine、四点恢复；
- 9 **若** 搜索队列非空 **则**
- 10 对每个非 EW 候选按 (38) 算 $\Delta_{\text{now}}, \Delta_{\text{fut}}$ ；取满足 (39) 者中字典序最优，否则选下一个 EW；
- 11 **否则**
- 12 每频道取最佳节点，开放 2-opt 排序取首站；首步 regret > 100 m 时回退贪心；
- 13 **若** 选定动作是严格保证清除 **则**
- 14 影子执行并取得后继动作 B 与策略缓存；在 K （式 (40)）内求 P 最小化
 $\|X - P\| + \|P - Q\|$ ；若严格更短则替换并锚定 B ； // 定理 6.11
- 15 执行动作：EW 处扫描全部 UNKNOWN 频道并顺路复测；direction 按 (23) 裁剪后验；near 原地保证清除；no_signal 对 LOCALIZING 频道置 lost（不裁剪后验）；Probe 失败则记录失败点并就地补测；
- 16 **若** 搜索队列已空 **则**
- 17 对 22 点全 no_signal 的频道执行空频道认证； // 推论 6.6
- 18 调用 /exit，输出统计量与诊断；

6.9 演练标定与正式测试结果

演练基线。最终版本在 42 轮演练中共清除 535 个源、累计虚拟时间 262090.6 s，加权平均 $\bar{T} = 489.89 \text{ s/源}$ ；单局 $\bar{T}$ 的均值 506.41 s、中位数 495.67 s、 $P_{90} = 649.83 \text{ s}$ 、最大 757.51 s。首次发现分布显示前八个检测点（阶段 A）已发现 89.72% 的源，但完整 22 点仍负责方向完备与空频道证明，因此最终版不以“删点”“跳过外环”“概率判空”等方式换取速度。

正式测试。三次正式测试结果见表 6（格式与题目表 1 一致）。三次测试均在全部 20 个频道取得定论（清除或认证为空）后主动调用 /exit 退出，故

$$N_{\text{cleared}} + N_{\text{empty}} = 20, \quad \frac{N_{\text{cleared}}}{N} = 100\% \quad (\text{三次均如此}), \quad (42)$$

平均定位清除时间分别为 354.23、572.39、497.01 s，三次算术均值 474.55 s，按源数加权 459.70 s。

表 6 问题 4 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
测试1 S3JF-AZ88-CUXJ-ZJW2	16	354.23 s	2.42 s
测试2 FVZM-3VMF-3CAG-34DD	11	572.39 s	3.39 s
测试3 57J4-6J79-KFND-U4HQ	12	497.01 s	3.01 s

三次移动距离依次为 23893.75、23476.39、22435.71 m，移动时间占虚拟总时间 84.31%、74.57%、75.24%（合计 77.87%）；三次共 40 次清除尝试、39 次成功、1 次失败（仅耗 3 s），其中 27 次（67.5%）由几何 Probe 完成，与式 (24) 的判断一致；测试 1 中 found = 16 在阶段 B 早期触发，其余两次未触发。

结果分析。平均定位清除时间与源数呈负相关，恰与第三问相反：三次总虚拟时间被压在 5667 ~ 6296 s 的窄区间内，而源数为 16, 11, 12，于是 $\bar{T} \approx T_{\text{total}}/N$ 随 N 单调下降。原因是第四问

的固定搜索成本占绝对主导——22 点骨架本身就要走约 $1.4 \sim 1.5 \times 10^4$ m，而第三问的八点骨架只需约 4×10^3 m；测试 1 更进一步， $\text{found} = 16$ 提前终止把搜索期移动由约 1.5×10^4 m 压到 8114 m，这是它 $\bar{T}$ 最低的主因。由此得一条可操作的推论：**第四问的优化重心应放在“尽早把 found 推到 16”与“压缩外环访问”，而不是提高单源定位精度**。三次合计移动 77.87%、检测 17.71%、切频 3.31%、清除 1.10%；移动占比仍主导但已低于第三问的 79.1%，原因是 22 点骨架需在每点扫描更多 UNKNOWN 频道，检测次数（635 次）显著高于第三问（310 次）。

6.10 第四问小结

(1) **策略**。以定理 6.2 的凸包充要条件刻画方向完备性，据此构造 22 点骨架并以 2 m 网格、997 m 邻域、8.6016 m 最小凸包余量的离线证书严格认证（定理 6.5），恢复“全点无信号即无源”的严格判空（推论 6.6）； no_signal 只置 lost 而不裁剪后验，失联由四点恢复扇处理（命题 6.7）；后验与 MEC 清除判据继承第一、三问，新增后验网格确定性清除给出有限步终止（定理 6.8）；在线层用固定最短搜索环、插入机会成本 (39) 与搜索后开放 2-opt 调度；最后叠加只吃确定性收益的 SAFE 局部支配层（定理 6.11），在保证清除区域内挪动清除点乃至实现零绕路清除。

(2) **有效性**。三次正式测试被清除干扰源个数的比例均为 100%，平均定位清除时间 354.23、572.39、497.01 s（加权 459.70 s），42 轮演练加权 489.89 s/源、 $P_{90} = 649.83$ s。

(3) **边界的诚实划定**。定理 6.2、6.5、6.8、6.11，命题 6.4、6.7、6.10 与推论 6.3、6.6 是可证明的保证；而表 5 中的全部奖励、门槛与可靠性 κ 、100 m 首步 regret 上限均为演练标定的工程参数； E 只是误差代理而非严格上界；2-opt 与固定环的最优性只对当前任务集合成立。本版的定位不是“激进追求更低平均值”，而是保留当前最好的策略，并且只吃确定性的局部支配收益。

7 模型检验与灵敏度分析

7.1 正确性检验

第一问在四个层面交叉验证：几何层面，对随机机构形用 10^{-3} m 网格暴力扫描判定点是否落在全部角域内，所得区域与裁剪输出多边形的对称差面积小于 10^{-6} 倍区域面积；解析层面，两检测点情形用命题 3.6 的闭式与裁剪结果对照，3000 组随机机构形的相对偏差不超过 9.3×10^{-14} ；统计层面，各 2×10^4 组算例中真实源全部落入 Ω ，与定理 3.2 相符；判据层面，以 0.02° 步长扫描 γ 并让 r_2/r_1 从 1:0.001 取到 1:7，失效区间恒为 $[90.02^\circ, 91.98^\circ]$ ，与定理 3.12 的开区间在步长精度内一致。

第二问的检验为：用细长条带近似重算整条决策链，零损失最优点仍落在 (828, 589) 至 (838, 575) 的同一平坦盆地内， J_{rob} 差异小于 0.2 m，说明网格分辨率与源采样密度已足够；并验证最优点始终落在 ∂L_c 的近端弧上而非内部，与“约束由近端决定”的论断一致。

第三、四问把每条几何保证都做成运行时可观测的自检量：(i) 骨架几何——第三问每局打印覆盖半径 992.057 m，第四问打印外环内切圆 1810.000 m、阶段 A 覆盖半径 1004.513 m、证书格半径 1.414214 m、距离余量 1.585786 m、凸包余量 7.187418 m，与解析值逐位一致，确保骨架未被误改；(ii) 后验非空——每次裁剪后校验 $\mathcal{F}_{\text{code}} \neq \emptyset$ ，为空说明读数不自洽（对应定理 3.2 的数据自检准则），立即抛错而非静默继续；(iii) 终态闭合——退出前校验 $N_{\text{cleared}} + N_{\text{empty}} = 20$ ；(iv) 第四问另在启动时穷举验证四点恢复扇（命题 6.7）与 SAFE 局部支配（在构造的严格 MEC 场景中检验替换点仍保证清除且两段路径不更长）。

7.2 灵敏度分析

对测向误差上界 δ ：位置不敏感、尺度线性。 δ 主要以 $\tan \delta$ 的形式进入定位直径，故最优点位置应与 δ 无关而直径按 $\tan \delta$ 缩放。表 7 证实： δ 取 $0.5^\circ, 1^\circ, 1.5^\circ$ 时最优点始终为 (824, 594) m， J_{rob} 之比 0.499 : 1 : 1.504 与 $\tan \delta$ 的理论比 0.500 : 1 : 1.501 相差不到 0.3%。这有直接的实际价值：若实测表明误差界应放宽，选点策略无需重算，只需把精度指标按比例放大；受影响的只有覆盖失效窗口，由 $(90^\circ, 92^\circ)$ 变为 $(90^\circ, 90^\circ + 2\delta)$ 。对第三、四问而言， δ 增大会按同一比例放大后验，从而推迟保证清除的触发时刻，但不改变搜索骨架与判空逻辑——后者只依赖接收半径，与测向精度无关。

对有效接收半径下界 R_{min} ：最具支配性的参数，且各问的敏感方式不同。第二问中 R_{min} 由 1000 增至 1300 m 时透镜半高由 672.6 涨到 1068.8 m，最优稳健直径由 119.55 降到 81.58 m，改

善 31.8%。第三问中 $R_{\min}$ 由 1000 降至 900 m 时, $n = 7$ 的可行半径区间由 [997.20, 2246.29] 收缩为 [1174.47, 2069.02] m, 临界值 $R_{\min}^{\text{crit}} = R_T \sin(\pi/7) = 780.99$ m, 低于此值七点外环无论半径取何值都不可行。第四问的敏感性更硬: 证书的距离余量只有 1.5858 m, $R_{\min}$ 哪怕下降 2 m, 定理 6.5 第一步即告失效, 必须重新执行离线全网格认证(程序为此设置了参数元组校验, 拒绝沿用旧证书)。反过来, 若在进行中通过“某点收到、某点收不到”积累出该源 R 的更紧下界, 第二问应立即重算透镜并把第二检测点外推, 第三、四问则可立即减点——这是各问一致的结论。

表 7 关键参数对第二问最优第二检测点与稳健直径的影响(零可靠性损失, 2 m 网格)

扰动参数	取值	最优点 S_2^*/m	J_{rob}/m	透镜半高/m
误差上界 δ	0.5°	(824, 594)	59.67	672.6
	1.0°	(824, 594)	119.55	672.6
	1.5°	(824, 594)	179.83	672.6
接收半径下界 $R_{\min}$	1000	(824, 594)	119.55	672.6
	1100	(962, 568)	101.90	813.9
	1200	(1068, 584)	87.75	944.7
	1300	(1094, 732)	81.58	1068.8

对先验、容差与决策阈值。先验 (A6) 只出现在把“放弃近端”折算为可靠性损失百分比这一步。若改用沿距离均匀的先验, 同样放弃 $r < 151.3$ m 的近端段, 读数将由 1.0 个百分点变为 8.9 个百分点; 两种先验下被放弃的几何区域、最优点与候选区域完全相同, 改变的只是刻度。近优容差 η 由 5% 增至 10% 时, 候选区域面积约增大一倍。与平均情形准则对照: 以 (A6) 先验加权求期望直径, 最优点为 (880, 510) m、期望直径 62.66 m, 而在最坏情形最优点处期望直径为 63.16 m, 仅劣于平均最优点 0.8%——本题的最坏情形解几乎同时是平均情形解, 最坏情形准则在此几乎不付出代价就买到了确定性保证。

第三问中 `route_guard_overrides` 均为 0, 说明保护器阈值 $R_0 = 250$ m 处于低介入工作点: 贪心调度的强制任务顺序与精确开放最短路的差距始终小于 250 m。第四问中若把 SAFE 层的采纳阈值从 0.01 m 放宽, 只会让浮点误差导致的“伪收益”被误采纳, 因此保留严格的 $L_{\text{new}} < L_{\text{old}} - 0.01$ m 判据; 若收紧该阈值, 只会让 SAFE 层更保守, 不影响正确性。

两个负面测试(说明第一问结论的工程价值)。其一, 若把清除判据由 $r_{\text{MEC}} \leq 20$ 误改为 $\rho_{\text{work}} \leq 20$, 由第一问的 Jung 界 $\rho_{\text{work}} \leq r_{\text{MEC}} \leq \frac{2}{\sqrt{3}}\rho_{\text{work}}$, $\rho_{\text{work}} = 20$ m 时 r_{MEC} 最坏可达 23.1 m, 即不存在任何半径 20 m 的圆能覆盖后验; 若进一步就在直径中点处清除, 由三点 120° 等边构形, 最远顶点到该点的距离可达 $\frac{\sqrt{3}}{2}D = 34.6$ m, 而程序仍认为“保证成功”。这类误判不会被清除失败的 3 s 惩罚及时暴露, 却会破坏完备性论证。其二, 若在第四问沿用第三问的八点骨架, 则 $G = (1800, 0)$ 、 $u = (1, 0)$ 的定向源在八点全部无信号, 程序会把该频道认证为空并提前退出, 造成漏源且无任何运行时报警——这正是第四问必须重做搜索层的根本原因。

8 模型的优点、不足与推广

优点。(i) 范式有据: 由题设“重复检测读数不变”直接推出有界确定性误差, 从而选定集员估计, 避免了对误差分布的无据假设, 全部结论可给严格证明。(ii) 刻画完全: 覆盖性给出充要条件(定理 3.9)而非单一结论, 并附有 $O(m)$ 的锐角计数快速否决(推论 3.10); 有界性同样是充要的(定理 3.3), 堵住了外框截断造成的虚假直径。(iii) 数值稳健: 叉积刻画绕开斜率发散与象限分段, 闭式解按 $(t - t')^2 + 4tt' \sin^2 \delta$ 组织以避免灾难性抵消, 实测与裁剪法在双精度下完全一致。(iv) 结构清晰: 第二问的“近端定约束、远端定精度”分离结构把二维极小极大问题的解压到一条边界弧上, 使候选区域有解析描述而非仅有数值点云。(v) 第四问的核心判据是充要的: 定理 6.2 把“对连续指向取全称量词”的方向完备性化为可有限检验的凸包包含关系, 由此既得到硬性下界(推论 6.3、命题 6.4), 又得到可复算的网格证书(定理 6.5), 而不是用“多布点应该就够了”的直觉替代证明。(vi) 保证与启发式的边界清晰: 四个问题都把可证明的命题与演练标定的参数分列, 不把工程取值包装为理论最优。(vii) 只吃确定性收益: 第四问最终放弃了所有能提高平均值却无法逐实例证明不退化的调度改动, 只保留严格局部支配(定理 6.11), 宁可提升小, 也不以长尾退化换平均收益。

不足。(i) 第二问用中轴线段代替极细扇形 u , 引入不超过 26 m 的几何误差, 虽远小于 $J_{\text{rob}} \approx$

119 m 的量级, 但在 $R_{\min}$ 显著增大、候选区域外推很远时应改用完整扇形复核。(ii) 命题 4.1 给的是理由与全网格验证而非完整证明, 贴近禁区边缘 ($|h| \leq 40$ m) 的缺点是例外。(iii) 模型未考虑地形遮挡、多径与非视距传播, 实际有效接收半径可能各向异性。(iv) 假定检测点坐标精确、机器狗按指令精确到位。(v) 第三问的八点骨架并非该族最小 (1 + 6 点即可严格覆盖), 第四问的 22 点也只验证了对 21 点子集的局部不可约性, 未给出全局最小方向完备点数; 命题 6.4 的下界 7 与实际使用的 14 个圆外点之间仍有可观差距。(vi) no_signal 给出的排除约束非凸, 本文出于保守性完全弃用 (第四问更是退化为析取), 损失了可观的负信息。(vii) 试探清除的成功概率 p 未被在线估计, 式 (24) 只用于定性论证; 调度可靠性 κ 是离线常数而非自适应量。(viii) 强制任务最短路与搜索后 2-opt 都只对“当前时刻”的任务集合精确, 不构成全局最优; SAFE 层只优化保证清除点, 不覆盖试探性动作的往返冗余。

推广。(i) 检测点定位误差: 把 S_i 的不确定集与角域做 Minkowski 和, Ω 仍为凸多边形, 算法框架完全不变。(ii) 逐点误差界: 若误差上界随距离或信噪比变化, 令 δ_i 逐点取值, 全部结论按 δ_i 重述, 定理 3.12 的失效窗口相应变为 $(90^\circ, 90^\circ + \delta_1 + \delta_2)$ 。(iii) 一般扇形辐射角: 若定向源的辐射半角由 90° 变为 ω , 定理 6.2 的凸包条件推广为“ G 到 $N_R(G)$ 中各点的方向集合的最大角间隙不超过 2ω ”, 只需把判据由 180° 换成 2ω 重新跑一遍网格认证, 骨架自动加密, 全部证明结构不变。(iv) 负信息的凸松弛: 把 $\|G - p\| > R_{\min}$ 的补集用若干半平面内逼近后并入裁剪序列, 可在保持保守性的前提下部分利用 no_signal; 第四问中还需与“背面”分支联合维护。(v) 沿示向线抵近: 沿示向方向前进不改变交会几何却直接缩短 r_1 , 且越过源时示向度读数会跳变约 180° , 这是一个几乎免费的强信号, 可用于把 r_1 迅速二分到式 (17) 的阈值以内。(vi) 自适应 p 与 κ : 在线统计各类试探清除的成功率, 把式 (24) 与 (39) 由固定常数升级为阈值自适应。(vii) 移动源: 若源可移动, 则 (A4) 失效, SAFE 层的“同一后状态”论证 (定理 6.11) 与后验的单调收缩都不再成立, 需改为带遗忘因子的集员滤波。

9 结论

第一问把测向误差按题设处理为有界确定性量, 用角域交集刻画干扰源的保证集: 角域经二维叉积化为 $2n$ 个线性半平面, 先由回收锥判定有界性 (定理 3.3), 再以 Sutherland-Hodgman 裁剪求有序顶点, 最后用旋转卡壳在 $O(m)$ 时间内扫描对踵点得直径, 总复杂度 $O(n^2)$; 两检测点另给出与裁剪结果在双精度下完全一致的闭式解 (命题 3.6)。对“以直径为直径的圆能否覆盖定位区域”, 答案是不一定: 覆盖圆存在当且仅当存在顶点对使其余顶点对之张角均不小于 90° (定理 3.9), 快速否决条件是内角小于 90° 的顶点不超过两个; 两检测点时充要条件为 $\gamma \notin (90^\circ, 92^\circ)$, $\gamma = 91^\circ$ 的算例中有顶点超出直径圆 0.307 m。三点 120° 布设时 $R^*/(D/2) = 1.133$ 几乎取到 Jung 界 1.1547, 随机算例中 $n \geq 3$ 的失效率高达 $14\% \sim 29\%$, 故工程上必须以最小覆盖圆作为不确定性包络。

第二问把源位置不确定性建模为先验可行集, 以“每一可能位置都被保证接收”与“避开交会禁区”为硬约束、以最坏源位置下的定位直径为目标建立稳健选址模型。求解揭示出近端源定约束、远端源定精度的分离结构, 最优点必落在近端接收圆边界弧上: $S_2^* \approx (836, 578)$ m (距 S_1 1016 m、与示向方向成 34.7°), $J_{\text{rob}} = 119.4$ m; 候选区域为关于示向线对称的两支窄带 (距 S_1 985 ~ 1018 m、成 $24.5^\circ \sim 41.4^\circ$)。若允许 1 个百分点的保证接收比例损失, 稳健直径可降至 99.4 m (改善 16.8%), 且该让步带确定性回退分支。

第三问把前两问的几何结论装配为搜索-定位-清除一体化在线系统: 八点骨架的最坏覆盖距离 992.057 m 严格小于最小有效接收半径, 故“某频道在八点全部无信号”即为该频道无源的严格证明; 后验取外接多边形保证 $\mathcal{F}_{\text{true}} \subseteq \mathcal{F}_{\text{code}}$, 于是“最小包围圆半径不超过 20 m 则在圆心清除必然成功”是严格结论; 由清除成本的不对称性导出试探清除的有利条件 $p > 3/(\Delta T + 3)$, 阈值仅约 6%。三次正式测试清除比例均为 100%, 平均定位清除时间 228.15、236.49、238.82 s (均值 234.49 s)。

第四问的关键是认识到定向源破坏的是搜索完备性而非定位与清除: 接收条件成为距离与半平面的合取, 距离覆盖不再蕴含可接收, no_signal 退化为析取。本文给出方向完备性的充要条件 $G \in \text{conv } N_{R_{\min}}(G)$ (定理 6.2), 由它导出“必须在目标圆外布点”与“圆外点不少于 7 个”, 并构造中心 + 990 m 内环七点 + 内切圆 1810 m 正十四边形外环的 22 点骨架, 用 2 m 网格、997 m 邻域、8.6016 m 最小凸包余量的离线证书严格认证 (距离余量 1.586 m、凸包余量 7.187 m),

从而恢复严格判空；对失联给出四点恢复扇并证明其必然重新接收；后验与 MEC 清除判据整体继承，并以后验网格确定性清除给出定位阶段的有限步终止；调度层用固定最短搜索环、插入机会成本与搜索后开放 2-opt，最后叠加严格局部支配的 SAFE 层实现零绕路清除。三次正式测试在全部 20 个频道取得定论后退出，被清除干扰源个数的比例均为 100%，平均定位清除时间 354.23、572.39、497.01 s（加权 459.70 s）；42 轮演练加权 489.89 s/源。结果分析表明第四问的成本结构由 22 点骨架的固定搜索开销主导，因此后续改进的方向是压缩外环规模与尽早触发 found = 16，而非继续提高单源定位精度。

四问贯通后可得一条可直接执行的作业准则：先用严格覆盖（或方向完备）的骨架把“哪些频道有源”变成可证明的事实；对每个有源频道，在保证接收的前提下往示向线侧前方走远使交会角接近 90°；交会后以最小覆盖圆半径而非直径的一半作为终止判据， $R^* \leq 20$ m 时停止测向、直趋圆心清除；面对可能存在的定向源，搜索点的布置标准不是“离得近”，而是“把每一个可能位置包住”。由式 (17) 的硬下界可知，当距离估计超过约 810 m 时继续增加测向次数收益急剧递减，应先沿示向方向前进以缩短 r 。

参考文献

[1] 2026 年高教社杯全国大学生数学建模竞赛 B 题《无线电干扰源的快速自动定位与清除》及其附录 1-4，见题目附件。

[2] Jung 定理：平面上直径为 D 的有界集必可被半径 $D/\sqrt{3}$ 的圆覆盖，界在等边三角形处取到。经典凸几何结果。

[3] Welzl E. Smallest enclosing disks (balls and ellipsoids)[M]. New Results and New Trends in Computer Science, LNCS 555, Springer, 1991: 359-370.

[4] Rockafellar R T. Convex Analysis[M]. Princeton University Press, 1970. (回收锥与严格分离定理)

[5] Toussaint G T. Solving geometric problems with the rotating calipers[C]. Proceedings of IEEE MELECON, 1983.

[6] Held M, Karp R M. A dynamic programming approach to sequencing problems[J]. Journal of the Society for Industrial and Applied Mathematics, 1962, 10(1): 196-210.

[7] Sutherland I E, Hodgman G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32-42.

A 支撑材料清单

表 8 支撑材料文件清单

文件名	内容说明
q1_locate.py	角域到半平面、Sutherland-Hodgman 裁剪、旋转卡壳求直径、内角与张角判据、Welzl 最小覆盖圆
q1_closedform.py	命题 3.6 的闭式顶点与对角线公式，及其与裁剪法的一致性对照
q1_experiments.py	基准算例、覆盖性对照表、失效率统计、Jung 界构形与 $D-\gamma$ 曲线
q2_model.py	先验可行集、接收透镜与交会禁区的判别、稳健直径 J_{rob} 的求值
q2_optimise.py	网格搜索最优点、候选区域生成、权衡下包络与边际率阈值
q2_sensitivity.py	对 δ 、 R_{min} 、先验与容差的灵敏度分析，平均情形对照
candidates.csv	全部可行网格点的 J_{rob} 、 ℓ 、 ψ 与候选区域标记
q3_v3_2_final.py	第三问完整机器狗程序：八点骨架、频道状态机、保守后验裁剪、最小包围圆清除判据、三类试探清除、统一在线评分、强制任务开放 TSP 保护器与官方 HTTP 接口调用
q4_v5_safe_final.py	第四问完整程序：22 点方向完备骨架与运行前证书自检、四点恢复扇、析取语义下的后验维护、后验网格确定性清除、固定最短搜索环与插入机会成本、搜索后开放 2-opt、SAFE 局部支配层
q4_certificate.py	离线方向完备认证：2 m 网格、997 m 邻域、局部凸包余量统计，以及 22 个 21 点子集的不可约性检验

(表 8 续上页)

文件名	内容说明
q3/q4_summary.json	单局统计汇总：清除数、认证空频道数、时间构成、骨架诊断量与调度诊断量
q3/q4_trace.jsonl	逐动作明文轨迹：动作类型、起讫坐标、移动距离、频道、检测结果、示向度、后验工作半径与最小包围圆半径
正式测试日志 1-6	由模拟器导出的第三、四问各三次正式测试加密行为日志（文件名未作任何修改）

全部数值结果均可由上述脚本一键复现。第四问的两条运行前自检（22 点方向完备证书余量、四点恢复扇枚举）在程序启动时自动执行，未通过则拒绝连接模拟器，因此表 6 中的每一局都可确认为是在证书有效的骨架下完成的。

附录 A 问题一程序: q1.py

```

1 """B 题问题 1: 测向交会定位."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from math import cos, hypot, isfinite, radians, sin
7 from typing import Literal, Sequence
8
9 Point = tuple[float, float]
10 Status = Literal["bounded", "empty", "unbounded_or_too_large"]
11
12
13 @dataclass(frozen=True)
14 class LocalizationResult:
15     status: Status
16     polygon: tuple[Point, ...]
17     diameter: float | None
18     diameter_pair: tuple[Point, Point] | None
19     diameter_circle_center: Point | None
20     diameter_circle_radius: float | None
21     covered_by_diameter_circle: bool | None
22     search_half_size: float
23
24
25 def _sub(a: Point, b: Point) -> Point:
26     return a[0] - b[0], a[1] - b[1]
27
28
29 def _cross(a: Point, b: Point) -> float:
30     return a[0] * b[1] - a[1] * b[0]
31
32
33 def _distance(a: Point, b: Point) -> float:
34     return hypot(a[0] - b[0], a[1] - b[1])
35
36
37 def _deduplicate_polygon(polygon: list[Point], eps: float) -> list[Point]:
38     if not polygon:
39         return []
40
41     points = [polygon[0]]
42     for p in polygon[1:]:
43         if _distance(p, points[-1]) > eps:
44             points.append(p)
45
46     if len(points) > 1 and _distance(points[0], points[-1]) <= eps:
47         points.pop()
48     return points
49
50
51 def clip_polygon_by_left_halfplane(
52     polygon: Sequence[Point],
53     line_point: Point,
54     line_direction: Point,
55     eps: float,
56 ) -> list[Point]:
57     if not polygon:
58         return []
59
60     def side(p: Point) -> float:
61         return _cross(line_direction, _sub(p, line_point))
62
63     result: list[Point] = []
64     prev = polygon[-1]
65     prev_value = side(prev)
66     prev_inside = prev_value >= -eps
67
68     for cur in polygon:
69         cur_value = side(cur)
70         cur_inside = cur_value >= -eps
71
72         if prev_inside != cur_inside:
73             denom = prev_value - cur_value
74             if abs(denom) > eps:
75                 t = prev_value / denom
76                 t = max(0.0, min(1.0, t))
77                 result.append(
78                     (
79                         prev[0] + t * (cur[0] - prev[0]),
80                         prev[1] + t * (cur[1] - prev[1]),
81                     )
82                 )
83
84         if cur_inside:
85             result.append(cur)
86
87     prev = cur
88     prev_value = cur_value
89     prev_inside = cur_inside
90
91     return _deduplicate_polygon(result, eps)
92
93
94 def _bearing_halfplanes(
95     station: Point, bearing_deg: float, error_deg: float
96 ) -> tuple[tuple[Point, Point], tuple[Point, Point]]:
97     low = radians(bearing_deg - error_deg)

```

```

98     high = radians(bearing_deg + error_deg)
99
100    d1 = cos(low), sin(low)
101    d2 = cos(high), sin(high)
102    return (station, d1), (station, (-d2[0], -d2[1]))
103
104
105    def intersect_bearing_sectors(
106        stations: Sequence[Point],
107        bearings_deg: Sequence[float],
108        error_deg: float = 1.0,
109        initial_half_size: float = 1800.0,
110        max_expansions: int = 24,
111        relative_eps: float = 1e-10,
112    ) -> tuple[Status, list[Point], float]:
113        if len(stations) != len(bearings_deg):
114            raise ValueError(" 检测点数量与示向度数量必须相同")
115        if not stations:
116            raise ValueError(" 至少需要一个检测点")
117        if not isfinite(error_deg) or not 0.0 < error_deg < 90.0:
118            raise ValueError(" 该凸角域算法要求 0 < error_deg < 90")
119        if not isfinite(initial_half_size) or initial_half_size <= 0.0:
120            raise ValueError("initial_half_size 必须为有限正数")
121        if not isinstance(max_expansions, int) or max_expansions < 0:
122            raise ValueError("max_expansions 必须为非负整数")
123        if not isfinite(relative_eps) or relative_eps <= 0.0:
124            raise ValueError("relative_eps 必须为正")
125
126        stations = [(float(x), float(y)) for x, y in stations]
127        bearings = [float(x) for x in bearings_deg]
128
129        if any(not isfinite(v) for p in stations for v in p):
130            raise ValueError(" 检测点坐标必须为有限数")
131        if any(not isfinite(v) for v in bearings):
132            raise ValueError(" 示向度必须为有限数")
133
134        bearings = [x % 360.0 for x in bearings]
135        limits = [
136            hp
137            for station, bearing in zip(stations, bearings)
138            for hp in _bearing_halfplanes(station, bearing, error_deg)
139        ]
140
141        half_size = float(initial_half_size)
142        last_polygon: list[Point] = []
143
144        for k in range(max_expansions + 1):
145            eps = relative_eps * max(1.0, half_size)
146            polygon: list[Point] = [
147                (-half_size, -half_size),
148                (half_size, -half_size),
149                (half_size, half_size),
150                (-half_size, half_size),
151            ]
152
153            for point, direction in limits:
154                polygon = clip_polygon_by_left_halfplane(
155                    polygon, point, direction, eps
156                )
157                if not polygon:
158                    break
159
160            last_polygon = polygon
161            if not polygon:
162                return "empty", [], half_size
163
164            tol = 20.0 * eps
165            on_box = any(
166                abs(abs(x) - half_size) <= tol
167                or abs(abs(y) - half_size) <= tol
168                for x, y in polygon
169            )
170            if not on_box:
171                return "bounded", polygon, half_size
172
173            if k < max_expansions:
174                half_size *= 2.0
175
176        return "unbounded_or_too_large", last_polygon, half_size
177
178
179    def polygon_diameter(polygon: Sequence[Point]) -> tuple[float, tuple[Point, Point]]:
180        if not polygon:
181            raise ValueError(" 空区域没有直径")
182        if len(polygon) == 1:
183            return 0.0, (polygon[0], polygon[0])
184
185        max_d2 = -1.0
186        pair = polygon[0], polygon[1]
187
188        for i, p in enumerate(polygon):
189            for q in polygon[i + 1 :]:
190                d2 = (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2
191                if d2 > max_d2:
192                    max_d2 = d2
193                    pair = p, q
194
195        return max_d2**0.5, pair
196

```

```

197
198 def diameter_circle_coverage(
199     polygon: Sequence[Point],
200     diameter: float,
201     diameter_pair: tuple[Point, Point],
202     relative_eps: float = 1e-10,
203 ) -> tuple[Point, float, bool]:
204     if not polygon:
205         raise ValueError(" 空区域无法判断直径圆覆盖")
206     if not isfinite(diameter) or diameter < 0.0:
207         raise ValueError("diameter 必须为有限非负数")
208     if not isfinite(relative_eps) or relative_eps <= 0.0:
209         raise ValueError("relative_eps 必须为正")
210
211     a, b = diameter_pair
212     pair_distance = _distance(a, b)
213     pair_tol = relative_eps * max(1.0, diameter, pair_distance)
214     if abs(pair_distance - diameter) > pair_tol:
215         raise ValueError("diameter_pair 两点距离必须等于 diameter")
216
217     center = (a[0] + b[0]) / 2.0, (a[1] + b[1]) / 2.0
218     radius = diameter / 2.0
219     tol = relative_eps * max(1.0, diameter, radius)
220     covered = all(_distance(center, p) <= radius + tol for p in polygon)
221     return center, radius, covered
222
223
224 def solve_localization(
225     stations: Sequence[Point],
226     bearings_deg: Sequence[float],
227     error_deg: float = 1.0,
228     initial_half_size: float = 1800.0,
229     max_expansions: int = 24,
230     relative_eps: float = 1e-10,
231 ) -> LocalizationResult:
232     status, polygon, half_size = intersect_bearing_sectors(
233         stations=stations,
234         bearings_deg=bearings_deg,
235         error_deg=error_deg,
236         initial_half_size=initial_half_size,
237         max_expansions=max_expansions,
238         relative_eps=relative_eps,
239     )
240
241     if status != "bounded" or not polygon:
242         return LocalizationResult(
243             status=status,
244             polygon=tuple(polygon),
245             diameter=None,
246             diameter_pair=None,
247             diameter_circle_center=None,
248             diameter_circle_radius=None,
249             covered_by_diameter_circle=None,
250             search_half_size=half_size,
251         )
252
253     diameter, pair = polygon_diameter(polygon)
254     center, radius, covered = diameter_circle_coverage(
255         polygon=polygon,
256         diameter=diameter,
257         diameter_pair=pair,
258         relative_eps=relative_eps,
259     )
260
261     return LocalizationResult(
262         status=status,
263         polygon=tuple(polygon),
264         diameter=diameter,
265         diameter_pair=pair,
266         diameter_circle_center=center,
267         diameter_circle_radius=radius,
268         covered_by_diameter_circle=covered,
269         search_half_size=half_size,
270     )
271
272
273 def plot_localization(
274     result: LocalizationResult,
275     stations: Sequence[Point],
276     bearings_deg: Sequence[float],
277     error_deg: float = 1.0,
278     save_path: str | None = None,
279 ) -> None:
280     if result.status != "bounded" or not result.polygon:
281         raise ValueError(" 只有非空有限区域可以绘图")
282
283     import matplotlib.pyplot as plt
284     from matplotlib.patches import Circle as CirclePatch
285
286     plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
287     plt.rcParams["axes.unicode_minus"] = False
288
289     fig, ax = plt.subplots(figsize=(8, 8))
290     closed = [*result.polygon, result.polygon[0]]
291     xs = [p[0] for p in closed]
292     ys = [p[1] for p in closed]
293
294     ax.fill(xs, ys, color="tab:blue", alpha=0.25, label=" 定位区域")
295     ax.plot(xs, ys, color="tab:blue")

```

```

296
297 ray_length = max(100.0, 1.2 * result.search_half_size)
298 for i, (station, bearing) in enumerate(zip(stations, bearings_deg), 1):
299     ax.scatter("station", marker="^", color="black", zorder=5)
300     ax.annotate(f"S{i}", station, xytext=(5, 5), textcoords="offset points")
301
302     for angle in (bearing - error_deg, bearing + error_deg):
303         rad = radians(angle)
304         end = (
305             station[0] + ray_length * cos(rad),
306             station[1] + ray_length * sin(rad),
307         )
308         ax.plot(
309             [station[0], end[0]],
310             [station[1], end[1]],
311             color="gray",
312             linestyle="--",
313             linewidth=0.8,
314         )
315
316 assert result.diameter_pair is not None
317 p1, p2 = result.diameter_pair
318 ax.plot(
319     [p1[0], p2[0]],
320     [p1[1], p2[1]],
321     color="tab:orange",
322     linewidth=2.0,
323     label=f"直径 D={result.diameter:.3f} m",
324 )
325
326 assert result.diameter_circle_center is not None
327 assert result.diameter_circle_radius is not None
328 circle_color = "tab:green" if result.covered_by_diameter_circle else "tab:red"
329 ax.add_patch(
330     CirclePatch(
331         result.diameter_circle_center,
332         result.diameter_circle_radius,
333         fill=False,
334         color=circle_color,
335         linewidth=2.0,
336         label=f"直径圆 R=D/2={result.diameter_circle_radius:.3f} m",
337     )
338 )
339 ax.scatter(*result.diameter_circle_center, marker="x", color=circle_color, zorder=5)
340
341 ax.set_aspect("equal", adjustable="box")
342 ax.set_xlabel("x / m")
343 ax.set_ylabel("y / m")
344 ax.grid(alpha=0.25)
345 ax.legend()
346 fig.tight_layout()
347
348 if save_path is None:
349     plt.show()
350 else:
351     fig.savefig(save_path, dpi=180, bbox_inches="tight")
352     plt.close(fig)
353
354
355 if __name__ == "__main__":
356     demo_stations = [
357         (-400.0, 0.0),
358         (600.0, 0.0),
359         (100.0, 500.0),
360     ]
361     demo_bearings = [0.0, 180.0, 270.0]
362
363     demo_result = solve_localization(demo_stations, demo_bearings)
364     print(f"状态: {demo_result.status}")
365     print(f"顶点数: {len(demo_result.polygon)}")
366     if demo_result.diameter is not None:
367         print(f"定位区域直径: {demo_result.diameter:.6f} m")
368         print(f"直径圆半径: {demo_result.diameter_circle_radius:.6f} m")
369         print(f"直径为 D 的圆能否覆盖: {demo_result.covered_by_diameter_circle}")

```

附录 B 问题二程序

B.1 主程序：q2.py

```

1  """B 题问题 2: 第二检测点选择与候选区域."""
2
3  from __future__ import annotations
4
5  import argparse
6  from dataclasses import dataclass, replace
7  from math import atan2, cos, degrees, isfinite, pi, radians, sin
8  from pathlib import Path
9  from typing import Sequence
10
11 import numpy as np
12 from scipy.spatial import ConvexHull, distance
13
14 from q1 import clip_polygon_by_left_halfplane, polygon_diameter
15 from q2_decision import (
16     TradeoffEnvelopePoint,
17     make_final_decision,
18     pareto_mask as _pareto_mask,
19     select_validation_indices,
20 )
21 from q2_evaluation import calculate_weighted_scores, evaluate_candidate_score
22 from q2_evidence import (
23     plot_reliability_tradeoff,
24     plot_result,
25     write_candidate_csv,
26     write_reliability_tradeoff_csv,
27 )
28
29 Point = tuple[float, float]
30
31
32 @dataclass(frozen=True)
33 class Q2Config:
34     target_radius: float = 1800.0
35     min_receive_radius: float = 1000.0
36     max_receive_radius: float = 1500.0
37     bearing_error_deg: float = 1.0
38     too_strong_radius: float = 5.0
39     source_radial_samples: int = 180
40     source_angular_samples: int = 41
41     candidate_axis_samples: int = 55
42     disk_sides: int = 144
43     validated_candidate_count: int = 24
44     bearing_error_samples: int = 5
45     reliability_loss_levels: tuple[float, ...] = (
46         0.0,
47         0.005,
48         0.01,
49         0.02,
50         0.03,
51         0.05,
52     )
53     perform_numerical_calibration: bool = True
54     numerical_calibration_factor: int = 2
55     tradeoff_improvement_tolerance_ratio: float = 0.01
56     local_refinement_axis_samples: int = 3
57     local_refinement_passes: int = 4
58     max_refinement_centers: int = 4
59     position_convergence_tolerance: float = 10.0
60     diameter_convergence_tolerance_ratio: float = 0.01
61     score_tolerance: float = 0.03
62     diameter_tolerance_ratio: float = 0.03
63     weight_not_safe: float = 0.24
64     weight_certain_failure: float = 0.31
65     weight_diameter: float = 0.35
66     weight_angle: float = 0.10
67
68     def validate(self) -> None:
69         positive_values = {
70             "target_radius": self.target_radius,
71             "min_receive_radius": self.min_receive_radius,
72             "max_receive_radius": self.max_receive_radius,
73             "bearing_error_deg": self.bearing_error_deg,
74             "too_strong_radius": self.too_strong_radius,
75             "score_tolerance": self.score_tolerance,
76             "diameter_tolerance_ratio": self.diameter_tolerance_ratio,
77             "position_convergence_tolerance": self.position_convergence_tolerance,
78         }
79         for name, value in positive_values.items():
80             if not isfinite(value) or value <= 0.0:
81                 raise ValueError(f"{name} 必须为有限正数")
82         if self.min_receive_radius > self.max_receive_radius:
83             raise ValueError("min_receive_radius 不能大于 max_receive_radius")
84         if not 0.0 < self.bearing_error_deg < 90.0:
85             raise ValueError("bearing_error_deg 必须位于 (0, 90) 度")
86         if (
87             not isfinite(self.tradeoff_improvement_tolerance_ratio)
88             or self.tradeoff_improvement_tolerance_ratio < 0.0
89         ):
90             raise ValueError("tradeoff_improvement_tolerance_ratio 必须为有限非负数")
91         if not isinstance(self.perform_numerical_calibration, bool):
92             raise ValueError("perform_numerical_calibration 必须为布尔值")
93         if (
94             not isinstance(self.numerical_calibration_factor, int)

```

```

95         or self.numerical_calibration_factor < 2
96     ):
97         raise ValueError("numerical_calibration_factor 必须为不小于 2 的整数")
98     if (
99         not isfinite(self.diameter_convergence_tolerance_ratio)
100         or self.diameter_convergence_tolerance_ratio < 0.0
101     ):
102         raise ValueError("diameter_convergence_tolerance_ratio 必须为有限非负数")
103     loss_levels = tuple(sorted({float(x) for x in self.reliability_loss_levels}))
104     if not loss_levels or abs(loss_levels[0]) > 1e-12:
105         raise ValueError("reliability_loss_levels 必须从 0 开始")
106     if any(not isfinite(x) or not 0.0 <= x < 1.0 for x in loss_levels):
107         raise ValueError("reliability_loss_levels 必须位于 [0, 1)")
108     integer_values = {
109         "source_radial_samples": self.source_radial_samples,
110         "source_angular_samples": self.source_angular_samples,
111         "candidate_axis_samples": self.candidate_axis_samples,
112         "disk_sides": self.disk_sides,
113         "validated_candidate_count": self.validated_candidate_count,
114         "bearing_error_samples": self.bearing_error_samples,
115         "local_refinement_axis_samples": self.local_refinement_axis_samples,
116     }
117     for name, value in integer_values.items():
118         if not isinstance(value, int) or value < 3:
119             raise ValueError(f"{name} 必须为不小于 3 的整数")
120     if (
121         not isinstance(self.local_refinement_passes, int)
122         or self.local_refinement_passes < 0
123     ):
124         raise ValueError("local_refinement_passes 必须为非负整数")
125     if (
126         not isinstance(self.max_refinement_centers, int)
127         or self.max_refinement_centers <= 0
128     ):
129         raise ValueError("max_refinement_centers 必须为正整数")
130     weights = np.array(
131         [
132             self.weight_not_safe,
133             self.weight_certain_failure,
134             self.weight_diameter,
135             self.weight_angle,
136         ],
137         dtype=float,
138     )
139     if np.any(~np.isfinite(weights)) or np.any(weights < 0.0):
140         raise ValueError(" 评分权重必须为有限非负数")
141     if not np.isclose(float(weights.sum()), 1.0, atol=1e-12):
142         raise ValueError(" 四项评分权重之和必须等于 1")
143
144 @dataclass(frozen=True)
145 class CandidateMetrics:
146     point: Point
147     local_u: float
148     local_v: float
149     safe_coverage: float
150     uncertain_coverage: float
151     certain_failure: float
152     angle_penalty: float
153     proxy_diameter: float
154     proxy_score: float
155     pareto: bool
156     refinement_level: int = 0
157     validated_bearing_worst: float | None = None
158     strong_signal_diameter: float | None = None
159     no_signal_diameter: float | None = None
160     validated_robust_diameter: float | None = None
161     validated_score: float | None = None
162     validated_pareto: bool = False
163
164
165 @dataclass(frozen=True)
166 class NumericalStabilityRecord:
167     role: str
168     point: Point
169     coarse_robust_diameter: float
170     default_robust_diameter: float
171     fine_robust_diameter: float
172     default_fine_relative_difference: float
173
174
175 @dataclass(frozen=True)
176 class Q2Result:
177     first_station: Point
178     first_bearing_deg: float
179     first_region_polygon: tuple[Point, ...]
180     first_region_diameter: float
181     source_samples: np.ndarray
182     candidates: tuple[CandidateMetrics, ...]
183     validated_candidates: tuple[CandidateMetrics, ...]
184     primary_safe_coverage: float
185     selected_reliability_loss: float
186     accepted_safe_coverage_floor: float
187     tradeoff_improvement_tolerance: float
188     reliability_tradeoff: tuple[TradeoffEnvelopePoint, ...]
189     reliability_feasible_candidates: tuple[CandidateMetrics, ...]
190     refinement_converged: bool
191     refinement_resolution: float
192     refinement_position_change: float

```

```

194     refinement_diameter_relative_change: float
195     numerical_calibration_applied: bool
196     numerical_error_ratio: float
197     numerical_stability_records: tuple[NumericalStabilityRecord, ...]
198     best_candidate: CandidateMetrics
199     near_optimal_candidates: tuple[CandidateMetrics, ...]
200     safe_near_optimal_candidates: tuple[CandidateMetrics, ...]
201     pareto_best_candidate: CandidateMetrics
202     pareto_near_optimal_candidates: tuple[CandidateMetrics, ...]
203     config: Q2Config
204
205
206 def _validate_station_and_bearing(station: Point, bearing_deg: float) -> Point:
207     clean_station = float(station[0]), float(station[1])
208     values = (*clean_station, float(bearing_deg))
209     if any(not isfinite(value) for value in values):
210         raise ValueError(" 检测点坐标和示向度必须为有限数")
211     return clean_station
212
213
214 def _directions(bearing_deg: float) -> tuple[np.ndarray, np.ndarray]:
215     angle = radians(bearing_deg)
216     forward = np.array([cos(angle), sin(angle)], dtype=float)
217     normal = np.array([-sin(angle), cos(angle)], dtype=float)
218     return forward, normal
219
220
221 def _regular_circle_polygon(
222     center: Point, radius: float, sides: int, circumscribed: bool = True
223 ) -> list[Point]:
224     r_poly = radius / cos(pi / sides) if circumscribed else radius
225     return [
226         (
227             center[0] + r_poly * cos(2.0 * pi * i / sides),
228             center[1] + r_poly * sin(2.0 * pi * i / sides),
229         )
230         for i in range(sides)
231     ]
232
233
234 def _bearing_halfplanes(
235     station: Point, bearing_deg: float, error_deg: float
236 ) -> tuple[tuple[Point, Point], tuple[Point, Point]]:
237     low = radians(bearing_deg - error_deg)
238     high = radians(bearing_deg + error_deg)
239     d_low = cos(low), sin(low)
240     d_high = cos(high), sin(high)
241     return (
242         (station, d_low),
243         (station, (-d_high[0], -d_high[1])),
244     )
245
246
247 def _clip_by_bearing(
248     polygon: Sequence[Point],
249     station: Point,
250     bearing_deg: float,
251     error_deg: float,
252     eps: float = 1e-8,
253 ) -> list[Point]:
254     out = list(polygon)
255     for p0, direction in _bearing_halfplanes(
256         station, bearing_deg, error_deg
257     ):
258         out = clip_polygon_by_left_halfplane(
259             out, p0, direction, eps
260         )
261         if not out:
262             break
263     return out
264
265
266 def _clip_by_disk(
267     polygon: Sequence[Point],
268     center: Point,
269     radius: float,
270     sides: int,
271     eps: float = 1e-8,
272 ) -> list[Point]:
273     out = list(polygon)
274     disk = _regular_circle_polygon(center, radius, sides, True)
275     for i, a in enumerate(disk):
276         b = disk[(i + 1) % len(disk)]
277         edge = b[0] - a[0], b[1] - a[1]
278         edge_len = float(np.hypot(*edge))
279         unit = edge[0] / edge_len, edge[1] / edge_len
280         out = clip_polygon_by_left_halfplane(
281             out, a, unit, eps
282         )
283         if not out:
284             break
285     return out
286
287
288 def build_first_feasible_polygon(
289     first_station: Point,
290     first_bearing_deg: float,
291     config: Q2Config,
292 ) -> list[Point]:

```

```

293 polygon = _regular_circle_polygon(
294     (0.0, 0.0), config.target_radius, config.disk_sides, True
295 )
296 polygon = _clip_by_bearing(
297     polygon,
298     first_station,
299     first_bearing_deg,
300     config.bearing_error_deg,
301 )
302 polygon = _clip_by_disk(
303     polygon,
304     first_station,
305     config.max_receive_radius,
306     config.disk_sides,
307 )
308 if not polygon:
309     raise ValueError(" 给定 S1 和 theta1 在目标圆域内没有可行干扰源位置")
310 return polygon
311
312 def sample_first_feasible_region(
313     first_station: Point,
314     first_bearing_deg: float,
315     config: Q2Config,
316 ) -> np.ndarray:
317     radial_t = (
318         np.arange(config.source_radial_samples, dtype=float) + 0.5
319     ) / config.source_radial_samples
320     r_squared = config.too_strong_radius**2 + radial_t * (
321         config.max_receive_radius**2 - config.too_strong_radius**2
322     )
323     radii = np.sqrt(r_squared)
324     angle_delta = np.linspace(
325         -config.bearing_error_deg,
326         config.bearing_error_deg,
327         config.source_angular_samples,
328     )
329     angles = np.deg2rad(first_bearing_deg + angle_delta)
330     r_grid, a_grid = np.meshgrid(radii, angles, indexing="ij")
331     sample_grid = np.column_stack(
332         [
333             first_station[0] + (r_grid * np.cos(a_grid)).ravel(),
334             first_station[1] + (r_grid * np.sin(a_grid)).ravel(),
335         ]
336     )
337     inside = np.einsum("ij,ij->i", sample_grid, sample_grid) <= (
338         config.target_radius**2 + 1e-7
339     )
340     points = sample_grid[inside]
341
342     s1 = np.asarray(first_station, dtype=float)
343     boundary: list[np.ndarray] = []
344     for angle in angles:
345         ray = np.array([np.cos(angle), np.sin(angle)], dtype=float)
346         span = _intersect_interval(
347             (config.too_strong_radius, config.max_receive_radius),
348             _ray_disk_interval(
349                 s1,
350                 ray,
351                 np.zeros(2, dtype=float),
352                 config.target_radius,
353             ),
354         ),
355     )
356     if span is None:
357         continue
358     edge_radii = [
359         max(span[0], config.too_strong_radius + 1e-7),
360         span[1],
361     ]
362     if span[0] <= config.min_receive_radius <= span[1]:
363         edge_radii.append(config.min_receive_radius)
364     for r in edge_radii:
365         boundary.append(s1 + r * ray)
366     if boundary:
367         points = np.vstack([points, np.asarray(boundary)])
368         points = np.unique(np.round(points, decimals=10), axis=0)
369     if len(points) < 20:
370         raise ValueError(
371             " 第一次可行区域的有效样本过少, 请提高 source_radial_samples 和 "
372             "source_angular_samples"
373         )
374     return points
375
376 def _ray_disk_interval(
377     origin: np.ndarray,
378     direction: np.ndarray,
379     center: np.ndarray,
380     radius: float,
381     eps: float = 1e-12,
382 ) -> tuple[float, float] | None:
383     delta = origin - center
384     proj = float(delta @ direction)
385     disc = proj**2 - (float(delta @ delta) - radius**2)
386     if disc < -eps:
387         return None
388     root = float(np.sqrt(max(0.0, disc)))
389     low = -proj - root
390     high = -proj + root

```

```

392     if high < -eps:
393         return None
394     return max(0.0, low), max(0.0, high)
395
396 def _intersect_interval(
397     first: tuple[float, float],
398     second: tuple[float, float] | None,
399 ) -> tuple[float, float] | None:
400     if second is None:
401         return None
402     low = max(first[0], second[0])
403     high = min(first[1], second[1])
404     if high <= low + 1e-12:
405         return None
406     return low, high
407
408
409
410 def _radial_area(interval: tuple[float, float] | None) -> float:
411     if interval is None:
412         return 0.0
413     return 0.5 * max(0.0, interval[1] ** 2 - interval[0] ** 2)
414
415
416 def _coverage_angular_quadrature(
417     first_station: Point,
418     first_bearing_deg: float,
419     config: Q2Config,
420 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
421     xg, wg = np.polynomial.legendre.leggauss(
422         config.source_angular_samples
423     )
424     mid = radians(first_bearing_deg)
425     half = radians(config.bearing_error_deg)
426     angles = mid + half * xg
427     quad_w = half * wg
428     rays = np.column_stack([np.cos(angles), np.sin(angles)])
429     s1 = np.asarray(first_station, dtype=float)
430     origin = np.zeros(2, dtype=float)
431     low = np.empty(len(rays), dtype=float)
432     high = np.empty(len(rays), dtype=float)
433     for i, ray in enumerate(rays):
434         hit = _ray_disk_interval(
435             s1,
436             ray,
437             origin,
438             config.target_radius,
439         )
440         span = _intersect_interval(
441             (config.too_strong_radius, config.max_receive_radius),
442             hit,
443         )
444         if span is None:
445             low[i] = 0.0
446             high[i] = 0.0
447         else:
448             low[i], high[i] = span
449     return rays, quad_w, low, high
450
451
452 def _continuous_coverages_for_candidate(
453     second_station: np.ndarray,
454     first_station: np.ndarray,
455     directions: np.ndarray,
456     angular_weights: np.ndarray,
457     lower_radius: np.ndarray,
458     upper_radius: np.ndarray,
459     config: Q2Config,
460 ) -> tuple[float, float, float]:
461     total = 0.0
462     safe_sum = 0.0
463     fail_sum = 0.0
464     delta = first_station - second_station
465     delta2 = float(delta @ delta)
466
467     for ray, w, low, high in zip(
468         directions,
469         angular_weights,
470         lower_radius,
471         upper_radius,
472     ):
473         span = float(low), float(high)
474         area = _radial_area(span)
475         if area <= 0.0:
476             continue
477         total += float(w) * area
478
479         near_span = (
480             float(low), min(float(high), config.min_receive_radius)
481         )
482         near_safe = None
483         if near_span[1] > near_span[0] + 1e-12:
484             near_safe = _intersect_interval(
485                 near_span,
486                 _ray_disk_interval(
487                     first_station,
488                     ray,
489                     second_station,
490                     config.min_receive_radius,

```

```

491         ),
492     )
493
494     far_low = max(float(low), config.min_receive_radius)
495     far_high = float(high)
496     far_safe: tuple[float, float] | None = None
497     if far_high > far_low + 1e-12:
498         proj = float(ray @ delta)
499         if delta2 <= 1e-18:
500             far_safe = far_low, far_high
501         elif proj < -1e-15:
502             cut = -delta2 / (2.0 * proj)
503             start = max(far_low, cut)
504             if far_high > start + 1e-12:
505                 far_safe = start, far_high
506
507     safe_sum += float(w) * (
508         _radial_area(near_safe) + _radial_area(far_safe)
509     )
510
511     received = _intersect_interval(
512         span,
513         _ray_disk_interval(
514             first_station,
515             ray,
516             second_station,
517             config.max_receive_radius,
518         ),
519     )
520     fail_sum += float(w) * (
521         area - _radial_area(received)
522     )
523
524     if total <= 0.0:
525         raise ValueError(" 第一次示向可行域面积为零")
526     safe = float(np.clip(safe_sum / total, 0.0, 1.0))
527     fail = float(np.clip(fail_sum / total, 0.0, 1.0))
528     uncertain = float(np.clip(1.0 - safe - fail, 0.0, 1.0))
529     uncertain = 1.0 - safe - fail
530     if uncertain < -1e-10:
531         raise RuntimeError(" 连续覆盖积分得到相互重叠的结果分类")
532     return safe, max(0.0, uncertain), fail
533
534
535 def _point_set_diameter(points: np.ndarray) -> float:
536     if len(points) <= 1:
537         return 0.0
538     if len(points) == 2:
539         return float(np.linalg.norm(points[0] - points[1]))
540     try:
541         hull = ConvexHull(points)
542         hull_points = points[hull.vertices]
543     except Exception:
544         hull_points = points
545     if len(hull_points) <= 1:
546         return 0.0
547     return float(np.max(distance.pdist(hull_points)))
548
549
550 def _point_in_convex_polygon(
551     point: np.ndarray, polygon: Sequence[Point], eps: float = 1e-8
552 ) -> bool:
553     if not polygon:
554         return False
555     for index, first in enumerate(polygon):
556         second = polygon[(index + 1) % len(polygon)]
557         edge_x = second[0] - first[0]
558         edge_y = second[1] - first[1]
559         cross = edge_x * (point[1] - first[1]) - edge_y * (
560             point[0] - first[0]
561         )
562         if cross < -eps:
563             return False
564     return True
565
566
567 def _segment_circle_intersections(
568     first: np.ndarray,
569     second: np.ndarray,
570     center: np.ndarray,
571     radius: float,
572     eps: float = 1e-10,
573 ) -> list[np.ndarray]:
574     segment = second - first
575     delta = first - center
576     a = float(segment @ segment)
577     if a <= eps:
578         return []
579     b = 2.0 * float(delta @ segment)
580     c = float(delta @ delta) - radius**2
581     disc = b**2 - 4.0 * a * c
582     if disc < -eps:
583         return []
584     root = np.sqrt(max(0.0, disc))
585     hits: list[np.ndarray] = []
586     for t in ((-b - root) / (2.0 * a), (-b + root) / (2.0 * a)):
587         if -eps <= t <= 1.0 + eps:
588             p = first + min(1.0, max(0.0, t)) * segment
589             if not hits or np.linalg.norm(p - hits[-1]) > eps:

```

```

590         hits.append(p)
591     return hits
592
593
594 def _circle_circle_intersections(
595     first_center: np.ndarray,
596     first_radius: float,
597     second_center: np.ndarray,
598     second_radius: float,
599     eps: float = 1e-10,
600 ) -> list[np.ndarray]:
601     delta = second_center - first_center
602     d = float(np.linalg.norm(delta))
603     if d <= eps:
604         return []
605     if d > first_radius + second_radius + eps:
606         return []
607     if d < abs(first_radius - second_radius) - eps:
608         return []
609     x = (
610         first_radius**2 - second_radius**2 + d**2
611     ) / (2.0 * d)
612     h2 = first_radius**2 - x**2
613     if h2 < -eps:
614         return []
615     base = first_center + x * delta / d
616     normal = np.array([-delta[1], delta[0]], dtype=float)
617     normal /= d
618     h = np.sqrt(max(0.0, h2))
619     if h <= eps:
620         return [base]
621     return [base + h * normal, base - h * normal]
622
623
624 def _polygon_diameter_excluding_disks(
625     polygon: Sequence[Point],
626     excluded_disks: Sequence[tuple[Point, float]],
627     circle_samples: int,
628     eps: float = 1e-8,
629 ) -> float:
630     if not polygon:
631         return 0.0
632     if not excluded_disks:
633         return polygon_diameter(polygon)[0]
634
635     clean_disks = [
636         (np.asarray(center, dtype=float), float(radius))
637         for center, radius in excluded_disks
638     ]
639
640     def allowed(point: np.ndarray) -> bool:
641         return all(
642             np.linalg.norm(point - center) >= radius - eps
643             for center, radius in clean_disks
644         )
645
646     outer_diameter, outer_pair = polygon_diameter(polygon)
647     if all(allowed(np.asarray(point, dtype=float)) for point in outer_pair):
648         return outer_diameter
649
650     candidates: list[np.ndarray] = []
651     polygon_arrays = [np.asarray(point, dtype=float) for point in polygon]
652     candidates.extend(point for point in polygon_arrays if allowed(point))
653
654     for index, first in enumerate(polygon_arrays):
655         second = polygon_arrays[(index + 1) % len(polygon_arrays)]
656         for center, radius in clean_disks:
657             for point in _segment_circle_intersections(
658                 first, second, center, radius, eps
659             ):
660                 if allowed(point):
661                     candidates.append(point)
662
663     angles = np.linspace(0.0, 2.0 * pi, circle_samples, endpoint=False)
664     circle_directions = np.column_stack([np.cos(angles), np.sin(angles)])
665     for center, radius in clean_disks:
666         for point in center + radius * circle_directions:
667             if _point_in_convex_polygon(point, polygon, eps) and allowed(point):
668                 candidates.append(point)
669
670     for first_index, (first_center, first_radius) in enumerate(clean_disks):
671         for second_center, second_radius in clean_disks[first_index + 1 :]:
672             for point in _circle_circle_intersections(
673                 first_center,
674                 first_radius,
675                 second_center,
676                 second_radius,
677                 eps,
678             ):
679                 if _point_in_convex_polygon(point, polygon, eps) and allowed(point):
680                     candidates.append(point)
681
682     if not candidates:
683         return 0.0
684     return _point_set_diameter(np.asarray(candidates, dtype=float))
685
686
687 def _clip_by_distance_comparison(
688     polygon: Sequence[Point],

```

```

689     first_station: Point,
690     second_station: Point,
691     eps: float = 1e-8,
692 ) -> list[Point]:
693     s1 = np.asarray(first_station, dtype=float)
694     s2 = np.asarray(second_station, dtype=float)
695     normal = 2.0 * (s1 - s2)
696     normal2 = float(normal @ normal)
697     if normal2 <= eps**2:
698         return []
699     rhs = float(s1 @ s1 - s2 @ s2)
700     p0 = rhs * normal / normal2
701     line_dir = np.array(
702         [normal[1], -normal[0]], dtype=float
703     )
704     line_dir /= np.linalg.norm(line_dir)
705     return clip_polygon_by_left_halfplane(
706         polygon,
707         (float(p0[0]), float(p0[1])),
708         (float(line_dir[0]), float(line_dir[1])),
709         eps,
710     )
711
712
713 def generate_candidate_points(
714     source_samples: np.ndarray,
715     first_station: Point,
716     first_bearing_deg: float,
717     config: Q2Config,
718 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
719     u_dir, v_dir = _directions(first_bearing_deg)
720     relative = source_samples - np.asarray(first_station, dtype=float)
721     u0 = relative @ u_dir
722     v0 = relative @ v_dir
723     pad = config.max_receive_radius
724     u_values = np.linspace(
725         float(u0.min() - pad),
726         float(u0.max() + pad),
727         config.candidate_axis_samples,
728     )
729     v_values = np.linspace(
730         float(v0.min() - pad),
731         float(v0.max() + pad),
732         config.candidate_axis_samples,
733     )
734     u_grid, v_grid = np.meshgrid(u_values, v_values, indexing="xy")
735     local = np.column_stack([u_grid.ravel(), v_grid.ravel()])
736     points = (
737         np.asarray(first_station, dtype=float)
738         + local[:, :1] * u_dir
739         + local[:, 1:] * v_dir
740     )
741     keep = np.linalg.norm(
742         points - np.asarray(first_station, dtype=float), axis=1
743     ) > 1e-8
744     return points[keep], local[keep, 0], local[keep, 1]
745
746
747 def _compute_raw_metrics(
748     candidate_points: np.ndarray,
749     candidate_u: np.ndarray,
750     candidate_v: np.ndarray,
751     source_samples: np.ndarray,
752     first_station: Point,
753     first_bearing_deg: float,
754     config: Q2Config,
755     first_region_diameter: float,
756 ) -> list[dict[str, float | Point]]:
757     s1 = np.asarray(first_station, dtype=float)
758     d1 = source_samples - s1
759     r1 = np.linalg.norm(d1, axis=1)
760     u1 = d1 / r1[:, None]
761     tan_err = np.tan(np.deg2rad(config.bearing_error_deg))
762     rays, quad_w, r_low, r_high = (
763         _coverage_angular_quadrature(
764             first_station,
765             first_bearing_deg,
766             config,
767         )
768     )
769     rows: list[dict[str, float | Point]] = []
770
771     for s2, u, v in zip(candidate_points, candidate_u, candidate_v):
772         d2 = source_samples - s2
773         r2 = np.linalg.norm(d2, axis=1)
774         has_bearing = (
775             (r2 > config.too_strong_radius)
776             & (r2 <= config.max_receive_radius)
777         )
778
779         safe, uncertain, failure = (
780             _continuous_coverages_for_candidate(
781                 s2,
782                 s1,
783                 rays,
784                 quad_w,
785                 r_low,
786                 r_high,
787                 config,

```

```

788     )
789 )
790
791 if np.any(has_bearing):
792     r2_sel = r2[has_bearing]
793     u2 = d2[has_bearing] / r2_sel[:, None]
794     cos_phi = np.abs(np.einsum("ij,ij->i", u1[has_bearing], u2))
795     cos_phi = np.clip(cos_phi, 0.0, 1.0)
796     sin_phi = np.sqrt(np.maximum(0.0, 1.0 - cos_phi**2))
797     angle_cost = float(np.mean(1.0 - sin_phi))
798     sin_safe = np.maximum(sin_phi, 1e-8)
799     r1_sel = r1[has_bearing]
800     linear_d = (
801         2.0
802         * tan_err
803         / sin_safe
804         * np.sqrt(
805             r1_sel**2
806             + r2_sel**2
807             + 2.0 * r1_sel * r2_sel * cos_phi
808         )
809     )
810     proxy_d = min(
811         first_region_diameter,
812         float(np.max(linear_d)),
813     )
814 else:
815     angle_cost = 1.0
816     proxy_d = first_region_diameter
817
818 rows.append(
819     {
820         "point": (float(s2[0]), float(s2[1])),
821         "local_u": float(u),
822         "local_v": float(v),
823         "safe_coverage": safe,
824         "uncertain_coverage": uncertain,
825         "certain_failure": failure,
826         "angle_penalty": angle_cost,
827         "proxy_diameter": proxy_d,
828     }
829 )
830 return rows
831
832
833 def _polygon_after_second_bearing(
834     candidate_base_polygon: Sequence[Point],
835     second_station: Point,
836     measured_bearing_deg: float,
837     config: Q2Config,
838 ) -> list[Point]:
839     return _clip_by_bearing(
840         candidate_base_polygon,
841         second_station,
842         measured_bearing_deg,
843         config.bearing_error_deg,
844     )
845
846
847 def _validate_candidate_on_grid(
848     record: dict[str, float | Point],
849     first_polygon: Sequence[Point],
850     source_samples: np.ndarray,
851     first_station: Point,
852     config: Q2Config,
853     first_region_diameter: float,
854 ) -> tuple[float, float, float, float]:
855     point = np.asarray(record["point"], dtype=float)
856     station = np.asarray(first_station, dtype=float)
857     r1 = np.linalg.norm(source_samples - station, axis=1)
858     source_from_second = source_samples - point
859     r2 = np.linalg.norm(source_from_second, axis=1)
860     second_station = float(point[0]), float(point[1])
861
862     strong_polygon = _clip_by_disk(
863         first_polygon,
864         second_station,
865         config.too_strong_radius,
866         config.disk_sides,
867     )
868     strong_signal_diameter = _polygon_diameter_excluding_disks(
869         strong_polygon,
870         [(first_station, config.too_strong_radius)],
871         config.disk_sides,
872     )
873
874     no_signal_outer = _clip_by_distance_comparison(
875         first_polygon, first_station, second_station
876     )
877     no_signal_diameter = _polygon_diameter_excluding_disks(
878         no_signal_outer,
879         [
880             (first_station, config.too_strong_radius),
881             (second_station, config.min_receive_radius),
882         ],
883         config.disk_sides,
884     )
885
886     candidate_base_polygon = _clip_by_disk(

```

```

887     first_polygon,
888     second_station,
889     config.max_receive_radius,
890     config.disk_sides,
891 )
892
893 possible = (
894     (r2 > config.too_strong_radius)
895     & (r2 <= config.max_receive_radius)
896 )
897 possible_indices = np.flatnonzero(possible)
898 bearing_grid_worst = 0.0
899
900 if len(possible_indices) and candidate_base_polygon:
901     error_values = np.linspace(
902         -config.bearing_error_deg,
903         config.bearing_error_deg,
904         config.bearing_error_samples,
905     )
906     for source_index in possible_indices:
907         source = source_samples[source_index]
908         true_bearing = degrees(
909             atan2(source[1] - point[1], source[0] - point[0])
910         )
911         for error in error_values:
912             polygon = _polygon_after_second_bearing(
913                 candidate_base_polygon,
914                 second_station,
915                 true_bearing + float(error),
916                 config,
917             )
918             if polygon:
919                 diameter_value = _polygon_diameter_excluding_disks(
920                     polygon,
921                     [
922                         (first_station, config.too_strong_radius),
923                         (second_station, config.too_strong_radius),
924                     ],
925                     config.disk_sides,
926                 )
927                 bearing_grid_worst = max(
928                     bearing_grid_worst, diameter_value
929                 )
930
931 robust_diameter = min(
932     first_region_diameter,
933     max(
934         bearing_grid_worst,
935         strong_signal_diameter,
936         no_signal_diameter,
937     ),
938 )
939 return (
940     bearing_grid_worst,
941     strong_signal_diameter,
942     no_signal_diameter,
943     robust_diameter,
944 )
945
946 def _build_candidate_metrics(
947     raw_records: Sequence[dict[str, float | Point]],
948     first_region_diameter: float,
949     score_weights: tuple[float, float, float, float],
950 ) -> tuple[
951     tuple[CandidateMetrics, ...],
952     np.ndarray,
953     np.ndarray,
954     np.ndarray,
955     np.ndarray,
956     np.ndarray,
957     np.ndarray,
958 ]:
959     safe = np.array([record["safe_coverage"] for record in raw_records])
960     failure = np.array([record["certain_failure"] for record in raw_records])
961     diameter = np.array([record["proxy_diameter"] for record in raw_records])
962     angle = np.array([record["angle_penalty"] for record in raw_records])
963     score = calculate_weighted_scores(
964         safe,
965         failure,
966         diameter,
967         angle,
968         first_region_diameter,
969         score_weights,
970     )
971     nondominated = _pareto_mask(
972         safe,
973         failure,
974         np.clip(diameter / max(first_region_diameter, 1e-9), 0.0, 1.0),
975         angle,
976     )
977
978 candidates = tuple(
979     CandidateMetrics(
980         point=record["point"],
981         local_u=float(record["local_u"]),
982         local_v=float(record["local_v"]),
983         safe_coverage=float(record["safe_coverage"]),
984         uncertain_coverage=float(record["uncertain_coverage"]),
985         certain_failure=float(record["certain_failure"]),

```

```

986         angle_penalty=float(record["angle_penalty"]),
987         proxy_diameter=float(record["proxy_diameter"]),
988         proxy_score=float(score[index]),
989         pareto=bool(nondominated[index]),
990         refinement_level=int(record.get("refinement_level", 0)),
991     )
992     for index, record in enumerate(raw_records)
993 )
994 return candidates, safe, failure, diameter, angle, score, nondominated
995
996 def _validate_selected_candidates(
997     selected_indices: Sequence[int],
998     candidates: Sequence[CandidateMetrics],
999     raw_records: Sequence[dict[str, float | Point]],
1000     first_polygon: Sequence[Point],
1001     source_samples: np.ndarray,
1002     first_station: Point,
1003     config: Q2Config,
1004     first_region_diameter: float,
1005     score_weights: tuple[float, float, float, float],
1006 ) -> List[CandidateMetrics]:
1007     validated: List[CandidateMetrics] = []
1008     for index in selected_indices:
1009         candidate = candidates[index]
1010         bearing_worst, strong_signal, no_signal, robust = (
1011             _validate_candidate_on_grid(
1012                 raw_records[index],
1013                 first_polygon,
1014                 source_samples,
1015                 first_station,
1016                 config,
1017                 first_region_diameter,
1018             )
1019         )
1020     )
1021     validated_score = evaluate_candidate_score(
1022         candidate.safe_coverage,
1023         candidate.certain_failure,
1024         robust,
1025         candidate.angle_penalty,
1026         first_region_diameter,
1027         score_weights,
1028     ).total
1029     validated.append(
1030         replace(
1031             candidate,
1032             validated_bearing_worst=bearing_worst,
1033             strong_signal_diameter=strong_signal,
1034             no_signal_diameter=no_signal,
1035             validated_robust_diameter=robust,
1036             validated_score=float(validated_score),
1037         )
1038     )
1039     return validated
1040
1041 def _make_decision_from_candidates(
1042     validated: Sequence[CandidateMetrics],
1043     maximum_safe_coverage: float,
1044     first_station: Point,
1045     config: Q2Config,
1046     improvement_tolerance_ratio: float | None = None,
1047 ):
1048     if not validated:
1049         raise RuntimeError(" 没有可用于离散几何验证的候选点")
1050     return make_final_decision(
1051         np.array([item.safe_coverage for item in validated]),
1052         np.array([item.certain_failure for item in validated]),
1053         np.array([float(item.validated_robust_diameter) for item in validated]),
1054         np.array([item.angle_penalty for item in validated]),
1055         np.array([float(item.validated_score) for item in validated]),
1056         np.array(
1057             [
1058                 np.hypot(
1059                     item.point[0] - first_station[0],
1060                     item.point[1] - first_station[1],
1061                 )
1062                 for item in validated
1063             ]
1064         ),
1065         config.reliability_loss_levels,
1066         (
1067             config.tradeoff_improvement_tolerance_ratio
1068             if improvement_tolerance_ratio is None
1069             else improvement_tolerance_ratio
1070         ),
1071         config.diameter_tolerance_ratio,
1072         config.score_tolerance,
1073         maximum_safe_coverage=maximum_safe_coverage,
1074     )
1075
1076 def _candidate_grid_steps(
1077     source_samples: np.ndarray,
1078     first_station: Point,
1079     first_bearing_deg: float,
1080     config: Q2Config,
1081 ) -> tuple[float, float]:
1082     forward, normal = _directions(first_bearing_deg)

```

```

1085 relative_sources = source_samples - np.asarray(first_station, dtype=float)
1086 source_u = relative_sources @ forward
1087 source_v = relative_sources @ normal
1088 denominator = config.candidate_axis_samples - 1
1089 return (
1090     float(source_u.max() - source_u.min() + 2.0 * config.max_receive_radius)
1091     / denominator,
1092     float(source_v.max() - source_v.min() + 2.0 * config.max_receive_radius)
1093     / denominator,
1094 )
1095
1096 def _generate_local_refinement_points(
1097     center: CandidateMetrics,
1098     first_station: Point,
1099     first_bearing_deg: float,
1100     step_u: float,
1101     step_v: float,
1102     axis_samples: int,
1103     known_points: set[tuple[float, float]],
1104 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
1105     offsets_u = np.linspace(-0.5 * step_u, 0.5 * step_u, axis_samples)
1106     offsets_v = np.linspace(-0.5 * step_v, 0.5 * step_v, axis_samples)
1107     uu, vv = np.meshgrid(
1108         center.local_u + offsets_u,
1109         center.local_v + offsets_v,
1110         indexing="xy",
1111     )
1112     local_u = uu.ravel()
1113     local_v = vv.ravel()
1114     forward, normal = _directions(first_bearing_deg)
1115     points = (
1116         np.asarray(first_station, dtype=float)
1117         + local_u[:, None] * forward
1118         + local_v[:, None] * normal
1119     )
1120     keep = []
1121     for point in points:
1122         key = (round(float(point[0]), 8), round(float(point[1]), 8))
1123         keep.append(
1124             key not in known_points
1125             and np.linalg.norm(point - np.asarray(first_station, dtype=float))
1126             > 1e-8
1127         )
1128     mask = np.asarray(keep, dtype=bool)
1129     return points[mask], local_u[mask], local_v[mask]
1130
1131
1132 def _select_refinement_centers(
1133     validated: Sequence[CandidateMetrics],
1134     near_optimal_indices: Sequence[int],
1135     step_u: float,
1136     step_v: float,
1137     maximum_centers: int,
1138 ) -> tuple[CandidateMetrics, ...]:
1139     ordered = sorted(
1140         (validated[index] for index in near_optimal_indices),
1141         key=lambda item: (
1142             float(item.validated_robust_diameter),
1143             item.certain_failure,
1144             item.angle_penalty,
1145             np.hypot(item.local_u, item.local_v),
1146         ),
1147     )
1148     separation = 0.75 * max(step_u, step_v)
1149     selected: list[CandidateMetrics] = []
1150     for candidate in ordered:
1151         if all(
1152             np.hypot(
1153                 candidate.point[0] - center.point[0],
1154                 candidate.point[1] - center.point[1],
1155             )
1156             > separation
1157             for center in selected
1158         ):
1159             selected.append(candidate)
1160             if len(selected) >= maximum_centers:
1161                 break
1162     return tuple(selected)
1163
1164
1165 def _select_decision_refinement_centers(
1166     validated: Sequence[CandidateMetrics],
1167     near_optimal_indices: Sequence[int],
1168     safe_near_optimal_indices: Sequence[int],
1169     step_u: float,
1170     step_v: float,
1171     maximum_centers: int,
1172 ) -> tuple[CandidateMetrics, ...]:
1173     main_budget = max(1, (maximum_centers + 1) // 2)
1174     safe_budget = max(0, maximum_centers - main_budget)
1175     main_centers = _select_refinement_centers(
1176         validated,
1177         near_optimal_indices,
1178         step_u,
1179         step_v,
1180         main_budget,
1181     )
1182     safe_centers = (

```

```

1184     _select_refinement_centers(
1185         validated,
1186         safe_near_optimal_indices,
1187         step_u,
1188         step_v,
1189         safe_budget,
1190     )
1191     if safe_budget
1192     else ()
1193 )
1194 selected: list[CandidateMetrics] = list(main_centers)
1195 selected_points = {candidate.point for candidate in selected}
1196 for candidate in safe_centers:
1197     if candidate.point not in selected_points:
1198         selected.append(candidate)
1199         selected_points.add(candidate.point)
1200
1201 if len(selected) < maximum_centers:
1202     combined_indices = tuple(
1203         dict.fromkeys(("near_optimal_indices", "safe_near_optimal_indices"))
1204     )
1205     extras = _select_refinement_centers(
1206         validated,
1207         combined_indices,
1208         step_u,
1209         step_v,
1210         maximum_centers,
1211     )
1212     for candidate in extras:
1213         if candidate.point not in selected_points:
1214             selected.append(candidate)
1215             selected_points.add(candidate.point)
1216         if len(selected) >= maximum_centers:
1217             break
1218 return tuple(selected)
1219
1220
1221 def _numerical_resolution_config(
1222     config: Q2Config,
1223     *,
1224     fine: bool,
1225 ) -> Q2Config:
1226     factor = config.numerical_calibration_factor
1227     if fine:
1228         return replace(
1229             config,
1230             source_radial_samples=config.source_radial_samples * factor,
1231             source_angular_samples=(config.source_angular_samples - 1) * factor + 1,
1232             bearing_error_samples=(config.bearing_error_samples - 1) * factor + 1,
1233             disk_sides=config.disk_sides * factor,
1234             perform_numerical_calibration=False,
1235             local_refinement_passes=0,
1236         )
1237     return replace(
1238         config,
1239         source_radial_samples=max(3, config.source_radial_samples // factor),
1240         source_angular_samples=max(
1241             3, (config.source_angular_samples - 1) // factor + 1
1242         ),
1243         bearing_error_samples=max(
1244             3, (config.bearing_error_samples - 1) // factor + 1
1245         ),
1246         disk_sides=max(12, config.disk_sides // factor),
1247         perform_numerical_calibration=False,
1248         local_refinement_passes=0,
1249     )
1250
1251
1252 def _build_numerical_validation_context(
1253     first_station: Point,
1254     first_bearing_deg: float,
1255     config: Q2Config,
1256 ) -> tuple[Q2Config, tuple[Point, ...], np.ndarray, float]:
1257     polygon = build_first_feasible_polygon(
1258         first_station, first_bearing_deg, config
1259     )
1260     first_diameter = _polygon_diameter_excluding_disks(
1261         polygon,
1262         [(first_station, config.too_strong_radius)],
1263         config.disk_sides,
1264     )
1265     samples = sample_first_feasible_region(
1266         first_station, first_bearing_deg, config
1267     )
1268     return config, tuple(polygon), samples, float(first_diameter)
1269
1270
1271 def _evaluate_stability_records(
1272     candidates_with_roles: Sequence[tuple[int, str]],
1273     validated: Sequence[CandidateMetrics],
1274     first_station: Point,
1275     coarse_context: tuple[Q2Config, tuple[Point, ...], np.ndarray, float],
1276     fine_context: tuple[Q2Config, tuple[Point, ...], np.ndarray, float],
1277 ) -> tuple[NumericalStabilityRecord, ...]:
1278
1279     def evaluate(
1280         candidate: CandidateMetrics,
1281         context: tuple[Q2Config, tuple[Point, ...], np.ndarray, float],
1282     ) -> float:

```

```

1283     resolution_config, polygon, samples, first_diameter = context
1284     return float(
1285         _validate_candidate_on_grid(
1286             {"point": candidate.point},
1287             polygon,
1288             samples,
1289             first_station,
1290             resolution_config,
1291             first_diameter,
1292         )[3]
1293     )
1294
1295 records: list[NumericalStabilityRecord] = []
1296 for index, role in candidates_with_roles:
1297     candidate = validated[index]
1298     default_diameter = float(candidate.validated_robust_diameter)
1299     coarse_diameter = evaluate(candidate, coarse_context)
1300     fine_diameter = evaluate(candidate, fine_context)
1301     relative_difference = abs(fine_diameter - default_diameter) / max(
1302         abs(fine_diameter), 1e-12
1303     )
1304     records.append(
1305         NumericalStabilityRecord(
1306             role=role,
1307             point=candidate.point,
1308             coarse_robust_diameter=coarse_diameter,
1309             default_robust_diameter=default_diameter,
1310             fine_robust_diameter=fine_diameter,
1311             default_fine_relative_difference=float(relative_difference),
1312         )
1313     )
1314 return tuple(records)
1315
1316
1317 def solve_second_station(
1318     first_station: Point,
1319     first_bearing_deg: float,
1320     config: Q2Config | None = None,
1321 ) -> Q2Result:
1322     active_config = config or Q2Config()
1323     active_config.validate()
1324     clean_station = _validate_station_and_bearing(first_station, first_bearing_deg)
1325     clean_bearing = float(first_bearing_deg) % 360.0
1326
1327     first_polygon = build_first_feasible_polygon(
1328         clean_station, clean_bearing, active_config
1329     )
1330     first_region_diameter = _polygon_diameter_excluding_disks(
1331         first_polygon,
1332         [(clean_station, active_config.too_strong_radius)],
1333         active_config.disk_sides,
1334     )
1335     source_samples = sample_first_feasible_region(
1336         clean_station, clean_bearing, active_config
1337     )
1338     candidate_points, candidate_u, candidate_v = generate_candidate_points(
1339         source_samples, clean_station, clean_bearing, active_config
1340     )
1341     raw_records = _compute_raw_metrics(
1342         candidate_points,
1343         candidate_u,
1344         candidate_v,
1345         source_samples,
1346         clean_station,
1347         clean_bearing,
1348         active_config,
1349         first_region_diameter,
1350     )
1351     score_weights = (
1352         active_config.weight_not_safe,
1353         active_config.weight_certain_failure,
1354         active_config.weight_diameter,
1355         active_config.weight_angle,
1356     )
1357     (
1358         candidates,
1359         safe_array,
1360         failure_array,
1361         diameter_array,
1362         angle_array,
1363         _,
1364         pareto,
1365     ) = _build_candidate_metrics(raw_records, first_region_diameter, score_weights)
1366
1367     selected_indices = select_validation_indices(
1368         safe_array,
1369         diameter_array,
1370         np.asarray([candidate.point for candidate in candidates], dtype=float),
1371         active_config.validated_candidate_count,
1372         active_config.reliability_loss_levels,
1373     )
1374     validated = _validate_selected_candidates(
1375         selected_indices,
1376         candidates,
1377         raw_records,
1378         first_polygon,
1379         source_samples,
1380         clean_station,
1381         active_config,

```

```

1382     first_region_diameter,
1383     score_weights,
1384 )
1385
1386 step_u, step_v = _candidate_grid_steps(
1387     source_samples,
1388     clean_station,
1389     clean_bearing,
1390     active_config,
1391 )
1392 known_points = {
1393     (round(item.point[0], 8), round(item.point[1], 8)) for item in candidates
1394 }
1395 all_records = list(raw_records)
1396 all_candidates = list(candidates)
1397 maximum_safe = max(item.safe_coverage for item in all_candidates)
1398 decision = _make_decision_from_candidates(
1399     validated,
1400     maximum_safe,
1401     clean_station,
1402     active_config,
1403 )
1404 previous_near = [
1405     validated[index] for index in decision.near_optimal_indices
1406 ]
1407 previous_best_diameter = float(
1408     validated[decision.best_index].validated_robust_diameter
1409 )
1410 refinement_converged = False
1411 refinement_resolution = max(step_u, step_v)
1412 refinement_position_change = float("inf")
1413 refinement_diameter_relative_change = float("inf")
1414 for refinement_level in range(1, active_config.local_refinement_passes + 1):
1415     centers = _select_decision_refinement_centers(
1416         validated,
1417         decision.near_optimal_indices,
1418         decision.safe_near_optimal_indices,
1419         step_u,
1420         step_v,
1421         active_config.max_refinement_centers,
1422     )
1423     point_parts: list[np.ndarray] = []
1424     u_parts: list[np.ndarray] = []
1425     v_parts: list[np.ndarray] = []
1426     temporary_known = set(known_points)
1427     for center in centers:
1428         points_part, u_part, v_part = _generate_local_refinement_points(
1429             center,
1430             clean_station,
1431             clean_bearing,
1432             step_u,
1433             step_v,
1434             active_config.local_refinement_axis_samples,
1435             temporary_known,
1436         )
1437         if len(points_part):
1438             point_parts.append(points_part)
1439             u_parts.append(u_part)
1440             v_parts.append(v_part)
1441             temporary_known.update(
1442                 (round(float(point[0]), 8), round(float(point[1]), 8))
1443                 for point in points_part
1444             )
1445     if not point_parts:
1446         break
1447     local_points = np.vstack(point_parts)
1448     local_u = np.concatenate(u_parts)
1449     local_v = np.concatenate(v_parts)
1450     local_records = _compute_raw_metrics(
1451         local_points,
1452         local_u,
1453         local_v,
1454         source_samples,
1455         clean_station,
1456         clean_bearing,
1457         active_config,
1458         first_region_diameter,
1459     )
1460     for record in local_records:
1461         record["refinement_level"] = float(refinement_level)
1462     local_candidates, *_ = _build_candidate_metrics(
1463         local_records,
1464         first_region_diameter,
1465         score_weights,
1466     )
1467     all_records.extend(local_records)
1468     all_candidates.extend(local_candidates)
1469     local_indices = tuple(range(len(local_records)))
1470     validated.extend(
1471         _validate_selected_candidates(
1472             local_indices,
1473             local_candidates,
1474             local_records,
1475             first_polygon,
1476             source_samples,
1477             clean_station,
1478             active_config,
1479             first_region_diameter,
1480             score_weights,

```

```

1481     )
1482 )
1483 for item in local_candidates:
1484     known_points.add((round(item.point[0], 8), round(item.point[1], 8)))
1485 maximum_safe = max(
1486     maximum_safe,
1487     max(item.safe_coverage for item in local_candidates),
1488 )
1489 decision = _make_decision_from_candidates(
1490     validated,
1491     maximum_safe,
1492     clean_station,
1493     active_config,
1494 )
1495 divisor = active_config.local_refinement_axis_samples - 1
1496 step_u /= divisor
1497 step_v /= divisor
1498 refinement_resolution = max(step_u, step_v)
1499 current_best = validated[decision.best_index]
1500 refinement_position_change = min(
1501     float(
1502         np.hypot(
1503             current_best.point[0] - previous.point[0],
1504             current_best.point[1] - previous.point[1],
1505         )
1506     )
1507     for previous in previous_near
1508 )
1509 refinement_diameter_relative_change = abs(
1510     float(current_best.validated_robust_diameter) - previous_best_diameter
1511 ) / max(previous_best_diameter, 1e-12)
1512 previous_best_diameter = float(current_best.validated_robust_diameter)
1513 previous_near = [
1514     validated[index] for index in decision.near_optimal_indices
1515 ]
1516 if (
1517     refinement_level >= 2
1518     and refinement_resolution
1519     <= active_config.position_convergence_tolerance
1520     and refinement_position_change
1521     <= active_config.position_convergence_tolerance
1522     and refinement_diameter_relative_change
1523     <= active_config.diameter_convergence_tolerance_ratio
1524 ):
1525     refinement_converged = True
1526     break
1527
1528 candidates, *_ = _build_candidate_metrics(
1529     all_records,
1530     first_region_diameter,
1531     score_weights,
1532 )
1533 candidate_by_point = {candidate.point: candidate for candidate in candidates}
1534 validated = [
1535     replace(
1536         item,
1537         proxy_score=candidate_by_point[item.point].proxy_score,
1538         pareto=candidate_by_point[item.point].pareto,
1539     )
1540     for item in validated
1541 ]
1542 maximum_safe = max(item.safe_coverage for item in candidates)
1543 decision = _make_decision_from_candidates(
1544     validated,
1545     maximum_safe,
1546     clean_station,
1547     active_config,
1548 )
1549
1550 numerical_calibration_applied = False
1551 numerical_error_ratio = active_config.tradeoff_improvement_tolerance_ratio
1552 numerical_stability_records: tuple[NumericalStabilityRecord, ...] = ()
1553 if active_config.perform_numerical_calibration:
1554     coarse_config = _numerical_resolution_config(active_config, fine=False)
1555     fine_config = _numerical_resolution_config(active_config, fine=True)
1556     coarse_context = _build_numerical_validation_context(
1557         clean_station, clean_bearing, coarse_config
1558     )
1559     fine_context = _build_numerical_validation_context(
1560         clean_station, clean_bearing, fine_config
1561     )
1562
1563 baseline_index = decision.tradeoff_points[0].best_index
1564 minimum_index = min(
1565     decision.tradeoff_points,
1566     key=lambda point: point.robust_diameter,
1567 ).best_index
1568 provisional_index = decision.best_index
1569 role_by_index: dict[int, list[str]] = {}
1570 for index, role in (
1571     (baseline_index, " 零可靠性损失最优点"),
1572     (minimum_index, " 最大探索范围内最小直径点"),
1573     (provisional_index, " 标定前推荐点"),
1574 ):
1575     role_by_index.setdefault(index, []).append(role)
1576 calibration_targets = tuple(
1577     (index, "\ ".join(roles))
1578     for index, roles in role_by_index.items()
1579 )

```

```

1580     numerical_stability_records = _evaluate_stability_records(
1581         calibration_targets,
1582         validated,
1583         clean_station,
1584         coarse_context,
1585         fine_context,
1586     )
1587     numerical_error_ratio = max(
1588         record.default_fine_relative_difference
1589         for record in numerical_stability_records
1590     )
1591     numerical_calibration_applied = True
1592     decision = _make_decision_from_candidates(
1593         validated,
1594         maximum_safe,
1595         clean_station,
1596         active_config,
1597         improvement_tolerance_ratio=numerical_error_ratio,
1598     )
1599
1600     calibrated_indices = set(role_by_index)
1601     while decision.best_index not in calibrated_indices:
1602         extra_records = _evaluate_stability_records(
1603             ((decision.best_index, " 标定后推荐点"),),
1604             validated,
1605             clean_station,
1606             coarse_context,
1607             fine_context,
1608         )
1609         numerical_stability_records += extra_records
1610         calibrated_indices.add(decision.best_index)
1611         numerical_error_ratio = max(
1612             numerical_error_ratio,
1613             extra_records[0].default_fine_relative_difference,
1614         )
1615         decision = _make_decision_from_candidates(
1616             validated,
1617             maximum_safe,
1618             clean_station,
1619             active_config,
1620             improvement_tolerance_ratio=numerical_error_ratio,
1621         )
1622
1623     validated = [
1624         replace(item, validated_pareto=bool(decision.pareto_mask[index]))
1625         for index, item in enumerate(validated)
1626     ]
1627     best = validated[decision.best_index]
1628     near_optimal = tuple(
1629         validated[index] for index in decision.near_optimal_indices
1630     )
1631     safe_near_optimal = tuple(
1632         validated[index] for index in decision.safe_near_optimal_indices
1633     )
1634     pareto_best = validated[decision.weighted_best_index]
1635     pareto_near_optimal = tuple(
1636         validated[index] for index in decision.weighted_near_optimal_indices
1637     )
1638     reliability_feasible = tuple(
1639         item
1640         for item in candidates
1641         if item.safe_coverage >= decision.accepted_safe_coverage_floor - 1e-12
1642     )
1643
1644     return Q2Result(
1645         first_station=clean_station,
1646         first_bearing_deg=clean_bearing,
1647         first_region_polygon=tuple(first_polygon),
1648         first_region_diameter=float(first_region_diameter),
1649         source_samples=source_samples,
1650         candidates=candidates,
1651         validated_candidates=tuple(validated),
1652         primary_safe_coverage=decision.maximum_safe_coverage,
1653         selected_reliability_loss=decision.selected_reliability_loss,
1654         accepted_safe_coverage_floor=decision.accepted_safe_coverage_floor,
1655         tradeoff_improvement_tolerance=decision.improvement_tolerance,
1656         reliability_tradeoff=decision.tradeoff_points,
1657         reliability_feasible_candidates=reliability_feasible,
1658         refinement_converged=refinement_converged,
1659         refinement_resolution=float(refinement_resolution),
1660         refinement_position_change=float(refinement_position_change),
1661         refinement_diameter_relative_change=float(
1662             refinement_diameter_relative_change
1663         ),
1664         numerical_calibration_applied=numerical_calibration_applied,
1665         numerical_error_ratio=float(numerical_error_ratio),
1666         numerical_stability_records=numerical_stability_records,
1667         best_candidate=best,
1668         near_optimal_candidates=near_optimal,
1669         safe_near_optimal_candidates=safe_near_optimal,
1670         pareto_best_candidate=pareto_best,
1671         pareto_near_optimal_candidates=pareto_near_optimal,
1672         config=active_config,
1673     )
1674
1675
1676 def _build_parser() -> argparse.ArgumentParser:
1677     parser = argparse.ArgumentParser(description="B 题问题 2 参数化第二检测点求解器")
1678     parser.add_argument("--s1-x", type=float, default=0.0, help=" 第一检测点 x 坐标")

```

```

1679 parser.add_argument("--s1-y", type=float, default=0.0, help=" 第一检测点 y 坐标")
1680 parser.add_argument("--bearing", type=float, default=0.0, help=" 第一次示向度/度")
1681 parser.add_argument(
1682     "--candidate-samples",
1683     type=int,
1684     default=55,
1685     help=" 候选网格每个坐标轴的采样数",
1686 )
1687 parser.add_argument(
1688     "--source-radial-samples",
1689     type=int,
1690     default=180,
1691     help=" 第一次可行域的径向样本数",
1692 )
1693 parser.add_argument(
1694     "--source-angular-samples",
1695     type=int,
1696     default=41,
1697     help=" 第一次可行域的角度样本数",
1698 )
1699 parser.add_argument(
1700     "--disk-sides",
1701     type=int,
1702     default=144,
1703     help=" 圆边界及排除圆弧的离散边数",
1704 )
1705 parser.add_argument(
1706     "--validated-candidates",
1707     type=int,
1708     default=24,
1709     help=" 按可靠性分层、代理直径和空间多样性选出的初始精验证数",
1710 )
1711 parser.add_argument(
1712     "--bearing-error-samples",
1713     type=int,
1714     default=5,
1715     help="[-1°, 1°] 内的确定性误差网格点数",
1716 )
1717 parser.add_argument(
1718     "--reliability-loss-levels",
1719     type=float,
1720     nargs="+",
1721     default=list(Q2Config().reliability_loss_levels),
1722     help=" 精验证探索分层及最大损失范围, 例如 0 .005 .01 .02 .03 .05",
1723 )
1724 parser.add_argument(
1725     "--skip-numerical-calibration",
1726     action="store_true",
1727     help=" 跳过粗/默认/加密复算, 仅供快速调试",
1728 )
1729 parser.add_argument(
1730     "--numerical-calibration-factor",
1731     type=int,
1732     default=2,
1733     help=" 关键候选点数值加密倍数, 默认 2",
1734 )
1735 parser.add_argument(
1736     "--tradeoff-improvement-tolerance-ratio",
1737     type=float,
1738     default=0.01,
1739     help=" 关闭数值标定时使用的后备相对容差",
1740 )
1741 parser.add_argument(
1742     "--local-refinement-samples",
1743     type=int,
1744     default=3,
1745     help=" 每轮局部加密网格的单轴点数",
1746 )
1747 parser.add_argument(
1748     "--local-refinement-passes",
1749     type=int,
1750     default=4,
1751     help=" 围绕当前最优层逐层缩小网格的轮数",
1752 )
1753 parser.add_argument(
1754     "--max-refinement-centers",
1755     type=int,
1756     default=4,
1757     help=" 每轮同时跟踪的相互分离近优区域数量",
1758 )
1759 parser.add_argument(
1760     "--position-convergence-tolerance",
1761     type=float,
1762     default=10.0,
1763     help=" 最优点坐标与局部网格分辨率的收敛阈值/米",
1764 )
1765 parser.add_argument(
1766     "--diameter-convergence-tolerance-ratio",
1767     type=float,
1768     default=0.01,
1769     help=" 相邻局部加密轮次稳健直径的相对收敛阈值",
1770 )
1771 parser.add_argument("--weight-not-safe", type=float, default=0.24)
1772 parser.add_argument("--weight-certain-failure", type=float, default=0.31)
1773 parser.add_argument("--weight-diameter", type=float, default=0.35)
1774 parser.add_argument("--weight-angle", type=float, default=0.10)
1775 parser.add_argument(
1776     "--score-tolerance",
1777     type=float,

```

```

1778         default=0.03,
1779         help=" 验证后无量纲综合评分的近优绝对容差",
1780     )
1781     parser.add_argument(
1782         "--diameter-tolerance-ratio",
1783         type=float,
1784         default=0.03,
1785         help=" 可靠接收候选区域相对最小直径的容差",
1786     )
1787     parser.add_argument(
1788         "--output-dir",
1789         type=Path,
1790         default=Path("output/q2"),
1791         help=" 结果输出目录",
1792     )
1793     return parser
1794
1795
1796 def main() -> None:
1797     args = _build_parser().parse_args()
1798     config = Q2Config(
1799         candidate_axis_samples=args.candidate_samples,
1800         source_radial_samples=args.source_radial_samples,
1801         source_angular_samples=args.source_angular_samples,
1802         disk_sides=args.disk_sides,
1803         validated_candidate_count=args.validated_candidates,
1804         bearing_error_samples=args.bearing_error_samples,
1805         reliability_loss_levels=tuple(args.reliability_loss_levels),
1806         perform_numerical_calibration=not args.skip_numerical_calibration,
1807         numerical_calibration_factor=args.numerical_calibration_factor,
1808         tradeoff_improvement_tolerance_ratio=(
1809             args.tradeoff_improvement_tolerance_ratio
1810         ),
1811         local_refinement_axis_samples=args.local_refinement_samples,
1812         local_refinement_passes=args.local_refinement_passes,
1813         max_refinement_centers=args.max_refinement_centers,
1814         position_convergence_tolerance=args.position_convergence_tolerance,
1815         diameter_convergence_tolerance_ratio=(
1816             args.diameter_convergence_tolerance_ratio
1817         ),
1818         score_tolerance=args.score_tolerance,
1819         diameter_tolerance_ratio=args.diameter_tolerance_ratio,
1820         weight_not_safe=args.weight_not_safe,
1821         weight_certain_failure=args.weight_certain_failure,
1822         weight_diameter=args.weight_diameter,
1823         weight_angle=args.weight_angle,
1824     )
1825     result = solve_second_station(
1826         first_station=(args.sl_x, args.sl_y),
1827         first_bearing_deg=args.bearing,
1828         config=config,
1829     )
1830     output_dir: Path = args.output_dir
1831     candidate_csv_path = output_dir / " 候选点评价.csv"
1832     selection_figure_path = output_dir / " 选点结果图.png"
1833     tradeoff_csv_path = output_dir / " 可靠性权衡数据.csv"
1834     tradeoff_figure_path = output_dir / " 可靠性与定位精度权衡曲线.png"
1835     write_candidate_csv(result, candidate_csv_path)
1836     plot_result(result, selection_figure_path)
1837     write_reliability_tradeoff_csv(result, tradeoff_csv_path)
1838     plot_reliability_tradeoff(result, tradeoff_figure_path)
1839
1840     best = result.best_candidate
1841     print(" 问题 2 参数化求解完成")
1842     print(f" 输入 S1: {result.first_station}")
1843     print(f" 输入示向度: {result.first_bearing_deg:.6f}°")
1844     print(f" 第一次可行区域直径: {result.first_region_diameter:.6f} m")
1845     print(f" 候选点数量: {len(result.candidates)}")
1846     print(f" 粗筛帕累托点数量: {sum(item.pareto for item in result.candidates)}")
1847     print(f" 主策略最大可靠覆盖率: {result.primary_safe_coverage:.6%}")
1848     print(
1849         " 自动选择的可靠性损失上限: "
1850         f"{100.0 * result.selected_reliability_loss:.3f} 个百分点"
1851     )
1852     print(
1853         " 主策略可接受覆盖率下限: "
1854         f"{result.accepted_safe_coverage_floor:.6%}"
1855     )
1856     print(f" 主策略最优候选点数量: {len(result.near_optimal_candidates)}")
1857     print(
1858         " 最高可靠覆盖近优候选点数量: "
1859         f"{len(result.safe_near_optimal_candidates)}"
1860     )
1861     print(
1862         " 可靠性可行区域网格点数量: "
1863         f"{len(result.reliability_feasible_candidates)}"
1864     )
1865     print(
1866         " 局部加密收敛: "
1867         f"{ '是' if result.refinement_converged else '否'}, "
1868         f" 最终分辨率 {result.refinement_resolution:.3f} m, "
1869         f" 最优点变化 {result.refinement_position_change:.3f} m, "
1870         " 稳健直径相对变化 "
1871         f"{result.refinement_diameter_relative_change:.3%}"
1872     )
1873     if result.numerical_calibration_applied:
1874         print(
1875             " 数值误差标定比例: "
1876             f"{result.numerical_error_ratio:.4%} (默认与加密配置最大相对差) "

```

```

1877     )
1878     for record in result.numerical_stability_records:
1879         print(
1880             f" {record.role}: S2=({record.point[0]:.3f}, "
1881             f"{record.point[1]:.3f}), J 粗/默认/加密 ="
1882             f"{record.coarse_robust_diameter:.3f}/"
1883             f"{record.default_robust_diameter:.3f}/"
1884             f"{record.fine_robust_diameter:.3f} m"
1885         )
1886     else:
1887         print(
1888             " 数值误差标定: 已跳过; 使用后备容差比例 "
1889             f"{result.numerical_error_ratio:.4%}"
1890         )
1891     print(f" 主策略最优 S2: ({best.point[0]:.6f}, {best.point[1]:.6f})")
1892     print(f" 局部坐标 (u, v): ({best.local_u:.6f}, {best.local_v:.6f})")
1893     print(f" 安全覆盖率: {best.safe_coverage:.6%}")
1894     print(f" 接收不确定覆盖率: {best.uncertain_coverage:.6%}")
1895     print(f" 必然失效覆盖率: {best.certain_failure:.6%}")
1896     print(f" 角度惩罚: {best.angle_penalty:.6f}")
1897     print(f" 离散几何验证最坏测向直径: {best.validated_bearing_worst:.6f} m")
1898     print(f" 过强信号后验直径: {best.strong_signal_diameter:.6f} m")
1899     print(f" 无信号后验直径: {best.no_signal_diameter:.6f} m")
1900     print(f" 离散验证后的稳健后验直径: {best.validated_robust_diameter:.6f} m")
1901     pareto_best = result.pareto_best_candidate
1902     print(
1903         " 辅助帕累托加权折中 S2: "
1904         f"({pareto_best.point[0]:.6f}, {pareto_best.point[1]:.6f})"
1905     )
1906     print(f" 辅助折中点综合评分: {pareto_best.validated_score:.6f}")
1907     print(f" 辅助帕累托近优点数量: {len(result.pareto_near_optimal_candidates)}")
1908     print(f" 候选点评价数据: {candidate_csv_path.resolve()}")
1909     print(f" 选点结果图: {selection_figure_path.resolve()}")
1910     print(f" 可靠性权衡数据: {tradeoff_csv_path.resolve()}")
1911     print(f" 可靠性权衡曲线: {tradeoff_figure_path.resolve()}")
1912
1913
1914 if __name__ == "__main__":
1915     main()

```

B.2 决策模块：q2_decision.py

```

1  """ 问题 2 的候选筛选与最终决策。"""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from math import isfinite
7  from typing import Sequence
8
9  import numpy as np
10
11
12 @dataclass(frozen=True)
13 class TradeoffEnvelopePoint:
14     allowed_loss: float
15     actual_loss: float
16     robust_diameter: float
17     best_index: int
18
19
20 @dataclass(frozen=True)
21 class FinalDecision:
22     pareto_mask: np.ndarray
23     maximum_safe_coverage: float
24     selected_reliability_loss: float
25     accepted_safe_coverage_floor: float
26     improvement_tolerance: float
27     tradeoff_points: tuple[TradeoffEnvelopePoint, ...]
28     best_index: int
29     near_optimal_indices: tuple[int, ...]
30     safe_near_optimal_indices: tuple[int, ...]
31     weighted_best_index: int
32     weighted_near_optimal_indices: tuple[int, ...]
33
34
35 def _validate_same_length(*arrays: np.ndarray) -> int:
36     sizes = {len(np.asarray(array)) for array in arrays}
37     if len(sizes) != 1:
38         raise ValueError(" 候选点指标数组长度必须一致")
39     n = sizes.pop()
40     if n == 0:
41         raise ValueError(" 候选点集不能为空")
42     return n
43
44
45 def _clean_reliability_loss_levels(
46     levels: Sequence[float],
47 ) -> tuple[float, ...]:
48     values = tuple(sorted({float(level) for level in levels}))
49     if not values or abs(values[0]) > 1e-12:
50         raise ValueError(" 可靠性损失层级必须从 0 开始")
51     if any(not isfinite(level) or not 0.0 <= level < 1.0 for level in values):
52         raise ValueError(" 可靠性损失层级必须为 [0, 1) 内的有限数")
53     return values
54
55
56 def pareto_mask(
57     safe_coverage: np.ndarray,
58     certain_failure: np.ndarray,
59     diameter_score: np.ndarray,
60     angle_penalty: np.ndarray,
61 ) -> np.ndarray:
62     safe = np.asarray(safe_coverage, dtype=float)
63     failure = np.asarray(certain_failure, dtype=float)
64     diameter = np.asarray(diameter_score, dtype=float)
65     angle = np.asarray(angle_penalty, dtype=float)
66     n = _validate_same_length(safe, failure, diameter, angle)
67     if any(
68         np.any(~np.isfinite(array))
69         for array in (safe, failure, diameter, angle)
70     ):
71         raise ValueError(" 帕累托指标不能包含非有限数")
72
73     mask = np.ones(n, dtype=bool)
74     eps = 1e-12
75     for i in range(n):
76         not_worse = (
77             (safe >= safe[i] - eps)
78             & (failure <= failure[i] + eps)
79             & (diameter <= diameter[i] + eps)
80             & (angle <= angle[i] + eps)
81         )
82         better = (
83             (safe > safe[i] + eps)
84             | (failure < failure[i] - eps)
85             | (diameter < diameter[i] - eps)
86             | (angle < angle[i] - eps)
87         )
88         if np.any(not_worse & better):
89             mask[i] = False
90     return mask
91
92
93 def select_validation_indices(
94     safe_coverage: np.ndarray,
95     proxy_diameter: np.ndarray,
96     candidate_points: np.ndarray,
97     validation_count: int,

```

```

98     reliability_loss_levels: Sequence[float],
99 ) -> tuple[int, ...]:
100     safe = np.asarray(safe_coverage, dtype=float)
101     diameter = np.asarray(proxy_diameter, dtype=float)
102     points = np.asarray(candidate_points, dtype=float)
103     n = _validate_same_length(safe, diameter, points)
104     if points.ndim != 2 or points.shape[1] != 2:
105         raise ValueError(" 候选点坐标必须是 n×2 数组")
106     if any(np.any(~np.isfinite(array)) for array in (safe, diameter, points)):
107         raise ValueError(" 验证分层指标不能包含非有限数")
108     if not isinstance(validation_count, int) or validation_count <= 0:
109         raise ValueError(" 验证候选数必须为正整数")
110     levels = _clean_reliability_loss_levels(reliability_loss_levels)
111
112     take = min(validation_count, n)
113     max_safe = float(np.max(safe))
114     losses = np.maximum(0.0, max_safe - safe)
115     bands: list[np.ndarray] = []
116     prev = 0.0
117     for level_i, level in enumerate(levels):
118         if level_i == 0:
119             group = np.flatnonzero(losses <= level + 1e-12)
120         else:
121             group = np.flatnonzero(
122                 (losses > prev + 1e-12) & (losses <= level + 1e-12)
123             )
124         if not len(group):
125             group = np.flatnonzero(losses <= level + 1e-12)
126         bands.append(group)
127         prev = level
128
129     chosen: list[int] = []
130     chosen_set: set[int] = set()
131
132     visits = [0 for _ in bands]
133
134     def choose(pool: np.ndarray, prefer_proxy: bool) -> int | None:
135         free = [int(idx) for idx in pool if int(idx) not in chosen_set]
136         if not free:
137             return None
138         pool_ids = {int(idx) for idx in pool}
139         chosen_here = [idx for idx in chosen if idx in pool_ids]
140         if prefer_proxy or not chosen_here:
141             return min(free, key=lambda idx: (diameter[idx], idx))
142         chosen_points = points[np.asarray(chosen_here, dtype=int)]
143         return max(
144             free,
145             key=lambda idx: (
146                 float(
147                     np.min(
148                         np.linalg.norm(chosen_points - points[idx], axis=1)
149                     )
150                 ),
151                 -float(diameter[idx]),
152                 -idx,
153             ),
154         )
155
156     while len(chosen) < take:
157         changed = False
158         for band_i, group in enumerate(bands):
159             seen = visits[band_i]
160             prefer_proxy = seen < 2 or seen % 2 == 1
161             idx = choose(group, prefer_proxy)
162             if idx is None:
163                 continue
164             chosen.append(idx)
165             chosen_set.add(idx)
166             visits[band_i] += 1
167             changed = True
168             if len(chosen) >= take:
169                 break
170         if not changed:
171             break
172
173     if len(chosen) < take:
174         all_ids = np.arange(n, dtype=int)
175         while len(chosen) < take:
176             idx = choose(all_ids, prefer_proxy=False)
177             if idx is None:
178                 break
179             chosen.append(idx)
180             chosen_set.add(idx)
181     return tuple(chosen)
182
183
184 def make_final_decision(
185     safe_coverage: np.ndarray,
186     certain_failure: np.ndarray,
187     robust_diameter: np.ndarray,
188     angle_penalty: np.ndarray,
189     weighted_score: np.ndarray,
190     movement_distance: np.ndarray,
191     reliability_loss_levels: Sequence[float],
192     improvement_tolerance_ratio: float,
193     diameter_tolerance_ratio: float,
194     score_tolerance: float,
195     maximum_safe_coverage: float | None = None,
196 ) -> FinalDecision:

```

```

197 safe = np.asarray(safe_coverage, dtype=float)
198 failure = np.asarray(certain_failure, dtype=float)
199 diameter = np.asarray(robust_diameter, dtype=float)
200 angle = np.asarray(angle_penalty, dtype=float)
201 score = np.asarray(weighted_score, dtype=float)
202 movement = np.asarray(movement_distance, dtype=float)
203 _validate_same_length(safe, failure, diameter, angle, score, movement)
204 if any(
205     np.any(~np.isfinite(array))
206     for array in (safe, failure, diameter, angle, score, movement)
207 ):
208     raise ValueError(" 决策指标不能包含非有限数")
209 levels = _clean_reliability_loss_levels(reliability_loss_levels)
210 if (
211     not isfinite(improvement_tolerance_ratio)
212     or improvement_tolerance_ratio < 0.0
213 ):
214     raise ValueError(" 定位改善数值容差比例必须为有限非负数")
215 if not isfinite(diameter_tolerance_ratio) or diameter_tolerance_ratio < 0.0:
216     raise ValueError(" 直径容差比例必须为有限非负数")
217 if not isfinite(score_tolerance) or score_tolerance < 0.0:
218     raise ValueError(" 评分容差必须为有限非负数")
219
220 observed_maximum = float(np.max(safe))
221 maximum_safe = (
222     observed_maximum
223     if maximum_safe_coverage is None
224     else float(maximum_safe_coverage)
225 )
226 if (
227     not isfinite(maximum_safe)
228     or maximum_safe < observed_maximum - 1e-10
229     or not 0.0 <= maximum_safe <= 1.0 + 1e-10
230 ):
231     raise ValueError(" 最大安全覆盖率必须覆盖全部验证候选且位于 [0, 1]")
232
233 losses = np.maximum(0.0, maximum_safe - safe)
234 maximum_allowed_loss = levels[-1]
235 actual_levels = sorted(
236     {
237         round(float(loss), 12)
238         for loss in losses
239         if loss <= maximum_allowed_loss + 1e-12
240     }
241 )
242 if not actual_levels or actual_levels[0] > 1e-12:
243     raise RuntimeError(" 最高可靠覆盖层没有经过精验证的候选点")
244
245 tradeoff_points: List[TradeoffEnvelopePoint] = []
246 for allowed_loss in actual_levels:
247     floor = max(0.0, maximum_safe - allowed_loss)
248     eligible = [
249         index
250         for index in range(len(safe))
251         if safe[index] >= floor - 1e-12
252     ]
253     if not eligible:
254         continue
255     best_index = min(
256         eligible,
257         key=lambda index: (
258             diameter[index],
259             failure[index],
260             angle[index],
261             movement[index],
262         ),
263     )
264     tradeoff_points.append(
265         TradeoffEnvelopePoint(
266             allowed_loss=float(allowed_loss),
267             actual_loss=float(maximum_safe - safe[best_index]),
268             robust_diameter=float(diameter[best_index]),
269             best_index=int(best_index),
270         )
271     )
272 baseline_diameter = tradeoff_points[0].robust_diameter
273 improvement_tolerance = improvement_tolerance_ratio * baseline_diameter
274 minimum_diameter = min(point.robust_diameter for point in tradeoff_points)
275 if baseline_diameter - minimum_diameter <= improvement_tolerance + 1e-12:
276     selected_index = tradeoff_points[0].best_index
277 else:
278     selected_index = min(
279         (
280             index
281             for index in range(len(safe))
282             if losses[index] <= maximum_allowed_loss + 1e-12
283             and diameter[index]
284             <= minimum_diameter + improvement_tolerance + 1e-12
285         ),
286         key=lambda index: (
287             losses[index],
288             diameter[index],
289             failure[index],
290             angle[index],
291             movement[index],
292         ),
293     )
294 selected_loss = float(losses[selected_index])
295

```

```

296     safe_floor = max(0.0, maximum_safe - selected_loss)
297     eligible = [
298         index for index in range(len(safe)) if safe[index] >= safe_floor - 1e-12
299     ]
300     best_index = min(
301         eligible,
302         key=lambda index: (
303             diameter[index],
304             failure[index],
305             angle[index],
306             movement[index],
307         ),
308     )
309     best_diameter = float(diameter[best_index])
310     near_optimal = tuple(
311         index
312         for index in eligible
313         if diameter[index]
314         <= (1.0 + diameter_tolerance_ratio) * best_diameter + 1e-12
315     )
316
317     maximum_reliability = [
318         index for index in range(len(safe)) if safe[index] >= maximum_safe - 1e-12
319     ]
320     safe_best_diameter = min(float(diameter[index]) for index in maximum_reliability)
321     safe_near_optimal = tuple(
322         index
323         for index in maximum_reliability
324         if diameter[index]
325         <= (1.0 + diameter_tolerance_ratio) * safe_best_diameter + 1e-12
326     )
327
328     nondominated = pareto_mask(safe, failure, diameter, angle)
329     pareto_indices = [int(index) for index in np.flatnonzero(nondominated)]
330     weighted_best_index = min(pareto_indices, key=lambda index: score[index])
331     weighted_best_score = float(score[weighted_best_index])
332     weighted_near_optimal = tuple(
333         index
334         for index in pareto_indices
335         if score[index] <= weighted_best_score + score_tolerance + 1e-12
336     )
337     return FinalDecision(
338         pareto_mask=nondominated,
339         maximum_safe_coverage=maximum_safe,
340         selected_reliability_loss=float(selected_loss),
341         accepted_safe_coverage_floor=safe_floor,
342         improvement_tolerance=float(improvement_tolerance),
343         tradeoff_points=tuple(tradeoff_points),
344         best_index=best_index,
345         near_optimal_indices=near_optimal,
346         safe_near_optimal_indices=safe_near_optimal,
347         weighted_best_index=weighted_best_index,
348         weighted_near_optimal_indices=weighted_near_optimal,
349     )

```

B.3 评价模块：q2_evaluation.py

```

1  """ 问题 2 候选点评分。"""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from math import isfinite
7
8  import numpy as np
9
10
11 @dataclass(frozen=True)
12 class ScoreBreakdown:
13     not_safe: float
14     certain_failure: float
15     relative_diameter: float
16     angle_penalty: float
17     total: float
18
19
20 def _validate_weights(weights: tuple[float, float, float, float]) -> None:
21     w = np.asarray(weights, dtype=float)
22     if np.any(~np.isfinite(w)) or np.any(w < 0.0):
23         raise ValueError(" 评分权重必须为有限非负数")
24     if not np.isclose(float(w.sum()), 1.0, atol=1e-12):
25         raise ValueError(" 四项评分权重之和必须等于 1")
26
27
28 def calculate_weighted_scores(
29     safe_coverage: np.ndarray,
30     certain_failure: np.ndarray,
31     robust_diameter: np.ndarray,
32     angle_penalty: np.ndarray,
33     reference_diameter: float,
34     weights: tuple[float, float, float, float],
35 ) -> np.ndarray:
36     parts = [
37         np.asarray(safe_coverage, dtype=float),
38         np.asarray(certain_failure, dtype=float),
39         np.asarray(robust_diameter, dtype=float),
40         np.asarray(angle_penalty, dtype=float),
41     ]
42     if len({array.shape for array in parts}) != 1:
43         raise ValueError(" 四项评价指标必须具有相同形状")
44     if not isfinite(reference_diameter) or reference_diameter <= 0.0:
45         raise ValueError(" 参考直径必须为有限正数")
46     if any(np.any(~np.isfinite(array)) for array in parts):
47         raise ValueError(" 评价指标不能包含非有限数")
48     _validate_weights(weights)
49
50     s, f, d, a = parts
51     relative_diameter = np.clip(d / reference_diameter, 0.0, 1.0)
52     return (
53         weights[0] * (1.0 - s)
54         + weights[1] * f
55         + weights[2] * relative_diameter
56         + weights[3] * a
57     )
58
59
60 def evaluate_candidate_score(
61     safe_coverage: float,
62     certain_failure: float,
63     robust_diameter: float,
64     angle_penalty: float,
65     reference_diameter: float,
66     weights: tuple[float, float, float, float],
67 ) -> ScoreBreakdown:
68     total = calculate_weighted_scores(
69         np.asarray([safe_coverage], dtype=float),
70         np.asarray([certain_failure], dtype=float),
71         np.asarray([robust_diameter], dtype=float),
72         np.asarray([angle_penalty], dtype=float),
73         reference_diameter,
74         weights,
75     )[0]
76     return ScoreBreakdown(
77         not_safe=1.0 - float(safe_coverage),
78         certain_failure=float(certain_failure),
79         relative_diameter=float(
80             np.clip(robust_diameter / reference_diameter, 0.0, 1.0)
81         ),
82         angle_penalty=float(angle_penalty),
83         total=float(total),
84     )

```

B.4 结果输出模块：q2_evidence.py

```

1  """ 问题 2 结果与图表输出。"""
2
3  from __future__ import annotations
4
5  import csv
6  from dataclasses import dataclass
7  from pathlib import Path
8  from typing import TYPE_CHECKING, Any
9
10 import numpy as np
11
12 if TYPE_CHECKING:
13     from q2 import Q2Result
14
15
16 @dataclass(frozen=True)
17 class ReliabilityTradeoffPoint:
18     allowed_loss: float
19     actual_loss: float
20     robust_diameter: float
21     relative_diameter: float
22     safe_coverage: float
23     point: tuple[float, float]
24
25
26 def build_reliability_tradeoff(
27     result: Q2Result,
28 ) -> tuple[ReliabilityTradeoffPoint, ...]:
29     if not result.reliability_tradeoff:
30         raise ValueError(" 没有可用的可靠性权衡数据")
31     baseline = result.reliability_tradeoff[0].robust_diameter
32     return tuple(
33         ReliabilityTradeoffPoint(
34             allowed_loss=envelope.allowed_loss,
35             actual_loss=envelope.actual_loss,
36             robust_diameter=envelope.robust_diameter,
37             relative_diameter=(
38                 1.0
39                 if baseline <= 1e-12
40                 else envelope.robust_diameter / baseline
41             ),
42             safe_coverage=result.validated_candidates[
43                 envelope.best_index
44             ].safe_coverage,
45             point=result.validated_candidates[envelope.best_index].point,
46         )
47         for envelope in result.reliability_tradeoff
48     )
49
50
51 def write_candidate_csv(result: Q2Result, output_path: Path) -> None:
52     output_path.parent.mkdir(parents=True, exist_ok=True)
53     fieldnames = [
54         "x 坐标 _ 米",
55         "y 坐标 _ 米",
56         "沿首次示向位移 u_ 米",
57         "侧向位移 v_ 米",
58         "安全覆盖率",
59         "不确定覆盖率",
60         "必然失效覆盖率",
61         "交会角惩罚",
62         "粗筛最坏直径 _ 米",
63         "粗筛加权评分",
64         "是否粗筛帕累托点",
65         "局部加密层级",
66         "验证最坏测向直径 _ 米",
67         "过强信号后验直径 _ 米",
68         "无信号后验直径 _ 米",
69         "验证稳健直径 _ 米",
70         "验证加权评分",
71         "是否验证帕累托点",
72         "是否可靠性可行区域",
73         "是否主候选区域",
74         "是否最高可靠覆盖近优区域",
75         "是否帕累托加权近优点",
76         "数值稳定性检查角色",
77         "粗配置稳健直径 _ 米",
78         "加密配置稳健直径 _ 米",
79         "默认与加密相对差",
80     ]
81     near_points = {candidate.point for candidate in result.near_optimal_candidates}
82     safe_near_points = {
83         candidate.point for candidate in result.safe_near_optimal_candidates
84     }
85     pareto_near_points = {
86         candidate.point for candidate in result.pareto_near_optimal_candidates
87     }
88     feasible_points = {
89         candidate.point for candidate in result.reliability_feasible_candidates
90     }
91     validated_by_point = {
92         candidate.point: candidate for candidate in result.validated_candidates
93     }
94     stability_by_point = {
95         record.point: record for record in result.numerical_stability_records
96     }
97     with output_path.open("w", encoding="utf-8-sig", newline="") as handle:

```

```

98     writer = csv.DictWriter(handle, fieldnames=fieldnames)
99     writer.writeheader()
100     for candidate in result.candidates:
101         validated = validated_by_point.get(candidate.point)
102         stability = stability_by_point.get(candidate.point)
103         writer.writerow(
104             {
105                 "x 坐标 _ 米": candidate.point[0],
106                 "y 坐标 _ 米": candidate.point[1],
107                 "沿首次示向位移 u_ 米": candidate.local_u,
108                 "侧向位移 v_ 米": candidate.local_v,
109                 "安全覆盖率": candidate.safe_coverage,
110                 "不确定覆盖率": candidate.uncertain_coverage,
111                 "必然失效覆盖率": candidate.certain_failure,
112                 "交会角惩罚": candidate.angle_penalty,
113                 "粗筛最坏直径 _ 米": candidate.proxy_diameter,
114                 "粗筛加权评分": candidate.proxy_score,
115                 "是否粗筛帕累托点": int(candidate.pareto),
116                 "局部加密层级": candidate.refinement_level,
117                 "验证最坏测向直径 _ 米": (
118                     "" if validated is None else validated.validated_bearing_worst
119                 ),
120                 "过强信号后验直径 _ 米": (
121                     "" if validated is None else validated.strong_signal_diameter
122                 ),
123                 "无信号后验直径 _ 米": (
124                     "" if validated is None else validated.no_signal_diameter
125                 ),
126                 "验证稳健直径 _ 米": (
127                     "" if validated is None else validated.validated_robust_diameter
128                 ),
129                 "验证加权评分": (
130                     "" if validated is None else validated.validated_score
131                 ),
132                 "是否验证帕累托点": (
133                     0 if validated is None else int(validated.validated_pareto)
134                 ),
135                 "是否可靠性可行区域": int(candidate.point in feasible_points),
136                 "是否主候选区域": int(candidate.point in near_points),
137                 "是否最高可靠覆盖近优区域": int(
138                     candidate.point in safe_near_points
139                 ),
140                 "是否帕累托加权近优点": int(
141                     candidate.point in pareto_near_points
142                 ),
143                 "数值稳定性检查角色": (
144                     "" if stability is None else stability.role
145                 ),
146                 "粗配置稳健直径 _ 米": (
147                     "" if stability is None else stability.coarse_robust_diameter
148                 ),
149                 "加密配置稳健直径 _ 米": (
150                     "" if stability is None else stability.fine_robust_diameter
151                 ),
152                 "默认与加密相对差": (
153                     ""
154                     if stability is None
155                     else stability.default_fine_relative_difference
156                 ),
157             }
158         )
159
160
161 def write_reliability_tradeoff_csv(result: Q2Result, output_path: Path) -> None:
162     output_path.parent.mkdir(parents=True, exist_ok=True)
163     fieldnames = [
164         "允许可靠性下降 _ 百分点",
165         "实际可靠性下降 _ 百分点",
166         "最优稳健定位直径 _ 米",
167         "相对稳健定位直径",
168         "定位直径改善 _ 百分比",
169         "对应安全覆盖率",
170         "对应点 x_ 米",
171         "对应点 y_ 米",
172         "是否自动选中",
173         "定位改善数值容差 _ 米",
174         "数值误差标定比例",
175         "是否执行数值加密标定",
176         "精验证候选总数",
177     ]
178     with output_path.open("w", encoding="utf-8-sig", newline="") as handle:
179         writer = csv.DictWriter(handle, fieldnames=fieldnames)
180         writer.writeheader()
181         for point in build_reliability_tradeoff(result):
182             writer.writerow(
183                 {
184                     "允许可靠性下降 _ 百分点": 100.0 * point.allowed_loss,
185                     "实际可靠性下降 _ 百分点": 100.0 * point.actual_loss,
186                     "最优稳健定位直径 _ 米": point.robust_diameter,
187                     "相对稳健定位直径": point.relative_diameter,
188                     "定位直径改善 _ 百分比": 100.0
189                     * (1.0 - point.relative_diameter),
190                     "对应安全覆盖率": point.safe_coverage,
191                     "对应点 x_ 米": point.point[0],
192                     "对应点 y_ 米": point.point[1],
193                     "是否自动选中": int(
194                         abs(point.allowed_loss - result.selected_reliability_loss)
195                         <= 1e-12
196                     ),
197                 }
198             )

```

```

197         "定位改善数值容差 _ 米": (
198             result.tradeoff_improvement_tolerance
199         ),
200         "数值误差标定比例": result.numerical_error_ratio,
201         "是否执行数值加密标定": int(
202             result.numerical_calibration_applied
203         ),
204         "精验证候选总数": len(result.validated_candidates),
205     }
206 )
207
208
209 def _configure_chinese_font(plt: Any) -> None:
210     plt.rcParams["font.sans-serif"] = [
211         "Microsoft YaHei",
212         "SimHei",
213         "DejaVu Sans",
214     ]
215     plt.rcParams["axes.unicode_minus"] = False
216
217
218 def plot_result(result: Q2Result, output_path: Path) -> None:
219     import matplotlib.pyplot as plt
220     from scipy.interpolate import griddata
221     from scipy.spatial import ConvexHull
222
223     _configure_chinese_font(plt)
224     candidates = result.candidates
225     x = np.array([candidate.point[0] for candidate in candidates])
226     y = np.array([candidate.point[1] for candidate in candidates])
227     safe = np.array([candidate.safe_coverage for candidate in candidates])
228     validated = result.validated_candidates
229     validated_safe = np.array([candidate.safe_coverage for candidate in validated])
230     validated_diameter = np.array(
231         [float(candidate.validated_robust_diameter) for candidate in validated]
232     )
233     validated_pareto = np.array(
234         [candidate.validated_pareto for candidate in validated], dtype=bool
235     )
236
237     figure, axes = plt.subplots(1, 2, figsize=(14, 6))
238     display_x = np.linspace(float(x.min()), float(x.max()), 180)
239     display_y = np.linspace(float(y.min()), float(y.max()), 180)
240     grid_x, grid_y = np.meshgrid(display_x, display_y)
241     display_safe = griddata((x, y), safe, (grid_x, grid_y), method="linear")
242     missing = ~np.isfinite(display_safe)
243     if np.any(missing):
244         nearest = griddata((x, y), safe, (grid_x, grid_y), method="nearest")
245         display_safe[missing] = nearest[missing]
246     safe_min = float(safe.min())
247     safe_max = float(safe.max())
248     if np.isclose(safe_min, safe_max):
249         color_levels = np.linspace(safe_min - 1e-6, safe_max + 1e-6, 3)
250     else:
251         color_levels = np.linspace(safe_min, safe_max, 13)
252     spatial = axes[0].contourf(
253         grid_x,
254         grid_y,
255         display_safe,
256         levels=color_levels,
257         cmap="Blues",
258         alpha=0.88,
259     )
260     figure.colorbar(
261         spatial,
262         ax=axes[0],
263         label="安全覆盖面积比例",
264     )
265     if safe_min() < result.accepted_safe_coverage_floor < safe.max():
266         axes[0].contour(
267             grid_x,
268             grid_y,
269             display_safe,
270             levels=[result.accepted_safe_coverage_floor],
271             colors=["tab:blue"],
272             linewidths=1.8,
273         )
274     feasible = result.reliability_feasible_candidates
275     axes[0].scatter(
276         [item.point[0] for item in feasible],
277         [item.point[1] for item in feasible],
278         s=8,
279         facecolors="none",
280         edgecolors="tab:blue",
281         linewidths=0.35,
282         alpha=0.45,
283         label=r"可靠性可行域  $A_{\{\varepsilon\}}$  节点",
284     )
285     axes[0].scatter(
286         *result.first_station,
287         marker="s",
288         s=80,
289         color="black",
290         label="S1",
291     )
292     axes[0].scatter(
293         *result.best_candidate.point,
294         marker="*",
295         s=220,

```

```

296     color="red",
297     edgecolor="black",
298     label=" 主策略最优 S2",
299 )
300 near = result.near_optimal_candidates
301 near_coordinates = np.asarray([item.point for item in near], dtype=float)
302 if len(near_coordinates) >= 3:
303     unassigned = set(range(len(near_coordinates)))
304     connection_radius = max(5.0 * result.refinement_resolution, 1e-8)
305     clusters: list[list[int]] = []
306     while unassigned:
307         seed = unassigned.pop()
308         cluster = [seed]
309         frontier = [seed]
310         while frontier:
311             current = frontier.pop()
312             neighbors = [
313                 index
314                 for index in tuple(unassigned)
315                 if np.linalg.norm(
316                     near_coordinates[index] - near_coordinates[current]
317                 )
318                 <= connection_radius
319             ]
320             for index in neighbors:
321                 unassigned.remove(index)
322                 cluster.append(index)
323                 frontier.append(index)
324             clusters.append(cluster)
325     region_label_used = False
326     for cluster in clusters:
327         if len(cluster) < 3:
328             continue
329         cluster_points = near_coordinates[cluster]
330         try:
331             hull = ConvexHull(cluster_points)
332         except Exception:
333             continue
334         polygon = cluster_points[hull.vertices]
335         axes[0].fill(
336             polygon[:, 0],
337             polygon[:, 1],
338             facecolor="tab:red",
339             edgecolor="tab:red",
340             alpha=0.13,
341             linewidth=1.0,
342             label=(r" 近优区域 $C^*$ 数值近似" if not region_label_used else None),
343         )
344         region_label_used = True
345     axes[0].scatter(
346         [item.point[0] for item in near],
347         [item.point[1] for item in near],
348         facecolors="none",
349         edgecolors="red",
350         s=45,
351         linewidths=0.8,
352         label=r" 近优区域 $C^*$ 的精确节点",
353     )
354     safe_near = result.safe_near_optimal_candidates
355     if safe_near:
356         axes[0].scatter(
357             [item.point[0] for item in safe_near],
358             [item.point[1] for item in safe_near],
359             facecolors="none",
360             edgecolors="tab:blue",
361             marker="s",
362             s=55,
363             linewidths=0.9,
364             label=" 最高可靠覆盖近优区域",
365         )
366     pareto_best = result.pareto_best_candidate
367     axes[0].scatter(
368         *pareto_best.point,
369         marker="D",
370         s=70,
371         color="tab:purple",
372         edgecolor="black",
373         label=" 辅助帕累托折中点",
374     )
375     axes[0].set_aspect("equal", adjustable="box")
376     axes[0].set_title(
377         " 主结论: 可靠性可行域、最优点与近优区域\n"
378         f" 自动选择可靠性损失 {100.0 * result.selected_reliability_loss:.2f} 个百分点"
379     )
380     axes[0].set_xlabel("x / m")
381     axes[0].set_ylabel("y / m")
382     axes[0].grid(alpha=0.2)
383     axes[0].legend(loc="best")
384
385     axes[1].scatter(
386         validated_safe[~validated_pareto],
387         validated_diameter[~validated_pareto],
388         c="lightgray",
389         s=25,
390         label=" 验证后被支配点",
391     )
392     pareto_failure = np.array(
393         [candidate.certain_failure for candidate in validated]
394     )[validated_pareto]

```

```

395 if len(pareto_failure) and np.ptp(pareto_failure) > 1e-12:
396     pareto_scatter = axes[1].scatter(
397         validated_safe[validated_pareto],
398         validated_diameter[validated_pareto],
399         c=pareto_failure,
400         cmap="plasma",
401         s=38,
402         label=" 验证后四目标帕累托点",
403     )
404     figure.colorbar(
405         pareto_scatter, ax=axes[1], label=" 必然失效覆盖比例"
406     )
407 else:
408     constant_failure = float(pareto_failure[0]) if len(pareto_failure) else 0.0
409     axes[1].scatter(
410         validated_safe[validated_pareto],
411         validated_diameter[validated_pareto],
412         color="#cf3f72",
413         s=38,
414         label=(
415             " 验证后四目标帕累托点"
416             f" (q_fail={constant_failure:.3f}) "
417         ),
418     )
419     axes[1].scatter(
420         result.best_candidate.safe_coverage,
421         result.best_candidate.validated_robust_diameter,
422         marker="*",
423         s=220,
424         color="red",
425         edgecolor="black",
426         label=" 主策略最优 S2",
427     )
428     axes[1].scatter(
429         pareto_best.safe_coverage,
430         pareto_best.validated_robust_diameter,
431         marker="D",
432         s=70,
433         color="tab:purple",
434         edgecolor="black",
435         label=" 辅助帕累托折中点",
436     )
437     axes[1].axvline(
438         result.accepted_safe_coverage_floor,
439         color="tab:blue",
440         linestyle="--",
441         linewidth=1.1,
442         label=" 主策略可靠性下限",
443     )
444     axes[1].set_title(" 辅助：精验证后的四目标帕累托二维投影")
445     axes[1].set_xlabel(" 安全覆盖率")
446     axes[1].set_ylabel(r" 稳健后验直径  $\bar{J}_{\{\text{rm rob}\}} / m$ ")
447     axes[1].grid(alpha=0.2)
448     axes[1].legend(loc="best")
449
450 figure.tight_layout()
451 output_path.parent.mkdir(parents=True, exist_ok=True)
452 figure.savefig(output_path, dpi=180, bbox_inches="tight")
453 plt.close(figure)
454
455
456 def plot_reliability_tradeoff(result: Q2Result, output_path: Path) -> None:
457     import matplotlib.pyplot as plt
458
459     _configure_chinese_font(plt)
460     points = build_reliability_tradeoff(result)
461     x = np.asarray([100.0 * point.allowed_loss for point in points])
462     y = np.asarray([point.relative_diameter for point in points])
463     selected_loss = 100.0 * result.selected_reliability_loss
464     selected_index = int(np.argmin(np.abs(x - selected_loss)))
465
466     figure, axis = plt.subplots(figsize=(8, 5.5))
467     axis.step(
468         x,
469         y,
470         where="post",
471         color="tab:blue",
472         linewidth=2.0,
473         label=" 最佳可达相对定位直径",
474     )
475     axis.scatter(x, y, s=28, color="tab:blue")
476     axis.axhline(1.0, color="gray", linestyle="--", linewidth=1.0)
477     axis.axvline(
478         selected_loss,
479         color="tab:red",
480         linestyle="--",
481         linewidth=1.2,
482         label=f" 自动选择 {selected_loss:.2f} 个百分点",
483     )
484     axis.scatter(
485         x[selected_index],
486         y[selected_index],
487         marker="*",
488         s=180,
489         color="tab:red",
490         edgecolor="black",
491         zorder=4,
492         label=" 最终决策位置",
493     )

```

```
494     axis.set_title(  
495         " 可靠性与定位精度权衡曲线\n"  
496         f" 基于 {len(result.validated_candidates)} 个分层及局部精验证候选"  
497     )  
498     axis.set_xlabel(" 允许安全覆盖率下降 / 百分点")  
499     axis.set_ylabel(" 相对稳健定位直径 (不降低可靠性时 =1) ")  
500     axis.grid(alpha=0.25)  
501     axis.legend(loc="best")  
502     figure.tight_layout()  
503     output_path.parent.mkdir(parents=True, exist_ok=True)  
504     figure.savefig(output_path, dpi=180, bbox_inches="tight")  
505     plt.close(figure)
```

附录 C 问题三程序: q3.py

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 from __future__ import annotations
5
6 import itertools
7 import json
8 import math
9 import random
10 import socket
11 import time
12 from dataclasses import dataclass, field
13 from pathlib import Path
14 from typing import Optional
15 from urllib.error import HTTPError, URLError
16 from urllib.request import Request, urlopen
17
18
19 BASE_URL = "http://127.0.0.1:2026"
20 ROBOT_ID = "*****"
21 ARENA_ID = "default"
22
23 HTTP_TIMEOUT_S = 10
24 HTTP_RETRIES = 3
25
26 TRACE_PATH = Path("q3_v3_trace.jsonl")
27 SUMMARY_PATH = Path("q3_v3_summary.json")
28
29
30 TARGET_RADIUS = 1800.0
31 MIN_RECEIVE_RADIUS = 1000.0
32 MAX_RECEIVE_RADIUS = 1500.0
33
34 BEARING_ERROR_DEG = 1.0
35 NEAR_RADIUS = 5.0
36 CLEAR_RADIUS = 20.0
37
38 SPEED = 5.0
39 MEASURE_SECONDS = 5.0
40 SWITCH_SECONDS = 1.0
41 CLEAR_SUCCESS_SECONDS = 5.0
42 CLEAR_FAILURE_SECONDS = 3.0
43
44 CHANNEL_COUNT = 20
45 SOURCE_MAX = 16
46
47
48 EW_OUTER_COUNT = 7
49 EW_RING_RADIUS = 1010.0
50
51 Q2_FAST_SIDE_M = 80.0
52
53 RECOVERY_FORWARD_M = 200.0
54 RECOVERY_SIDE_M = 350.0
55
56 STRONG_PROBE_RADIUS_M = 80.0
57 WEAK_PROBE_RADIUS_M = 180.0
58
59 GUARANTEED_CLEAR_BONUS_M = 320.0
60 STRONG_PROBE_BONUS_M = 150.0
61 WEAK_PROBE_BONUS_M = 60.0
62 EW_PROGRESS_BONUS_M = 15.0
63 Q2_FAST_INFO_BONUS_M = 45.0
64
65 MAX_OPPORTUNISTIC_PER_EW = 4
66 MIN_OPPORTUNISTIC_BASELINE_M = 220.0
67 MIN_OPPORTUNISTIC_SIN = 0.20
68 MAX_OPPORTUNISTIC_CENTER_DISTANCE_M = 1650.0
69
70 DENSE_FOUND_THRESHOLD = 13
71 DENSE_MAX_OPPORTUNISTIC_PER_EW = 5
72
73 GEOM_PROBE_MIN_SIN = 0.65
74 GEOM_PROBE_MAX_ERROR_M = 24.0
75 GEOM_PROBE_BONUS_M = 170.0
76
77 FAILED_CLEAR_AVOID_RADIUS_M = 8.0
78
79 MAX_LOGICAL_ACTIONS = 2500
80
81 MANDATORY_ROUTE_MAX_NODES = 12
82 MANDATORY_ROUTE_MIN_REGRET_M = 250.0
83
84
85 Point = tuple[float, float]
86
87
88 def distance(a: Point, b: Point) -> float:
89     return math.hypot(a[0] - b[0], a[1] - b[1])
90
91
92 def cross(a: Point, b: Point) -> float:
93     return a[0] * b[1] - a[1] * b[0]
94
95
96 def sub(a: Point, b: Point) -> Point:
97     return a[0] - b[0], a[1] - b[1]

```

```

98
99
100 def clip_left(
101     polygon: list[Point],
102     line_point: Point,
103     line_direction: Point,
104     eps: float = 1e-9,
105 ) -> list[Point]:
106     if not polygon:
107         return []
108
109     def value(p: Point) -> float:
110         return cross(line_direction, sub(p, line_point))
111
112     out: list[Point] = []
113     last = polygon[-1]
114     last_value = value(last)
115     last_inside = last_value >= -eps
116
117     for point in polygon:
118         point_value = value(point)
119         point_inside = point_value >= -eps
120
121         if last_inside != point_inside:
122             span = last_value - point_value
123             if abs(span) > 1e-15:
124                 t = last_value / span
125                 t = min(1.0, max(0.0, t))
126                 out.append(
127                     (
128                         last[0] + t * (point[0] - last[0]),
129                         last[1] + t * (point[1] - last[1]),
130                     )
131                 )
132
133         if point_inside:
134             out.append(point)
135
136         last = point
137         last_value = point_value
138         last_inside = point_inside
139
140     trimmed: list[Point] = []
141     for p in out:
142         if not trimmed or distance(trimmed[-1], p) > 1e-8:
143             trimmed.append(p)
144
145     if len(trimmed) > 1 and distance(trimmed[0], trimmed[-1]) <= 1e-8:
146         trimmed.pop()
147
148     return trimmed
149
150
151 def circumscribed_disk(
152     center: Point,
153     radius: float,
154     sides: int,
155 ) -> list[Point]:
156     outer_radius = radius / math.cos(math.pi / sides)
157     phase = math.pi / sides
158
159     return [
160         (
161             center[0]
162             + outer_radius * math.cos(phase + 2.0 * math.pi * i / sides),
163             center[1]
164             + outer_radius * math.sin(phase + 2.0 * math.pi * i / sides),
165         )
166         for i in range(sides)
167     ]
168
169
170 def clip_by_convex_polygon(
171     subject: list[Point],
172     clipper: list[Point],
173 ) -> list[Point]:
174     out = subject
175
176     for i in range(len(clipper)):
177         a = clipper[i]
178         b = clipper[(i + 1) % len(clipper)]
179
180         out = clip_left(
181             out,
182             a,
183             (b[0] - a[0], b[1] - a[1]),
184         )
185
186         if not out:
187             break
188
189     return out
190
191
192 TARGET_POLY = circumscribed_disk(
193     (0.0, 0.0),
194     TARGET_RADIUS,
195     72,
196 )

```

```

197
198
199 def update_polygon_with_direction(
200     polygon: Optional[List[Point]],
201     station: Point,
202     bearing_deg: float,
203 ) -> List[Point]:
204     poly = TARGET_POLY.copy() if polygon is None else List(polygon)
205
206     lower = math.radians(bearing_deg - BEARING_ERROR_DEG)
207     upper = math.radians(bearing_deg + BEARING_ERROR_DEG)
208
209     poly = clip_left(
210         poly,
211         station,
212         (math.cos(lower), math.sin(lower)),
213     )
214
215     poly = clip_left(
216         poly,
217         station,
218         (-math.cos(upper), -math.sin(upper)),
219     )
220
221     poly = clip_by_convex_polygon(
222         poly,
223         circumscribed_disk(
224             station,
225             MAX_RECEIVE_RADIUS,
226             32,
227         ),
228     )
229
230     return poly
231
232
233 def polygon_diameter(
234     polygon: List[Point],
235 ) -> tuple[float, tuple[Point, Point]]:
236     if not polygon:
237         raise ValueError(" 空多边形没有直径")
238
239     if len(polygon) == 1:
240         return 0.0, (polygon[0], polygon[0])
241
242     max_sq = -1.0
243     farthest = (polygon[0], polygon[1])
244
245     for i, p in enumerate(polygon):
246         for q in polygon[i + 1:]:
247             sq = (
248                 (p[0] - q[0]) ** 2
249                 + (p[1] - q[1]) ** 2
250             )
251
252             if sq > max_sq:
253                 max_sq = sq
254                 farthest = (p, q)
255
256     return math.sqrt(max_sq), farthest
257
258
259 @dataclass(frozen=True)
260 class Circle:
261     center: Point
262     radius: float
263
264
265 def circle_from_two(a: Point, b: Point) -> Circle:
266     center = (
267         (a[0] + b[0]) / 2.0,
268         (a[1] + b[1]) / 2.0,
269     )
270     return Circle(center, distance(center, a))
271
272
273 def circle_from_three(
274     a: Point,
275     b: Point,
276     c: Point,
277 ) -> Optional[Circle]:
278     ax, ay = a
279     bx, by = b
280     cx, cy = c
281
282     denom = 2.0 * (
283         ax * (by - cy)
284         + bx * (cy - ay)
285         + cx * (ay - by)
286     )
287
288     if abs(denom) <= 1e-12:
289         return None
290
291     a2 = ax * ax + ay * ay
292     b2 = bx * bx + by * by
293     c2 = cx * cx + cy * cy
294
295     ux = (

```

```

296         a2 * (by - cy)
297         + b2 * (cy - ay)
298         + c2 * (ay - by)
299     ) / denom
300
301     uy = (
302         a2 * (cx - bx)
303         + b2 * (ax - cx)
304         + c2 * (bx - ax)
305     ) / denom
306
307     center = (ux, uy)
308     return Circle(center, distance(center, a))
309
310
311 def circle_contains(circle: Circle, point: Point) -> bool:
312     return distance(circle.center, point) <= circle.radius + 1e-7
313
314
315 def minimum_enclosing_circle(points: List[Point]) -> Circle:
316     if not points:
317         raise ValueError(" 空点集没有最小包围圆")
318
319     pts = list(dict.fromkeys(points))
320     random.Random(20260912).shuffle(pts)
321
322     mec: Optional[Circle] = None
323
324     for i, p in enumerate(pts):
325         if mec is not None and circle_contains(mec, p):
326             continue
327
328         mec = Circle(p, 0.0)
329
330         for j in range(i):
331             q = pts[j]
332
333             if circle_contains(mec, q):
334                 continue
335
336             mec = circle_from_two(p, q)
337
338             for k in range(j):
339                 r = pts[k]
340
341                 if circle_contains(mec, r):
342                     continue
343
344                 circle3 = circle_from_three(p, q, r)
345
346                 if circle3 is not None:
347                     mec = circle3
348             else:
349                 pairs = [
350                     circle_from_two(p, q),
351                     circle_from_two(p, r),
352                     circle_from_two(q, r),
353                 ]
354
355                 valid_pairs = [
356                     item
357                     for item in pairs
358                     if circle_contains(item, p)
359                     and circle_contains(item, q)
360                     and circle_contains(item, r)
361                 ]
362
363                 mec = min(
364                     valid_pairs,
365                     key=lambda item: item.radius,
366                 )
367
368     assert mec is not None
369
370     return Circle(
371         mec.center,
372         max(distance(mec.center, p) for p in pts),
373     )
374
375
376 @dataclass(frozen=True)
377 class TargetInfo:
378     center: Point
379     work_radius: float
380     mec_center: Point
381     mec_radius: float
382
383
384 def target_info_from_polygon(polygon: List[Point]) -> TargetInfo:
385     diam, ends = polygon_diameter(polygon)
386
387     center = (
388         (ends[0][0] + ends[1][0]) / 2.0,
389         (ends[0][1] + ends[1][1]) / 2.0,
390     )
391
392     work_radius = diam / 2.0
393
394     enclosing = minimum_enclosing_circle(polygon)

```

```

395
396     return TargetInfo(
397         center=center,
398         work_radius=work_radius,
399         mec_center=enclosing.center,
400         mec_radius=enclosing.radius,
401     )
402
403
404 @dataclass(frozen=True)
405 class GeometricProbe:
406     point: Point
407     sin_phi: float
408     error_proxy_m: float
409
410
411 def point_in_convex_polygon(
412     point: Point,
413     polygon: list[Point],
414     eps: float = 1e-7,
415 ) -> bool:
416     if not polygon:
417         return False
418
419     orientation = 0
420
421     for i in range(len(polygon)):
422         a = polygon[i]
423         b = polygon[(i + 1) % len(polygon)]
424
425         value = cross(
426             (b[0] - a[0], b[1] - a[1]),
427             (point[0] - a[0], point[1] - a[1]),
428         )
429
430         if abs(value) <= eps:
431             continue
432
433         edge_sign = 1 if value > 0 else -1
434
435         if orientation == 0:
436             orientation = edge_sign
437         elif edge_sign != orientation:
438             return False
439
440     return True
441
442
443 def bearing_pair_probe(
444     first: "DirectionObservation",
445     second: "DirectionObservation",
446 ) -> Optional[GeometricProbe]:
447     angle1 = math.radians(first.bearing_deg)
448     angle2 = math.radians(second.bearing_deg)
449
450     ray1 = (math.cos(angle1), math.sin(angle1))
451     ray2 = (math.cos(angle2), math.sin(angle2))
452
453     denom = cross(ray1, ray2)
454     sin_phi = abs(denom)
455
456     if sin_phi < GEOM_PROBE_MIN_SIN:
457         return None
458
459     offset = (
460         second.station[0] - first.station[0],
461         second.station[1] - first.station[1],
462     )
463
464     dist1 = cross(offset, ray2) / denom
465     dist2 = cross(offset, ray1) / denom
466
467     if dist1 <= 0.0 or dist2 <= 0.0:
468         return None
469
470     point = (
471         first.station[0] + dist1 * ray1[0],
472         first.station[1] + dist1 * ray1[1],
473     )
474
475     err_est = (
476         math.tan(math.radians(BEARING_ERROR_DEG))
477         * (abs(dist1) + abs(dist2))
478         / sin_phi
479     )
480
481     return GeometricProbe(
482         point=point,
483         sin_phi=sin_phi,
484         error_proxy_m=err_est,
485     )
486
487
488 def build_backbone() -> list[tuple[int, Point]]:
489     sites = [(0.0, 0.0)]
490
491     for i in range(EW_OUTER_COUNT):
492         theta = 2.0 * math.pi * i / EW_OUTER_COUNT
493

```

```

494         sites.append(
495             (
496                 EW_RING_RADIUS * math.cos(theta),
497                 EW_RING_RADIUS * math.sin(theta),
498             )
499         )
500     return list(enumerate(sites))
501
502
503
504 def analytic_cover_radius() -> float:
505     half_gap = math.pi / EW_OUTER_COUNT
506
507     center_gap = EW_RING_RADIUS / (2.0 * math.cos(half_gap))
508
509     boundary_gap = math.sqrt(
510         TARGET_RADIUS**2
511         + EW_RING_RADIUS**2
512         - 2.0
513         * TARGET_RADIUS
514         * EW_RING_RADIUS
515         * math.cos(half_gap)
516     )
517
518     return max(center_gap, boundary_gap)
519
520
521 class OfficialClient:
522     def __init__(self) -> None:
523         self.request_counter = 0
524
525     def next_request_id(self, prefix: str) -> str:
526         self.request_counter += 1
527         return (
528             f"{prefix}-"
529             f"{int(time.time() * 1000)}-"
530             f"{self.request_counter}"
531         )
532
533     def base(self, request_id: str) -> dict:
534         return {
535             "arena_id": ARENA_ID,
536             "robot_id": ROBOT_ID,
537             "request_id": request_id,
538         }
539
540     def action_payload(
541         self,
542         request_id: str,
543         position: Point,
544         channel: int,
545     ) -> dict:
546         payload = self.base(request_id)
547
548         payload["position"] = {
549             "x": float(position[0]),
550             "y": float(position[1]),
551         }
552
553         payload["channel"] = int(channel)
554
555         return payload
556
557     def post(self, path: str, payload: dict) -> dict:
558         body = json.dumps(
559             payload,
560             ensure_ascii=False,
561         ).encode("utf-8")
562
563         last_exc: Optional[Exception] = None
564
565         for retry_no in range(1, HTTP_RETRIES + 1):
566             req = Request(
567                 BASE_URL + path,
568                 data=body,
569                 headers={"Content-Type": "application/json"},
570                 method="POST",
571             )
572
573             try:
574                 with urlopen(
575                     req,
576                     timeout=HTTP_TIMEOUT_S,
577                 ) as resp:
578                     return json.loads(
579                         resp.read().decode("utf-8")
580                     )
581
582             except HTTPError as exc:
583                 message = ""
584
585                 try:
586                     message = exc.read().decode(
587                         "utf-8",
588                         errors="replace",
589                     )
590             except Exception:
591                 pass
592

```

```

593         raise RuntimeError(
594             f"HTTP {exc.code}: {message}"
595         ) from exc
596
597     except (
598         URLError,
599         TimeoutError,
600         socket.timeout,
601         ConnectionError,
602     ) as exc:
603         last_exc = exc
604         print(
605             f"[网络异常] attempt={retry_no}, "
606             f"复用同一 request_id: {exc}"
607         )
608
609     assert last_exc is not None
610     raise last_exc
611
612     def enter(self) -> dict:
613         rid = self.next_request_id("enter")
614         return self.post("/enter", self.base(rid))
615
616     def measure(
617         self,
618         position: Point,
619         channel: int,
620     ) -> dict:
621         rid = self.next_request_id("measure")
622
623         return self.post(
624             "/measure",
625             self.action_payload(
626                 rid,
627                 position,
628                 channel,
629             ),
630         )
631
632     def clear(
633         self,
634         position: Point,
635         channel: int,
636     ) -> dict:
637         rid = self.next_request_id("clear")
638
639         return self.post(
640             "/clear",
641             self.action_payload(
642                 rid,
643                 position,
644                 channel,
645             ),
646         )
647
648     def exit(self) -> dict:
649         rid = self.next_request_id("exit")
650         return self.post("/exit", self.base(rid))
651
652
653 @dataclass
654 class DirectionObservation:
655     station: Point
656     bearing_deg: float
657
658
659 @dataclass
660 class ChannelRecord:
661     state: str = "UNKNOWN"
662
663     directions: list[DirectionObservation] = field(
664         default_factory=list
665     )
666
667     polygon: Optional[list[Point]] = None
668
669     strong_position: Optional[Point] = None
670
671     measured_positions: list[Point] = field(
672         default_factory=list
673     )
674
675     no_signal_ew_indices: set[int] = field(
676         default_factory=set
677     )
678
679     lost: bool = False
680
681     failed_clear_points: list[Point] = field(
682         default_factory=list
683     )
684
685
686 @dataclass
687 class Ledger:
688     move_distance: float = 0.0
689     move_seconds: float = 0.0
690
691     measure_count: int = 0

```

```

692     measure_seconds: float = 0.0
693
694     switch_count: int = 0
695     switch_seconds: float = 0.0
696
697     clear_success: int = 0
698     clear_fail: int = 0
699     clear_seconds: float = 0.0
700
701     @property
702     def total_seconds(self) -> float:
703         return (
704             self.move_seconds
705             + self.measure_seconds
706             + self.switch_seconds
707             + self.clear_seconds
708         )
709
710
711 class Q3V3Agent:
712     def __init__(self) -> None:
713         self.client = OfficialClient()
714
715         self.position: Point = (0.0, 0.0)
716         self.current_measure_channel = 1
717
718         self.channels = {
719             ch: ChannelRecord()
720             for ch in range(1, CHANNEL_COUNT + 1)
721         }
722
723         self.pending_ew = build_backbone()
724
725         self.ledger = Ledger()
726
727         self.trace_sequence = 0
728         self.remaining_real_duration_s: Optional[float] = None
729
730         self.early_stop_search_by_found16 = False
731         self.geom_probe_attempts = 0
732         self.geom_probe_success = 0
733
734         self.route_guard_overrides = 0
735         self.route_guard_regret_saved_m_est = 0.0
736
737
738     @property
739     def cleared_count(self) -> int:
740         return sum(
741             record.state == "CLEARED"
742             for record in self.channels.values()
743         )
744
745     @property
746     def empty_count(self) -> int:
747         return sum(
748             record.state == "CERTIFIED_EMPTY"
749             for record in self.channels.values()
750         )
751
752     @property
753     def found_count(self) -> int:
754         return sum(
755             record.state in {"LOCALIZING", "CLEARED"}
756             for record in self.channels.values()
757         )
758
759     def active_channels(self) -> List[int]:
760         return [
761             ch
762             for ch, record in self.channels.items()
763             if record.state == "LOCALIZING"
764         ]
765
766     def trace(self, event: dict) -> None:
767         self.trace_sequence += 1
768
769         row = {
770             "seq": self.trace_sequence,
771             "virtual_time_est": self.ledger.total_seconds,
772             "position": {
773                 "x": self.position[0],
774                 "y": self.position[1],
775             },
776             "cleared": self.cleared_count,
777             "empty": self.empty_count,
778             "ew_left": len(self.pending_ew),
779             **event,
780         }
781
782         with TRACE_PATH.open(
783             "a",
784             encoding="utf-8",
785         ) as handle:
786             handle.write(
787                 json.dumps(
788                     row,
789                     ensure_ascii=False,
790                 )

```

```

791         + "\n"
792     )
793
794     def account_move(self, target: Point) -> float:
795         moved = distance(self.position, target)
796
797         self.ledger.move_distance += moved
798         self.ledger.move_seconds += moved / SPEED
799
800         self.position = target
801
802         return moved
803
804
805     def target_info(self, channel: int) -> TargetInfo:
806         record = self.channels[channel]
807
808         if record.strong_position is not None:
809             return TargetInfo(
810                 center=record.strong_position,
811                 work_radius=0.0,
812                 mec_center=record.strong_position,
813                 mec_radius=5.0,
814             )
815
816         if not record.polygon:
817             raise RuntimeError(
818                 f"频道 {channel} 没有可用后验"
819             )
820
821         return target_info_from_polygon(record.polygon)
822
823
824     def measure_at(
825         self,
826         position: Point,
827         channel: int,
828         *,
829         ew_index: Optional[int] = None,
830         mark_lost_on_no_signal: bool = True,
831         reason: str = "",
832     ) -> tuple[str, Optional[float]]:
833         start = self.position
834
835         resp = self.client.measure(
836             position,
837             channel,
838         )
839
840         if resp.get("accepted") is not True:
841             raise RuntimeError(
842                 f"/measure rejected: {resp}"
843             )
844
845         travelled = self.account_move(position)
846
847         channel_changed = (
848             self.current_measure_channel
849             != channel
850         )
851
852         if channel_changed:
853             self.ledger.switch_count += 1
854             self.ledger.switch_seconds += SWITCH_SECONDS
855
856         self.current_measure_channel = channel
857
858         self.ledger.measure_count += 1
859         self.ledger.measure_seconds += MEASURE_SECONDS
860
861         rec = self.channels[channel]
862         rec.measured_positions.append(position)
863
864         measure_result = resp["measure_result"]
865         svd: Optional[float] = None
866
867         if measure_result == "direction":
868             svd = float(resp["svd_deg"])
869
870             rec.state = "LOCALIZING"
871             rec.lost = False
872
873             rec.directions.append(
874                 DirectionObservation(
875                     station=position,
876                     bearing_deg=svd,
877                 )
878             )
879
880             polygon = update_polygon_with_direction(
881                 rec.polygon,
882                 position,
883                 svd,
884             )
885
886             if not polygon:
887                 raise RuntimeError(
888                     f"频道 {channel} 后验意外为空"
889                 )

```

```

890         rec.polygon = polygon
891
892     elif measure_result == "near":
893         rec.state = "LOCALIZING"
894         rec.lost = False
895         rec.strong_position = position
896
897     elif measure_result == "no_signal":
898         if (
899             rec.state == "UNKNOWN"
900             and ew_index is not None
901         ):
902             rec.no_signal_ew_indices.add(
903                 ew_index
904             )
905
906         if (
907             rec.state == "LOCALIZING"
908             and mark_lost_on_no_signal
909         ):
910             rec.lost = True
911
912     else:
913         raise RuntimeError(
914             f"未知 measure_result: {measure_result!r}"
915         )
916
917     extra_log = {}
918
919     if (
920         rec.state == "LOCALIZING"
921         and rec.polygon
922     ):
923         target = self.target_info(channel)
924
925         extra_log = {
926             "work_radius": target.work_radius,
927             "mec_radius": target.mec_radius,
928             "direction_count": len(
929                 rec.directions
930             ),
931         }
932
933     self.trace(
934         {
935             "action": "measure",
936             "channel": channel,
937             "reason": reason,
938             "from": {
939                 "x": start[0],
940                 "y": start[1],
941             },
942             "to": {
943                 "x": position[0],
944                 "y": position[1],
945             },
946             "move_m": travelled,
947             "switch": channel_changed,
948             "result": measure_result,
949             "bearing": svd,
950             **extra_log,
951         }
952     )
953
954     if measure_result == "near":
955         self.clear_at(
956             position,
957             channel,
958             reason="near 后原地保证清除",
959             measure_after_failure=False,
960         )
961
962     self.apply_found16_shortcut()
963
964     return measure_result, svd
965
966
967 def clear_at(
968     self,
969     position: Point,
970     channel: int,
971     *,
972     reason: str,
973     measure_after_failure: bool,
974 ) -> bool:
975     start = self.position
976
977     resp = self.client.clear(
978         position,
979         channel,
980     )
981
982     if resp.get("accepted") is not True:
983         raise RuntimeError(
984             f"/clear rejected: {resp}"
985         )
986
987     travelled = self.account_move(position)

```

```

989
990     cleared = (
991         resp.get("clear_result")
992         == "success"
993     )
994
995     rec = self.channels[channel]
996
997     if cleared:
998         self.ledger.clear_success += 1
999         self.ledger.clear_seconds += CLEAR_SUCCESS_SECONDS
1000
1001         rec.state = "CLEARED"
1002         rec.lost = False
1003
1004     else:
1005         self.ledger.clear_fail += 1
1006         self.ledger.clear_seconds += CLEAR_FAILURE_SECONDS
1007
1008         rec.failed_clear_points.append(
1009             position
1010         )
1011
1012     self.trace(
1013         {
1014             "action": "clear",
1015             "channel": channel,
1016             "reason": reason,
1017             "from": {
1018                 "x": start[0],
1019                 "y": start[1],
1020             },
1021             "to": {
1022                 "x": position[0],
1023                 "y": position[1],
1024             },
1025             "move_m": travelled,
1026             "result": (
1027                 "success"
1028                 if cleared
1029                 else resp.get(
1030                     "clear_result",
1031                     "fail",
1032                 )
1033             ),
1034         }
1035     )
1036
1037     if (
1038         not cleared
1039         and measure_after_failure
1040     ):
1041         self.measure_at(
1042             position,
1043             channel,
1044             mark_lost_on_no_signal=True,
1045             reason="clear 失败后原地补测",
1046         )
1047
1048     self.apply_found16_shortcut()
1049
1050     return cleared
1051
1052
1053 def q2_fast_candidates(
1054     self,
1055     channel: int,
1056 ) -> tuple[Point, Point]:
1057     target = self.target_info(channel)
1058     obs = self.channels[channel].directions[-1]
1059
1060     angle = math.radians(
1061         obs.bearing_deg
1062     )
1063
1064     side = (
1065         -math.sin(angle),
1066         math.cos(angle),
1067     )
1068
1069     center = target.center
1070
1071     return (
1072         (
1073             center[0]
1074             + Q2_FAST_SIDE_M * side[0],
1075             center[1]
1076             + Q2_FAST_SIDE_M * side[1],
1077         ),
1078         (
1079             center[0]
1080             - Q2_FAST_SIDE_M * side[0],
1081             center[1]
1082             - Q2_FAST_SIDE_M * side[1],
1083         ),
1084     )
1085
1086 def q2_fast_information(
1087     self,

```

```

1088     channel: int,
1089     point: Point,
1090 ) -> float:
1091     target = self.target_info(channel)
1092     obs = self.channels[channel].directions[-1]
1093
1094     v1 = (
1095         target.center[0] - obs.station[0],
1096         target.center[1] - obs.station[1],
1097     )
1098
1099     v2 = (
1100         target.center[0] - point[0],
1101         target.center[1] - point[1],
1102     )
1103
1104     len1 = math.hypot(v1[0], v1[1])
1105     len2 = math.hypot(v2[0], v2[1])
1106
1107     if len1 <= 1e-9 or len2 <= 1e-9:
1108         return 1.0
1109
1110     return min(
1111         1.0,
1112         abs(cross(v1, v2)) / (len1 * len2),
1113     )
1114
1115 def recovery_candidates(
1116     self,
1117     channel: int,
1118 ) -> tuple[Point, Point]:
1119     obs = self.channels[channel].directions[-1]
1120
1121     angle = math.radians(
1122         obs.bearing_deg
1123     )
1124
1125     heading = (
1126         math.cos(angle),
1127         math.sin(angle),
1128     )
1129
1130     side = (
1131         -math.sin(angle),
1132         math.cos(angle),
1133     )
1134
1135     base = obs.station
1136
1137     return (
1138         (
1139             base[0]
1140             + RECOVERY_FORWARD_M * heading[0]
1141             + RECOVERY_SIDE_M * side[0],
1142             base[1]
1143             + RECOVERY_FORWARD_M * heading[1]
1144             + RECOVERY_SIDE_M * side[1],
1145         ),
1146         (
1147             base[0]
1148             + RECOVERY_FORWARD_M * heading[0]
1149             - RECOVERY_SIDE_M * side[0],
1150             base[1]
1151             + RECOVERY_FORWARD_M * heading[1]
1152             - RECOVERY_SIDE_M * side[1],
1153         ),
1154     )
1155
1156 def apply_found16_shortcut(self) -> None:
1157     if self.found_count < SOURCE_MAX:
1158         return
1159
1160     updated = False
1161
1162     for record in self.channels.values():
1163         if record.state == "UNKNOWN":
1164             record.state = "CERTIFIED_EMPTY"
1165             updated = True
1166
1167     if self.pending_ew:
1168         self.pending_ew.clear()
1169         updated = True
1170
1171     if updated:
1172         self.early_stop_search_by_found16 = True
1173         self.trace(
1174             {
1175                 "event": "found16_cancel_search",
1176                 "found_count": self.found_count,
1177             }
1178         )
1179
1180 def best_geometric_probe(
1181     self,
1182     channel: int,
1183 ) -> Optional[GeometricProbe]:
1184     rec = self.channels[channel]
1185
1186     if len(rec.directions) < 2 or not rec.polygon:

```

```

1187         return None
1188
1189     best_probe: Optional[GeometricProbe] = None
1190
1191     for first, second in itertools.combinations(
1192         rec.directions,
1193         2,
1194     ):
1195         probe = bearing_pair_probe(
1196             first,
1197             second,
1198         )
1199
1200         if probe is None:
1201             continue
1202
1203         if (
1204             probe.error_proxy_m
1205             > GEOM_PROBE_MAX_ERROR_M
1206         ):
1207             continue
1208
1209         if not point_in_convex_polygon(
1210             probe.point,
1211             rec.polygon,
1212         ):
1213             continue
1214
1215         if (
1216             probe.point[0] ** 2
1217             + probe.point[1] ** 2
1218             > TARGET_RADIUS ** 2 + 1e-6
1219         ):
1220             continue
1221
1222         if (
1223             best_probe is None
1224             or probe.error_proxy_m
1225             < best_probe.error_proxy_m
1226         ):
1227             best_probe = probe
1228
1229     return best_probe
1230
1231
1232     def already_measured_near(
1233         self,
1234         channel: int,
1235         point: Point,
1236         radius: float = 1.0,
1237     ) -> bool:
1238         return any(
1239             distance(point, old) <= radius
1240             for old
1241             in self.channels[channel].measured_positions
1242         )
1243
1244     def opportunistic_score(
1245         self,
1246         channel: int,
1247         ew: Point,
1248     ) -> Optional[float]:
1249         rec = self.channels[channel]
1250
1251         if (
1252             rec.state != "LOCALIZING"
1253             or not rec.directions
1254             or rec.strong_position is not None
1255         ):
1256             return None
1257
1258         if self.already_measured_near(
1259             channel,
1260             ew,
1261         ):
1262             return None
1263
1264         if rec.lost:
1265             return 100.0
1266
1267         target = self.target_info(channel)
1268
1269         if (
1270             target.mec_radius <= CLEAR_RADIUS
1271             or (
1272                 len(rec.directions) >= 2
1273                 and target.work_radius <= WEAK_PROBE_RADIUS_M
1274             )
1275             or self.best_geometric_probe(channel) is not None
1276         ):
1277             return None
1278
1279         if (
1280             distance(
1281                 ew,
1282                 target.center,
1283             )
1284             > MAX_OPPORTUNISTIC_CENTER_DISTANCE_M
1285         ):

```

```

1286         return None
1287
1288     obs = rec.directions[-1]
1289
1290     base_len = distance(
1291         ew,
1292         obs.station,
1293     )
1294
1295     if base_len < MIN_OPPORTUNISTIC_BASELINE_M:
1296         return None
1297
1298     first_vec = (
1299         target.center[0] - obs.station[0],
1300         target.center[1] - obs.station[1],
1301     )
1302
1303     second_vec = (
1304         target.center[0] - ew[0],
1305         target.center[1] - ew[1],
1306     )
1307
1308     first_len = math.hypot(first_vec[0], first_vec[1])
1309     second_len = math.hypot(second_vec[0], second_vec[1])
1310
1311     if first_len <= 1e-9 or second_len <= 1e-9:
1312         sin_phi = 1.0
1313     else:
1314         sin_phi = min(
1315             1.0,
1316             abs(cross(first_vec, second_vec)) / (first_len * second_len),
1317         )
1318
1319     if sin_phi < MIN_OPPORTUNISTIC_SIN:
1320         return None
1321
1322     return (
1323         10.0 * sin_phi
1324         + min(base_len, 1500.0) / 1500.0
1325     )
1326
1327 def execute_ew(
1328     self,
1329     ew_index: int,
1330     ew: Point,
1331 ) -> None:
1332     unknown_channels = [
1333         ch
1334         for ch, record in self.channels.items()
1335         if record.state == "UNKNOWN"
1336     ]
1337
1338     ranked: list[tuple[float, int]] = []
1339
1340     for ch in self.active_channels():
1341         score = self.opportunistic_score(
1342             ch,
1343             ew,
1344         )
1345
1346         if score is not None:
1347             ranked.append(
1348                 (score, ch)
1349             )
1350
1351     ranked.sort(
1352         reverse=True
1353     )
1354
1355     limit = (
1356         DENSE_MAX_OPPORTUNISTIC_PER_EW
1357         if self.found_count >= DENSE_FOUND_THRESHOLD
1358         else MAX_OPPORTUNISTIC_PER_EW
1359     )
1360
1361     extra_channels = [
1362         ch
1363         for _, ch
1364         in ranked[
1365             :limit
1366         ]
1367     ]
1368
1369     channels_to_measure = list(
1370         dict.fromkeys(
1371             unknown_channels
1372             + extra_channels
1373         )
1374     )
1375
1376     if self.current_measure_channel in channels_to_measure:
1377         channels_to_measure.remove(
1378             self.current_measure_channel
1379         )
1380     channels_to_measure.insert(
1381         0,
1382         self.current_measure_channel,
1383     )
1384

```

```

1385     if not channels_to_measure:
1386
1387         return
1388
1389     for ch in channels_to_measure:
1390         is_new = (
1391             self.channels[ch].state
1392             == "UNKNOWN"
1393         )
1394
1395         self.measure_at(
1396             ew,
1397             ch,
1398             ew_index=(
1399                 ew_index
1400                 if is_new
1401                 else None
1402             ),
1403             mark_lost_on_no_signal=is_new,
1404             reason=(
1405                 "EW 存在性扫描"
1406                 if is_new
1407                 else "EW 零移动成本顺路定位"
1408             ),
1409         )
1410
1411         if self.cleared_count >= SOURCE_MAX:
1412             return
1413
1414
1415     def certify_empty(self) -> None:
1416         if self.pending_ew:
1417             return
1418
1419         all_ew = set(
1420             range(EW_OUTER_COUNT + 1)
1421         )
1422
1423         for record in self.channels.values():
1424             if (
1425                 record.state == "UNKNOWN"
1426                 and all_ew.issubset(
1427                     record.no_signal_ew_indices
1428                 )
1429             ):
1430                 record.state = "CERTIFIED_EMPTY"
1431
1432
1433     def too_close_to_failed_clear(
1434         self,
1435         channel: int,
1436         point: Point,
1437     ) -> bool:
1438         return any(
1439             distance(point, old)
1440             <= FAILED_CLEAR_AVOID_RADIUS_M
1441             for old
1442             in self.channels[channel].failed_clear_points
1443         )
1444
1445
1446     @staticmethod
1447     def mandatory_action_id(action):
1448         kind = action[0]
1449
1450         if kind == "EW":
1451             return ("EW", int(action[1]))
1452
1453         if kind == "CLEAR":
1454             _, channel, _, reason, _ = action
1455
1456             if (
1457                 reason == "MEC<=20m 保证清除"
1458                 or reason == "near 保证清除"
1459             ):
1460                 return ("CLEAR", int(channel))
1461
1462         return None
1463
1464     @staticmethod
1465     def mandatory_action_point(action) -> Point:
1466         kind = action[0]
1467
1468         if kind == "EW":
1469             return action[2]
1470
1471         if kind == "CLEAR":
1472             return action[2]
1473
1474         raise ValueError("非强制动作没有 mandatory point")
1475
1476     def _mandatory_open_tsp_first(
1477         self,
1478         nodes: list[tuple[tuple[str, int], Point]],
1479     ) -> tuple[
1480         Optional[tuple[str, int]],
1481         float,
1482         dict[tuple[str, int], float],
1483     ]:

```

```

1484     n = len(nodes)
1485
1486     if n == 0:
1487         return None, 0.0, {}
1488
1489     points = [point for _, point in nodes]
1490
1491     pair_dist = [
1492         [
1493             distance(points[i], points[j])
1494             for j in range(n)
1495         ]
1496         for i in range(n)
1497     ]
1498
1499     start_dist = [
1500         distance(self.position, point)
1501         for point in points
1502     ]
1503
1504     memo: dict[tuple[int, int], float] = {}
1505
1506     def tail_cost(
1507         last: int,
1508         remaining_mask: int,
1509     ) -> float:
1510         if remaining_mask == 0:
1511             return 0.0
1512
1513         key = (last, remaining_mask)
1514
1515         cached = memo.get(key)
1516
1517         if cached is not None:
1518             return cached
1519
1520         best = float("inf")
1521
1522         mask = remaining_mask
1523
1524         while mask:
1525             bit = mask & -mask
1526             nxt = bit.bit_length() - 1
1527
1528             candidate = (
1529                 pair_dist[last][nxt]
1530                 + tail_cost(
1531                     nxt,
1532                     remaining_mask ^ bit,
1533                 )
1534             )
1535
1536             if candidate < best:
1537                 best = candidate
1538
1539             mask ^= bit
1540
1541         memo[key] = best
1542         return best
1543
1544     # 状态压缩 DP
1545     full_mask = (1 << n) - 1
1546
1547     forced_first_cost: dict[
1548         tuple[str, int],
1549         float,
1550     ] = {}
1551
1552     best_id: Optional[tuple[str, int]] = None
1553     best_cost = float("inf")
1554
1555     for first in range(n):
1556         remaining = full_mask ^ (1 << first)
1557
1558         total = (
1559             start_dist[first]
1560             + tail_cost(
1561                 first,
1562                 remaining,
1563             )
1564         )
1565
1566         node_id = nodes[first][0]
1567         forced_first_cost[node_id] = total
1568
1569         if total < best_cost:
1570             best_cost = total
1571             best_id = node_id
1572
1573     return (
1574         best_id,
1575         best_cost,
1576         forced_first_cost,
1577     )
1578
1579     def apply_mandatory_route_guard(
1580         self,
1581         selected,
1582         candidates,

```

```

1583 ):
1584     if selected is None:
1585         return None
1586
1587     selected_score, selected_action = selected
1588     selected_id = self.mandatory_action_id(
1589         selected_action
1590     )
1591
1592     if selected_id is None:
1593         return selected
1594
1595     if self.found_count >= 15:
1596         return selected
1597
1598     mandatory_candidates = []
1599
1600     seen_ids = set()
1601
1602     for item in candidates:
1603         _, action = item
1604
1605         node_id = self.mandatory_action_id(
1606             action
1607         )
1608
1609         if (
1610             node_id is None
1611             or node_id in seen_ids
1612         ):
1613             continue
1614
1615         seen_ids.add(node_id)
1616
1617         mandatory_candidates.append(
1618             (
1619                 node_id,
1620                 self.mandatory_action_point(
1621                     action
1622                 ),
1623                 item,
1624             )
1625         )
1626
1627     if (
1628         len(mandatory_candidates) <= 1
1629         or len(mandatory_candidates)
1630             > MANDATORY_ROUTE_MAX_NODES
1631     ):
1632         return selected
1633
1634     nodes = [
1635         (node_id, point)
1636         for node_id, point, _
1637         in mandatory_candidates
1638     ]
1639
1640     (
1641         best_first_id,
1642         best_total_m,
1643         forced_first_cost,
1644     ) = self._mandatory_open_tsp_first(
1645         nodes
1646     )
1647
1648     if (
1649         best_first_id is None
1650         or best_first_id == selected_id
1651     ):
1652         return selected
1653
1654     selected_total_m = forced_first_cost.get(
1655         selected_id
1656     )
1657
1658     if selected_total_m is None:
1659         return selected
1660
1661     regret_m = (
1662         selected_total_m
1663         - best_total_m
1664     )
1665
1666     if (
1667         regret_m
1668         < MANDATORY_ROUTE_MIN_REGRET_M
1669     ):
1670         return selected
1671
1672     replacement = next(
1673         (
1674             item
1675             for node_id, _, item
1676             in mandatory_candidates
1677             if node_id == best_first_id
1678         ),
1679         None,
1680     )
1681

```

```

1682     if replacement is None:
1683         return selected
1684
1685     self.route_guard_overrides += 1
1686     self.route_guard_regret_saved_m_est += (
1687         regret_m
1688     )
1689
1690     self.trace(
1691         {
1692             "event": "mandatory_route_guard_override",
1693             "from_task": list(selected_id),
1694             "to_task": list(best_first_id),
1695             "route_regret_m": regret_m,
1696             "route_regret_seconds_equiv": (
1697                 regret_m / SPEED
1698             ),
1699             "mandatory_node_count": len(
1700                 mandatory_candidates
1701             ),
1702         }
1703     )
1704
1705     return replacement
1706
1707
1708 def choose_action(self):
1709     pool = []
1710
1711     for ew_index, ew in self.pending_ew:
1712         pool.append(
1713             (
1714                 distance(
1715                     self.position,
1716                     ew,
1717                 )
1718                 - EW_PROGRESS_BONUS_M,
1719                 (
1720                     "EW",
1721                     ew_index,
1722                     ew,
1723                     " 完备搜索",
1724                 ),
1725             )
1726         )
1727
1728     for ch in self.active_channels():
1729         rec = self.channels[ch]
1730
1731         if rec.strong_position is not None:
1732             pool.append(
1733                 (
1734                     distance(
1735                         self.position,
1736                         rec.strong_position,
1737                     )
1738                     - GUARANTEED_CLEAR_BONUS_M,
1739                     (
1740                         "CLEAR",
1741                         ch,
1742                         rec.strong_position,
1743                         "near 保证清除",
1744                         False,
1745                     ),
1746                 )
1747             )
1748             continue
1749
1750         target = self.target_info(ch)
1751
1752         if rec.lost:
1753             if self.pending_ew:
1754                 continue
1755
1756             recovery_points = self.recovery_candidates(ch)
1757
1758             recovery_point = min(
1759                 recovery_points,
1760                 key=lambda p: distance(
1761                     self.position,
1762                     p,
1763                 ),
1764             )
1765
1766             pool.append(
1767                 (
1768                     distance(
1769                         self.position,
1770                         recovery_point,
1771                     ),
1772                     (
1773                         "MEASURE",
1774                         ch,
1775                         recovery_point,
1776                         " 最后 EW 后的保证接收恢复点",
1777                     ),
1778                 )
1779             )
1780

```

```

1781         continue
1782
1783     if (
1784         target.mec_radius
1785         <= CLEAR_RADIUS - 1e-6
1786         and not self.too_close_to_failed_clear(
1787             ch,
1788             target.mec_center,
1789         )
1790     ):
1791         pool.append(
1792             (
1793                 distance(
1794                     self.position,
1795                     target.mec_center,
1796                 )
1797                 - GUARANTEED_CLEAR_BONUS_M,
1798                 (
1799                     "CLEAR",
1800                     ch,
1801                     target.mec_center,
1802                     "MEC<=20m 保证清除",
1803                     False,
1804                 ),
1805             )
1806         )
1807
1808         continue
1809
1810     direction_count = len(
1811         rec.directions
1812     )
1813
1814     probe = self.best_geometric_probe(ch)
1815
1816     if (
1817         probe is not None
1818         and not self.too_close_to_failed_clear(
1819             ch,
1820             probe.point,
1821         )
1822     ):
1823         pool.append(
1824             (
1825                 distance(
1826                     self.position,
1827                     probe.point,
1828                 )
1829                 - GEOM_PROBE_BONUS_M,
1830                 (
1831                     "CLEAR",
1832                     ch,
1833                     probe.point,
1834                     (
1835                         " 两线交点 Probe "
1836                         f"err≈{probe.error_proxy_m:.1f}m, "
1837                         f"sin={probe.sin_phi:.2f}"
1838                     ),
1839                     True,
1840                 ),
1841             )
1842         )
1843
1844     if (
1845         direction_count >= 2
1846         and target.work_radius
1847         <= STRONG_PROBE_RADIUS_M
1848         and not self.too_close_to_failed_clear(
1849             ch,
1850             target.center,
1851         )
1852     ):
1853         pool.append(
1854             (
1855                 distance(
1856                     self.position,
1857                     target.center,
1858                 )
1859                 - STRONG_PROBE_BONUS_M,
1860                 (
1861                     "CLEAR",
1862                     ch,
1863                     target.center,
1864                     (
1865                         " 强 Probe Clear "
1866                         f"workR={target.work_radius:.1f}"
1867                     ),
1868                     True,
1869                 ),
1870             )
1871         )
1872
1873         continue
1874
1875     if (
1876         direction_count >= 2
1877         and target.work_radius
1878         <= WEAK_PROBE_RADIUS_M
1879         and not self.too_close_to_failed_clear(

```

```

1880         ch,
1881         target.center,
1882     )
1883 ):
1884     pool.append(
1885         (
1886             distance(
1887                 self.position,
1888                 target.center,
1889             )
1890             - WEAK_PROBE_BONUS_M,
1891             (
1892                 "CLEAR",
1893                 ch,
1894                 target.center,
1895                 (
1896                     " 弱 Probe Clear "
1897                     f"workR={target.work_radius:.1f}"
1898                 ),
1899                 True,
1900             ),
1901         )
1902     )
1903
1904
1905 if direction_count == 1:
1906     for point in self.q2_fast_candidates(ch):
1907         gain = self.q2_fast_information(
1908             ch,
1909             point,
1910         )
1911
1912         pool.append(
1913             (
1914                 distance(
1915                     self.position,
1916                     point,
1917                 )
1918                 - Q2_FAST_INFO_BONUS_M
1919                 * gain,
1920             (
1921                 "MEASURE",
1922                 ch,
1923                 point,
1924                 (
1925                     "Q2-fast 第二测点 "
1926                     f"sin≈{gain:.2f}"
1927                 ),
1928             ),
1929         )
1930     )
1931
1932     pool.append(
1933         (
1934             distance(
1935                 self.position,
1936                 target.center,
1937             ),
1938             (
1939                 "MEASURE",
1940                 ch,
1941                 target.center,
1942                 (
1943                     " 直接逼近后验中心 "
1944                     f"workR={target.work_radius:.1f}, "
1945                     f"mecR={target.mec_radius:.1f}"
1946                 ),
1947             ),
1948         )
1949     )
1950
1951 if not pool:
1952     return None
1953
1954 chosen = min(
1955     pool,
1956     key=lambda item: item[0],
1957 )
1958
1959 return self.apply_mandatory_route_guard(
1960     chosen,
1961     pool,
1962 )
1963
1964
1965 def run(self) -> None:
1966     for path in (TRACE_PATH, SUMMARY_PATH):
1967         try:
1968             path.unlink()
1969         except FileNotFoundError:
1970             pass
1971
1972     cover = analytic_cover_radius()
1973
1974     print("=" * 80)
1975     print("Q3 V3.2 FINAL - MANDATORY ROUTE GUARD")
1976     print(
1977         f"8 点连续覆盖半径 = "
1978         f"{cover:.3f} m"

```

```

1979     )
1980     print("=" * 80)
1981
1982     if cover > MIN_RECEIVE_RADIUS:
1983         raise RuntimeError(
1984             " 当前骨架不满足 1000m 完备覆盖"
1985         )
1986
1987     enter = self.client.enter()
1988
1989     if enter.get("accepted") is not True:
1990         raise RuntimeError(
1991             f"/enter rejected: {enter}"
1992         )
1993
1994     self.remaining_real_duration_s = float(
1995         enter["remaining_real_duration_s"]
1996     )
1997
1998     self.trace(
1999         {
2000             "action": "enter",
2001             "remaining_real_duration_s": (
2002                 self.remaining_real_duration_s
2003             ),
2004         }
2005     )
2006
2007     for logical_step in range(
2008         1,
2009         MAX_LOGICAL_ACTIONS + 1,
2010     ):
2011         self.apply_found16_shortcut()
2012         self.certify_empty()
2013
2014         if self.cleared_count >= SOURCE_MAX:
2015             print(" 已清除 16 个源, 提前结束。")
2016             break
2017
2018         if (
2019             self.cleared_count
2020             + self.empty_count
2021             == CHANNEL_COUNT
2022         ):
2023             print(
2024                 " 全部 20 频道已清除或严格判空。"
2025             )
2026             break
2027
2028         chosen = self.choose_action()
2029
2030         if chosen is None:
2031             raise RuntimeError(
2032                 " 无动作但尚未满足终止条件"
2033             )
2034
2035         score, action = chosen
2036         kind = action[0]
2037
2038         if kind == "EW":
2039             _, ew_index, ew, reason = action
2040
2041             print(
2042                 f"\n[{{logical_step}}] EW{{ew_index}} "
2043                 f"score={{score:.1f}} "
2044                 f"({{ew[0]:.1f}},{{ew[1]:.1f}})"
2045             )
2046
2047             self.pending_ew.remove(
2048                 (ew_index, ew)
2049             )
2050
2051             self.execute_ew(
2052                 ew_index,
2053                 ew,
2054             )
2055
2056         elif kind == "MEASURE":
2057             _, ch, point, reason = action
2058
2059             print(
2060                 f"\n[{{logical_step}}] MEASURE "
2061                 f"ch={{ch}} score={{score:.1f}} "
2062                 f"{{reason}}"
2063             )
2064
2065             self.measure_at(
2066                 point,
2067                 ch,
2068                 mark_lost_on_no_signal=True,
2069                 reason=reason,
2070             )
2071
2072         elif kind == "CLEAR":
2073             (
2074                 _,
2075                 ch,
2076                 point,
2077                 reason,

```

```

2078         measure_after_failure,
2079     ) = action
2080
2081     print(
2082         f"\n[{logical_step}] CLEAR "
2083         f"ch={ch} score={score:.1f} "
2084         f"{reason}"
2085     )
2086
2087     is_geom_probe = (
2088         reason.startswith(" 两线交点 Probe")
2089     )
2090
2091     if is_geom_probe:
2092         self.geom_probe_attempts += 1
2093
2094     clear_ok = self.clear_at(
2095         point,
2096         ch,
2097         reason=reason,
2098         measure_after_failure=(
2099             measure_after_failure
2100         ),
2101     )
2102
2103     if is_geom_probe and clear_ok:
2104         self.geom_probe_success += 1
2105
2106     else:
2107         raise RuntimeError(
2108             f" 未知动作: {kind}"
2109         )
2110
2111     print(
2112         f"   cleared={self.cleared_count}, "
2113         f"found={self.found_count}, "
2114         f"empty={self.empty_count}, "
2115         f"active={len(self.active_channels())}, "
2116         f"EW_left={len(self.pending_ew)}, "
2117         f"T≈{self.ledger.total_seconds:.1f}s"
2118     )
2119
2120     else:
2121         raise RuntimeError(
2122             " 超过最大逻辑动作数, 疑似循环"
2123         )
2124
2125     exit_response = self.client.exit()
2126
2127     summary = {
2128         "cleared": self.cleared_count,
2129         "found": self.found_count,
2130         "certified_empty": self.empty_count,
2131         "early_stop_search_by_found16": (
2132             self.early_stop_search_by_found16
2133         ),
2134         "geom_probe_attempts": self.geom_probe_attempts,
2135         "geom_probe_success": self.geom_probe_success,
2136         "route_guard_overrides": self.route_guard_overrides,
2137         "route_guard_regret_saved_m_est": (
2138             self.route_guard_regret_saved_m_est
2139         ),
2140         "route_guard_regret_saved_s_est": (
2141             self.route_guard_regret_saved_m_est
2142             / SPEED
2143         ),
2144         "move_distance_m": self.ledger.move_distance,
2145         "movement_seconds": self.ledger.move_seconds,
2146         "measure_count": self.ledger.measure_count,
2147         "measure_seconds": self.ledger.measure_seconds,
2148         "switch_count": self.ledger.switch_count,
2149         "switch_seconds": self.ledger.switch_seconds,
2150         "clear_success": self.ledger.clear_success,
2151         "clear_fail": self.ledger.clear_fail,
2152         "clear_seconds": self.ledger.clear_seconds,
2153         "estimated_virtual_time": self.ledger.total_seconds,
2154         "average_seconds_per_cleared": (
2155             self.ledger.total_seconds
2156             / self.cleared_count
2157             if self.cleared_count
2158             else None
2159         ),
2160         "backbone_cover_radius": cover,
2161         "exit_reason": exit_response.get(
2162             "exit_reason"
2163         ),
2164     }
2165
2166     SUMMARY_PATH.write_text(
2167         json.dumps(
2168             summary,
2169             ensure_ascii=False,
2170             indent=2,
2171         ),
2172         encoding="utf-8",
2173     )
2174
2175     self.trace(
2176         {

```

```
2177         "action": "exit",
2178         "exit_reason": exit_response.get(
2179             "exit_reason"
2180         ),
2181         "summary": summary,
2182     }
2183 )
2184
2185 print("\n" + "=" * 80)
2186 print("Q3 V3.2 FINAL RESULT")
2187 print("=" * 80)
2188
2189 for key, value in summary.items():
2190     print(f"{key:28s}: {value}")
2191
2192 print(
2193     f"\n 明文轨迹: {TRACE_PATH.resolve()}"
2194 )
2195
2196 print(
2197     f" 明文汇总: {SUMMARY_PATH.resolve()}"
2198 )
2199
2200
2201 def main() -> None:
2202     agent = Q3V3Agent()
2203     agent.run()
2204
2205
2206 if __name__ == "__main__":
2207     main()
```

附录 D 问题四程序：q4.py

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """B 题问题 4 程序。"""
4
5 from __future__ import annotations
6
7 import copy
8 import itertools
9 import json
10 import math
11 import random
12 import socket
13 import sys
14 import time
15 from dataclasses import dataclass, field
16 from pathlib import Path
17 from typing import Optional
18 from urllib.error import HTTPError, URLError
19 from urllib.request import Request, urlopen
20
21
22 # 接口配置
23
24 BASE_URL = "http://127.0.0.1:2026"
25 ROBOT_ID = "*****" # 提交版隐藏队号
26 ARENA_ID = "default"
27
28 HTTP_TIMEOUT_S = 10
29 HTTP_RETRIES = 3
30
31 # 每轮单独保存日志。
32 RUN_STAMP = (
33     time.strftime("%Y%m%d_%H%M%S")
34     + f"_{time.time_ns() % 1_000_000_000:09d}"
35 )
36 OUTPUT_DIR = Path(__file__).resolve().parent / " 第四问输出" / RUN_STAMP
37 TRACE_PATH = OUTPUT_DIR / "q4_trace.jsonl"
38 SUMMARY_PATH = OUTPUT_DIR / "q4_summary.json"
39
40
41 # 题目常量
42
43 TARGET_RADIUS = 1800.0
44 MIN_RECEIVE_RADIUS = 1000.0
45 MAX_RECEIVE_RADIUS = 1500.0
46
47 BEARING_ERROR_DEG = 1.0
48 NEAR_RADIUS = 5.0
49 CLEAR_RADIUS = 20.0
50
51 SPEED = 5.0
52 MEASURE_SECONDS = 5.0
53 SWITCH_SECONDS = 1.0
54 CLEAR_SUCCESS_SECONDS = 5.0
55 CLEAR_FAILURE_SECONDS = 3.0
56
57 CHANNEL_COUNT = 20
58 SOURCE_MAX = 16
59
60
61 # 策略参数
62
63 # 搜索点：中心 1 点、内环 7 点、外环 14 点。
64 # 外环给目标圆留 10m 余量。
65 INNER_SEARCH_COUNT = 7
66 INNER_SEARCH_RADIUS_M = 990.0
67 OUTER_SEARCH_COUNT = 14
68 OUTER_HULL_MARGIN_M = 10.0
69 OUTER_PHASE_DEG = 0.7
70 OUTER_SEARCH_RADIUS_M = (TARGET_RADIUS + OUTER_HULL_MARGIN_M) / math.cos(
71     math.pi / OUTER_SEARCH_COUNT
72 )
73 SEARCH_POINT_COUNT = 1 + INNER_SEARCH_COUNT + OUTER_SEARCH_COUNT
74
75 # 离线搜索覆盖证书。
76 CERTIFICATE_GRID_STEP_M = 2.0
77 CERTIFICATE_NEIGHBOR_RADIUS_M = 997.0
78 CERTIFICATE_MIN_HULL_MARGIN_M = 8.601631088485874
79 CERTIFICATE_VERSION = "q4-22-local-hull-2m-v1"
80 SCHEDULER_VERSION = "q4-v5-safe-final-local-dominance-v1"
81
82 # 第一次 direction 后的侧移量。
83 Q2_FAST_SIDE_M = 80.0
84
85 # 定向源失联后的四点恢复。
86 RECOVERY_FORWARD_M = 200.0
87 RECOVERY_SIDE_M = 350.0
88
89 # Probe clear 使用工作半径。
90 STRONG_PROBE_RADIUS_M = 80.0
91 WEAK_PROBE_RADIUS_M = 180.0
92
93 # 后验较小时切换到网格清除。
94 # 网格最远覆盖约 19.6m。
95 SWEEP_TRIGGER_DIRECTION_COUNT = 6
96 SWEEP_TRIGGER_WORK_RADIUS_M = 120.0
97 SWEEP_GRID_STEP_M = CLEAR_RADIUS * math.sqrt(2.0) * 0.98

```

```

98
99 # 调度奖励（等价米）
100 GUARANTEED_CLEAR_BONUS_M = 320.0
101 STRONG_PROBE_BONUS_M = 150.0
102 WEAK_PROBE_BONUS_M = 60.0
103 SWEEP_CLEAR_BONUS_M = 210.0
104 EW_PROGRESS_BONUS_M = 300.0
105 INNER_EW_PROGRESS_BONUS_M = 500.0
106 Q2_FAST_INFO_BONUS_M = 45.0
107
108 # 2-opt 首步允许的最大额外代价。
109 POST_ROUTE_FIRST_STEP_REGRET_M = 100.0
110
111 # EW 顺路复测上限。
112 MAX_OPPORTUNISTIC_PER_EW = SOURCE_MAX
113 MIN_OPPORTUNISTIC_BASELINE_M = 220.0
114 MIN_OPPORTUNISTIC_SIN = 0.15
115 MAX_OPPORTUNISTIC_CENTER_DISTANCE_M = 1200.0
116
117 # 两条示向线的几何 probe。
118 # 失败后原地补测。
119 GEOM_PROBE_MIN_SIN = 0.45
120 GEOM_PROBE_MAX_ERROR_M = 50.0
121 GEOM_PROBE_BONUS_M = 260.0
122
123 # 演练数据得到的调度可靠度。
124 # 只参与调度评分。
125 GEOM_PROBE_RELIABILITY = 0.88
126 STRONG_PROBE_RELIABILITY = 0.65
127 WEAK_PROBE_RELIABILITY = 0.50
128 Q2_MEASURE_RELIABILITY = 0.45
129 RECOVERY_RELIABILITY = 0.70
130
131 # 避免重复 clear 同一点。
132 FAILED_CLEAR_AVOID_RADIUS_M = 8.0
133
134 # SAFE 层参数。
135 # 清除半径留 5cm 余量。
136 SAFE_CLEAR_MARGIN_M = 0.05
137
138 # 小于 1cm 的收益忽略。
139 SAFE_MIN_ROUTE_GAIN_M = 0.01
140
141 MAX_LOGICAL_ACTIONS = 20000
142
143
144 Point = tuple[float, float]
145
146
147 # 几何工具
148
149 def distance(a: Point, b: Point) -> float:
150     return math.hypot(a[0] - b[0], a[1] - b[1])
151
152
153 def open_route_length(
154     start: Point,
155     points: list[Point],
156 ) -> float:
157     """ 从 start 出发且不返回起点的开放路线长度。"""
158     total = 0.0
159     current = start
160
161     for point in points:
162         total += distance(current, point)
163         current = point
164
165     return total
166
167
168 def optimized_open_order(
169     start: Point,
170     points: list[Point],
171 ) -> list[int]:
172     """ 最近邻生成初解，再用开放路线 2-opt 消除明显折返。"""
173     if not points:
174         return []
175
176     pending = list(range(len(points)))
177     order: list[int] = []
178     current = start
179
180     while pending:
181         chosen = min(
182             pending,
183             key=lambda index: (
184                 distance(current, points[index]),
185                 index,
186             ),
187         )
188         pending.remove(chosen)
189         order.append(chosen)
190         current = points[chosen]
191
192     # 2-opt 只比较反折段两端。
193     improved = True
194     while improved:
195         improved = False
196

```

```

197     for i in range(len(order) - 1):
198         previous = start if i == 0 else points[order[i - 1]]
199
200     for j in range(i + 1, len(order)):
201         first = points[order[i]]
202         last = points[order[j]]
203         following = (
204             points[order[j + 1]]
205             if j + 1 < len(order)
206             else None
207         )
208
209         old_length = distance(previous, first)
210         new_length = distance(previous, last)
211
212         if following is not None:
213             old_length += distance(last, following)
214             new_length += distance(first, following)
215
216         if new_length + 1e-9 < old_length:
217             order[i:j + 1] = reversed(order[i:j + 1])
218             improved = True
219             break
220
221     if improved:
222         break
223
224     return order
225
226
227 def cross(a: Point, b: Point) -> float:
228     return a[0] * b[1] - a[1] * b[0]
229
230
231 def sub(a: Point, b: Point) -> Point:
232     return a[0] - b[0], a[1] - b[1]
233
234
235 def clip_left(
236     polygon: list[Point],
237     line_point: Point,
238     line_direction: Point,
239     eps: float = 1e-9,
240 ) -> list[Point]:
241     if not polygon:
242         return []
243
244     def value(p: Point) -> float:
245         return cross(line_direction, sub(p, line_point))
246
247     result: list[Point] = []
248     prev = polygon[-1]
249     prev_value = value(prev)
250     prev_in = prev_value >= -eps
251
252     for cur in polygon:
253         cur_value = value(cur)
254         cur_in = cur_value >= -eps
255
256         if prev_in != cur_in:
257             denom = prev_value - cur_value
258             if abs(denom) > 1e-15:
259                 t = prev_value / denom
260                 t = min(1.0, max(0.0, t))
261                 result.append(
262                     (
263                         prev[0] + t * (cur[0] - prev[0]),
264                         prev[1] + t * (cur[1] - prev[1]),
265                     )
266                 )
267
268         if cur_in:
269             result.append(cur)
270
271     prev = cur
272     prev_value = cur_value
273     prev_in = cur_in
274
275     clean: list[Point] = []
276     for p in result:
277         if not clean or distance(clean[-1], p) > 1e-8:
278             clean.append(p)
279
280     if len(clean) > 1 and distance(clean[0], clean[-1]) <= 1e-8:
281         clean.pop()
282
283     return clean
284
285
286 def circumscribed_disk(
287     center: Point,
288     radius: float,
289     sides: int,
290 ) -> list[Point]:
291     """ 圆盘的外接正多边形; 只会保守放大。 """
292     polygon_radius = radius / math.cos(math.pi / sides)
293     phase = math.pi / sides
294
295     return [

```

```

296     (
297         center[0]
298         + polygon_radius * math.cos(phase + 2.0 * math.pi * i / sides),
299         center[1]
300         + polygon_radius * math.sin(phase + 2.0 * math.pi * i / sides),
301     )
302     for i in range(sides)
303 ]
304
305
306 def clip_by_convex_polygon(
307     subject: list[Point],
308     clipper: list[Point],
309 ) -> list[Point]:
310     poly = subject
311
312     for i in range(len(clipper)):
313         a = clipper[i]
314         b = clipper[(i + 1) % len(clipper)]
315
316         poly = clip_left(
317             poly,
318             a,
319             (b[0] - a[0], b[1] - a[1]),
320         )
321
322         if not poly:
323             break
324
325     return poly
326
327
328 TARGET_POLY = circumscribed_disk(
329     (0.0, 0.0),
330     TARGET_RADIUS,
331     72,
332 )
333
334
335 def update_polygon_with_direction(
336     polygon: Optional[list[Point]],
337     station: Point,
338     bearing_deg: float,
339 ) -> list[Point]:
340     """ 根据目标圆、测向角域和 1500m 接收范围更新保守后验。"""
341     poly = TARGET_POLY.copy() if polygon is None else list(polygon)
342
343     lower = math.radians(bearing_deg - BEARING_ERROR_DEG)
344     upper = math.radians(bearing_deg + BEARING_ERROR_DEG)
345
346     poly = clip_left(
347         poly,
348         station,
349         (math.cos(lower), math.sin(lower)),
350     )
351
352     poly = clip_left(
353         poly,
354         station,
355         (-math.cos(upper), -math.sin(upper)),
356     )
357
358     poly = clip_by_convex_polygon(
359         poly,
360         circumscribed_disk(
361             station,
362             MAX_RECEIVE_RADIUS,
363             32,
364         ),
365     )
366
367     return poly
368
369
370 def polygon_diameter(
371     polygon: list[Point],
372 ) -> tuple[float, tuple[Point, Point]]:
373     if not polygon:
374         raise ValueError(" 空多边形没有直径")
375
376     if len(polygon) == 1:
377         return 0.0, (polygon[0], polygon[0])
378
379     best_sq = -1.0
380     best_pair = (polygon[0], polygon[1])
381
382     for i, p in enumerate(polygon):
383         for q in polygon[i + 1:]:
384             sq = (
385                 (p[0] - q[0]) ** 2
386                 + (p[1] - q[1]) ** 2
387             )
388
389             if sq > best_sq:
390                 best_sq = sq
391                 best_pair = (p, q)
392
393     return math.sqrt(best_sq), best_pair
394

```

```

395
396 @dataclass(frozen=True)
397 class Circle:
398     center: Point
399     radius: float
400
401
402 def circle_from_two(a: Point, b: Point) -> Circle:
403     center = (
404         (a[0] + b[0]) / 2.0,
405         (a[1] + b[1]) / 2.0,
406     )
407     return Circle(center, distance(center, a))
408
409
410 def circle_from_three(
411     a: Point,
412     b: Point,
413     c: Point,
414 ) -> Optional[Circle]:
415     ax, ay = a
416     bx, by = b
417     cx, cy = c
418
419     det = 2.0 * (
420         ax * (by - cy)
421         + bx * (cy - ay)
422         + cx * (ay - by)
423     )
424
425     if abs(det) <= 1e-12:
426         return None
427
428     a2 = ax * ax + ay * ay
429     b2 = bx * bx + by * by
430     c2 = cx * cx + cy * cy
431
432     ux = (
433         a2 * (by - cy)
434         + b2 * (cy - ay)
435         + c2 * (ay - by)
436     ) / det
437
438     uy = (
439         a2 * (cx - bx)
440         + b2 * (ax - cx)
441         + c2 * (bx - ax)
442     ) / det
443
444     center = (ux, uy)
445     return Circle(center, distance(center, a))
446
447
448 def circle_contains(circle: Circle, point: Point) -> bool:
449     return distance(circle.center, point) <= circle.radius + 1e-7
450
451
452 def minimum_enclosing_circle(points: List[Point]) -> Circle:
453     if not points:
454         raise ValueError(" 空点集没有最小包围圆")
455
456     unique = list(dict.fromkeys(points))
457     random.Random(20260912).shuffle(unique)
458
459     circle: Optional[Circle] = None
460
461     for i, p in enumerate(unique):
462         if circle is not None and circle_contains(circle, p):
463             continue
464
465         circle = Circle(p, 0.0)
466
467         for j in range(i):
468             q = unique[j]
469
470             if circle_contains(circle, q):
471                 continue
472
473             circle = circle_from_two(p, q)
474
475             for k in range(j):
476                 r = unique[k]
477
478                 if circle_contains(circle, r):
479                     continue
480
481                 candidate = circle_from_three(p, q, r)
482
483                 if candidate is not None:
484                     circle = candidate
485             else:
486                 options = [
487                     circle_from_two(p, q),
488                     circle_from_two(p, r),
489                     circle_from_two(q, r),
490                 ]
491
492                 valid = [
493                     item

```

```

494         for item in options
495             if circle_contains(item, p)
496                 and circle_contains(item, q)
497                 and circle_contains(item, r)
498         ]
499
500         circle = min(
501             valid,
502             key=Lambda item: item.radius,
503         )
504
505     assert circle is not None
506
507     return Circle(
508         circle.center,
509         max(distance(circle.center, p) for p in unique),
510     )
511
512 @dataclass(frozen=True)
513 class TargetInfo:
514     center: Point
515     work_radius: float
516     mec_center: Point
517     mec_radius: float
518
519
520 def target_info_from_polygon(polygon: list[Point]) -> TargetInfo:
521     diameter, pair = polygon_diameter(polygon)
522
523     # 工作中心取直径端点中点。
524     center = (
525         (pair[0][0] + pair[1][0]) / 2.0,
526         (pair[0][1] + pair[1][1]) / 2.0,
527     )
528
529     work_radius = diameter / 2.0
530
531     mec = minimum_enclosing_circle(polygon)
532
533     return TargetInfo(
534         center=center,
535         work_radius=work_radius,
536         mec_center=mec.center,
537         mec_radius=mec.radius,
538     )
539
540
541
542 @dataclass(frozen=True)
543 class GeometricProbe:
544     point: Point
545     sin_phi: float
546     error_proxy_m: float
547
548
549 def point_in_convex_polygon(
550     point: Point,
551     polygon: list[Point],
552     eps: float = 1e-7,
553 ) -> bool:
554     """ 判断点是否在凸多边形内。 """
555     if not polygon:
556         return False
557
558     sign = 0
559
560     for i in range(len(polygon)):
561         a = polygon[i]
562         b = polygon[(i + 1) % len(polygon)]
563
564         value = cross(
565             (b[0] - a[0], b[1] - a[1]),
566             (point[0] - a[0], point[1] - a[1]),
567         )
568
569         if abs(value) <= eps:
570             continue
571
572         current = 1 if value > 0 else -1
573
574         if sign == 0:
575             sign = current
576         elif current != sign:
577             return False
578
579     return True
580
581
582 def point_segment_distance(
583     point: Point,
584     a: Point,
585     b: Point,
586 ) -> float:
587     """ 点到闭线段的欧氏距离。 """
588     vx = b[0] - a[0]
589     vy = b[1] - a[1]
590     vv = vx * vx + vy * vy
591
592

```

```

593     if vv <= 1e-18:
594         return distance(point, a)
595
596     t = (
597         (point[0] - a[0]) * vx
598         + (point[1] - a[1]) * vy
599     ) / vv
600     t = min(1.0, max(0.0, t))
601
602     projection = (
603         a[0] + t * vx,
604         a[1] + t * vy,
605     )
606     return distance(point, projection)
607
608
609 def point_to_convex_polygon_distance(
610     point: Point,
611     polygon: list[Point],
612 ) -> float:
613     """ 计算点到凸多边形的最短距离。 """
614     if not polygon:
615         return math.inf
616
617     if point_in_convex_polygon(point, polygon):
618         return 0.0
619
620     return min(
621         point_segment_distance(
622             point,
623             polygon[i],
624             polygon[(i + 1) % len(polygon)],
625         )
626         for i in range(len(polygon))
627     )
628
629
630 def segment_disk_parameter_interval(
631     start: Point,
632     end: Point,
633     center: Point,
634     radius: float,
635 ) -> Optional[tuple[float, float]]:
636     """ 返回线段落在给定圆盘内的参数区间。 """
637     dx = end[0] - start[0]
638     dy = end[1] - start[1]
639     fx = start[0] - center[0]
640     fy = start[1] - center[1]
641
642     aa = dx * dx + dy * dy
643     cc = fx * fx + fy * fy - radius * radius
644
645     if aa <= 1e-18:
646         if cc <= 1e-12:
647             return 0.0, 1.0
648         return None
649
650     bb = 2.0 * (fx * dx + fy * dy)
651     discriminant = bb * bb - 4.0 * aa * cc
652
653     if discriminant < -1e-10:
654         return None
655
656     discriminant = max(0.0, discriminant)
657     root = math.sqrt(discriminant)
658
659     t1 = (-bb - root) / (2.0 * aa)
660     t2 = (-bb + root) / (2.0 * aa)
661
662     lo = max(0.0, min(t1, t2))
663     hi = min(1.0, max(t1, t2))
664
665     if lo > hi + 1e-12:
666         return None
667
668     return lo, hi
669
670
671 def bearing_pair_probe(
672     first: "DirectionObservation",
673     second: "DirectionObservation",
674 ) -> Optional[GeometricProbe]:
675     """ 由两条中心示向线生成几何试探点和误差代理。 """
676     a = math.radians(first.bearing_deg)
677     b = math.radians(second.bearing_deg)
678
679     d1 = (math.cos(a), math.sin(a))
680     d2 = (math.cos(b), math.sin(b))
681
682     denominator = cross(d1, d2)
683     sin_phi = abs(denominator)
684
685     if sin_phi < GEOM_PROBE_MIN_SIN:
686         return None
687
688     delta = (
689         second.station[0] - first.station[0],
690         second.station[1] - first.station[1],
691     )

```

```

692
693     t1 = cross(delta, d2) / denominator
694     t2 = cross(delta, d1) / denominator
695
696     if t1 <= 0.0 or t2 <= 0.0:
697         return None
698
699     point = (
700         first.station[0] + t1 * d1[0],
701         first.station[1] + t1 * d1[1],
702     )
703
704     error_proxy = (
705         math.tan(math.radians(BEARING_ERROR_DEG))
706         * (abs(t1) + abs(t2))
707         / sin_phi
708     )
709
710     return GeometricProbe(
711         point=point,
712         sin_phi=sin_phi,
713         error_proxy_m=error_proxy,
714     )
715
716
717 # 搜索骨架
718
719 def build_backbone() -> list[tuple[int, Point]]:
720     """ 返回中心、内环 7 点、外环 14 点；编号固定用于严格判空。"""
721     points: list[Point] = [(0.0, 0.0)]
722
723     for i in range(INNER_SEARCH_COUNT):
724         angle = 2.0 * math.pi * i / INNER_SEARCH_COUNT
725         points.append(
726             (
727                 INNER_SEARCH_RADIUS_M * math.cos(angle),
728                 INNER_SEARCH_RADIUS_M * math.sin(angle),
729             )
730         )
731
732     for i in range(OUTER_SEARCH_COUNT):
733         angle = (
734             math.radians(OUTER_PHASE_DEG)
735             + 2.0 * math.pi * i / OUTER_SEARCH_COUNT
736         )
737         points.append(
738             (
739                 OUTER_SEARCH_RADIUS_M * math.cos(angle),
740                 OUTER_SEARCH_RADIUS_M * math.sin(angle),
741             )
742         )
743
744     return list(enumerate(points))
745
746
747 def search_phase(ew_index: int) -> int:
748     """ 先完成中心/内环快速筛查，再开放外环完备搜索。"""
749     return 0 if ew_index <= INNER_SEARCH_COUNT else 1
750
751
752 def phase_a_omni_cover_radius() -> float:
753     """ 中心 + 内环 7 点的距离覆盖半径，仅用于诊断。"""
754     alpha = math.pi / INNER_SEARCH_COUNT
755     inner_seam = INNER_SEARCH_RADIUS_M / (2.0 * math.cos(alpha))
756     boundary_seam = math.sqrt(
757         TARGET_RADIUS**2
758         + INNER_SEARCH_RADIUS_M**2
759         - 2.0
760         * TARGET_RADIUS
761         * INNER_SEARCH_RADIUS_M
762         * math.cos(alpha)
763     )
764     return max(inner_seam, boundary_seam)
765
766
767 def validate_direction_complete_backbone() -> dict[str, float]:
768     """ 检查当前 22 点搜索骨架是否仍满足离线证书参数。"""
769     # 参数变动后需要重新做覆盖认证。
770     cert_params = (
771         7,
772         990.0,
773         14,
774         10.0,
775         0.7,
776         2.0,
777         997.0,
778     )
779     params_now = (
780         INNER_SEARCH_COUNT,
781         INNER_SEARCH_RADIUS_M,
782         OUTER_SEARCH_COUNT,
783         OUTER_HULL_MARGIN_M,
784         OUTER_PHASE_DEG,
785         CERTIFICATE_GRID_STEP_M,
786         CERTIFICATE_NEIGHBOR_RADIUS_M,
787     )
788     if params_now != cert_params:
789         raise RuntimeError(
790             " 搜索骨架参数已变化，22 点局部凸包证书失效； "

```

```

791         " 请重新执行离线全网格认证"
792     )
793     if SEARCH_POINT_COUNT != 22:
794         raise RuntimeError(" 方向完备搜索点数应为 22")
795
796     outer_apothem = OUTER_SEARCH_RADIUS_M * math.cos(
797         math.pi / OUTER_SEARCH_COUNT
798     )
799     omni_cover = phase_a_omni_cover_radius()
800     cell_radius = CERTIFICATE_GRID_STEP_M / math.sqrt(2.0)
801     distance_slack = (
802         MIN_RECEIVE_RADIUS
803         - CERTIFICATE_NEIGHBOR_RADIUS_M
804         - cell_radius
805     )
806     hull_slack = CERTIFICATE_MIN_HULL_MARGIN_M - cell_radius
807
808     if outer_apothem + 1e-6 < TARGET_RADIUS:
809         raise RuntimeError(" 外环多边形没有包含目标圆")
810     if distance_slack <= 0.0 or hull_slack <= 0.0:
811         raise RuntimeError("22 点方向完备证书没有正安全余量")
812
813     return {
814         "outer_apothem_m": outer_apothem,
815         "phase_a_omni_cover_radius_m": omni_cover,
816         "certificate_cell_radius_m": cell_radius,
817         "certificate_distance_slack_m": distance_slack,
818         "certificate_hull_slack_m": hull_slack,
819     }
820
821
822 def run_geometry_self_test(samples: int = 20000) -> None:
823     """ 不连接模拟器，随机复核搜索完备性和四点恢复命题。"""
824     metrics = validate_direction_complete_backbone()
825     points = [point for _, point in build_backbone()]
826     rng = random.Random(20260912)
827
828     for _ in range(samples):
829         radius = TARGET_RADIUS * math.sqrt(rng.random())
830         angle = 2.0 * math.pi * rng.random()
831         source = (radius * math.cos(angle), radius * math.sin(angle))
832         emission = 2.0 * math.pi * rng.random()
833         direction = (math.cos(emission), math.sin(emission))
834
835         detected = any(
836             distance(point, source) <= MIN_RECEIVE_RADIUS + 1e-7
837             and (
838                 (point[0] - source[0]) * direction[0]
839                 + (point[1] - source[1]) * direction[1]
840             ) >= -1e-7
841             for point in points
842         )
843         if not detected:
844             raise AssertionError((source, emission))
845
846     # 在局部坐标系中检查恢复点。
847     recovery = (
848         (RECOVERY_FORWARD_M, RECOVERY_SIDE_M),
849         (RECOVERY_FORWARD_M, -RECOVERY_SIDE_M),
850         (0.0, RECOVERY_SIDE_M),
851         (0.0, -RECOVERY_SIDE_M),
852     )
853     for r_i in range(6, 1501, 3):
854         for error_i in range(-100, 101, 5):
855             error = math.radians(error_i / 100.0)
856             source = (r_i * math.cos(error), r_i * math.sin(error))
857             worst_radius = max(MIN_RECEIVE_RADIUS, float(r_i))
858
859             for beta_i in range(0, 360, 3):
860                 beta = math.radians(beta_i)
861                 emission = (math.cos(beta), math.sin(beta))
862                 station_visible = (
863                     -source[0] * emission[0]
864                     - source[1] * emission[1]
865                 ) >= -1e-10
866                 if not station_visible:
867                     continue
868
869                 recovered = any(
870                     distance(point, source) <= worst_radius + 1e-7
871                     and (
872                         (point[0] - source[0]) * emission[0]
873                         + (point[1] - source[1]) * emission[1]
874                     ) >= -1e-7
875                     for point in recovery
876                 )
877                 if not recovered:
878                     raise AssertionError((source, beta_i))
879
880     test_pts = [
881         (10.0, 0.0),
882         (1.0, 0.0),
883         (2.0, 0.0),
884     ]
885     route_order = optimized_open_order(
886         (0.0, 0.0),
887         test_pts,
888     )
889     if sorted(route_order) != [0, 1, 2]:

```

```

890         raise AssertionError("2-opt 路线遗漏或重复节点")
891     if open_route_length(
892         (0.0, 0.0),
893         [test_pts[index] for index in route_order],
894     ) > 10.0 + 1e-9:
895         raise AssertionError(" 开放路线优化产生了额外折返")
896
897     scheduler = Q4Agent()
898     remaining_route = scheduler.remaining_search_route()
899     if (
900         len(remaining_route) != SEARCH_POINT_COUNT
901         or len({index for index, _ in remaining_route})
902         != SEARCH_POINT_COUNT
903     ):
904         raise AssertionError(" 未来搜索路线遗漏或重复 EW")
905
906     # 简单检查搜索期机会成本。
907     scheduler.channels[1].state = "LOCALIZING"
908     scheduler.channels[1].strong_position = remaining_route[1][1]
909     if scheduler.choose_action()[1][0] != "EW":
910         raise AssertionError(" 未来路线机会成本未能阻止提前折返")
911
912     scheduler.channels[1].strong_position = scheduler.position
913     if scheduler.choose_action()[1][0] != "CLEAR":
914         raise AssertionError(" 当前位置保证清除被错误延后")
915
916     marginal = scheduler.post_search_marginal_costs(
917         [
918             (0.0, ("CLEAR", 1, (10.0, 0.0), " 测试", False)),
919             (0.0, ("CLEAR", 2, (20.0, 0.0), " 测试", False)),
920         ],
921         (0.0, 0.0),
922     )
923     if marginal[1] > 1e-9 or abs(marginal[2] - 10.0) > 1e-9:
924         raise AssertionError(" 联合任务路线边际成本计算错误")
925
926     # SAFE 层自检。
927     # 新 clear 点必须保持严格可清除且路线不变长。
928     safe_agent = Q4Agent()
929     safe_agent.pending_ew.clear()
930     safe_agent.planned_ew.clear()
931     safe_agent.channels[1].state = "LOCALIZING"
932     safe_agent.channels[1].polygon = [
933         (9.0, -1.0),
934         (11.0, -1.0),
935         (11.0, 1.0),
936         (9.0, 1.0),
937     ]
938
939     safe_info = safe_agent.target_info(1)
940     old_target = safe_info.mec_center
941     start = (0.0, 0.0)
942     anchor = (20.0, 0.0)
943
944     candidate, _, _ = safe_agent.best_safe_clear_point_between(
945         1,
946         start,
947         old_target,
948         anchor,
949     )
950
951     if not safe_agent.point_is_strict_guaranteed_clear(
952         1,
953         candidate,
954     ):
955         raise AssertionError("SAFE clear 点失去严格 20m 保证")
956
957     old_path = (
958         distance(start, old_target)
959         + distance(old_target, anchor)
960     )
961     new_path = (
962         distance(start, candidate)
963         + distance(candidate, anchor)
964     )
965
966     if new_path > old_path + 1e-8:
967         raise AssertionError("SAFE 局部支配产生了更长路径")
968
969     print("Q4 geometry self-test passed")
970     for key, value in metrics.items():
971         print(f"{key}: {value:.6f}")
972
973
974 # HTTP 客户端
975
976 class OfficialClient:
977     def __init__(self) -> None:
978         self.request_counter = 0
979
980     def next_request_id(self, prefix: str) -> str:
981         self.request_counter += 1
982         return (
983             f"{prefix}-"
984             f"{int(time.time() * 1000)}-"
985             f"{self.request_counter}"
986         )
987
988     def base(self, request_id: str) -> dict:

```

```

989         return {
990             "arena_id": ARENA_ID,
991             "robot_id": ROBOT_ID,
992             "request_id": request_id,
993         }
994
995     def action_payload(
996         self,
997         request_id: str,
998         position: Point,
999         channel: int,
1000     ) -> dict:
1001         payload = self.base(request_id)
1002
1003         payload["position"] = {
1004             "x": float(position[0]),
1005             "y": float(position[1]),
1006         }
1007
1008         payload["channel"] = int(channel)
1009
1010         return payload
1011
1012     def post(self, path: str, payload: dict) -> dict:
1013         """ 发送请求; 重试时保持原 request_id 和请求体。 """
1014         raw_body = json.dumps(
1015             payload,
1016             ensure_ascii=False,
1017         ).encode("utf-8")
1018
1019         last_error: Optional[Exception] = None
1020
1021         for attempt in range(1, HTTP_RETRIES + 1):
1022             request = Request(
1023                 BASE_URL + path,
1024                 data=raw_body,
1025                 headers={"Content-Type": "application/json"},
1026                 method="POST",
1027             )
1028
1029             try:
1030                 with urlopen(
1031                     request,
1032                     timeout=HTTP_TIMEOUT_S,
1033                 ) as response:
1034                     return json.loads(
1035                         response.read().decode("utf-8")
1036                     )
1037
1038             except HTTPError as exc:
1039                 text = ""
1040
1041                 try:
1042                     text = exc.read().decode(
1043                         "utf-8",
1044                         errors="replace",
1045                     )
1046             except Exception:
1047                 pass
1048
1049             raise RuntimeError(
1050                 f"HTTP {exc.code}: {text}"
1051             ) from exc
1052
1053             except (
1054                 URLError,
1055                 TimeoutError,
1056                 socket.timeout,
1057                 ConnectionError,
1058             ) as exc:
1059                 last_error = exc
1060                 print(
1061                     f"[网络异常] attempt={attempt}, "
1062                     f"复用同一 request_id: {exc}"
1063                 )
1064
1065         assert last_error is not None
1066         raise last_error
1067
1068     def enter(self) -> dict:
1069         rid = self.next_request_id("enter")
1070         return self.post("/enter", self.base(rid))
1071
1072     def measure(
1073         self,
1074         position: Point,
1075         channel: int,
1076     ) -> dict:
1077         rid = self.next_request_id("measure")
1078
1079         return self.post(
1080             "/measure",
1081             self.action_payload(
1082                 rid,
1083                 position,
1084                 channel,
1085             ),
1086         )
1087

```

```

1088     def clear(
1089         self,
1090         position: Point,
1091         channel: int,
1092     ) -> dict:
1093         rid = self.next_request_id("clear")
1094
1095         return self.post(
1096             "/clear",
1097             self.action_payload(
1098                 rid,
1099                 position,
1100                 channel,
1101             ),
1102         )
1103
1104     def exit(self) -> dict:
1105         rid = self.next_request_id("exit")
1106         return self.post("/exit", self.base(rid))
1107
1108
1109 # 状态记录
1110
1111 @dataclass
1112 class DirectionObservation:
1113     station: Point
1114     bearing_deg: float
1115
1116
1117 @dataclass
1118 class ChannelRecord:
1119     state: str = "UNKNOWN"
1120     # UNKNOWN / LOCALIZING / CLEARED / CERTIFIED_EMPTY
1121
1122     directions: list[DirectionObservation] = field(
1123         default_factory=list
1124     )
1125
1126     polygon: Optional[list[Point]] = None
1127
1128     strong_position: Optional[Point] = None
1129
1130     measured_positions: list[Point] = field(
1131         default_factory=list
1132     )
1133
1134     no_signal_ew_indices: set[int] = field(
1135         default_factory=set
1136     )
1137
1138     lost: bool = False
1139
1140     # 四点恢复队列。
1141     recovery_generation: int = -1
1142     recovery_pending: list[Point] = field(default_factory=list)
1143
1144     sweep_pending: list[Point] = field(default_factory=list)
1145     sweep_polygon_version: int = -1
1146
1147     failed_clear_points: list[Point] = field(
1148         default_factory=list
1149     )
1150
1151
1152 @dataclass
1153 class Ledger:
1154     move_distance: float = 0.0
1155     move_seconds: float = 0.0
1156
1157     measure_count: int = 0
1158     measure_seconds: float = 0.0
1159
1160     switch_count: int = 0
1161     switch_seconds: float = 0.0
1162
1163     clear_success: int = 0
1164     clear_fail: int = 0
1165     clear_seconds: float = 0.0
1166
1167     @property
1168     def total_seconds(self) -> float:
1169         return (
1170             self.move_seconds
1171             + self.measure_seconds
1172             + self.switch_seconds
1173             + self.clear_seconds
1174         )
1175
1176
1177 # 主控制器
1178
1179 class Q4Agent:
1180     def __init__(self) -> None:
1181         self.client = OfficialClient()
1182
1183         self.position: Point = (0.0, 0.0)
1184         self.current_measure_channel = 1
1185
1186         self.channels = {

```

```

1187         ch: ChannelRecord()
1188         for ch in range(1, CHANNEL_COUNT + 1)
1189     }
1190
1191     self.pending_ew = build_backbone()
1192     self.planned_ew: List[tuple[int, Point]] = []
1193
1194     self.ledger = Ledger()
1195
1196     self.trace_sequence = 0
1197     self.remaining_real_duration_s: Optional[float] = None
1198     self.server_virtual_time_s: Optional[float] = None
1199     self.action_count_by_category: dict[str, int] = {}
1200     self.move_distance_by_category_m: dict[str, float] = {}
1201
1202     # 搜索统计
1203     self.early_stop_search_by_found16 = False
1204     self.geom_probe_attempts = 0
1205     self.geom_probe_success = 0
1206     self.search_visit_count = 0
1207     self.search_completed_virtual_time_s: Optional[float] = None
1208     self.opportunistic_measure_count = 0
1209     self.opportunistic_receive_count = 0
1210     self.recovery_attempts = 0
1211     self.recovery_receive_count = 0
1212     self.first_discovery_virtual_time_s: dict[int, float] = {}
1213
1214     # 路线统计
1215     self.search_insertion_action_count = 0
1216     self.search_insertion_detour_m = 0.0
1217     self.search_insertion_future_cost_m = 0.0
1218     self.search_insertion_count_by_category: dict[str, int] = {}
1219     self.future_ew_deferred_states: set[tuple[int, int, bool]] = set()
1220     self.post_search_route_plan_count = 0
1221
1222     # SAFE 层统计
1223     # 保存原 V5 下一动作，便于下一步汇合。
1224     self.safe_forced_action: Optional[tuple[float, tuple]] = None
1225     self.safe_forced_channels_patch = None
1226     self.safe_forced_planned_ew_patch = None
1227
1228     self.safe_clear_shift_count = 0
1229     self.safe_clear_shift_saved_m = 0.0
1230     self.safe_clear_segment_hit_count = 0
1231
1232
1233     # 日志与统计
1234
1235     @property
1236     def cleared_count(self) -> int:
1237         return sum(
1238             record.state == "CLEARED"
1239             for record in self.channels.values()
1240         )
1241
1242     @property
1243     def empty_count(self) -> int:
1244         return sum(
1245             record.state == "CERTIFIED_EMPTY"
1246             for record in self.channels.values()
1247         )
1248
1249     @property
1250     def found_count(self) -> int:
1251         """ 统计已经实际发现的有源频道数。 """
1252         return sum(
1253             record.state in {"LOCALIZING", "CLEARED"}
1254             for record in self.channels.values()
1255         )
1256
1257     def active_channels(self) -> List[int]:
1258         return [
1259             ch
1260             for ch, record in self.channels.items()
1261             if record.state == "LOCALIZING"
1262         ]
1263
1264     def trace(self, event: dict) -> None:
1265         self.trace_sequence += 1
1266
1267         row = {
1268             "seq": self.trace_sequence,
1269             "virtual_time_est": self.ledger.total_seconds,
1270             "position": {
1271                 "x": self.position[0],
1272                 "y": self.position[1],
1273             },
1274             "cleared": self.cleared_count,
1275             "empty": self.empty_count,
1276             "ew_left": len(self.pending_ew),
1277             **event,
1278         }
1279
1280         with TRACE_PATH.open(
1281             "a",
1282             encoding="utf-8",
1283         ) as handle:
1284             handle.write(
1285                 json.dumps(

```

```

1286         row,
1287         ensure_ascii=False,
1288     )
1289     + "\n"
1290 )
1291
1292 def account_move(self, target: Point) -> float:
1293     moved = distance(self.position, target)
1294
1295     self.ledger.move_distance += moved
1296     self.ledger.move_seconds += moved / SPEED
1297
1298     self.position = target
1299
1300     return moved
1301
1302 @staticmethod
1303 def reason_category(reason: str) -> str:
1304     for prefix, category in (
1305         ("EW", "search_waypoint"),
1306         ("Q2", "q2_measure"),
1307         ("直接", "direct_refine"),
1308         ("定向源四点", "directional_recovery"),
1309         ("MEC", "mec_clear"),
1310         ("两线", "geometric_probe_clear"),
1311         ("强 Probe", "strong_probe_clear"),
1312         ("弱 Probe", "weak_probe_clear"),
1313         ("后验网格", "posterior_sweep_clear"),
1314         ("near", "near_clear"),
1315         ("clear 失败", "post_clear_measure"),
1316     ):
1317         if reason.startswith(prefix):
1318             return category
1319     return "other"
1320
1321 def account_action_category(
1322     self,
1323     reason: str,
1324     moved: float,
1325 ) -> None:
1326     category = self.reason_category(reason)
1327     self.action_count_by_category[category] = (
1328         self.action_count_by_category.get(category, 0) + 1
1329     )
1330     self.move_distance_by_category_m[category] = (
1331         self.move_distance_by_category_m.get(category, 0.0) + moved
1332     )
1333
1334 # 后验更新
1335
1336 def target_info(self, channel: int) -> TargetInfo:
1337     record = self.channels[channel]
1338
1339     if record.strong_position is not None:
1340         return TargetInfo(
1341             center=record.strong_position,
1342             work_radius=0.0,
1343             mec_center=record.strong_position,
1344             mec_radius=5.0,
1345         )
1346
1347     if not record.polygon:
1348         raise RuntimeError(
1349             f"频道 {channel} 没有可用后验"
1350         )
1351
1352     return target_info_from_polygon(record.polygon)
1353
1354 # measure
1355
1356 def measure_at(
1357     self,
1358     position: Point,
1359     channel: int,
1360     *,
1361     ew_index: Optional[int] = None,
1362     mark_lost_on_no_signal: bool = True,
1363     reason: str = "",
1364 ) -> tuple[str, Optional[float]]:
1365     before = self.position
1366
1367     response = self.client.measure(
1368         position,
1369         channel,
1370     )
1371
1372     if response.get("accepted") is not True:
1373         raise RuntimeError(
1374             f"/measure rejected: {response}"
1375         )
1376
1377     if "virtual_time_s" in response:
1378         self.server_virtual_time_s = float(response["virtual_time_s"])
1379
1380     moved = self.account_move(position)
1381     self.account_action_category(reason, moved)
1382
1383     switched = (
1384         self.current_measure_channel

```

```

1385         != channel
1386     )
1387
1388     if switched:
1389         self.ledger.switch_count += 1
1390         self.ledger.switch_seconds += SWITCH_SECONDS
1391
1392     self.current_measure_channel = channel
1393
1394     self.ledger.measure_count += 1
1395     self.ledger.measure_seconds += MEASURE_SECONDS
1396
1397     record = self.channels[channel]
1398     was_unknown = record.state == "UNKNOWN"
1399     record.measured_positions.append(position)
1400
1401     result = response["measure_result"]
1402     bearing: Optional[float] = None
1403
1404     if result == "direction":
1405         bearing = float(response["svd_deg"])
1406
1407         record.state = "LOCALIZING"
1408         record.lost = False
1409         record.recovery_pending.clear()
1410
1411         record.directions.append(
1412             DirectionObservation(
1413                 station=position,
1414                 bearing_deg=bearing,
1415             )
1416         )
1417
1418         polygon = update_polygon_with_direction(
1419             record.polygon,
1420             position,
1421             bearing,
1422         )
1423
1424         if not polygon:
1425             raise RuntimeError(
1426                 f" 频道 {channel} 后验意外为空"
1427             )
1428
1429         record.polygon = polygon
1430
1431     elif result == "near":
1432         record.state = "LOCALIZING"
1433         record.lost = False
1434         record.recovery_pending.clear()
1435         record.strong_position = position
1436
1437     elif result == "no_signal":
1438         if (
1439             record.state == "UNKNOWN"
1440             and ew_index is not None
1441         ):
1442             record.no_signal_ew_indices.add(
1443                 ew_index
1444             )
1445
1446         if (
1447             record.state == "LOCALIZING"
1448             and mark_lost_on_no_signal
1449         ):
1450             record.lost = True
1451
1452     else:
1453         raise RuntimeError(
1454             f" 未知 measure_result: {result!r}"
1455         )
1456
1457     if was_unknown and result in {"direction", "near"}:
1458         self.first_discovery_virtual_time_s[channel] = (
1459             self.ledger.total_seconds
1460         )
1461
1462     diagnostic = {}
1463
1464     if (
1465         record.state == "LOCALIZING"
1466         and record.polygon
1467     ):
1468         info = self.target_info(channel)
1469
1470         diagnostic = {
1471             "work_radius": info.work_radius,
1472             "mec_radius": info.mec_radius,
1473             "direction_count": len(
1474                 record.directions
1475             ),
1476         }
1477
1478     self.trace(
1479         {
1480             "action": "measure",
1481             "channel": channel,
1482             "reason": reason,
1483             "from": {

```

```

1484         "x": before[0],
1485         "y": before[1],
1486     },
1487     "to": {
1488         "x": position[0],
1489         "y": position[1],
1490     },
1491     "move_m": moved,
1492     "switch": switched,
1493     "result": result,
1494     "bearing": bearing,
1495     **diagnostic,
1496 }
1497 )
1498
1499 # near 后直接 clear。
1500 if result == "near":
1501     self.clear_at(
1502         position,
1503         channel,
1504         reason="near 后原地保证清除",
1505         measure_after_failure=False,
1506     )
1507
1508     self.apply_found16_shortcut()
1509
1510     return result, bearing
1511
1512 # clear
1513
1514 def clear_at(
1515     self,
1516     position: Point,
1517     channel: int,
1518     *,
1519     reason: str,
1520     measure_after_failure: bool,
1521 ) -> bool:
1522     before = self.position
1523
1524     response = self.client.clear(
1525         position,
1526         channel,
1527     )
1528
1529     if response.get("accepted") is not True:
1530         raise RuntimeError(
1531             f"/clear rejected: {response}"
1532         )
1533
1534     if "virtual_time_s" in response:
1535         self.server_virtual_time_s = float(response["virtual_time_s"])
1536
1537     moved = self.account_move(position)
1538     self.account_action_category(reason, moved)
1539
1540     success = (
1541         response.get("clear_result")
1542         == "success"
1543     )
1544
1545     record = self.channels[channel]
1546
1547     if success:
1548         self.ledger.clear_success += 1
1549         self.ledger.clear_seconds += CLEAR_SUCCESS_SECONDS
1550
1551         record.state = "CLEARED"
1552         record.lost = False
1553
1554     else:
1555         # SAFE clear 意外失败时取消缓存动作。
1556         self.safe_forced_action = None
1557         self.safe_forced_channels_patch = None
1558         self.safe_forced_planned_ew_patch = None
1559
1560         self.ledger.clear_fail += 1
1561         self.ledger.clear_seconds += CLEAR_FAILURE_SECONDS
1562
1563         record.failed_clear_points.append(
1564             position
1565         )
1566
1567     self.trace(
1568         {
1569             "action": "clear",
1570             "channel": channel,
1571             "reason": reason,
1572             "from": {
1573                 "x": before[0],
1574                 "y": before[1],
1575             },
1576             "to": {
1577                 "x": position[0],
1578                 "y": position[1],
1579             },
1580             "move_m": moved,
1581             "result": (
1582                 "success"

```

```

1583         if success
1584         else response.get(
1585             "clear_result",
1586             "fail",
1587         )
1588     ),
1589 }
1590 )
1591
1592 if (
1593     not success
1594     and measure_after_failure
1595 ):
1596     self.measure_at(
1597         position,
1598         channel,
1599         mark_lost_on_no_signal=True,
1600         reason="clear 失败后原地补测",
1601     )
1602
1603     self.apply_found16_shortcut()
1604
1605     return success
1606
1607 # Q2-fast 与恢复
1608
1609 def q2_fast_candidates(
1610     self,
1611     channel: int,
1612 ) -> tuple[Point, Point]:
1613     info = self.target_info(channel)
1614     observation = self.channels[channel].directions[-1]
1615
1616     theta = math.radians(
1617         observation.bearing_deg
1618     )
1619
1620     normal = (
1621         -math.sin(theta),
1622         math.cos(theta),
1623     )
1624
1625     center = info.center
1626
1627     return (
1628         (
1629             center[0]
1630             + Q2_FAST_SIDE_M * normal[0],
1631             center[1]
1632             + Q2_FAST_SIDE_M * normal[1],
1633         ),
1634         (
1635             center[0]
1636             - Q2_FAST_SIDE_M * normal[0],
1637             center[1]
1638             - Q2_FAST_SIDE_M * normal[1],
1639         ),
1640     )
1641
1642 def q2_fast_information(
1643     self,
1644     channel: int,
1645     point: Point,
1646 ) -> float:
1647     info = self.target_info(channel)
1648     last = self.channels[channel].directions[-1]
1649
1650     a = (
1651         info.center[0] - last.station[0],
1652         info.center[1] - last.station[1],
1653     )
1654
1655     b = (
1656         info.center[0] - point[0],
1657         info.center[1] - point[1],
1658     )
1659
1660     na = math.hypot(a[0], a[1])
1661     nb = math.hypot(b[0], b[1])
1662
1663     if na <= 1e-9 or nb <= 1e-9:
1664         return 1.0
1665
1666     return min(
1667         1.0,
1668         abs(cross(a, b)) / (na * nb),
1669     )
1670
1671 def recovery_candidates(
1672     self,
1673     channel: int,
1674 ) -> tuple[Point, Point, Point, Point]:
1675     """ 根据最后一次成功示向生成四个恢复测点。 """
1676     last = self.channels[channel].directions[-1]
1677
1678     theta = math.radians(
1679         last.bearing_deg
1680     )
1681

```

```

1682     forward = (
1683         math.cos(theta),
1684         math.sin(theta),
1685     )
1686
1687     normal = (
1688         -math.sin(theta),
1689         math.cos(theta),
1690     )
1691
1692     s = last.station
1693
1694     forward_pair = (
1695         (
1696             s[0]
1697             + RECOVERY_FORWARD_M * forward[0]
1698             + RECOVERY_SIDE_M * normal[0],
1699             s[1]
1700             + RECOVERY_FORWARD_M * forward[1]
1701             + RECOVERY_SIDE_M * normal[1],
1702         ),
1703         (
1704             s[0]
1705             + RECOVERY_FORWARD_M * forward[0]
1706             - RECOVERY_SIDE_M * normal[0],
1707             s[1]
1708             + RECOVERY_FORWARD_M * forward[1]
1709             - RECOVERY_SIDE_M * normal[1],
1710         ),
1711     )
1712
1713     lateral_pair = (
1714         (
1715             s[0] + RECOVERY_SIDE_M * normal[0],
1716             s[1] + RECOVERY_SIDE_M * normal[1],
1717         ),
1718         (
1719             s[0] - RECOVERY_SIDE_M * normal[0],
1720             s[1] - RECOVERY_SIDE_M * normal[1],
1721         ),
1722     )
1723
1724     return forward_pair + lateral_pair
1725
1726 def apply_found16_shortcut(self) -> None:
1727     """ 已发现 16 个频道时, 直接结束剩余未知频道搜索。"""
1728     if self.found_count < SOURCE_MAX:
1729         return
1730
1731     changed = False
1732
1733     for record in self.channels.values():
1734         if record.state == "UNKNOWN":
1735             record.state = "CERTIFIED_EMPTY"
1736             changed = True
1737
1738     if self.pending_ew:
1739         self.pending_ew.clear()
1740         self.planned_ew.clear()
1741         changed = True
1742
1743     if changed:
1744         self.early_stop_search_by_found16 = True
1745         self.trace(
1746             {
1747                 "event": "found16_cancel_search",
1748                 "found_count": self.found_count,
1749             }
1750         )
1751
1752 def best_geometric_probe(
1753     self,
1754     channel: int,
1755 ) -> Optional[GeometricProbe]:
1756     record = self.channels[channel]
1757
1758     if len(record.directions) < 2 or not record.polygon:
1759         return None
1760
1761     best: Optional[GeometricProbe] = None
1762
1763     # 观测数很少, 直接枚举组合。
1764     for first, second in itertools.combinations(
1765         record.directions,
1766         2,
1767     ):
1768         candidate = bearing_pair_probe(
1769             first,
1770             second,
1771         )
1772
1773         if candidate is None:
1774             continue
1775
1776         if (
1777             candidate.error_proxy_m
1778             > GEOM_PROBE_MAX_ERROR_M
1779         ):
1780             continue

```

```

1781         # 交点必须在当前后验内。
1782         if not point_in_convex_polygon(
1783             candidate.point,
1784             record.polygon,
1785         ):
1786             continue
1787
1788         if (
1789             candidate.point[0] ** 2
1790             + candidate.point[1] ** 2
1791             > TARGET_RADIUS ** 2 + 1e-6
1792         ):
1793             continue
1794
1795         if (
1796             best is None
1797             or candidate.error_proxy_m
1798             < best.error_proxy_m
1799         ):
1800             best = candidate
1801
1802         return best
1803
1804     # EW 顺路复测
1805
1806     def already_measured_near(
1807         self,
1808         channel: int,
1809         point: Point,
1810         radius: float = 1.0,
1811     ) -> bool:
1812         return any(
1813             distance(point, old) <= radius
1814             for old
1815             in self.channels[channel].measured_positions
1816         )
1817
1818     def opportunistic_score(
1819         self,
1820         channel: int,
1821         ew: Point,
1822     ) -> Optional[float]:
1823         record = self.channels[channel]
1824
1825         if (
1826             record.state != "LOCALIZING"
1827             or not record.directions
1828             or record.strong_position is not None
1829         ):
1830             return None
1831
1832         if self.already_measured_near(
1833             channel,
1834             ew,
1835         ):
1836             return None
1837
1838         # lost 频道优先重捕获。
1839         if record.lost:
1840             return 100.0
1841
1842         info = self.target_info(channel)
1843
1844         # 已接近清除时不再额外 EW 复测。
1845         if (
1846             info.mec_radius <= CLEAR_RADIUS
1847             or (
1848                 len(record.directions) >= 2
1849                 and info.work_radius <= STRONG_PROBE_RADIUS_M
1850             )
1851             or self.best_geometric_probe(channel) is not None
1852         ):
1853             return None
1854
1855         if (
1856             distance(
1857                 ew,
1858                 info.center,
1859             )
1860             > MAX_OPPORTUNISTIC_CENTER_DISTANCE_M
1861         ):
1862             return None
1863
1864         last = record.directions[-1]
1865
1866         baseline = distance(
1867             ew,
1868             last.station,
1869         )
1870
1871         if baseline < MIN_OPPORTUNISTIC_BASELINE_M:
1872             return None
1873
1874         v1 = (
1875             info.center[0] - last.station[0],
1876             info.center[1] - last.station[1],
1877         )
1878
1879

```

```

1880     v2 = (
1881         info.center[0] - ew[0],
1882         info.center[1] - ew[1],
1883     )
1884
1885     n1 = math.hypot(v1[0], v1[1])
1886     n2 = math.hypot(v2[0], v2[1])
1887
1888     if n1 <= 1e-9 or n2 <= 1e-9:
1889         sin_phi = 1.0
1890     else:
1891         sin_phi = min(
1892             1.0,
1893             abs(cross(v1, v2)) / (n1 * n2),
1894         )
1895
1896     if sin_phi < MIN_OPPORTUNISTIC_SIN:
1897         return None
1898
1899     # 先看交会角，再看基线。
1900     return (
1901         10.0 * sin_phi
1902         + min(baseline, 1500.0) / 1500.0
1903     )
1904
1905 def execute_ew(
1906     self,
1907     ew_index: int,
1908     ew: Point,
1909 ) -> None:
1910     self.search_visit_count += 1
1911     unknown = [
1912         ch
1913         for ch, record in self.channels.items()
1914         if record.state == "UNKNOWN"
1915     ]
1916
1917     active_scores: list[tuple[float, int]] = []
1918
1919     for ch in self.active_channels():
1920         score = self.opportunistic_score(
1921             ch,
1922             ew,
1923         )
1924
1925         if score is not None:
1926             active_scores.append(
1927                 (score, ch)
1928             )
1929
1930     active_scores.sort(
1931         reverse=True
1932     )
1933
1934     opportunistic = [
1935         ch
1936         for _, ch
1937         in active_scores[
1938             :MAX_OPPORTUNISTIC_PER_EW
1939         ]
1940     ]
1941
1942     todo = list(
1943         dict.fromkeys(
1944             unknown
1945             + opportunistic
1946         )
1947     )
1948
1949     # 当前频道优先。
1950     if self.current_measure_channel in todo:
1951         todo.remove(
1952             self.current_measure_channel
1953         )
1954         todo.insert(
1955             0,
1956             self.current_measure_channel,
1957         )
1958
1959     if not todo:
1960         return
1961
1962     for ch in todo:
1963         is_unknown = (
1964             self.channels[ch].state
1965             == "UNKNOWN"
1966         )
1967
1968         result, _ = self.measure_at(
1969             ew,
1970             ch,
1971             ew_index=(
1972                 ew_index
1973                 if is_unknown
1974                 else None
1975             ),
1976             # EW 上的 no_signal 不改变 active 状态。
1977             mark_lost_on_no_signal=is_unknown,
1978             reason=(

```

```

1979         "EW 存在性扫描"
1980         if is_unknown
1981             else "EW 零移动成本顺路定位"
1982     ),
1983 )
1984
1985     if not is_unknown:
1986         self.opportunistic_measure_count += 1
1987         if result in {"direction", "near"}:
1988             self.opportunistic_receive_count += 1
1989
1990     # 找到 16 个源后停止剩余 UNKNOWN 扫描。
1991     if self.found_count >= SOURCE_MAX:
1992         return
1993
1994 # 空频道判定
1995
1996 def certify_empty(self) -> None:
1997     if self.pending_ew:
1998         return
1999
2000     required = set(range(SEARCH_POINT_COUNT))
2001
2002     for record in self.channels.values():
2003         if (
2004             record.state == "UNKNOWN"
2005             and required.issubset(
2006                 record.no_signal_ew_indices
2007             )
2008         ):
2009             record.state = "CERTIFIED_EMPTY"
2010
2011 # clear 去重
2012
2013 def too_close_to_failed_clear(
2014     self,
2015     channel: int,
2016     point: Point,
2017 ) -> bool:
2018     return any(
2019         distance(point, old)
2020         <= FAILED_CLEAR_AVOID_RADIUS_M
2021         for old
2022         in self.channels[channel].failed_clear_points
2023     )
2024
2025 def build_posterior_sweep(
2026     self,
2027     channel: int,
2028 ) -> List[Point]:
2029     """ 后验足够小时生成固定步长的清除网格。"""
2030     polygon = self.channels[channel].polygon
2031     if not polygon:
2032         return []
2033
2034     xmin = min(point[0] for point in polygon)
2035     xmax = max(point[0] for point in polygon)
2036     ymin = min(point[1] for point in polygon)
2037     ymax = max(point[1] for point in polygon)
2038
2039     nx = max(1, math.ceil((xmax - xmin) / SWEEP_GRID_STEP_M))
2040     ny = max(1, math.ceil((ymax - ymin) / SWEEP_GRID_STEP_M))
2041     xs = [xmin + i * SWEEP_GRID_STEP_M for i in range(nx + 1)]
2042     ys = [ymin + i * SWEEP_GRID_STEP_M for i in range(ny + 1)]
2043
2044     points: List[Point] = []
2045     for row, y in enumerate(ys):
2046         row_points = [(x, y) for x in xs]
2047         if row % 2:
2048             row_points.reverse()
2049         points.extend(row_points)
2050
2051     # Sweep 保持原 V5 顺序。
2052     return points
2053
2054 # SAFE 局部路径优化
2055
2056 def is_strict_guaranteed_clear_action(
2057     self,
2058     action: tuple,
2059 ) -> bool:
2060     if action[0] != "CLEAR":
2061         return False
2062
2063     reason = action[3]
2064     return (
2065         reason.startswith("MEC<=20m 保证清除")
2066         or reason.startswith("near 保证清除")
2067     )
2068
2069 def point_is_strict_guaranteed_clear(
2070     self,
2071     channel: int,
2072     point: Point,
2073 ) -> bool:
2074     """ 检查给定位置是否仍满足严格清除条件。"""
2075     record = self.channels[channel]
2076     limit = CLEAR_RADIUS - SAFE_CLEAR_MARGIN_M
2077

```

```

2078         if record.strong_position is not None:
2079             return (
2080                 distance(
2081                     point,
2082                     record.strong_position,
2083                 )
2084                 + NEAR_RADIUS
2085                 <= CLEAR_RADIUS - SAFE_CLEAR_MARGIN_M
2086             )
2087
2088         if not record.polygon:
2089             return False
2090
2091         return all(
2092             distance(point, vertex) <= limit + 1e-9
2093             for vertex in record.polygon
2094         )
2095
2096     def clear_segment_feasible_point(
2097         self,
2098         channel: int,
2099         start: Point,
2100         end: Point,
2101         preferred: Point,
2102     ) -> Optional[Point]:
2103         """ 若路径线段穿过严格清除区域，返回可用清除点。"""
2104         record = self.channels[channel]
2105         limit = CLEAR_RADIUS - SAFE_CLEAR_MARGIN_M
2106
2107         lo = 0.0
2108         hi = 1.0
2109
2110         constraints: list[tuple[Point, float]] = []
2111
2112         if record.strong_position is not None:
2113             constraints.append(
2114                 (
2115                     record.strong_position,
2116                     CLEAR_RADIUS
2117                     - NEAR_RADIUS
2118                     - SAFE_CLEAR_MARGIN_M,
2119                 )
2120             )
2121         elif record.polygon:
2122             constraints.extend(
2123                 (vertex, limit)
2124                 for vertex in record.polygon
2125             )
2126         else:
2127             return None
2128
2129         for center, radius in constraints:
2130             interval = segment_disk_parameter_interval(
2131                 start,
2132                 end,
2133                 center,
2134                 radius,
2135             )
2136             if interval is None:
2137                 return None
2138
2139             lo = max(lo, interval[0])
2140             hi = min(hi, interval[1])
2141
2142             if lo > hi + 1e-12:
2143                 return None
2144
2145             dx = end[0] - start[0]
2146             dy = end[1] - start[1]
2147             dd = dx * dx + dy * dy
2148
2149             if dd <= 1e-18:
2150                 candidate = start
2151             else:
2152                 preferred_t = (
2153                     (preferred[0] - start[0]) * dx
2154                     + (preferred[1] - start[1]) * dy
2155                 ) / dd
2156                 t = min(hi, max(lo, preferred_t))
2157                 candidate = (
2158                     start[0] + t * dx,
2159                     start[1] + t * dy,
2160                 )
2161
2162             if self.point_is_strict_guaranteed_clear(
2163                 channel,
2164                 candidate,
2165             ):
2166                 return candidate
2167
2168         return None
2169
2170     def clear_mec_safe_radius(
2171         self,
2172         channel: int,
2173         original_target: Point,
2174     ) -> float:
2175         """ 计算以原 MEC 中心为圆心的安全移动半径。"""
2176         record = self.channels[channel]

```

```

2177
2178     if record.strong_position is not None:
2179         if distance(
2180             original_target,
2181             record.strong_position,
2182         ) <= 1e-6:
2183             return max(
2184                 0.0,
2185                 CLEAR_RADIUS
2186                 - NEAR_RADIUS
2187                 - SAFE_CLEAR_MARGIN_M,
2188             )
2189         return 0.0
2190
2191     if not record.polygon:
2192         return 0.0
2193
2194     info = self.target_info(channel)
2195
2196     if distance(
2197         original_target,
2198         info.mec_center,
2199     ) > 1e-5:
2200         return 0.0
2201
2202     return max(
2203         0.0,
2204         CLEAR_RADIUS
2205         - info.mec_radius
2206         - SAFE_CLEAR_MARGIN_M,
2207     )
2208
2209 def best_safe_clear_point_between(
2210     self,
2211     channel: int,
2212     start: Point,
2213     original_target: Point,
2214     next_target: Optional[Point],
2215 ) -> tuple[Point, float, bool]:
2216     """ 在保证清除成功的前提下寻找更短的清除位置。"""
2217     if next_target is None:
2218         # 最后一动作尽量把 clear 点往当前位置移动。
2219         radius = self.clear_mec_safe_radius(
2220             channel,
2221             original_target,
2222         )
2223         old_length = distance(
2224             start,
2225             original_target,
2226         )
2227
2228         if radius <= 1e-12 or old_length <= 1e-12:
2229             return original_target, 0.0, False
2230
2231         shift = min(radius, old_length)
2232         candidate = (
2233             original_target[0]
2234             + (
2235                 start[0] - original_target[0]
2236             )
2237             * shift
2238             / old_length,
2239             original_target[1]
2240             + (
2241                 start[1] - original_target[1]
2242             )
2243             * shift
2244             / old_length,
2245         )
2246
2247         if not self.point_is_strict_guaranteed_clear(
2248             channel,
2249             candidate,
2250         ):
2251             return original_target, 0.0, False
2252
2253         new_length = distance(start, candidate)
2254         saved = old_length - new_length
2255
2256         if saved <= SAFE_MIN_ROUTE_GAIN_M:
2257             return original_target, 0.0, False
2258
2259         return candidate, saved, False
2260
2261     old_length = (
2262         distance(start, original_target)
2263         + distance(original_target, next_target)
2264     )
2265
2266     # 优先找路径线段上的可清除点。
2267     seg_point = self.clear_segment_feasible_point(
2268         channel,
2269         start,
2270         next_target,
2271         original_target,
2272     )
2273
2274     if seg_point is not None:
2275         new_length = (

```

```

2276         distance(start, seg_point)
2277         + distance(seg_point, next_target)
2278     )
2279     saved = old_length - new_length
2280
2281     if saved > SAFE_MIN_ROUTE_GAIN_M:
2282         return seg_point, saved, True
2283
2284     # 否则在 MEC 安全圆上找更短点。
2285     radius = self.clear_mec_safe_radius(
2286         channel,
2287         original_target,
2288     )
2289
2290     if radius <= 1e-12:
2291         return original_target, 0.0, False
2292
2293     best_point = original_target
2294     best_length = old_length
2295
2296     # 这里只需找到严格更短的可行点。
2297     # 找不到就回退原目标。
2298     for index in range(180):
2299         angle = 2.0 * math.pi * index / 180.0
2300         candidate = (
2301             original_target[0]
2302             + radius * math.cos(angle),
2303             original_target[1]
2304             + radius * math.sin(angle),
2305         )
2306
2307         if not self.point_is_strict_guaranteed_clear(
2308             channel,
2309             candidate,
2310         ):
2311             continue
2312
2313         cand_len = (
2314             distance(start, candidate)
2315             + distance(candidate, next_target)
2316         )
2317
2318         if cand_len + 1e-9 < best_length:
2319             best_length = cand_len
2320             best_point = candidate
2321
2322     saved = old_length - best_length
2323
2324     if saved <= SAFE_MIN_ROUTE_GAIN_M:
2325         return original_target, 0.0, False
2326
2327     return best_point, saved, False
2328
2329 @staticmethod
2330 def shadow_apply_guaranteed_clear_success(
2331     shadow: "Q4Agent",
2332     action: tuple,
2333 ) -> None:
2334     """ 在副本状态中模拟一次必成功清除，不发送 HTTP 请求。"""
2335     _, channel, point, _, _ = action
2336     shadow.position = point
2337
2338     record = shadow.channels[channel]
2339     record.state = "CLEARED"
2340     record.lost = False
2341     record.recovery_pending.clear()
2342     record.sweep_pending.clear()
2343
2344     shadow.apply_found16_shortcut()
2345     shadow.certify_empty()
2346
2347 def safe_transform_selected_action(
2348     self,
2349     chosen: tuple[float, tuple],
2350 ) -> tuple[float, tuple]:
2351     """ 保持原 V5 动作顺序，只尝试缩短已选保证清除动作的路径。"""
2352     score, action = chosen
2353
2354     if not self.is_strict_guaranteed_clear_action(action):
2355         return chosen
2356
2357     _, channel, original_target, reason, measure_after_failure = action
2358
2359     # 先复核原 clear 点。
2360     if not self.point_is_strict_guaranteed_clear(
2361         channel,
2362         original_target,
2363     ):
2364         return chosen
2365
2366     shadow = copy.deepcopy(self)
2367     self.shadow_apply_guaranteed_clear_success(
2368         shadow,
2369         action,
2370     )
2371
2372     next_chosen = shadow._choose_action_v5()
2373     next_target = (
2374         next_chosen[1][2]

```

```

2375         if next_chosen is not None
2376         else None
2377     )
2378
2379     new_target, saved_m, segment_hit = (
2380         self.best_safe_clear_point_between(
2381             channel,
2382             self.position,
2383             original_target,
2384             next_target,
2385         )
2386     )
2387
2388     if saved_m <= SAFE_MIN_ROUTE_GAIN_M:
2389         return chosen
2390
2391     # 新点再核验一次。
2392     if not self.point_is_strict_guaranteed_clear(
2393         channel,
2394         new_target,
2395     ):
2396         return chosen
2397
2398     # 保存 shadow 中的下一动作和计划。
2399     if next_chosen is not None:
2400         self.safe_forced_action = next_chosen
2401         self.safe_forced_channels_patch = copy.deepcopy(
2402             shadow.channels
2403         )
2404         self.safe_forced_planned_ew_patch = copy.deepcopy(
2405             shadow.planned_ew
2406         )
2407
2408     self.safe_clear_shift_count += 1
2409     self.safe_clear_shift_saved_m += saved_m
2410
2411     if segment_hit:
2412         self.safe_clear_segment_hit_count += 1
2413
2414     self.trace(
2415         {
2416             "event": "safe_clear_local_dominance",
2417             "channel": channel,
2418             "reason": reason,
2419             "old_target": {
2420                 "x": original_target[0],
2421                 "y": original_target[1],
2422             },
2423             "new_target": {
2424                 "x": new_target[0],
2425                 "y": new_target[1],
2426             },
2427             "next_anchor": (
2428                 {
2429                     "x": next_target[0],
2430                     "y": next_target[1],
2431                 }
2432                 if next_target is not None
2433                 else None
2434             ),
2435             "saved_m_est": saved_m,
2436             "segment_zero_detour": segment_hit,
2437         }
2438     )
2439
2440     new_action = (
2441         "CLEAR",
2442         channel,
2443         new_target,
2444         reason + " [SAFE 局部支配点]",
2445         measure_after_failure,
2446     )
2447
2448     # 保留原 V5 分数。
2449     return score, new_action
2450
2451 # 任务选择
2452
2453 def search_route_information_key(
2454     self,
2455     route: list[tuple[int, Point]],
2456 ) -> tuple[int, float]:
2457     """ 等长环路中，让可顺路形成大交会角的 EW 尽量提前。"""
2458     total_delay = 0
2459     total_quality = 0.0
2460
2461     for ch in self.active_channels():
2462         opportunities = []
2463
2464         for step, (_, point) in enumerate(route):
2465             score = self.opportunistic_score(ch, point)
2466
2467             if score is not None:
2468                 opportunities.append((step, -score))
2469
2470         if opportunities:
2471             first_step, negative_score = min(opportunities)
2472             total_delay += first_step
2473             total_quality -= negative_score

```

```

2474         return total_delay, -total_quality
2475
2476
2477 def plan_current_search_phase(self) -> None:
2478     """ 为当前搜索环生成固定开放路线。"""
2479     if not self.pending_ew:
2480         self.planned_ew.clear()
2481         return
2482
2483     current_phase = min(
2484         search_phase(index)
2485         for index, _ in self.pending_ew
2486     )
2487     phase_entries = sorted(
2488         (
2489             item
2490             for item in self.pending_ew
2491             if search_phase(item[0]) == current_phase
2492         ),
2493         key=lambda item: item[0],
2494     )
2495
2496     if len(phase_entries) <= 1:
2497         self.planned_ew = phase_entries
2498         return
2499
2500     routes: List[List[tuple[int, Point]]] = []
2501     count = len(phase_entries)
2502
2503     for start in range(count):
2504         routes.append(
2505             [
2506                 phase_entries[(start + offset) % count]
2507                 for offset in range(count)
2508             ]
2509         )
2510         routes.append(
2511             [
2512                 phase_entries[(start - offset) % count]
2513                 for offset in range(count)
2514             ]
2515         )
2516
2517     lengths = [
2518         open_route_length(
2519             self.position,
2520             [point for _, point in route],
2521         )
2522         for route in routes
2523     ]
2524     shortest_length = min(lengths)
2525     shortest_routes = [
2526         route
2527         for route, length in zip(routes, lengths)
2528         if length <= shortest_length + 1e-7
2529     ]
2530
2531     self.planned_ew = min(
2532         shortest_routes,
2533         key=self.search_route_information_key,
2534     )
2535
2536 def next_search_waypoint(self) -> Optional[tuple[int, Point]]:
2537     if not self.pending_ew:
2538         return None
2539
2540     pending_indices = {
2541         index
2542         for index, _ in self.pending_ew
2543     }
2544     self.planned_ew = [
2545         item
2546         for item in self.planned_ew
2547         if item[0] in pending_indices
2548     ]
2549
2550     if not self.planned_ew:
2551         self.plan_current_search_phase()
2552
2553     return self.planned_ew[0]
2554
2555 def mark_search_waypoint_done(self, ew_index: int) -> None:
2556     self.planned_ew = [
2557         item
2558         for item in self.planned_ew
2559         if item[0] != ew_index
2560     ]
2561
2562 def remaining_search_route(self) -> List[tuple[int, Point]]:
2563     """ 返回当前固定环之后的完整剩余搜索路线，不改变实际计划。"""
2564     if not self.pending_ew:
2565         return []
2566
2567     # 确保当前搜索环已有路线。
2568     self.next_search_waypoint()
2569
2570     pending_indices = {
2571         index
2572         for index, _ in self.pending_ew

```

```

2573     }
2574     route = [
2575         item
2576         for item in self.planned_ew
2577         if item[0] in pending_indices
2578     ]
2579     used_indices = {
2580         index
2581         for index, _ in route
2582     }
2583     cursor = route[-1][1] if route else self.position
2584
2585     phases_left = sorted(
2586         {
2587             search_phase(index)
2588             for index, _ in self.pending_ew
2589             if index not in used_indices
2590         }
2591     )
2592
2593     for phase in phases_left:
2594         entries = sorted(
2595             (
2596                 item
2597                 for item in self.pending_ew
2598                 if item[0] not in used_indices
2599                 and search_phase(item[0]) == phase
2600             ),
2601             key=lambda item: item[0],
2602         )
2603
2604         if len(entries) <= 1:
2605             best_route = entries
2606         else:
2607             count = len(entries)
2608             cyclic_routes = []
2609
2610             for start in range(count):
2611                 cyclic_routes.append(
2612                     [
2613                         entries[(start + offset) % count]
2614                         for offset in range(count)
2615                     ]
2616                 )
2617                 cyclic_routes.append(
2618                     [
2619                         entries[(start - offset) % count]
2620                         for offset in range(count)
2621                     ]
2622                 )
2623
2624             best_route = min(
2625                 cyclic_routes,
2626                 key=lambda candidate: (
2627                     open_route_length(
2628                         cursor,
2629                         [point for _, point in candidate],
2630                     ),
2631                     self.search_route_information_key(candidate),
2632                 ),
2633             )
2634
2635             route.extend(best_route)
2636             used_indices.update(index for index, _ in best_route)
2637             if best_route:
2638                 cursor = best_route[-1][1]
2639
2640     return route
2641
2642     def best_future_search_insertion_detour(
2643         self,
2644         target: Point,
2645         route: Optional[list[tuple[int, Point]]] = None,
2646     ) -> float:
2647         """ 估计目标点插入后续搜索路线的最小绕路。 """
2648         if route is None:
2649             route = self.remaining_search_route()
2650         if not route:
2651             return 0.0
2652
2653         previous = route[0][1]
2654         best = math.inf
2655
2656         for _, next_point in route[1:]:
2657             detour = (
2658                 distance(previous, target)
2659                 + distance(target, next_point)
2660                 - distance(previous, next_point)
2661             )
2662             best = min(best, max(0.0, detour))
2663             previous = next_point
2664
2665         # 最后一个 EW 后直接接目标点。
2666         best = min(best, distance(previous, target))
2667         return best
2668
2669     def post_search_marginal_costs(
2670         self,
2671         candidates: list[tuple[float, tuple]],

```

```

2672     start: Point,
2673 ) -> dict[int, float]:
2674     """ 估计搜索结束后各频道任务的联合路线边际成本。"""
2675     per_channel: dict[int, tuple[float, tuple]] = {}
2676
2677     for item in candidates:
2678         action = item[1]
2679         if action[0] == "EW":
2680             continue
2681
2682         channel = action[1]
2683         old = per_channel.get(channel)
2684         if old is None or item[0] < old[0]:
2685             per_channel[channel] = item
2686
2687     channels = list(per_channel)
2688     if not channels:
2689         return {}
2690
2691     points = [
2692         per_channel[channel][1][2]
2693         for channel in channels
2694     ]
2695     full_order = optimized_open_order(start, points)
2696     full_length = open_route_length(
2697         start,
2698         [points[index] for index in full_order],
2699     )
2700     marginal: dict[int, float] = {}
2701
2702     for removed_index, channel in enumerate(channels):
2703         reduced_points = [
2704             point
2705             for index, point in enumerate(points)
2706             if index != removed_index
2707         ]
2708         reduced_order = optimized_open_order(start, reduced_points)
2709         reduced_length = open_route_length(
2710             start,
2711             [reduced_points[index] for index in reduced_order],
2712         )
2713         marginal[channel] = max(
2714             0.0,
2715             full_length - reduced_length,
2716         )
2717
2718     return marginal
2719
2720 def remaining_ew_can_refine(self, channel: int) -> bool:
2721     """ 是否存在无需额外移动且满足原顺路门槛的未来 EW。"""
2722     return any(
2723         self.opportunistic_score(channel, point) is not None
2724         for _, point in self.pending_ew
2725     )
2726
2727 def search_insertion_detour(self, target: Point) -> float:
2728     next_ew = self.next_search_waypoint()
2729
2730     if next_ew is None:
2731         return 0.0
2732
2733     _, next_point = next_ew
2734     return max(
2735         0.0,
2736         distance(self.position, target)
2737         + distance(target, next_point)
2738         - distance(self.position, next_point),
2739     )
2740
2741 def action_travel_cost(self, target: Point) -> float:
2742     """ 计算当前动作的移动代价。"""
2743     if not self.pending_ew:
2744         return distance(self.position, target)
2745
2746     return self.search_insertion_detour(target)
2747
2748 def search_action_priority(
2749     self,
2750     item: tuple[float, tuple],
2751     current_detour: float,
2752     future_detour: float,
2753     can_refine_on_ew: bool,
2754 ) -> Optional[tuple[float, float, float, float]]:
2755     """ 给出搜索期联合动作优先级; None 表示先继续搜索路线。"""
2756     if can_refine_on_ew:
2757         future_detour = 0.0
2758
2759     action = item[1]
2760     reason = action[3]
2761     reliability = 1.0
2762
2763     if reason.startswith(" 两线"):
2764         reliability = GEOM_PROBE_RELIABILITY
2765     elif reason.startswith(" 强 Probe"):
2766         reliability = STRONG_PROBE_RELIABILITY
2767     elif reason.startswith(" 弱 Probe"):
2768         reliability = WEAK_PROBE_RELIABILITY
2769     elif reason.startswith("Q2"):
2770         reliability = Q2_MEASURE_RELIABILITY

```

```

2771         elif action[0] == "RECOVERY":
2772             reliability = RECOVERY_RELIABILITY
2773
2774         future_detour *= reliability
2775
2776         # score = 绕路 - 收益。
2777         route_regret = item[0] - future_detour
2778         if route_regret > 1e-9:
2779             return None
2780
2781         return (
2782             route_regret,
2783             item[0],
2784             current_detour,
2785             future_detour,
2786         )
2787
2788     def choose_post_search_action(
2789         self,
2790         candidates: list[tuple[float, tuple]],
2791     ) -> tuple[float, tuple]:
2792         """ 搜索结束后按当前任务池生成开放路线并选第一步。"""
2793         per_channel: dict[int, tuple[float, tuple]] = {}
2794
2795         for item in candidates:
2796             action = item[1]
2797             channel = action[1]
2798             old = per_channel.get(channel)
2799
2800             if old is None or item[0] < old[0]:
2801                 per_channel[channel] = item
2802
2803         tasks = list(per_channel.values())
2804         points = [item[1][2] for item in tasks]
2805         order = optimized_open_order(self.position, points)
2806
2807         if not order:
2808             raise RuntimeError(" 搜索后仍有活动频道但任务路线为空")
2809
2810         self.post_search_route_plan_count += 1
2811         planned = tasks[order[0]]
2812         greedy = min(candidates, key=lambda item: item[0])
2813         plan_dist = distance(
2814             self.position,
2815             planned[1][2],
2816         )
2817         greedy_distance = distance(
2818             self.position,
2819             greedy[1][2],
2820         )
2821
2822         if (
2823             planned[0]
2824             > greedy[0] + POST_ROUTE_FIRST_STEP_REGRET_M
2825             or plan_dist
2826             > greedy_distance + POST_ROUTE_FIRST_STEP_REGRET_M
2827         ):
2828             return greedy
2829
2830         return planned
2831
2832     def _choose_action_v5(self):
2833         candidates = []
2834
2835         # 每个搜索环固定路线。
2836         next_ew = self.next_search_waypoint()
2837         if next_ew is not None:
2838             ew_index, ew = next_ew
2839             current_search_phase = search_phase(ew_index)
2840             progress_bonus = (
2841                 INNER_EW_PROGRESS_BONUS_M
2842                 if current_search_phase == 0
2843                 else EW_PROGRESS_BONUS_M
2844             )
2845             candidates.append(
2846                 (
2847                     distance(
2848                         self.position,
2849                         ew,
2850                     )
2851                     - progress_bonus,
2852                     (
2853                         "EW",
2854                         ew_index,
2855                         ew,
2856                         " 完备搜索",
2857                     ),
2858                 )
2859             )
2860
2861         # LOCALIZING 频道
2862         for ch in self.active_channels():
2863             record = self.channels[ch]
2864
2865             if record.strong_position is not None:
2866                 candidates.append(
2867                     (
2868                         self.action_travel_cost(record.strong_position)
2869                         - GUARANTEED_CLEAR_BONUS_M,

```

```

2870         (
2871             "CLEAR",
2872             ch,
2873             record.strong_position,
2874             "near 保证清除",
2875             False,
2876         ),
2877     )
2878 )
2879 continue
2880
2881 info = self.target_info(ch)
2882
2883 # lost 频道按四点恢复。
2884 if record.lost:
2885     generation = len(record.directions)
2886     if record.recovery_generation != generation:
2887         record.recovery_generation = generation
2888         recovery_points = self.recovery_candidates(ch)
2889         # 先走前向恢复点, 再补侧向点。
2890         forward_pair = sorted(
2891             recovery_points[:2],
2892             key=lambda point: distance(self.position, point),
2893         )
2894         lateral_pair = sorted(
2895             recovery_points[2:],
2896             key=lambda point: distance(self.position, point),
2897         )
2898         record.recovery_pending = forward_pair + lateral_pair
2899
2900     if not record.recovery_pending:
2901         raise RuntimeError(
2902             f" 频道 {ch} 四点恢复扇全部失败; "
2903             " 这与 180 度覆盖及 1000m 最小半径约束矛盾"
2904         )
2905
2906     recovery = record.recovery_pending[0]
2907
2908     candidates.append(
2909         (
2910             self.action_travel_cost(recovery),
2911             (
2912                 "RECOVERY",
2913                 ch,
2914                 recovery,
2915                 " 定向源四点恢复扇",
2916             ),
2917         )
2918     )
2919
2920     continue
2921
2922 # 保证 clear
2923 if (
2924     info.mec_radius
2925     <= CLEAR_RADIUS - 1e-6
2926     and not self.too_close_to_failed_clear(
2927         ch,
2928         info.mec_center,
2929     )
2930 ):
2931     candidates.append(
2932         (
2933             self.action_travel_cost(info.mec_center)
2934             - GUARANTEED_CLEAR_BONUS_M,
2935             (
2936                 "CLEAR",
2937                 ch,
2938                 info.mec_center,
2939                 "MEC<=20m 保证清除",
2940                 False,
2941             ),
2942         )
2943     )
2944
2945     continue
2946
2947 n_direction = len(
2948     record.directions
2949 )
2950
2951 # 几何 probe
2952 geometric_probe = self.best_geometric_probe(ch)
2953
2954 if (
2955     geometric_probe is not None
2956     and not self.too_close_to_failed_clear(
2957         ch,
2958         geometric_probe.point,
2959     )
2960 ):
2961     candidates.append(
2962         (
2963             self.action_travel_cost(geometric_probe.point)
2964             - GEOM_PROBE_BONUS_M,
2965             (
2966                 "CLEAR",
2967                 ch,
2968                 geometric_probe.point,

```

```

2969         (
2970             " 两线交点 Probe "
2971             f"err≈{geometric_probe.error_proxy_m:.1f}m, "
2972             f"sin={geometric_probe.sin_phi:.2f}"
2973         ),
2974         True,
2975     ),
2976 )
2977 )
2978
2979 # probe clear
2980 if (
2981     n_direction >= 2
2982     and info.work_radius
2983     <= STRONG_PROBE_RADIUS_M
2984     and not self.too_close_to_failed_clear(
2985         ch,
2986         info.center,
2987     )
2988 ):
2989     candidates.append(
2990         (
2991             self.action_travel_cost(info.center)
2992             - STRONG_PROBE_BONUS_M,
2993             (
2994                 "CLEAR",
2995                 ch,
2996                 info.center,
2997                 (
2998                     " 强 Probe Clear "
2999                     f"workR={info.work_radius:.1f}"
3000                 ),
3001                 True,
3002             ),
3003         )
3004     )
3005
3006     continue
3007
3008 if (
3009     n_direction >= 2
3010     and info.work_radius
3011     <= WEAK_PROBE_RADIUS_M
3012     and not self.too_close_to_failed_clear(
3013         ch,
3014         info.center,
3015     )
3016 ):
3017     candidates.append(
3018         (
3019             self.action_travel_cost(info.center)
3020             - WEAK_PROBE_BONUS_M,
3021             (
3022                 "CLEAR",
3023                 ch,
3024                 info.center,
3025                 (
3026                     " 弱 Probe Clear "
3027                     f"workR={info.work_radius:.1f}"
3028                 ),
3029                 True,
3030             ),
3031         )
3032     )
3033
3034     # 同时保留普通 refine。
3035
3036 # 后验较小时改用 Sweep。
3037 if (
3038     n_direction >= SWEEP_TRIGGER_DIRECTION_COUNT
3039     and info.work_radius <= SWEEP_TRIGGER_WORK_RADIUS_M
3040 ):
3041     if record.sweep_polygon_version != n_direction:
3042         record.sweep_polygon_version = n_direction
3043         record.sweep_pending = self.build_posterior_sweep(ch)
3044
3045     if record.sweep_pending:
3046         sweep_point = min(
3047             record.sweep_pending,
3048             key=Lambda point: distance(self.position, point),
3049         )
3050         candidates.append(
3051             (
3052                 self.action_travel_cost(sweep_point)
3053                 - SWEEP_CLEAR_BONUS_M,
3054                 (
3055                     "SWEEP_CLEAR",
3056                     ch,
3057                     sweep_point,
3058                     " 后验网格保证清除",
3059                     False,
3060                 ),
3061             )
3062         )
3063
3064 # 首次 direction 后加入 Q2-fast。
3065 if n_direction == 1:
3066     for point in self.q2_fast_candidates(ch):
3067         info_gain = self.q2_fast_information(

```

```

3068         ch,
3069         point,
3070     )
3071
3072     candidates.append(
3073         (
3074             self.action_travel_cost(point)
3075             - Q2_FAST_INFO_BONUS_M
3076             * info_gain,
3077             (
3078                 "MEASURE",
3079                 ch,
3080                 point,
3081                 (
3082                     "Q2-fast 第二测点 "
3083                     f"sin≈{info_gain:.2f}"
3084                 ),
3085             ),
3086         )
3087     )
3088
3089     # 始终保留后验中心候选。
3090     candidates.append(
3091         (
3092             self.action_travel_cost(info.center),
3093             (
3094                 "MEASURE",
3095                 ch,
3096                 info.center,
3097                 (
3098                     " 直接逼近后验中心 "
3099                     f"workR={info.work_radius:.1f}, "
3100                     f"mecR={info.mec_radius:.1f}"
3101                 ),
3102             ),
3103         )
3104     )
3105
3106     if not candidates:
3107         return None
3108
3109     if self.pending_ew:
3110         ew_candidate = next(
3111             item
3112             for item in candidates
3113             if item[1][0] == "EW"
3114         )
3115         due_actions = []
3116         remaining_route = self.remaining_search_route()
3117         refine_cache: dict[int, bool] = {}
3118         route_end = remaining_route[-1][1]
3119         marginal_cost = self.post_search_marginal_costs(
3120             candidates,
3121             route_end,
3122         )
3123
3124         for item in candidates:
3125             action = item[1]
3126
3127             if action[0] == "EW":
3128                 continue
3129
3130             current_detour = self.search_insertion_detour(action[2])
3131             future_detour = self.best_future_search_insertion_detour(
3132                 action[2],
3133                 remaining_route,
3134             )
3135             future_detour = min(
3136                 future_detour,
3137                 marginal_cost.get(
3138                     action[1],
3139                     future_detour,
3140                 ),
3141             )
3142
3143             # 有顺路 EW 时, measure/recovery 的未来移动代价按 0。
3144             can_refine_on_ew = False
3145             if action[0] in {"MEASURE", "RECOVERY"}:
3146                 channel = action[1]
3147                 if channel not in refine_cache:
3148                     refine_cache[channel] = (
3149                         self.remaining_ew_can_refine(channel)
3150                     )
3151                 can_refine_on_ew = refine_cache[channel]
3152             # score = 绕路 - 收益。
3153             # 当前净代价不高于未来插入代价时才提前做。
3154             priority = self.search_action_priority(
3155                 item,
3156                 current_detour,
3157                 future_detour,
3158                 can_refine_on_ew,
3159             )
3160
3161             if priority is not None:
3162                 due_actions.append(
3163                     (*priority, item)
3164                 )
3165             elif can_refine_on_ew:
3166                 record = self.channels[action[1]]

```

```

3167         self.future_ew_deferred_states.add(
3168             (
3169                 action[1],
3170                 len(record.directions),
3171                 record.last,
3172             )
3173         )
3174
3175     if not due_actions:
3176         return ew_candidate
3177
3178     (
3179         -,
3180         -,
3181         current_detour,
3182         future_detour,
3183         chosen,
3184     ) = min(
3185         due_actions,
3186         key=lambda entry: entry[:3],
3187     )
3188     self.search_insertion_action_count += 1
3189     self.search_insertion_detour_m += current_detour
3190     self.search_insertion_future_cost_m += future_detour
3191     category = self.reason_category(chosen[1][3])
3192     self.search_insertion_count_by_category[category] = (
3193         self.search_insertion_count_by_category.get(
3194             category,
3195             0,
3196         )
3197         + 1
3198     )
3199     return chosen
3200
3201     return self.choose_post_search_action(candidates)
3202
3203 def choose_action(self):
3204     # SAFE 层有缓存动作时先执行缓存。
3205     if self.safe_forced_action is not None:
3206         if self.safe_forced_channels_patch is not None:
3207             self.channels = copy.deepcopy(
3208                 self.safe_forced_channels_patch
3209             )
3210
3211         if self.safe_forced_planned_ew_patch is not None:
3212             self.planned_ew = copy.deepcopy(
3213                 self.safe_forced_planned_ew_patch
3214             )
3215
3216         chosen = self.safe_forced_action
3217
3218         self.safe_forced_action = None
3219         self.safe_forced_channels_patch = None
3220         self.safe_forced_planned_ew_patch = None
3221
3222         return chosen
3223
3224     # 先按原 V5 选动作，再由 SAFE 层尝试缩短 clear 路径。
3225     chosen = self._choose_action_v5()
3226
3227     if chosen is None:
3228         return None
3229
3230     return self.safe_transform_selected_action(chosen)
3231
3232 # 运行
3233
3234 def run(self) -> None:
3235     # 墙钟时间只做统计。
3236     wall_start = time.perf_counter()
3237
3238     for path in (TRACE_PATH, SUMMARY_PATH):
3239         try:
3240             path.unlink()
3241         except FileNotFoundError:
3242             pass
3243
3244     validation = validate_direction_complete_backbone()
3245
3246     print("=" * 80)
3247     print("Q4 DIRECTION-COMPLETE SEARCH")
3248     print(
3249         f"{SEARCH_POINT_COUNT} 点局部凸包方向完备搜索"
3250     )
3251     print(
3252         " 阶段 A 诊断覆盖半径 = "
3253         f"{validation['phase_a_omni_cover_radius_m']:.3f} m"
3254     )
3255     print(
3256         " 外环正十四边形内切半径 = "
3257         f"{validation['outer_apothem_m']:.3f} m"
3258     )
3259     print(
3260         " 证书距离/凸包安全余量 = "
3261         f"{validation['certificate_distance_slack_m']:.3f} / "
3262         f"{validation['certificate_hull_slack_m']:.3f} m"
3263     )
3264     print("=" * 80)
3265

```

```

3266     enter = self.client.enter()
3267
3268     if enter.get("accepted") is not True:
3269         raise RuntimeError(
3270             f"/enter rejected: {enter}"
3271         )
3272
3273     OUTPUT_DIR.mkdir(parents=True, exist_ok=False)
3274
3275     self.remaining_real_duration_s = float(
3276         enter["remaining_real_duration_s"]
3277     )
3278     if "virtual_time_s" in enter:
3279         self.server_virtual_time_s = float(enter["virtual_time_s"])
3280
3281     self.trace(
3282         {
3283             "action": "enter",
3284             "remaining_real_duration_s": (
3285                 self.remaining_real_duration_s
3286             ),
3287         }
3288     )
3289
3290     for logical_step in range(
3291         1,
3292         MAX_LOGICAL_ACTIONS + 1,
3293     ):
3294         self.apply_found16_shortcut()
3295         self.certify_empty()
3296
3297         if self.cleared_count >= SOURCE_MAX:
3298             print(" 已清除 16 个源，提前结束。")
3299             break
3300
3301         if (
3302             self.cleared_count
3303             + self.empty_count
3304             == CHANNEL_COUNT
3305         ):
3306             print(
3307                 " 全部 20 频道已清除或严格判空。"
3308             )
3309             break
3310
3311         chosen = self.choose_action()
3312
3313         if chosen is None:
3314             raise RuntimeError(
3315                 " 无动作但尚未满足终止条件"
3316             )
3317
3318         score, action = chosen
3319         kind = action[0]
3320
3321         if kind == "EW":
3322             _, ew_index, ew, reason = action
3323
3324             print(
3325                 f"\n[{{logical_step}}] EW{{ew_index}} "
3326                 f"score={{score:.1f}} "
3327                 f"({{ew[0]:.1f}},{{ew[1]:.1f}})"
3328             )
3329
3330             self.pending_ew.remove(
3331                 (ew_index, ew)
3332             )
3333             self.mark_search_waypoint_done(ew_index)
3334
3335             self.execute_ew(
3336                 ew_index,
3337                 ew,
3338             )
3339
3340         elif kind in {"MEASURE", "RECOVERY"}:
3341             _, ch, point, reason = action
3342
3343             print(
3344                 f"\n[{{logical_step}}] MEASURE "
3345                 f"ch={{ch}} score={{score:.1f}} "
3346                 f"{{reason}}"
3347             )
3348
3349             if kind == "RECOVERY":
3350                 record = self.channels[ch]
3351                 record.recovery_pending.remove(point)
3352
3353             if kind == "RECOVERY":
3354                 self.recovery_attempts += 1
3355
3356             result, _ = self.measure_at(
3357                 point,
3358                 ch,
3359                 mark_lost_on_no_signal=True,
3360                 reason=reason,
3361             )
3362
3363             if (
3364                 kind == "RECOVERY"

```

```

3365         and result in {"direction", "near"}
3366     ):
3367         self.recovery_receive_count += 1
3368
3369     elif kind in {"CLEAR", "SWEEP_CLEAR"}:
3370         (
3371             -,
3372             ch,
3373             point,
3374             reason,
3375             measure_after_failure,
3376         ) = action
3377
3378     if kind == "SWEEP_CLEAR":
3379         self.channels[ch].sweep_pending.remove(point)
3380
3381     print(
3382         f"\n[{{logical_step}}] CLEAR "
3383         f"ch={{ch}} score={{score:.1f}} "
3384         f"reason="
3385     )
3386
3387     is_geom_probe = (
3388         reason.startswith(" 两线交点 Probe")
3389     )
3390
3391     if is_geom_probe:
3392         self.geom_probe_attempts += 1
3393
3394     clear_ok = self.clear_at(
3395         point,
3396         ch,
3397         reason=reason,
3398         measure_after_failure=(
3399             measure_after_failure
3400         ),
3401     )
3402
3403     if is_geom_probe and clear_ok:
3404         self.geom_probe_success += 1
3405
3406     else:
3407         raise RuntimeError(
3408             f" 未知动作: {{kind}}"
3409         )
3410
3411     if (
3412         not self.pending_ew
3413         and self.search_completed_virtual_time_s is None
3414     ):
3415         self.search_completed_virtual_time_s = (
3416             self.ledger.total_seconds
3417         )
3418
3419     print(
3420         f"   cleared={{self.cleared_count}}, "
3421         f"found={{self.found_count}}, "
3422         f"empty={{self.empty_count}}, "
3423         f"active={{len(self.active_channels())}}, "
3424         f"EW_left={{len(self.pending_ew)}}, "
3425         f"T≈{{self.ledger.total_seconds:.1f}}s"
3426     )
3427
3428     else:
3429         raise RuntimeError(
3430             " 超过最大逻辑动作数, 疑似循环"
3431         )
3432
3433     exit_response = self.client.exit()
3434     if "virtual_time_s" in exit_response:
3435         self.server_virtual_time_s = float(
3436             exit_response["virtual_time_s"]
3437         )
3438
3439     # 统计到/exit 完成为止。
3440     wall_runtime_s = (
3441         time.perf_counter() - wall_start
3442     )
3443
3444     summary = {
3445         "cleared": self.cleared_count,
3446         "found": self.found_count,
3447         "certified_empty": self.empty_count,
3448         "early_stop_search_by_found16": (
3449             self.early_stop_search_by_found16
3450         ),
3451         "geom_probe_attempts": self.geom_probe_attempts,
3452         "geom_probe_success": self.geom_probe_success,
3453         "search_certificate_version": CERTIFICATE_VERSION,
3454         "scheduler_version": SCHEDULER_VERSION,
3455         "search_visit_count": self.search_visit_count,
3456         "search_completed_virtual_time_s": (
3457             self.search_completed_virtual_time_s
3458         ),
3459         "opportunistic_measure_count": (
3460             self.opportunistic_measure_count
3461         ),
3462         "opportunistic_receive_count": (
3463             self.opportunistic_receive_count

```

```

3464         ),
3465         "recovery_attempts": self.recovery_attempts,
3466         "recovery_receive_count": self.recovery_receive_count,
3467         "search_insertion_action_count": (
3468             self.search_insertion_action_count
3469         ),
3470         "search_insertion_detour_m": (
3471             self.search_insertion_detour_m
3472         ),
3473         "search_insertion_future_cost_m": (
3474             self.search_insertion_future_cost_m
3475         ),
3476         "search_insertion_count_by_category": (
3477             self.search_insertion_count_by_category
3478         ),
3479         "future_ew_deferred_state_count": len(
3480             self.future_ew_deferred_states
3481         ),
3482         "post_search_route_plan_count": (
3483             self.post_search_route_plan_count
3484         ),
3485         "safe_clear_shift_count": (
3486             self.safe_clear_shift_count
3487         ),
3488         "safe_clear_shift_saved_m_est": (
3489             self.safe_clear_shift_saved_m
3490         ),
3491         "safe_clear_shift_saved_s_est": (
3492             self.safe_clear_shift_saved_m / SPEED
3493         ),
3494         "safe_clear_segment_hit_count": (
3495             self.safe_clear_segment_hit_count
3496         ),
3497         "first_discovery_virtual_time_s": (
3498             self.first_discovery_virtual_time_s
3499         ),
3500         "move_distance_m": self.ledger.move_distance,
3501         "movement_seconds": self.ledger.move_seconds,
3502         "measure_count": self.ledger.measure_count,
3503         "measure_seconds": self.ledger.measure_seconds,
3504         "switch_count": self.ledger.switch_count,
3505         "switch_seconds": self.ledger.switch_seconds,
3506         "clear_success": self.ledger.clear_success,
3507         "clear_fail": self.ledger.clear_fail,
3508         "clear_seconds": self.ledger.clear_seconds,
3509         "estimated_virtual_time": self.ledger.total_seconds,
3510         "simulator_virtual_time_s": self.server_virtual_time_s,
3511         "program_wall_runtime_s": wall_runtime_s,
3512         "program_wall_runtime_min": (
3513             wall_runtime_s / 60.0
3514         ),
3515         "estimated_time_error_s": (
3516             self.ledger.total_seconds - self.server_virtual_time_s
3517             if self.server_virtual_time_s is not None
3518             else None
3519         ),
3520         "action_count_by_category": self.action_count_by_category,
3521         "move_distance_by_category_m": (
3522             self.move_distance_by_category_m
3523         ),
3524         "average_seconds_per_cleared": (
3525             self.ledger.total_seconds
3526             / self.cleared_count
3527             if self.cleared_count
3528             else None
3529         ),
3530         "search_point_count": SEARCH_POINT_COUNT,
3531         **validation,
3532         "exit_reason": exit_response.get(
3533             "exit_reason"
3534         ),
3535     }
3536
3537     SUMMARY_PATH.write_text(
3538         json.dumps(
3539             summary,
3540             ensure_ascii=False,
3541             indent=2,
3542         ),
3543         encoding="utf-8",
3544     )
3545
3546     self.trace(
3547         {
3548             "action": "exit",
3549             "exit_reason": exit_response.get(
3550                 "exit_reason"
3551             ),
3552             "summary": summary,
3553         }
3554     )
3555
3556     print("\n" + "=" * 80)
3557     print("Q4 RESULT")
3558     print("=" * 80)
3559
3560     for key, value in summary.items():
3561         print(f"{key:28s}: {value}")
3562

```

```
3563     print(
3564         "\n 程序真实运行时间: "
3565         f"{wall_runtime_s:.3f} s "
3566         f"({wall_runtime_s / 60.0:.3f} min)"
3567     )
3568
3569     print(
3570         f"\n 明文轨迹: {TRACE_PATH.resolve()}"
3571     )
3572
3573     print(
3574         f" 明文汇总: {SUMMARY_PATH.resolve()}"
3575     )
3576
3577
3578 def main() -> None:
3579     if "--self-test" in sys.argv:
3580         run_geometry_self_test()
3581         return
3582
3583     agent = Q4Agent()
3584     try:
3585         agent.run()
3586     except KeyboardInterrupt:
3587         print("\n 程序被手动中止。若已经成功进入测试，请在模拟器中确认测试状态。")
3588         raise SystemExit(130)
3589     except Exception as exc:
3590         print("\n" + "=" * 80)
3591         print(" 第四问程序未能继续执行")
3592         print("=" * 80)
3593         print(str(exc))
3594         print(
3595             "\n 若错误发生在 /enter: 这不是搜索算法错误，而是模拟器接口尚未建立。 \n"
3596             " 检查顺序: 模拟器进程仍在运行 -> 界面显示接口已就绪 -> "
3597             "BASE_URL 为 127.0.0.1:2026 -> 防火墙/代理未拦截本机端口。"
3598         )
3599         raise SystemExit(1)
3600
3601
3602 if __name__ == "__main__":
3603     main()
```